FPT UNIVERSITY

FPT UNIVERSITY
SoftwareDesignDocument

Frevia

GroupBar	
Group Members	Pham Kha Vy - CE182254 Nguyen Tien Thuan - CE181024 Tran Phat Tai - CE181632 Vo Nhut Tin - CE180619 Nguyen Lu Nhat Huy - CE181196
Supervisor	Mrs. Cao Hong Nga
Ext Supervisor	
Capstone Project code	FRE

- Can Tho, March 2025 -

Table of Contents

1 Product Overview **1**

2 User Requirement **3**

2.1 Actors 3

2.2 Use Cases 4

2.2.1 Diagram(s) 4

2.2.2 Use Case Description 10

2.2.3 Use Case Specification 17

2.2.1 Authentication 17

2.2.3.1 UC-01 Login with Google 17

2.2.3.2 UC-02 Login 19

2.2.3.3 UC-03 Logout 21

2.2.3.4 UC-04 Register 22

2.2.3.5 UC-05 Forgot Password 25

2.2.2 Role Management 27

2.2.3.1 UC-06.01 View Role List 27

2.2.3.2 UC-06.02 View Role Detail 28

2.2.3.3 UC-06.03 Create Role 30

2.2.3.4 UC-06.04 Update Role 31

2.2.3.5 UC-06.05 Delete Role 33

2.2.3.6 UC-06.06 Update Role Permissions 34

2.2.3 Manage Freelancer Profile 36

2.2.3.1 UC-10.1 View Detailed Freelancer Profile 36

2.2.3.2 UC-10.2 Update Your Detailed Freelancer Profile . 37

2.2.4 Manage Freelancer Skills 39

2.2.3.1 UC-11.1 View List Freelancer Skill 39

2.2.3.2 UC-11.2 Add Freelancer Skill 40

2.2.3.3 UC-11.3 Delete Freelancer Skill 41

2.2.5 Manage Freelancer Portfolio 42

2.2.3.1 UC-12.1 View Portfolio List 42

2.2.3.2 UC-12.2 Add New Portfolio 43

2.2.3.3 UC-12.3 View Portfolio Detail 45

2.2.3.4 UC-12.4 Update Portfolio 46

2.2.3.5 UC-12.5 Delete Portfolio 47

2.2.6 Browse Jobs 48

2.2.3.1 UC-03.1 View List Job 48

2.2.3.2 UC-03.2 View Job Detail 50

2.2.3.3 UC-06.2 Post New Job 52

2.2.3.4 UC-06.3 Edit Job 54
2.2.3.5 UC-06.4 Delete Job 56
2.2.3.6 UC-06.5 Change Job Status 58
2.2.3.7 UC-06.6 View List Bookmarked Job 60
2.2.3.8 UC-06.7 Bookmark Job 62
2.2.3.9 UC-06.8 Remove Bookmark 64
2.2.7 Manage Contract 65
2.2.3.1 UC-13.01 Create Contract 65
2.2.3.2 UC-13.02 Update Contract 67
2.2.3.3 UC-13.03 Complete Contract 68
2.2.3.4 UC-13.04 View Contract List 70
2.2.3.5 UC-13.05 View Contract Detail 71
2.2.3.6 UC-13.06 Sign Contract 73
2.2.3.7 UC-13.07 Cancel Contract 74
2.2.3.8 UC-13.08 View Contract Files 76
2.2.3.9 UC-13.09 Upload Contract File 77
2.2.3.10 UC-13.10 Delete Contract File 79
2.2.8 Manage Forum 80
2.2.3.1 UC-04.1 View Forum Categories 80
2.2.3.2 UC-04.2 View Forum Category Detail 82
2.2.3.3 UC-04.3 View Post List 84
2.2.3.4 UC-04.4 View Post Detail 86
2.2.3.5 UC-04.5 Create Post 88
2.2.3.6 UC-04.6 Edit Post 91
2.2.3.7 UC-04.7 Delete Post 93
2.2.3.8 UC-04.8 Like Post 95
2.2.3.9 UC-04.9 Unlike Post 97
2.2.3.10 UC-04.10 View Like Post Detail 99
2.2.3.11 UC-04.11 View Post Comments 101
2.2.3.12 UC-04.12 Add Comment 103
2.2.3.13 UC-04.13 Edit Comment 105
2.2.3.14 UC-04.14 Delete Comment 107
2.2.3.15 UC-04.15 Like Comment 109
2.2.3.16 UC-04.16 Unlike Comment 111
2.2.3.17 UC-04.17 Report Content 113
2.2.9 Manage Milestone 116
2.2.3.1 UC-24.01 View Milestone List 116
2.2.3.2 UC-24.02 View Milestone Detail 118
2.2.3.3 UC-24.03 Create Milestone 120

2.2.3.4 UC-24.04 Update Milestone 122
2.2.3.5 UC-24.05 Delete Milestone 123
2.2.3.6 UC-24.06 Request Milestone Changes 124
2.2.3.7 UC-24.07 Approve Milestone 126
2.2.3.8 UC-24.08 Update Milestone Progress 128
2.2.10 Manage Milestone File 129
2.2.3.1 UC-25.01 View Milestone Files 129
2.2.3.2 UC-25.02 Upload Milestone File 130
2.2.3.3 UC-25.03 Delete Milestone File 132
2.2.11 Manage Milestone Submission 133
2.2.3.1 UC-26.01 Submit Milestone 133
2.2.3.2 UC-26.02 Delete Milestone Submission 135
2.2.3.3 UC-26.03 View Milestone Submission List 136
2.2.3.4 UC-26.04 View Milestone Submission Detail 137
2.2.12 Manage Conversation 139
2.2.3.1 UC-05.1 View Conversations List 139
2.2.3.2 UC-05.2 Start Conversation 141
2.2.3.3 UC-05.3 Conversation Detail 143
2.2.3.4 UC-05.4 Send Message 145
2.2.3.5 UC-05.5 Mark Message as Read 148
2.2.3.6 UC-05.6 Delete/Recall Message 150
2.2.3.7 UC-05.7 Pin/Unpin Conversation 152
2.2.3.8 UC-05.8 Send File 155
2.2.13 Manage Saved Search 158
2.2.3.1 UC View Saved Searches 158
2.2.3.2 UC View Saved Search Detail 159
2.2.3.3 UC Create Saved Search 161
2.2.3.4 UC Edit Saved Search 162
2.2.3.5 UC Delete Saved Search 164
2.2.3.6 UC View Job Alerts 166
2.2.3.7 UC View Job Alert Detail 167
2.2.3.8 UC Create Job Alert 168
2.2.3.9 UC Edit Job Alert 170
2.2.3.10 UC Delete Job Alert 171
2.2.14 Manage Permission 173
2.2.3.1 UC-15.01 View Permission List 173
2.2.3.2 UC-15.02 View Permission Detail 175
2.2.15 Session Management 177
2.2.3.1 UC-16.01 View Session List 177

2.2.3.2 UC-16.02 View Session Detail 179
2.2.3.3 UC-16.03 Revoke Session 181
2.2.16 Forum Category Management 183
2.2.3.1 UC-17.01 Create Forum Category 183
2.2.3.2 UC-17.02 Update Forum Category 185
2.2.3.3 UC-17.03 Delete Forum Category 187
2.2.3.4 UC-14.01 View Forum Category List 189
2.2.3.5 UC-14.02 View Forum Category Detail 191
2.2.3.6 UC-17.06 Restore Forum Category 193
2.2.17 Manage Dispute 195
2.2.3.1 UC-33.01 Open Dispute 195
2.2.3.2 UC-33.02 View Dispute List 197
2.2.3.3 UC-33.03 View Dispute Detail 198
2.2.3.4 UC-33.04 Respond to Dispute 200
2.2.3.5 UC-33.05 Respond to Dispute Resolution 202
2.2.3.6 UC-33.06 Propose Dispute Resolution 204
2.2.18 Manage Moderation 206
2.2.3.1 UC-06.1 View Moderation Queue 206
2.2.3.2 UC-06.2 Moderation Item Detail 209
2.2.3.3 UC-06.3 Resolve Moderation 210
2.2.3.4 UC-06.4 View Reports Queue 214
2.2.3.5 UC-06.5 Update Report Status 217
2.2.19 Manage ID Verification 218
2.2.3.1 UC-06.6 View ID Verification Requests 218
2.2.3.2 UC-06.7 ID Verification Detail 221
2.2.3.3 UC-06.8 Approve ID Verification 223
2.2.3.4 UC-06.9 Reject ID Verification 226
2.2.20 Manage Forum Trash 229
2.2.3.1 UC-06.10 View Deleted Posts / Comments List . . . 229
2.2.3.2 UC-06.11 View Deleted Posts / Comments Detail . 232
2.2.3.3 UC-06.12 Restore Deleted Posts / Comments 234
2.2.21 Manage Proposal 237
2.2.3.1 UC View My Proposals 237
2.2.3.2 UC View My Proposal Detail 238
2.2.3.3 UC Submit Proposal 240
2.2.3.4 UC Edit Proposal 242
2.2.3.5 UC Withdraw Proposal 244
2.2.3.6 UC View Job Proposals 246
2.2.3.7 UC View Proposal Detail 248

2.2.3.8 UC Accept Proposal . . . 250
2.2.3.9 UC Reject Proposal . . . 252
2.2.22 Manage User . . . 255
2.2.3.1 UC-17.1 View User List . . . 255
2.2.3.2 UC-17.2 View User Detail . . . 257
2.2.3.3 UC-17.3 Create User . . . 260
2.2.3.4 UC-17.4 Edit User . . . 264
2.2.3.5 UC-17.5 Edit Client Profile . . . 267
2.2.3.6 UC-17.6 Edit Freelancer Basics . . . 270
2.2.3.7 UC-17.7 Edit Freelancer Skills . . . 273
2.2.3.8 UC-17.8 Edit Freelancer Portfolio . . . 277
2.2.23 Manage Payment . . . 281
2.2.3.1 UC-43.01 Connect Stripe Account . . . 281
2.2.3.2 UC-43.02 View Payout History . . . 283
2.2.3.3 UC-43.03 View Payout Detail . . . 284
2.2.3.4 UC-43.04 Fund Milestone . . . 285
2.2.3.5 UC-43.05 Refund Payment . . . 287
2.2.3.6 UC-43.06 Pay Platform Fee . . . 289
2.2.3.7 UC-43.07 Release Payment . . . 290
2.2.3.8 UC-43.08 Pay Dispute Fee . . . 291
2.2.3.9 UC-43.09 View Payment History . . . 293
2.2.3.10 UC-43.10 View Payment Detail . . . 294
2.2.3.11 UC-43.11 Execute Dispute Resolution Payment . . . 295
2.2.3.12 UC-43.12 Handle Stripe Webhook Event . . . 298
2.2.3.13 UC-43.13 Sync Payment Intent Status . . . 299
2.2.24 Manage Advertisement Banner . . . 301
2.2.3.1 UC-18.1 Advertisement Banners List . . . 301
2.2.3.2 UC-18.2 Banner Detail . . . 304
2.2.3.3 UC-18.3 Create Banner . . . 307
2.2.3.4 UC-18.4 Edit Banner . . . 311
2.2.3.5 UC-18.5 Delete Banner . . . 315
2.2.25 Manage Skill . . . 317
2.2.3.1 UC-19.1 View List Skill . . . 317
2.2.3.2 UC-19.2 View Detail Skill . . . 320
2.2.3.3 UC-19.3 Create New Skill . . . 322
2.2.3.4 UC-19.4 Edit Skill . . . 325
2.2.3.5 UC-19.5 Delete Skill . . . 328
2.2.3.6 UC-19.6 Restore Skill . . . 331
3 Business Rule . . . 334

4 Functional Requirement 340
4.1 System Functional Overview 340
4.1.1 Screen Flow 340
4.1.2 Screen Details 342
4.1.3 Screen Authorization 346
4.1.4 Non-Screen Functions 348
4.1.5 Entity Relationship Diagram 363
4.1.5.1 Overall Entity Relationship Diagram 363
4.1.5.2 Authentication & Sessions 364
4.1.5.3 Role-Based Access Control 365
4.1.5.4 Profile Core 366
4.1.5.5 Profile History & Social Links 367
4.1.5.6 Identity Verification & Profile Visits 367
4.1.5.7 Portfolio & Skills 368
4.1.5.8 Favorites & Follows 369
4.1.5.9 Job Posting 370
4.1.5.10 Job Discovery & Alerts 371
4.1.5.11 Proposal 372
4.1.5.12 Contract 373
4.1.5.13 Milestone & Project Progress 374
4.1.5.14 Milestone Submission 375
4.1.5.15 Escrow, Payments & Certificates 376
4.1.5.16 Dispute Core 377
4.1.5.17 Dispute Resolution 378
4.1.5.18 Reviews & Reputation 379
4.1.5.19 Forum Posts & Comments 380
4.1.5.20 Forum Likes 381
4.1.5.21 Forum Reports & Moderation 382
4.1.5.22 Messaging & Notifications 383
4.1.5.23 Administration & System 384
4.2 Manage Forum 384
4.3 Manage Conversation 384
4.3.1 Start Conversation 384
4.3.2 View Conversation List 387
4.3.3 View Conversation Detail 389
4.3.4 Send Message 391
4.3.5 Send File 392
4.3.6 Mark Conversation as Read 394
4.3.7 Pin Conversation 396

4.3.8 Delete Message . . . 398
4.4 Moderation . . . 400
4.4.1 View Moderation Queue . . . 400
4.4.2 View Moderation Item Detail . . . 402
4.4.3 Resolve Moderation Item . . . 404
4.5 ID Verification . . . 407
4.5.1 View Verification Request List . . . 407
4.5.2 View ID Verification Request Detail . . . 409
4.5.3 Approve or Reject ID Verification . . . 411
4.6 Trash . . . 414
4.6.1 View Deleted Post List . . . 414
4.6.2 View Deleted Post Detail . . . 416
4.6.3 Restore Deleted Post . . . 417
4.6.4 View Deleted Comment List . . . 420
4.6.5 View Deleted Comment Detail . . . 422
4.6.6 Restore Deleted Comment . . . 423
5 Non-Functional Requirement . . . 426
5.1 Performance & Scalability . . . 426
5.2 Security & Privacy . . . 426
5.3 Availability & Reliability . . . 427
5.4 Usability & User Experience . . . 427

1 Product Overview

The Frevia project is a comprehensive freelancing platform that enables clients, freelancers, and administrators to build trusted professional relationships and efficiently manage the entire project lifecycle. Designed to connect businesses with skilled talent worldwide, Frevia provides intelligent job matching, structured contract workflows, and secure payment solutions to facilitate successful collaborations.

Through AI-powered recommendations, real-time communication, and advanced moderation mechanisms, the platform helps users discover suitable opportunities, deliver high-quality work, and maintain transparent interactions.

Frevia serves three primary stakeholder groups: clients seeking specialized expertise, freelancers showcasing their skills and services, and administrators responsible for maintaining platform integrity and operational efficiency. The platform offers an intuitive user experience, dynamic communication channels, and comprehensive management capabilities to support collaboration, professional growth, and secure transactions.

Key features of Frevia include:

- **Smart Job Matching and AI Recommendations:** The platform analyzes freelancer skills, experience, and preferences to recommend relevant job opportunities. AI-powered profile scoring provides quality assessments, while job alerts and saved searches help users stay informed about new opportunities.
- **End-to-End Contract and Milestone Management:** Clients and freelancers can negotiate project terms, define milestones and tasks, monitor progress, and securely manage payments. Integrated contract signing, activity logs, and file-sharing capabilities ensure transparency throughout the project lifecycle.
- **Community and Forum Engagement:** Frevia includes a community forum where users can discuss industry topics, exchange knowledge, and seek professional advice. Features such as categories, posts, comments, reactions, and reporting mechanisms encourage active participation and self-moderation.
- **Identity Verification and Content Moderation:** Freelancers can verify their identities through document submission, while administrators utilize moderation tools, banned keyword management, and AI-assisted violation detection to maintain a secure and trustworthy environment.
- **Wallet, Payments, and Gas Fee Subsidies:** Users can manage account balances, withdraw earnings, and review transaction histories. The platform also supports gas fee sub-

sidy mechanisms for blockchain-based transactions, improving payment accessibility and reducing operational costs.

- **Administrative Management Tools:** Administrators have access to comprehensive management functionalities, including user administration, role and permission control, system configuration, advertisement management, activity monitoring, and dispute resolution, ensuring fair and reliable platform operations.

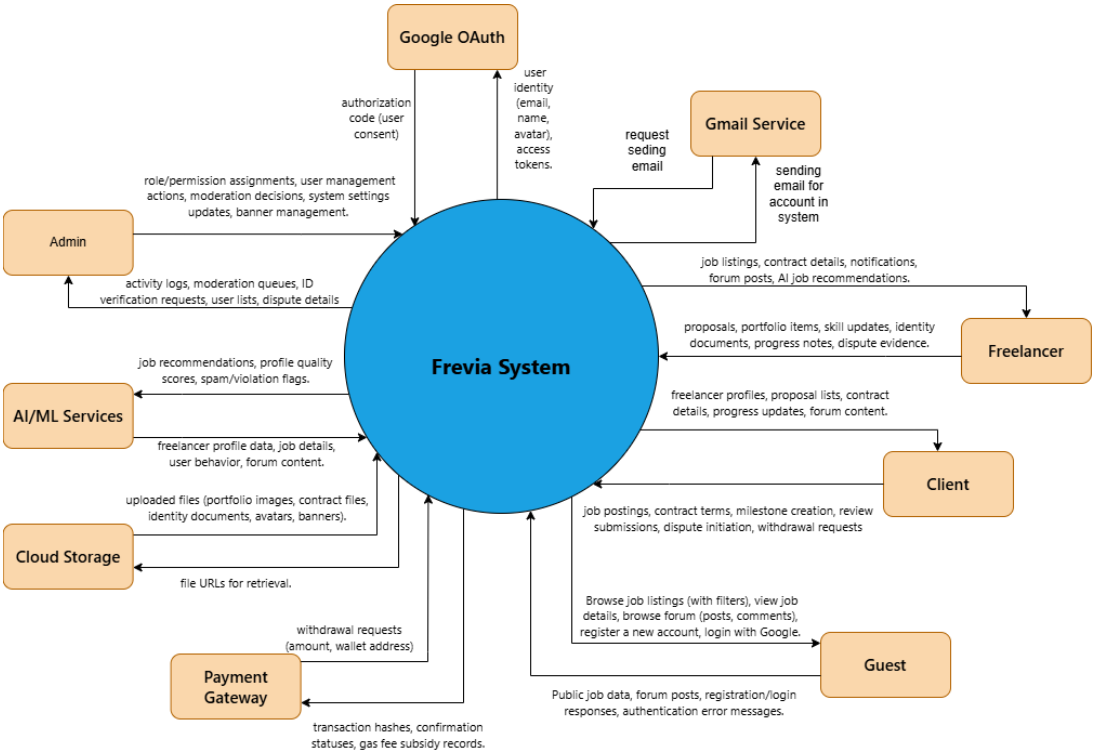


Figure 1: Context diagram

2 User Requirement

2.1 Actors

Table 1: Actors of the Frevia Platform

No	Actor	Description
1	Guest	Guests are unregistered visitors who have not yet created an account on the platform. They can browse publicly available information such as job listings, freelancer profiles, forum content, and platform features. Guests may also register a new account or authenticate using supported login methods to access platform services.
2	Client	Clients are individuals or organizations seeking professional services from freelancers. They can create and manage job postings, review proposals, hire freelancers, establish contracts, manage escrow payments, monitor project progress, initiate disputes, and provide reviews upon project completion.
3	Freelancer	Freelancers are service providers who offer professional skills and expertise through the platform. They can create and manage profiles, portfolios, and skills; search and apply for jobs; submit proposals; collaborate with clients; complete project tasks and milestones; and receive payments after successful project delivery.
4	Admin	Administrators are responsible for managing and maintaining the platform's operations, security, and compliance. Their responsibilities include user management, role and permission management, identity verification, content moderation, dispute resolution, system configuration, and monitoring platform activities to ensure a secure and reliable environment.

2.2 Use Cases

2.2.1 Diagram(s)

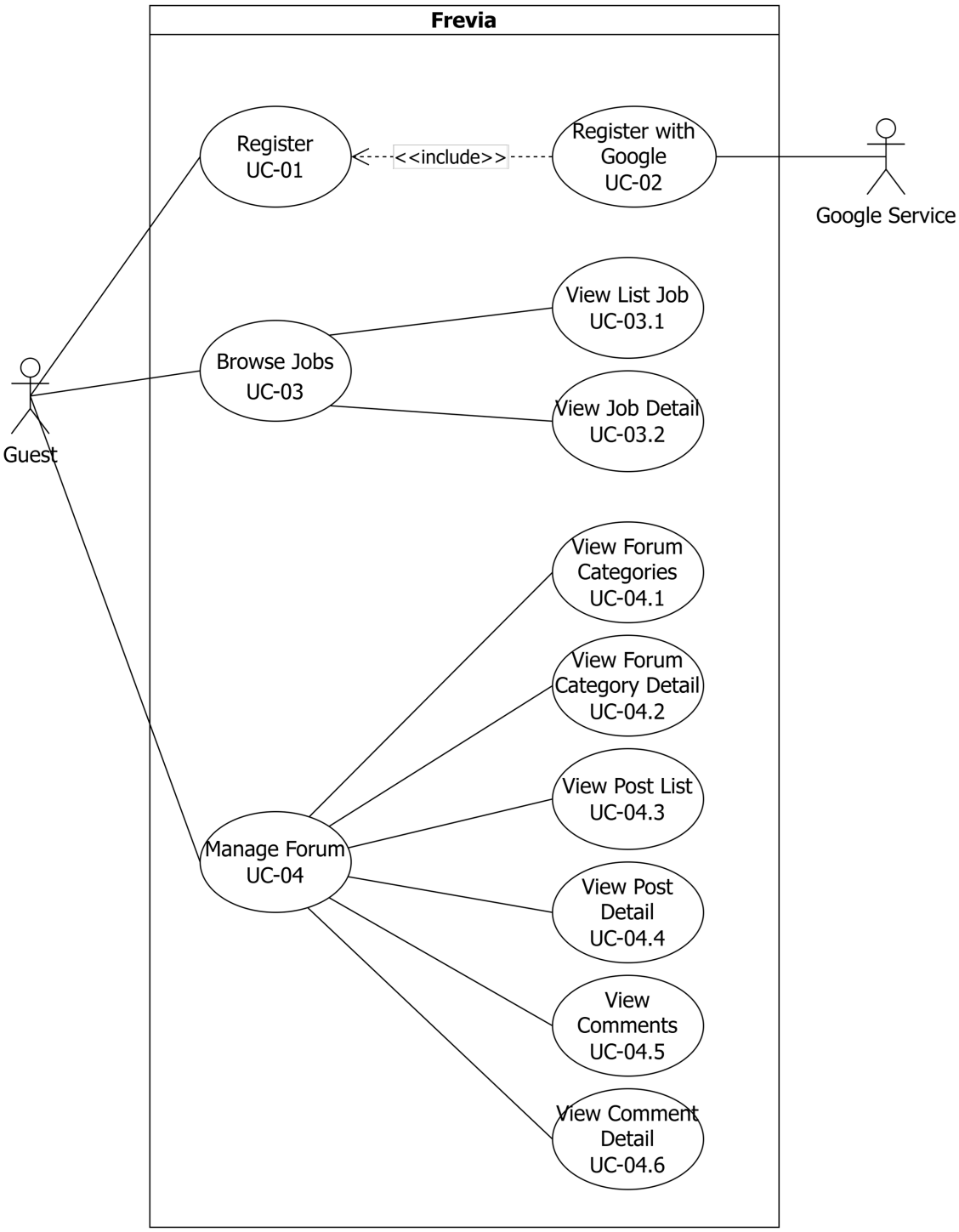


Figure 2: Guest Usecase

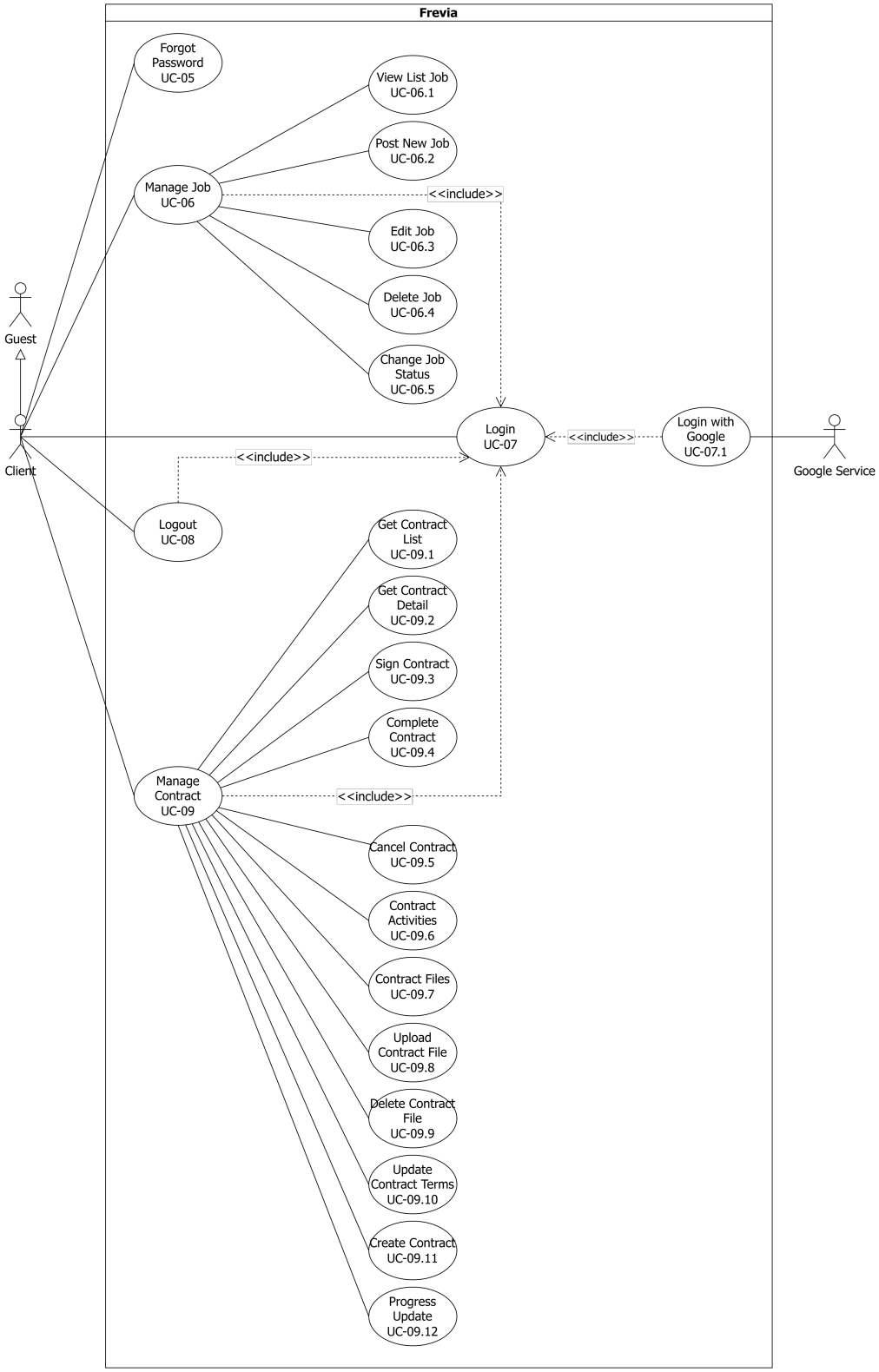


Figure 3: Client Usecase

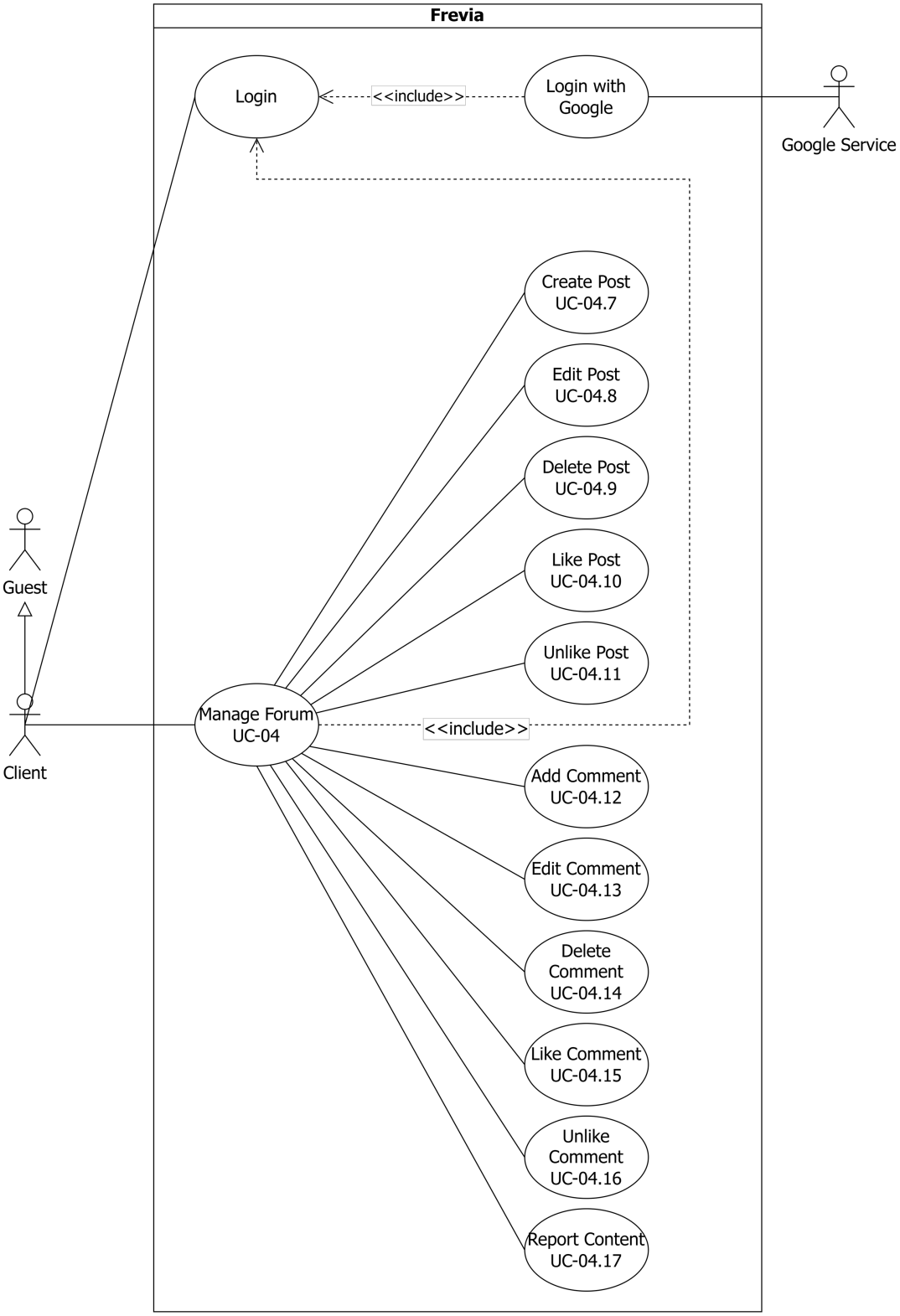


Figure 4: Client Usecase

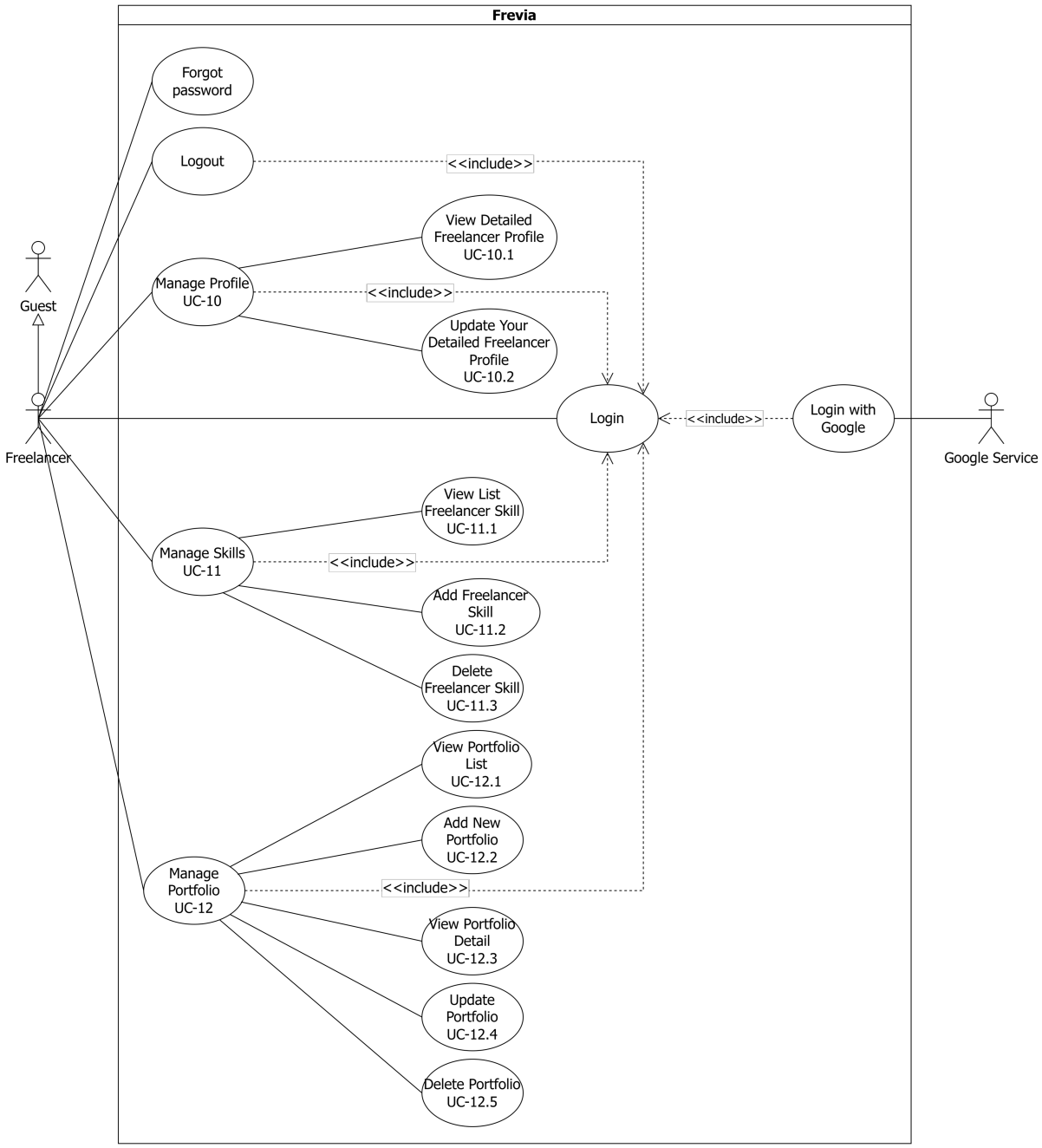


Figure 5: Freelancer Usecase

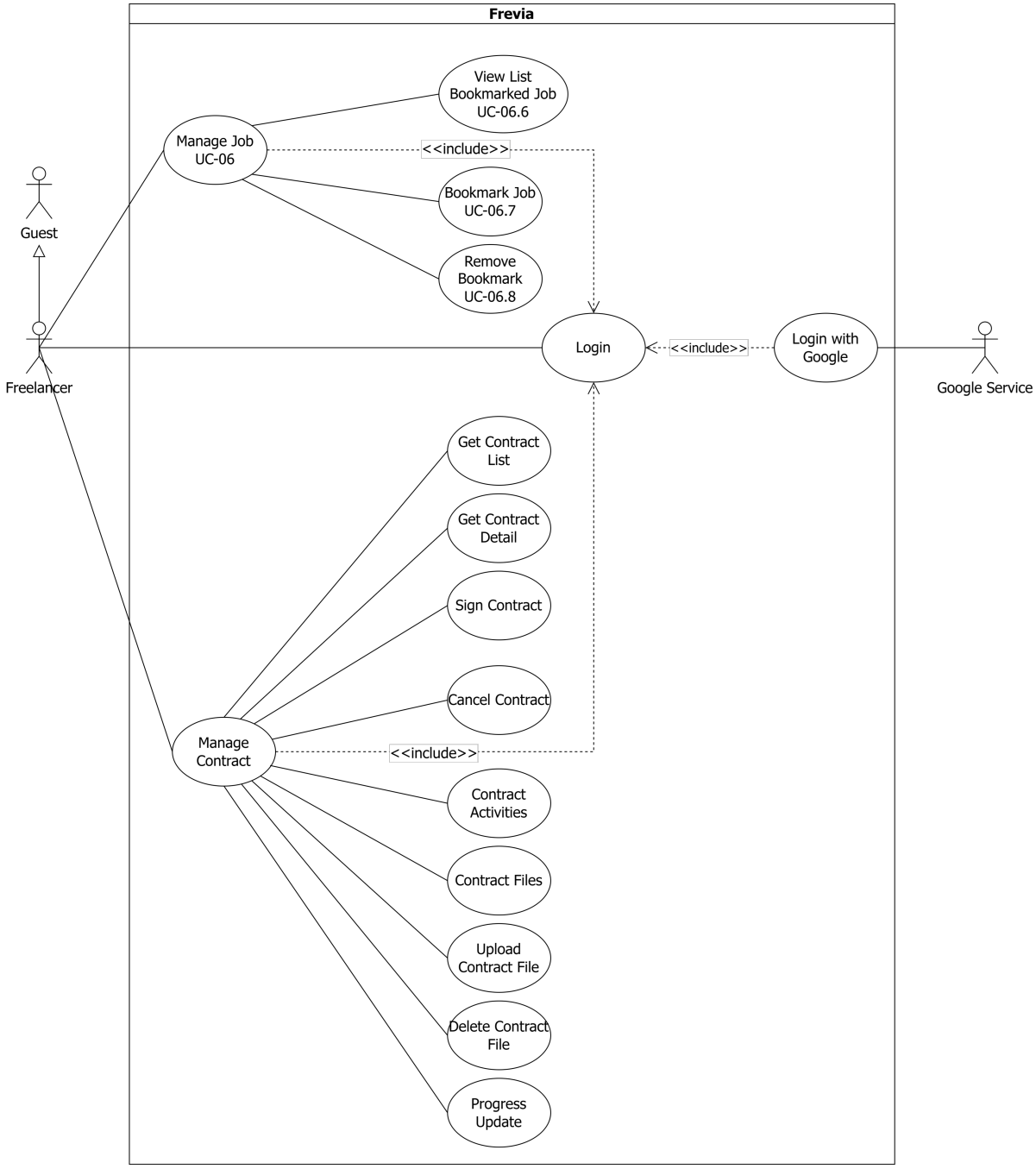


Figure 6: Freelancer Usecase

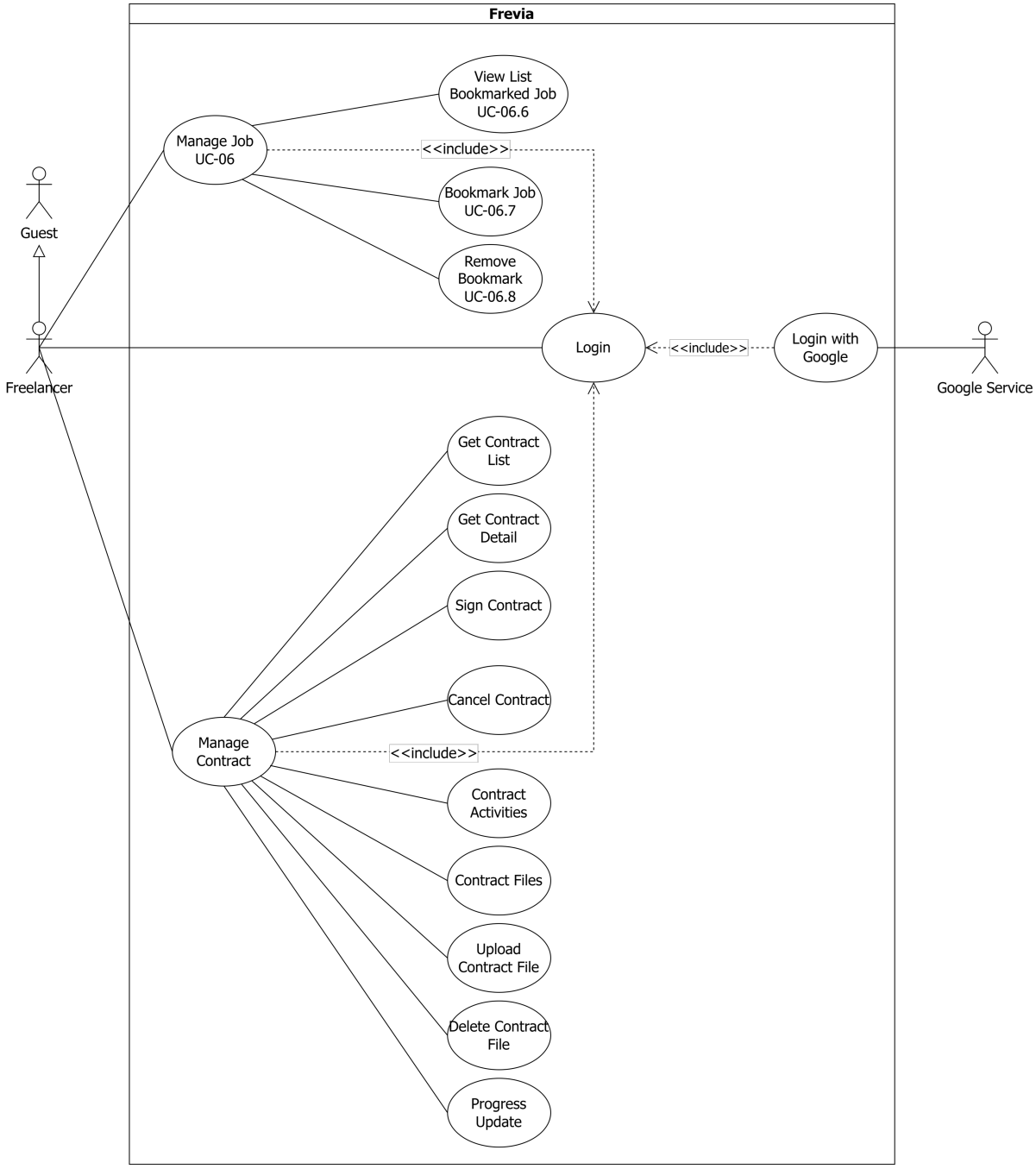


Figure 7: Freelancer Usecase

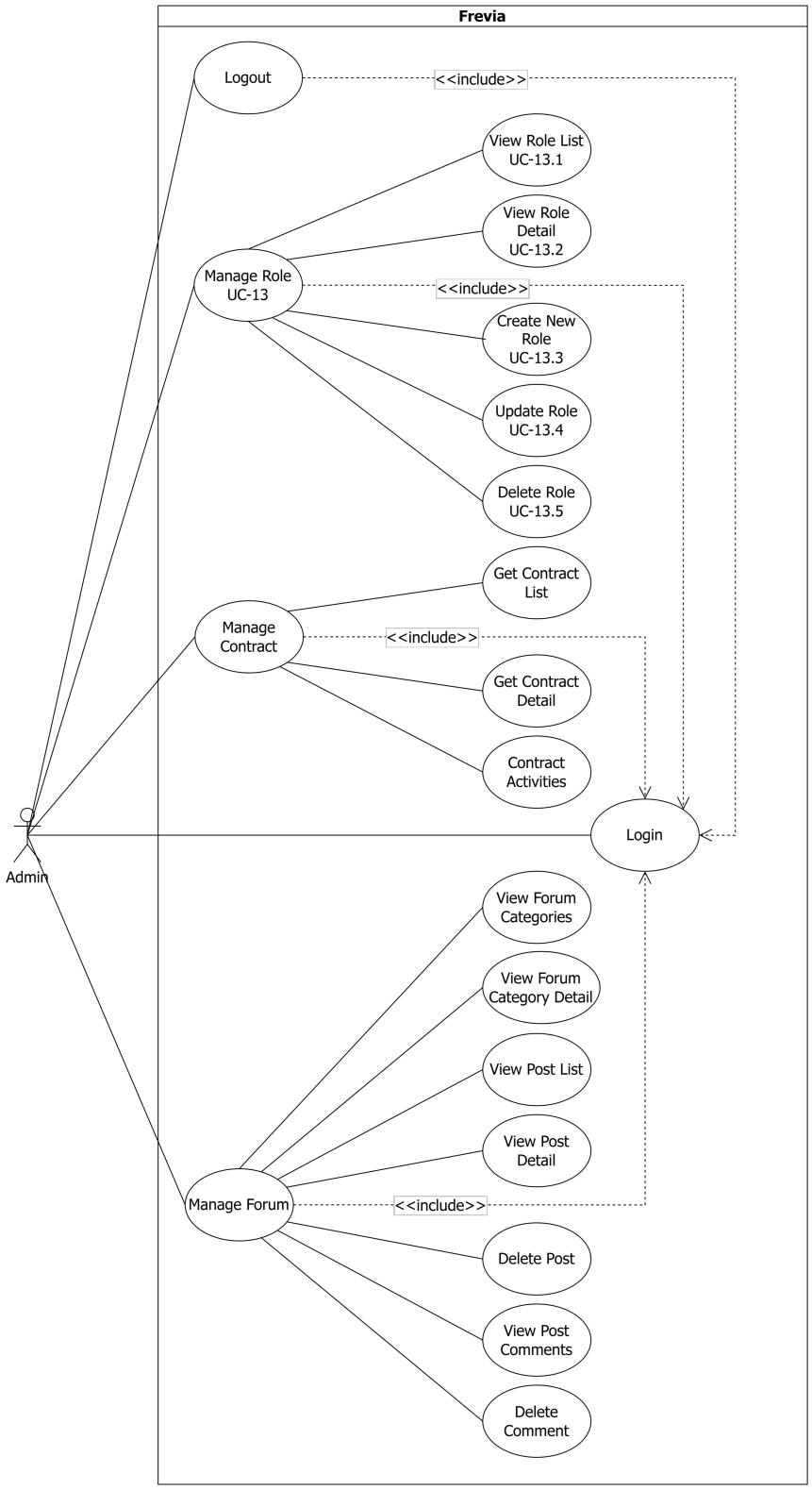


Figure 8: Admin Usecase

2.2.2 Use Case Description

ID	Use Case	Actors	Description
UC-01	Login with Google	Guest	Allows users to login/register using Google OAuth.
UC-02	Login	Guest	Allows users to log in using email/password and JWT authentication.
UC-03	Logout	Client, Expert, Freelancer, Admin	Allows users to logout and terminate authentication session.
UC-04	Register	Guest	Allows users to create new accounts using email and password.
UC-05	Forgot Password	Guest	Allows users to request password reset via email verification link.
UC-06.01	View Role List	Admin	Retrieve list of all roles along with their assigned permissions.
UC-06.02	View Role Detail	Admin	View detailed information of a specific role including its permissions.
UC-06.03	Create Role	Admin	Create a new role with name, description, and a list of permission IDs.
UC-06.04	Update Role	Admin	Update a role's name, description, and replace all assigned permissions.
UC-06.05	Delete Role	Admin	Delete a role from the system.
UC-06.06	Update Role Permissions	Admin	Assign route access rights to roles.
UC-15.01	View Permission List	Admin	Retrieve all permissions (id, name, path, method, module) for admin to assign to roles.
UC-15.02	View Permission Detail	Admin	View detailed information of a specific permission.
UC-16.01	View Session List	Client, Expert, Freelancer, Admin	View a list of active login sessions.
UC-16.02	View Session Detail	Client, Expert, Freelancer, Admin	View login session information (device, IP address, login time, approximate location, etc.).
UC-16.03	Revoke Session	Client, Expert, Freelancer, Admin	Log out of a specific device remotely.

Continued on next page

ID	Use Case	Actors	Description
UC-17.01	Create Forum Category	Admin	Allows admins to create a new forum category.
UC-17.02	Update Forum Category	Admin	Allows admins to update an existing forum category.
UC-17.03	Delete Forum Category	Admin	Allows admins to soft-delete a forum category.
UC-14.01	View Forum Category List	Admin	Allows admins to view the list of forum categories.
UC-14.02	View Forum Category Detail	Admin	Allows admins to view the details of a specific forum category.
UC-17.06	Restore Forum Category	Admin	Restore a deleted Category forum by setting deletedAt back to null.
UC-03	Job Browsing	Guest, Client, Freelancer	Enables guests to browse and search for available job listings.
UC-03.1	View List Job	Guest, Client, Freelancer	Retrieve paginated job list with filters (category, budget, skills, status, keywords) and sorting options.
UC-03.2	View Job Detail	Guest, Client, Freelancer	Get full job details: title, description, budget, skills, client info, proposals count, attachments, timeline.
UC-04	Manage Forum	Guest, Client, Freelancer, Admin	Manages forum operations including viewing, creating, editing, and moderating posts and comments.
UC-04.1	View Forum Categories	Guest, Client, Freelancer, Admin	Displays all available forum categories for browsing.
UC-04.2	View Forum Category Detail	Guest, Client, Freelancer, Admin	Shows detailed information of a specific forum category and its sub-forums.
UC-04.3	View Post List	Guest, Client, Freelancer, Admin	Retrieves paginated list of posts within a forum category with filters and sorting.
UC-04.4	View Post Detail	Guest, Client, Freelancer, Admin	Displays full post content, author info, creation date, and associated tags.

Continued on next page

ID	Use Case	Actors	Description
UC-04.5	View Comments	Guest, Client, Freelancer, Admin	Retrieves all comments related to a specific forum post.
UC-04.6	View Comment Detail	Guest, Client, Freelancer, Admin	Shows detailed information of a specific comment, including content, author, and timestamps.
UC-04.7	Create Post	Client, Freelancer	Allows freelancers to create new discussion posts or questions in the forum.
UC-04.8	Edit Post	Client, Freelancer	Allows freelancers to edit their own forum posts.
UC-04.9	Delete Post	Client, Freelancer, Admin	Allows post owners or admins to delete a forum post.
UC-04.10	Like Post	Client, Freelancer	Allows freelancers to like a forum post.
UC-04.11	Unlike Post	Client, Freelancer	Allows freelancers to remove their like from a forum post.
UC-04.12	Add Comment	Client, Freelancer	Allows freelancers to add comments to a forum post.
UC-04.13	Edit Comment	Client, Freelancer	Allows freelancers to edit their own comments.
UC-04.14	Delete Comment	Client, Freelancer	Allows comment owners or admins to delete a comment.
UC-04.15	Like Comment	Client, Freelancer	Allows freelancers to like a comment.
UC-04.16	Unlike Comment	Client, Freelancer	Allows freelancers to remove their like from a comment.
UC-04.17	Report Content	Client, Freelancer	Allows freelancers to report inappropriate content in forum posts or comments.
UC-06	Manage Job	Client, Freelancer	Manages job postings including creation, editing, deletion, and status changes.
UC-06.1	View List Job	Guest, Client, Freelancer	Retrieve paginated job list with filters and sorting options.
UC-06.2	Post New Job	Client	Allows clients to create and publish new job postings.
UC-06.3	Edit Job	Client	Allows clients to edit their own job postings.

Continued on next page

ID	Use Case	Actors	Description
UC-06.4	Delete Job	Client	Allows clients to delete their own job postings.
UC-06.5	Change Job Status	Client	Allows clients to update job status (Open, In Progress, Closed, Cancelled).
UC-06.6	View List Bookmarked Job	Freelancer	Retrieves list of jobs bookmarked by the freelancer.
UC-06.7	Bookmark Job	Freelancer	Allows freelancers to save/bookmark a job for later reference.
UC-06.8	Remove Bookmark	Freelancer	Allows freelancers to remove a bookmarked job from their list.
UC-13	Manage Contract	Client, Freelancer, Admin	Manages contractual agreements, milestones, deliverables, and file exchanges.
UC-13.01	Create Contract	Client	Allows clients to create and initiate a new contract with a freelancer.
UC-13.02	Update Contract	Client	Allows clients to update contract terms, budget, milestones, or deadlines before signing.
UC-13.03	Complete Contract	Client, Freelancer	Marks an active contract as completed after all milestones are approved and settled.
UC-13.04	View Contract List	Client, Freelancer, Admin	Retrieves paginated list of contracts with status filters and sorting.
UC-13.05	View Contract Detail	Client, Freelancer, Admin	Displays full contract details including terms, parties, status, and milestones.
UC-13.06	Sign Contract	Freelancer, Client	Allows users to digitally accept and sign a contract to activate it.
UC-13.07	Cancel Contract	Client, Freelancer	Allows either party to cancel or terminate a contract prior to completion.
UC-13.08	View Contract Files	Client, Freelancer, Admin	Manages and accesses all files and documentation associated with a contract.
UC-13.09	Upload Contract File	Client, Freelancer	Allows users to upload documents and deliverables to the contract workspace.

Continued on next page

ID	Use Case	Actors	Description
UC-13.10	Delete Contract File	Client, Freelancer, Admin	Allows the uploader or admin to delete files from a contract workspace.
UC-10	Manage Profile	Freelancer	Manages freelancer profile information and settings.
UC-10.1	View Detailed Freelancer Profile	Freelancer	Displays full freelancer profile including skills, portfolio, experience, and ratings.
UC-10.2	Update Your Detailed Freelancer Profile	Freelancer	Allows freelancers to edit and update their profile information.
UC-11	Manage Skills	Freelancer	Manages freelancer skill set and proficiency levels.
UC-11.1	View List Freelancer Skill	Freelancer	Retrieve the list of skills added by the freelancer.
UC-11.2	Add Freelancer Skill	Freelancer	Allows freelancers to add new skills to their profile.
UC-11.3	Delete Freelancer Skill	Freelancer	Allows freelancers to remove skills from their profile.
UC-12	Manage Portfolio	Freelancer	Manages freelancer portfolio projects and work samples.
UC-12.1	View Portfolio List	Freelancer	Retrieve the list of portfolios created by the freelancer.
UC-12.2	Add New Portfolio	Freelancer	Allows freelancers to add new portfolio items with images and descriptions.
UC-12.3	View Portfolio Detail	Freelancer	Displays full details of a portfolio item including media, description, and technologies used.
UC-12.4	Update Portfolio	Freelancer	Allows freelancers to edit and update their portfolio items.
UC-12.5	Delete Portfolio	Freelancer	Allows freelancers to remove portfolio items.
UC-33	Manage Dispute	Client, Freelancer, Admin	Manages formal dispute arbitration on funded contract milestones.

Continued on next page

ID	Use Case	Actors	Description
UC-33.01	Open Dispute	Client, Freelancer	Allows a client or freelancer to initiate a formal dispute on a funded milestone.
UC-33.02	View Dispute List	Client, Freelancer, Admin	Retrieves paginated overview of disputes with status filtering and deadline badges.
UC-33.03	View Dispute Detail	Client, Freelancer, Admin	Displays full dispute case records, claims, evidence, fees, and resolutions.
UC-33.04	Respond to Dispute	Client, Freelancer	Allows the respondent to submit a rebuttal statement and counter-evidence.
UC-33.05	Respond to Dispute Resolution	Client, Freelancer	Allows involved parties to accept or reject the arbitrator's proposed decision.
UC-33.06	Propose Dispute Resolution	Admin	Allows an administrator/arbitrator to evaluate claims and propose a fund split decision.
UC-43	Manage Payment	Client, Freelancer, Admin	Handles platform financial transactions, escrow deposits, payouts, and fees via Stripe.
UC-43.01	Connect Stripe Account	Freelancer	Connects a freelancer's bank account via Stripe Connect Express to receive payouts.
UC-43.02	View Payout History	Freelancer, Admin	Retrieves paginated history of all payout transfers sent to the freelancer's bank.
UC-43.03	View Payout Detail	Freelancer, Admin	Displays detailed ledger breakdown, fees, and arrival estimates for a payout transfer.
UC-43.04	Fund Milestone	Client	Deposits milestone funds into platform escrow using credit card or saved payment methods.
UC-43.05	Refund Payment	Client, Admin	Returns escrowed milestone funds to the client upon contract cancellation or dispute.
UC-43.06	Pay Platform Fee	Client	Processes the platform engagement fee upon contract initiation via Stripe.
UC-43.07	Release Payment	Client	Authorizes the release of escrowed milestone funds to the freelancer upon work approval.

Continued on next page

ID	Use Case	Actors	Description
UC-43.08	Pay Dispute Fee	Client, Freelancer	Processes arbitration fee payment within the 24-hour dispute window via Stripe.
UC-43.09	View Payment History	Client, Freelancer, Admin	Displays a comprehensive, paginated ledger of all account financial transactions.
UC-43.10	View Payment Detail	Client, Freelancer, Admin	Shows itemized receipt details, fees, and downloadable invoice for a transaction.
UC-43.11	Execute Dispute Resolution Payment	Admin, System	Executes split payouts (client refund and freelancer transfer) upon dispute finalization.
UC-43.12	Handle Stripe Webhook Event	Stripe, System	Listens for and verifies asynchronous Stripe webhook events to reconcile database state.
UC-43.13	Sync Payment Intent Status	Client, System	Synchronizes real-time payment intent and checkout status directly with the Stripe API.

2.2.3 Use Case Specification

2.2.1 Authentication

2.2.3.1 UC-01 Login with Google

UC ID and Name	UC-01 Login with Google		
Created By:	ThuanNT	Date Created:	20/09/2026
Primary Actor:	Guest	Secondary Actors:	Google OAuth Server
Trigger:	When a user clicks the “Login with Google” button on the login or registration page.		
Description:	Allows users to login/register using Google OAuth.		
Preconditions:	**PRE-1** The user is not currently authenticated. **PRE-2** The system is connected to the Google OAuth Server.		
Post-conditions:	**POST-1** The user is authenticated and a JWT is issued and stored in an httpOnly cookie. **POST-2** If the Google account is new to the system, a user account is automatically created with the Client role by default. **POST-3** The user is redirected to their role-appropriate dashboard.		

Normal Flow:	**A. Successful Google Login** 1. Guest clicks the “Login with Google” button. 2. The system redirects the Guest to the Google OAuth consent screen. 3. Guest grants permission on the Google consent screen. 4. Google redirects back to the system with an authorization code. 5. The system exchanges the authorization code for a Google access token. 6. The system retrieves the user’s profile information (email, name) from Google. 7. The system checks whether an account with the returned email already exists. 8. The system loads the existing user record (or creates a new user if not found). 9. The system issues a JWT and stores it in an httpOnly cookie. 10. The system redirects the Guest to the role-appropriate dashboard.
Alternative Flows:	**A.3 Google Permission Denied** 1. Guest denies permission on the Google consent screen. 2. The system displays the message “Google login was cancelled. Please try again.” 3. Return to step A.1. **A.7 New Google Account** 1. The system detects that no account exists for the retrieved Google email. 2. The system creates a new user account with the email and name from Google, assigned the Client role. 3. Proceed to step A.9. **A.7 Account Suspended** 1. The system detects that the matched account is suspended or banned. 2. The system displays the message “Your account has been suspended. Please contact support.” 3. The use case ends.
Exceptions:	**EX-01: Google OAuth Failure** 1. Communication with the Google OAuth Server fails during code exchange or profile retrieval. 2. The system displays the message “Authentication with Google failed. Please try again.” 3. The use case ends.

	EX-02: Database Connection Failure 1. The system fails to query or persist user session data in the database. 2. The system displays the message “Login failed. Please try again later.” 3. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-UA-01: Google OAuth shall use the standard OAuth 2.0 authorization code flow. BR-UA-02: A Google account not previously registered in the system shall be automatically registered with the Client role. BR-UA-03: Suspended or banned accounts shall not be permitted to authenticate. BR-UA-04: JWT tokens shall be stored in secure httpOnly cookies.
Other Information:	Auto-registered accounts via Google OAuth do not require email verification, as Google has already verified the email address. The redirect destination after login is determined by the user’s role (Client or Freelancer dashboard).
Assumptions:	Google OAuth 2.0 credentials are configured in the system. The Google OAuth Server is accessible at the time of login.

2.2.3.2 UC-02 Login

UC ID and Name	UC-02 Login		
Created By:	ThuanNT	Date Created:	20/09/2026
Primary Actor:	Guest	Secondary Actors:	N/A
Trigger:	When a user submits their email and password on the login page.		
Description:	Allows users to log in using email/password and JWT authentication.		
Preconditions:	**PRE-1** The user is not currently authenticated. **PRE-2** The user has a registered account with email and password.		
Post-conditions:	**POST-1** The user is authenticated and a JWT is issued and stored in an httpOnly cookie. **POST-2** An active session record is created in the database. **POST-3** The user is redirected to their role-appropriate dashboard.		

Normal Flow:	**A. Successful Login** 1. Guest navigates to the login page. 2. Guest enters their email and password. 3. Guest clicks the “Login” button. 4. The system validates that both fields are not empty. 5. The system looks up the account by email. 6. The system verifies the provided password against the stored hash. 7. The system checks that the account’s email has been verified and the account is active. 8. The system creates a session record, issues a JWT, and stores it in an httpOnly cookie. 9. The system redirects the Guest to the role-appropriate dashboard.
Alternative Flows:	**A.4 Required Fields Missing** 1. The system detects that the email or password field is empty. 2. The system displays the message “Email and password are required.” 3. Return to step A.2. **A.5 Account Not Found** 1. The system finds no account associated with the provided email. 2. The system displays the message “Invalid email or password.” 3. Return to step A.2. **A.5 Account Suspended** 1. The system finds that the matched account has been suspended or banned. 2. The system displays the message “Your account has been suspended. Please contact support.” 3. The use case ends. **A.6 Incorrect Password** 1. The system detects that the provided password does not match the stored hash. 2. The system displays the message “Invalid email or password.” 3. Return to step A.2. **A.7 Email Not Verified** 1. The system detects that the account’s email has not been verified. 2. The system displays the message “Please verify your email before logging in. Resend verification email?” 3. Return to step A.2.

Exceptions:	**EX-01: Database Connection Failure** 1. The system fails to connect to the database to verify credentials or create a session. 2. The system displays the message “Login failed. Please try again later.” 3. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-UA-05: JWT shall be stored exclusively in an httpOnly cookie. BR-UA-06: The system shall not distinguish between “account not found” and “incorrect password” in error messages to prevent account enumeration. BR-UA-07: Accounts with unverified emails shall not be permitted to log in. BR-UA-08: Suspended accounts shall not be permitted to authenticate.
Other Information:	The redirect destination after login is determined by the user’s role (Client, Freelancer, or Admin dashboard).
Assumptions:	Passwords are stored as hashed values in the database.

2.2.3.3 UC-03 Logout

UC ID and Name	UC-03 Logout		
Created By:	ThuanNT	Date Created:	20/09/2026
Primary Actor:	Client, Expert, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When an authenticated user clicks the “Logout” button.		
Description:	Allows users to logout and terminate authentication session.		
Preconditions:	**PRE-1** The user is currently authenticated with a valid JWT stored in an httpOnly cookie.		
Post-conditions:	**POST-1** The session record is deleted from the database and the token is invalidated. **POST-2** The httpOnly cookie is cleared on the client side. **POST-3** The user is redirected to the login page.		

Normal Flow:	**A. Successful Logout** 1. Authenticated user clicks the “Logout” button. 2. The system sends a logout request to the server with the session credentials. 3. The server validates the JWT and identifies the active session. 4. The server deletes the session from the database. 5. The server clears the httpOnly cookie. 6. The system redirects the user to the login page.
Alternative Flows:	N/A
Exceptions:	**EX-01: Database Connection Failure** 1. The server fails to delete the session in the database. 2. The system displays the message “Logout failed. Please try again later.” 3. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-UA-09: Logout must invalidate the session on the server side before clearing the cookie to prevent token reuse. BR-UA-10: Terminated sessions shall be rejected on all subsequent authenticated requests.
Other Information:	Terminating the server-side session ensures that any refresh tokens or access tokens derived from the session can no longer access protected endpoints.
Assumptions:	The database is accessible to process session deletion.

2.2.3.4 UC-04 Register

UC ID and Name	UC-04 Register		
Created By:	ThuanNT	Date Created:	20/09/2026
Primary Actor:	Guest	Secondary Actors:	Email Server
Trigger:	When a guest submits the registration form on the registration page.		
Description:	Allows users to create new accounts using email and password.		
Preconditions:	**PRE-1** The user is not currently authenticated. **PRE-2** The system is connected to the email server.		
Post-conditions:	**POST-1** A new user account is created in the database with the selected role (Client or Freelancer) and an active email verification status. **POST-2** The guest is redirected to the appropriate onboarding or dashboard page.		

Normal Flow:	**A. Successful Registration** 1. Guest navigates to the registration page. 2. Guest enters their full name, email address, password, and password confirmation. 3. Guest selects their role (Client or Freelancer). 4. Guest clicks the "Send OTP" button. 5. The system validates the input fields. 6. The system checks that the email is not already registered. 7. The system generates a time-limited OTP and sends it to the provided email address. 8. Guest enters the received OTP code. 9. Guest checks the "I agree to the Terms of Service and Privacy Policy" checkbox. 10. Guest clicks the "Continue to Profile" button. 11. The system validates the OTP. 12. The system creates the new user account and sets the email verification status to verified. 13. The system redirects the Guest to the appropriate onboarding or dashboard page.
Alternative Flows:	**A.5 Required Fields Missing** 1. The system detects that a required field is empty. 2. The system displays the message "All fields are required." 3. Return to step A.2. **A.5 Invalid Email Format** 1. The system detects that the entered email does not match a valid format. 2. The system displays the message "Please enter a valid email address." 3. Return to step A.2. **A.5 Password Too Weak** 1. The system detects that the password does not meet the minimum strength requirements. 2. The system displays the message "Password must be at least 8 characters and include at least one uppercase letter, one number, and one special character." 3. Return to step A.2. **A.5 Password Confirmation Mismatch** 1. The system detects that the password confirmation does not match the password. 2. The system displays the message "Passwords do not match." 3. Return to step A.2.

	A.6 Email Already Registered 1. The system finds an existing account associated with the entered email. 2. The system displays the message “This email is already registered. Please log in or use a different email.” 3. Return to step A.2. **A.11 OTP Expired** 1. The system detects that the OTP has expired. 2. The system displays the message “OTP has expired. Please request a new one.” 3. Return to step A.4. **A.11 OTP Not Found** 1. The system detects that the entered OTP does not match any issued code. 2. The system displays the message “Invalid OTP. Please check your email and try again.” 3. Return to step A.8. **A.10 Terms of Service Not Accepted** 1. Guest clicks “Continue to Profile” without checking the Terms of Service checkbox. 2. The system displays the message “You must agree to the Terms of Service and Privacy Policy to continue.” 3. Return to step A.9.
Exceptions:	**EX-01: Email Server Failure** 1. The system fails to send the OTP email. 2. The system displays the message “Unable to send OTP. Please try again later.” 3. The use case ends. **EX-02: Database Connection Failure** 1. The system fails to create the user account in the database. 2. The system displays the message “Registration failed. Please try again later.” 3. The use case ends.
Priority:	High
Frequency of Use:	Medium

Business Rules:	BR-UA-11: Only Guest users may register a new account. BR-UA-12: Email must be unique across all accounts in the system. BR-UA-13: The user must select either the Client or Freelancer role at registration; no default role is assigned. BR-UA-14: Passwords must be stored as hashed values; plain-text passwords shall never be stored. BR-UA-15: The guest must accept the Terms of Service and Privacy Policy before the account is created. BR-UA-16: Email verification is performed inline via OTP during registration; the account is activated upon successful OTP validation.
Other Information:	The OTP sent via email shall expire after 10 minutes. If expired, the guest may click "Send OTP" again to receive a new code. Each OTP is single-use and is invalidated immediately after successful verification.
Assumptions:	The email server is configured and accessible at the time of registration. Password hashing is handled server-side before storage.

2.2.3.5 UC-05 Forgot Password

UC ID and Name	UC-05 Forgot Password		
Created By:	ThuanNT	Date Created:	20/09/2026
Primary Actor:	Guest	Secondary Actors:	Email Server
Trigger:	When a user clicks the "Forgot Password" link on the login page and submits their email address.		
Description:	Allows users to request password reset via email verification link.		
Preconditions:	**PRE-1** The user is not currently authenticated. **PRE-2** The system is connected to the email server.		
Post-conditions:	**POST-1** A password reset link is sent to the provided email address if an account exists. **POST-2** The user is informed to check their email regardless of whether the account exists (to prevent email enumeration). **POST-3** Upon visiting the reset link and submitting a new password, the account password is updated and all existing JWT sessions are invalidated.		

Normal Flow:	**A. Successful Password Reset** 1. Guest clicks the “Forgot Password” link on the login page. 2. The system displays the Forgot Password form. 3. Guest enters their registered email address. 4. Guest clicks the “Send Reset Link” button. 5. The system validates the email format. 6. The system looks up the account by email. 7. The system generates a time-limited password reset token. 8. The system sends the password reset link to the provided email address. 9. The system displays the message “If an account exists for this email, a reset link has been sent.” 10. Guest opens the email and clicks the reset link. 11. The system validates the reset token. 12. The system displays the Reset Password form. 13. Guest enters and confirms the new password. 14. Guest clicks the “Reset Password” button. 15. The system validates the new password and updates the account password. 16. The system invalidates all existing sessions associated with the account. 17. The system displays the message “Password reset successful. You may now log in with your new password.”
Alternative Flows:	**A.5 Invalid Email Format** 1. The system detects that the submitted email does not match a valid format. 2. The system displays the message “Please enter a valid email address.” 3. Return to step A.3. **A.6 Account Not Found** 1. The system finds no account associated with the provided email. 2. The system displays the message “If an account exists for this email, a reset link has been sent.” 3. No email is sent; the use case ends. **A.11 Reset Token Not Found** 1. The system detects that the reset token does not exist. 2. The system displays the message “This reset link is invalid. Please request a new one.” 3. Return to step A.10. **A.11 Reset Token Expired** 1. The system detects that the reset token has expired. 2. The system displays the message “This reset link has expired. Please request a new one.” 3. Return to step A.10.

	A.13 Password Validation Failure 1. The system detects that the new password does not meet the strength requirements or the confirmation does not match. 2. The system displays the message “Passwords do not match or do not meet the requirements.” 3. Return to step A.13.
Exceptions:	**EX-01: Database Connection Failure** 1. The system fails to generate or store the reset token. 2. The system displays the message “An error occurred. Please try again later.” 3. The use case ends. **EX-02: Email Server Failure** 1. The system fails to send the reset email. 2. The system displays the message “Unable to send reset email. Please try again later.” 3. The use case ends.
Priority:	High
Frequency of Use:	Low
Business Rules:	BR-UA-17: The system shall return the same response message regardless of whether the submitted email is registered, to prevent email enumeration. BR-UA-18: Password reset tokens shall expire after 1 hour. BR-UA-19: Each reset token is single-use; it is invalidated immediately after the password is successfully reset. BR-UA-20: Upon successful password reset, all active JWTs and sessions for the account shall be invalidated to terminate any existing sessions.
Other Information:	This use case applies only to accounts registered via email and password. Accounts authenticated exclusively via Google OAuth do not have a stored password and shall be directed to use Google Login instead.
Assumptions:	The email server is configured and accessible. A token storage mechanism is available to persist and validate reset tokens.

2.2.2 Role Management

2.2.3.1 UC-06.01 View Role List

UC ID and Name	UC-06.01 View Role List		
Created By:	ThuanNT	Date Created:	20/09/2026

Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin navigates to the Role Management section from the admin dashboard.		
Description:	Retrieve list of all roles along with their assigned permissions.		
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The system is connected to the database.		
Post-conditions:	**POST-1** A list of all active roles is displayed along with their assigned permissions. **POST-2** The admin can select a role to view its details (UC-06.02) or manage permissions (UC-06.06).		
Normal Flow:	**A. View Role List** 1. Admin navigates to the Role Management page (`/admin/roles`). 2. The system sends `GET /api/roles` to retrieve roles. 3. The system queries the database for all active (non-deleted) roles and their associated permissions. 4. The system displays the role list including role names, descriptions, and assigned permission badges. 5. Admin selects a role to view its details (UC-06.02).		
Alternative Flows:	**A.4 No Roles Found** 1. The system finds no active roles in the database. 2. The system displays the message “No roles available.” 3. Return to step A.1.		
Exceptions:	**EX-01: Database Connection Failure** 1. The system fails to retrieve the role list from the database. 2. The system displays the message “Unable to load role list. Please try again later.” 3. The use case ends.		
Priority:	Medium		
Frequency of Use:	High		
Business Rules:	BR-RM-01: Only authenticated Admins can access the Role Management section. BR-RM-02: Each role displayed shall include its name, description, and list of assigned permissions. BR-RM-03: Soft-deleted roles shall not appear in the active role list.		
Other Information:	Roles are displayed in ascending order by creation date. Soft-deleted roles are excluded from the active catalogue.		
Assumptions:	Roles and permissions are maintained in the system database.		

2.2.3.2 UC-06.02 View Role Detail

UC ID and Name	UC-06.02 View Role Detail		
Created By:	ThuanNT	Date Created:	20/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin selects a specific role from the role list, or navigates directly to a role detail URL.		
Description:	View detailed information of a specific role including its permissions.		
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The selected role exists and has not been soft-deleted.		
Post-conditions:	**POST-1** The detailed information of the selected role is displayed, including name, description, and assigned permissions.		
Normal Flow:	**A. View Role Detail** 1. Admin selects a role from the role list or navigates directly to `/admin/roles/:id`. 2. The system validates that the role exists and is not soft-deleted. 3. The system sends `GET /api/roles/:id`. 4. The system queries the role details and all permissions assigned to the role. 5. The system displays the role name, description, and the list of assigned permissions.		
Alternative Flows:	**A.2 Role Not Found** 1. The system determines the role does not exist or is soft-deleted. 2. The system displays the message “Role not found.” 3. The admin returns to the role list.		
Exceptions:	**EX-01: Database Connection Failure** 1. The system fails to load role details from the database. 2. The system displays the message “Unable to load role details. Please try again later.” 3. The use case ends.		
Priority:	Medium		
Frequency of Use:	High		
Business Rules:	BR-RM-04: Only authenticated Admins can view role details. BR-RM-05: Role details shall include name, description, and the complete list of assigned permissions. BR-RM-06: Soft-deleted roles are not accessible via the detail view.		
Other Information:	If the admin navigates directly to a role detail URL, the system validates role existence and active status before rendering the page.		
Assumptions:	Roles and their associated permissions are maintained in the system database.		

2.2.3.3 UC-06.03 Create Role

UC ID and Name	UC-06.03 Create Role		
Created By:	ThuanNT	Date Created:	20/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks the “Create Role” button on the Role Management page.		
Description:	Create a new role with name, description, and a list of permission IDs.		
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The system is connected to the database.		
Post-conditions:	**POST-1** A new role record is created in the database with the provided name, description, and associated permissions. **POST-2** The new role appears in the active role list.		
Normal Flow:	**A. Create Role** 1. Admin clicks the “Create Role” button. 2. The system displays the Create Role form with fields for role name, description, and permission assignment. 3. Admin enters the role name, description, and selects the desired permissions. 4. Admin clicks the “Save” button. 5. The system validates the input data. 6. The system sends `POST /api/roles` with `{ name, description, permissionIds }`. 7. The system creates the new role record and assigns the selected permissions in the database. 8. The system displays a success message and redirects the admin to the role detail or role list page.		
Alternative Flows:	**A.4 Admin Cancels Creation** 1. Admin clicks the “Cancel” button. 2. The system discards all unsaved data and returns to the Role Management page. 3. The use case ends. **A.5 Required Fields Missing** 1. The system detects the role name is empty. 2. The system displays the message “Role name is required.” 3. Return to step A.3. **A.5 Duplicate Role Name** 1. The system detects that a role with the same name already exists. 2. The system displays the message “A role with this name already exists.” 3. Return to step A.3.		

	A.5 System Role Name Not Allowed 1. The system detects that the entered name matches a reserved system role name (Admin, Client, or Freelancer). 2. The system displays the message “Cannot use a system role name.” 3. Return to step A.3.
Exceptions:	**EX-01: Database Connection Failure** 1. The system fails to create the role in the database. 2. The system displays the message “Unable to create role. Please try again later.” 3. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-RM-07: Only authenticated Admins can create roles. BR-RM-08: Role name is a required field and must be unique across all active roles. BR-RM-09: System role names (Admin, Client, Freelancer) cannot be used for custom roles. BR-RM-10: Permission IDs assigned must correspond to valid, active permissions.
Other Information:	Permissions can be assigned during role creation and can be updated later via UC-06.06 Update Role Permissions.
Assumptions:	The admin remains authenticated throughout the creation process.

2.2.3.4 UC-06.04 Update Role

UC ID and Name	UC-06.04 Update Role		
Created By:	ThuanNT	Date Created:	20/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin selects the “Edit” option on a specific role from the Role Management page.		
Description:	Update a role’s name, description, and replace all assigned permissions.		
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The selected role exists and has not been soft-deleted.		
Post-conditions:	**POST-1** The role’s name, description, and assigned permissions are updated in the database. **POST-2** Users assigned to this role immediately inherit the updated permissions.		

Normal Flow:	**A. Update Role** 1. Admin selects the “Edit” option on a role. 2. The system retrieves the current role information and displays it in an editable form. 3. Admin modifies the role name, description, and/or selects/deselects permissions. 4. Admin clicks the “Save Changes” button. 5. The system validates the updated input data. 6. The system sends `PUT /api/roles/:id` with `{ name, description, permissionIds }`. 7. The system updates the role record and replaces its assigned permissions in a database transaction. 8. The system displays the updated role detail page with a success notification.
Alternative Flows:	**A.2 Role Not Found** 1. The system determines the role does not exist or is soft-deleted. 2. The system displays the message “Role not found.” 3. Return to step A.1. **A.4 Admin Cancels Update** 1. Admin clicks the “Cancel” button. 2. The system discards all unsaved changes and returns to the role detail page. 3. The use case ends. **A.5 Required Fields Missing** 1. The system detects the role name is empty. 2. The system displays the message “Role name is required.” 3. Return to step A.3. **A.5 Duplicate Role Name** 1. The system detects that the updated name conflicts with another existing active role. 2. The system displays the message “A role with this name already exists.” 3. Return to step A.3. **A.5 System Role Name Not Allowed** 1. The system detects that the updated name matches a system role name (Admin, Client, or Freelancer). 2. The system displays the message “Cannot use a system role name.” 3. Return to step A.3.
Exceptions:	**EX-01: Database Connection Failure** 1. The system fails to update the role in the database. 2. The system displays the message “Unable to update role. Please try again later.” 3. The use case ends.
Priority:	Medium

Frequency of Use:	Low
Business Rules:	BR-RM-11: Only authenticated Admins can update roles. BR-RM-12: Role name is a required field and must remain unique across all active roles. BR-RM-13: Updating a role's permissions replaces the role's assigned permission set. BR-RM-14: System role names (Admin, Client, Freelancer) cannot be used for custom roles.
Other Information:	The original role ID remains unchanged after the update. The updated permissions apply immediately to all active users.
Assumptions:	The admin remains authenticated throughout the update process.

2.2.3.5 UC-06.05 Delete Role

UC ID and Name	UC-06.05 Delete Role		
Created By:	ThuanNT	Date Created:	20/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin selects the "Delete" option on a specific role from the Role Management page.		
Description:	Delete a role from the system.		
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The selected role exists in the system. **PRE-3** The selected role has not already been soft-deleted.		
Post-conditions:	**POST-1** The role is soft-deleted by setting the `deleted_at` field to the current timestamp. **POST-2** The deleted role no longer appears in the active role list. **POST-3** Users previously assigned to the deleted role are no longer granted its permissions.		
Normal Flow:	**A. Delete Role** 1. Admin selects the "Delete" option on a role. 2. The system displays a confirmation dialog: "Are you sure you want to delete this role? This action cannot be undone." 3. Admin confirms the deletion request. 4. The system sends `DELETE /api/roles/:id`. 5. The system performs a soft delete by setting the `deleted_at` field of the selected role to the current timestamp. 6. The system removes the role from the active role list. 7. The system redirects the admin to the Role Management page.		

Alternative Flows:	**A.1 Role Not Found** 1. The system determines the selected role does not exist. 2. The system displays the message “Role not found.” 3. Return to step A.1. **A.1 Role Already Deleted** 1. The system determines the role has already been soft-deleted. 2. The system displays the message “This role has already been deleted.” 3. Return to step A.1. **A.1 Role Currently Assigned to Users** 1. The system detects the role is assigned to one or more active users. 2. The system displays the message “This role cannot be deleted while it is assigned to active users.” 3. Return to step A.1. **A.3 Admin Cancels Deletion** 1. Admin clicks the “Cancel” button in the confirmation dialog. 2. The system closes the dialog. 3. The role remains unchanged. 4. The use case ends.
Exceptions:	**EX-01: Database Connection Failure** 1. The system displays the message “Unable to delete role. Please try again later.” 2. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-RM-15: Only authenticated Admins can delete roles. BR-RM-16: Role deletion shall be implemented as a soft delete using the `deleted_at` field. BR-RM-17: Soft-deleted roles shall not appear in the active role list. BR-RM-18: A role that is currently assigned to active users shall not be deleted.
Other Information:	Deleted roles remain stored in the database for auditing and recovery purposes.
Assumptions:	The admin remains authenticated throughout the deletion process.

2.2.3.6 UC-06.06 Update Role Permissions

UC ID and Name	UC-06.06 Update Role Permissions		
Created By:	ThuanNT	Date Created:	20/09/2026

Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin selects a role on the Assign Role page and clicks the “Save permissions” button.		
Description:	Assign route access rights to roles.		
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The selected role exists and has not been soft-deleted. **PRE-3** Permissions are predefined in the system.		
Post-conditions:	**POST-1** All existing permissions of the role are replaced with the newly selected set. **POST-2** The role’s route access rights take effect immediately for all users assigned to the role.		
Normal Flow:	**A. Update Role Permissions** 1. Admin navigates to the Assign Role page. 2. Admin selects a role from the role dropdown. 3. The system retrieves the role detail including its current permissions. 4. The system retrieves all active permissions and displays them grouped by module, pre-checking the role’s current permissions. 5. Admin selects or deselects permissions individually or by module. 6. Admin clicks the “Save permissions” button. 7. The system validates that all selected permissions exist and are active. 8. The system sends `PUT /api/roles/:id/permissions` with the selected `permissionIds`. 9. The system removes existing permission associations and creates new permission associations for the selected permissions in a transaction. 10. The system displays the message “Updated permissions for [role name]” and the role detail reflects the new permission set.		
Alternative Flows:	**A.2 No Role Selected** 1. Admin does not select a role before proceeding. 2. The system displays the message “Choose a role to start assigning permissions.” 3. Return to step A.1. **A.3 Role Not Found** 1. The system determines the selected role does not exist or is soft-deleted. 2. The system displays the message “Role not found.” 3. Return to step A.2.		

	A.6 Clear All Permissions 1. Admin deselects all permissions and clicks the “Save permissions” button. 2. The system validates the empty selection. 3. The system removes all existing permission associations of the role. 4. The role is left with no route access rights. 5. Proceed to step A.10. **A.7 Invalid Permission Selected** 1. The system detects that one or more selected permissions do not exist or are inactive. 2. The system displays the message “One or more selected permissions are invalid.” 3. Return to step A.5.
Exceptions:	**EX-01: Database Connection Failure** 1. The system fails to replace the role’s permission associations. 2. The system displays the message “Unable to update role permissions. Please try again later.” 3. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-RM-19: Only authenticated Admins can update role permissions. BR-RM-20: Each permission maps to a server route defined by an HTTP method and path within a module; assigning permissions grants the role access to those routes. BR-RM-21: Updating a role’s permissions shall replace the role’s entire existing permission set with the newly selected set. BR-RM-22: All selected permission IDs must exist and be active in the permissions table. BR-RM-23: Permission changes take effect immediately for all users currently assigned to the role.
Other Information:	The Assign Role page lists permissions grouped by module and supports filtering by HTTP method and module. An empty permission set clears all route access for the role.
Assumptions:	Permissions are predefined and managed separately in the system. The admin remains authenticated throughout the process.

2.2.3 Manage Freelancer Profile

2.2.3.1 UC-10.1 View Detailed Freelancer Profile

UC ID and Name	UC-10.1 View Detailed Freelancer Profile		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Guest, Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user navigates to a freelancer's profile page.		
Description:	Allows guests or registered users to retrieve and view all profile information of a freelancer, including basic account data and detailed professional records merged from both the profiles and freelancer_profiles tables.		
Preconditions:	**PRE-1** The selected freelancer profile exists and is active in the database.		
Post-conditions:	**POST-1** The freelancer's detailed profile information is successfully displayed.		
Normal Flow:	**A. View Detailed Freelancer Profile** 1. User navigates to the selected freelancer's profile page. 2. The system retrieves the user's basic profile data from the `profiles` table. 3. The system retrieves the user's professional data from the `freelancer_profiles` table. 4. The system merges the retrieved data. 5. The system displays the complete detailed freelancer profile to the user.		
Alternative Flows:	N/A		
Exceptions:	**EX-01 Profile Not Found** 1. The system displays the message "Freelancer profile not found." 2. The use case ends. **EX-02 Database Connection Failure** 1. The system displays the message "Unable to load profile details. Please try again later." 2. The use case ends.		
Priority:	High		
Frequency of Use:	High		
Business Rules:	BR-PROF-01: Both basic details (from profiles) and professional details (from freelancer_profiles) must be presented together in a single view.		
Other Information:	N/A		
Assumptions:	The database correctly links the profiles and freelancer_profiles tables via a unique user identifier.		

2.2.3.2 UC-10.2 Update Your Detailed Freelancer Profile

UC ID and Name	UC-10.2 Update Your Detailed Freelancer Profile		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When an authenticated freelancer clicks the “Edit Profile” button on their own profile page.		
Description:	Allows a freelancer to update their personal and professional details such as display name, bio, title, education, certifications, languages, and availability status.		
Preconditions:	**PRE-1** The freelancer is authenticated and logged into the system. **PRE-2** The freelancer is viewing their own profile settings page.		
Post-conditions:	**POST-1** The updated profile information is saved in the profiles and freelancer_profiles tables. **POST-2** The updated information is visible on the freelancer’s public profile page.		
Normal Flow:	**A. Update Detailed Freelancer Profile** 1. Freelancer clicks the “Edit Profile” button. 2. The system displays the profile editing form populated with current data. 3. Freelancer modifies fields: display_name, bio, title, education, certifications, languages, and availability_status. 4. Freelancer clicks the “Save Changes” button. 5. The system validates the input data for all modified fields. 6. The system updates the corresponding records in the `profiles` and `freelancer_profiles` tables. 7. The system displays the message: “Profile updated successfully.” 8. The system redirects the user to their updated profile page.		
Alternative Flows:	**A.4 Cancel Update** 1. Freelancer clicks the “Cancel” button. 2. The system discards all unsaved changes. 3. Return to step A.1.		
Exceptions:	**EX-01 Invalid Input Data** 1. The system highlights the error fields and displays a warning message: “Please enter valid information for all fields.” 2. The use case ends. **EX-02 Database Connection Failure** 1. The system displays the message: “Profile update failed. Please try again later.” 2. The use case ends.		
Priority:	High		
Frequency of Use:	Medium		

Business Rules:	BR-PROF-02: Freelancers can only update their own profile. BR-PROF-03: Mandatory fields (e.g., display_name, title) cannot be left empty.
Other Information:	Some updates (like availability_status) may immediately impact search visibility for clients.
Assumptions:	The freelancer remains authenticated throughout the process.

2.2.4 Manage Freelancer Skills

2.2.3.1 UC-11.1 View List Freelancer Skill

UC ID and Name	UC-11.1 View List Freelancer Skill		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Guest, Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user accesses the skills section of a freelancer's profile page.		
Description:	Allows users to retrieve and view the complete list of skills added by a specific freelancer, including their proficiency levels.		
Preconditions:	**PRE-1** The system is connected to the database. **PRE-2** The freelancer profile exists.		
Post-conditions:	**POST-1** The list of skills and corresponding proficiency levels associated with the freelancer is displayed.		
Normal Flow:	**A. View List Freelancer Skill** 1. User accesses the freelancer's profile or skills section. 2. The system queries the database to retrieve all skills associated with the freelancer's ID. 3. The system loads the skill names and their respective proficiency levels. 4. The system displays the list of skills to the user.		
Alternative Flows:	N/A		
Exceptions:	**EX-01 No Skills Found** 1. The system displays the message "No skills added yet." 2. The use case ends. **EX-02 Database Connection Failure** 1. The system displays the message "Unable to load skills. Please try again later." 2. The use case ends.		
Priority:	Medium		
Frequency of Use:	High		

Business Rules:	BR-SKILL-01: All skills belonging to the freelancer must be displayed along with their proficiency level.
Other Information:	N/A
Assumptions:	Skills are properly linked to the freelancer's ID in the database.

2.2.3.2 UC-11.2 Add Freelancer Skill

UC ID and Name	UC-11.2 Add Freelancer Skill		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the freelancer submits a new skill through the skill input form.		
Description:	Allows an authenticated freelancer to add a new skill to their profile along with an indicating proficiency level.		
Preconditions:	**PRE-1** The freelancer is authenticated and logged in. **PRE-2** The freelancer is on their profile or skills settings page.		
Post-conditions:	**POST-1** A new skill record is created and associated with the freelancer in the database. **POST-2** The newly added skill is displayed in the freelancer's skill list.		
Normal Flow:	**A. Add Freelancer Skill** 1. Freelancer clicks the "Add Skill" button. 2. The system displays a form to input the skill. 3. Freelancer enters or selects a skill name and selects a proficiency level. 4. Freelancer clicks the "Save" button. 5. The system validates the selected skill and proficiency level. 6. The system checks if the skill already exists in the freelancer's profile. 7. The system inserts the new skill record into the database. 8. The system displays the message: "Skill added successfully." 9. The system updates the displayed list of skills.		
Alternative Flows:	**A.4 Cancel Addition** 1. Freelancer clicks the "Cancel" button. 2. The system discards the input. 3. Return to step A.1.		
Exceptions:	**EX-01 Missing Required Information** 1. The system displays the message "Please select a skill and a proficiency level." 2. The use case ends.		

	EX-02 Skill Already Exists 1. The system displays the message “This skill has already been added to your profile.” 2. The use case ends. **EX-03 Database Connection Failure** 1. The system displays the message “Unable to add skill. Please try again later.” 2. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-SKILL-02: A freelancer cannot add duplicate skills to their profile. BR-SKILL-03: Both skill name and proficiency level are required.
Other Information:	Proficiency levels are typically predefined (e.g., Beginner, Intermediate, Expert).
Assumptions:	The system provides a standardized list of skills.

2.2.3.3 UC-11.3 Delete Freelancer Skill

UC ID and Name	UC-11.3 Delete Freelancer Skill		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the freelancer clicks the “Delete” option next to a specific skill.		
Description:	Allows a freelancer to permanently remove (hard delete) a skill from their profile.		
Preconditions:	**PRE-1** The freelancer is authenticated and logged in. **PRE-2** The skill to be deleted currently exists in the freelancer’s profile.		
Post-conditions:	**POST-1** The specific skill record is permanently deleted from the database. **POST-2** The skill is removed from the freelancer’s displayed skill list.		

Normal Flow:	**A. Delete Freelancer Skill** 1. Freelancer clicks the "Delete" option for a specific skill. 2. The system prompts a confirmation dialog. 3. Freelancer clicks the "Confirm" button. 4. The system executes a hard delete operation on the database for the selected skill record. 5. The system displays the message: "Skill removed successfully." 6. The system refreshes the skill list.
Alternative Flows:	**A.3 Cancel Deletion** 1. Freelancer clicks the "Cancel" button in the confirmation dialog. 2. The system closes the dialog. 3. Return to step A.1.
Exceptions:	**EX-01 Skill Not Found** 1. The system displays the message "Skill not found or already deleted." 2. The use case ends. **EX-02 Database Connection Failure** 1. The system displays the message "Unable to remove skill. Please try again later." 2. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-SKILL-04: Skill deletion must be a hard delete operation; the record is permanently removed from the mapping table. BR-SKILL-05: Only the owning freelancer can delete a skill from their profile.
Other Information:	N/A
Assumptions:	Deleting a skill from a freelancer's profile does not delete the core skill definition from the system's global master list of skills.

2.2.5 Manage Freelancer Portfolio

2.2.3.1 UC-12.1 View Portfolio List

UC ID and Name	UC-12.1 View Portfolio List		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the freelancer accesses the portfolio section of their profile.		

Description:	Allows the freelancer to retrieve and view the list of portfolio items they have created.
Preconditions:	**PRE-1** The system is connected to the database. **PRE-2** The freelancer profile exists and the user is authenticated.
Post-conditions:	**POST-1** A list of the freelancer's active portfolios is displayed.
Normal Flow:	**A. Get Portfolio List** 1. Freelancer navigates to their portfolio section. 2. The system retrieves all portfolios associated with the freelancer's ID. 3. The system filters out any soft-deleted portfolios (where deleted_at is not null). 4. The system loads portfolio summaries (e.g., thumbnail, title, short description). 5. The system displays the portfolio list.
Alternative Flows:	N/A
Exceptions:	**EX-01 No Portfolios Found** 1. The system displays the message "No portfolios available." 2. The use case ends. **EX-02 Database Connection Failure** 1. The system displays the message "Unable to load portfolios. Please try again later." 2. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-PORT-01: Only portfolios with deleted_at = NULL shall be displayed.
Other Information:	N/A
Assumptions:	Portfolios are linked correctly to the freelancer ID.

2.2.3.2 UC-12.2 Add New Portfolio

UC ID and Name	UC-12.2 Add New Portfolio		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the freelancer submits a new portfolio through the creation form.		
Description:	Allows an authenticated freelancer to create a new portfolio entry, optionally uploading media files (images, documents, links).		

Preconditions:	**PRE-1** The freelancer is authenticated and logged in. **PRE-2** The freelancer is viewing their own portfolio management section.
Post-conditions:	**POST-1** A new portfolio record is created in the database. **POST-2** Associated media files are uploaded and linked to the new portfolio. **POST-3** The new portfolio is visible in the freelancer's portfolio list.
Normal Flow:	**A. Add New Portfolio** 1. Freelancer clicks the "Add Portfolio" button. 2. The system displays the Add Portfolio form. 3. Freelancer enters portfolio details (title, description, role, technologies used). 4. Freelancer optionally uploads media files or provides external links. 5. Freelancer clicks the "Save" button. 6. The system validates the input data and media file formats/sizes. 7. The system uploads and stores the media files. 8. The system creates a new portfolio record in the database. 9. The system displays the message: "Portfolio created successfully." 10. The system redirects the freelancer to the updated portfolio list.
Alternative Flows:	**A.5 Cancel Creation** 1. Freelancer clicks the "Cancel" button. 2. The system discards the input and any pending uploads. 3. Return to step A.1.
Exceptions:	**EX-01 Required Fields Missing** 1. The system displays the message "Portfolio title is required." 2. The use case ends. **EX-02 Invalid Media Upload** 1. The system displays the message "Invalid file format or size limit exceeded." 2. The use case ends. **EX-03 Database or Storage Failure** 1. The system displays the message "Unable to create portfolio. Please try again later." 2. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-PORT-02: Portfolio title is a required field. BR-PORT-03: Media uploads must conform to system-defined size and type restrictions.

Other Information:	N/A
Assumptions:	The system has sufficient storage to handle media file uploads.

2.2.3.3 UC-12.3 View Portfolio Detail

UC ID and Name	UC-12.3 View Portfolio Detail		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the freelancer selects a specific portfolio item from the portfolio list.		
Description:	Allows the freelancer to view detailed information of a specific portfolio, including its full description and attached media.		
Preconditions:	**PRE-1** The selected portfolio item exists. **PRE-2** The portfolio has not been soft-deleted.		
Post-conditions:	**POST-1** The complete details of the portfolio are displayed successfully.		
Normal Flow:	**A. Get Portfolio Detail** 1. Freelancer clicks on a portfolio item from the list. 2. The system retrieves the portfolio details using the portfolio ID. 3. The system retrieves any associated media links/files. 4. The system displays the portfolio title, description, skills used, and media gallery.		
Alternative Flows:	N/A		
Exceptions:	**EX-01 Portfolio Not Found** 1. The system displays the message “Portfolio not found.” 2. The use case ends. **EX-02 Database Connection Failure** 1. The system displays the message “Unable to load portfolio details. Please try again later.” 2. The use case ends.		
Priority:	High		
Frequency of Use:	High		
Business Rules:	BR-PORT-04: Only portfolios with deleted_at = NULL can be accessed.		
Other Information:	N/A		
Assumptions:	Media URLs associated with the portfolio are valid and accessible.		

2.2.3.4 UC-12.4 Update Portfolio

UC ID and Name	UC-12.4 Update Portfolio		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the freelancer submits modified details for an existing portfolio.		
Description:	Allows a freelancer to update the information and/or replace the media files of an existing portfolio.		
Preconditions:	**PRE-1** The freelancer is authenticated and logged in. **PRE-2** The freelancer is the owner of the selected portfolio. **PRE-3** The portfolio exists and is not soft-deleted.		
Post-conditions:	**POST-1** The portfolio information is updated in the database. **POST-2** New media is stored, and old removed media is unlinked/deleted. **POST-3** The updated portfolio details are visible.		
Normal Flow:	**A. Update Portfolio** 1. Freelancer clicks the "Edit" option for a portfolio. 2. The system loads the existing portfolio details into an editable form. 3. Freelancer modifies portfolio fields and/or uploads new media. 4. Freelancer clicks the "Save Changes" button. 5. The system validates the updated data and new media files. 6. The system updates the portfolio record in the database. 7. The system updates associated media files. 8. The system displays the message: "Portfolio updated successfully." 9. The system redirects the freelancer to the updated portfolio list view.		
Alternative Flows:	**A.4 Cancel Update** 1. Freelancer clicks the "Cancel" button. 2. The system discards all unsaved changes. 3. Return to step A.1.		
Exceptions:	**EX-01 Required Fields Missing** 1. The system displays the message "Portfolio title is required." 2. The use case ends. **EX-02 Invalid Media Upload** 1. The system displays the message "Invalid file format or size limit exceeded." 2. The use case ends.		

	EX-03 Database Connection Failure 1. The system displays the message “Unable to update portfolio. Please try again later.” 2. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-PORT-05: Only the portfolio owner can edit it. BR-PORT-06: Portfolio title remains a required field.
Other Information:	N/A
Assumptions:	Old media files removed during the update may be permanently deleted from storage to save space.

2.2.3.5 UC-12.5 Delete Portfolio

UC ID and Name	UC-12.5 Delete Portfolio		
Created By:	Nguyen Lu Nhat Huy	Date Created:	10/09/2026
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the freelancer clicks the “Delete” option next to a specific portfolio.		
Description:	Allows a freelancer to remove a portfolio from their profile by performing a soft delete operation.		
Preconditions:	**PRE-1** The freelancer is authenticated and logged in. **PRE-2** The freelancer is the owner of the portfolio. **PRE-3** The portfolio exists and is not already deleted.		
Post-conditions:	**POST-1** The portfolio is soft-deleted by updating the deleted_at field. **POST-2** The portfolio is no longer visible in the portfolio list.		
Normal Flow:	**A. Delete Portfolio** 1. Freelancer clicks the “Delete” option next to a portfolio. 2. The system displays a confirmation dialog. 3. Freelancer clicks the “Confirm” button. 4. The system performs a soft delete by updating the deleted_at field for the portfolio record. 5. The system displays the message: “Portfolio deleted successfully.” 6. The system removes the portfolio from the visible list.		

Alternative Flows:	**A.3 Cancel Deletion** 1. Freelancer clicks the “Cancel” button in the confirmation dialog. 2. The system closes the dialog. 3. Return to step A.1.
Exceptions:	**EX-01 Portfolio Not Found** 1. The system displays the message “Portfolio not found.” 2. The use case ends. **EX-02 Database Connection Failure** 1. The system displays the message “Unable to delete portfolio. Please try again later.” 2. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-PORT-07: Portfolio deletion must be a soft delete (updating the deleted_at column). BR-PORT-08: Soft-deleted portfolios must not be displayed.
Other Information:	Soft-deleted portfolios can potentially be restored by the freelancer if an archive page is implemented.
Assumptions:	The soft delete does not immediately remove associated media files from storage.

2.2.6 Browse Jobs

2.2.3.1 UC-03.1 View List Job

UC ID and Name	UC-03.1 View List Job		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Guest, Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user accesses the Jobs page from the navigation menu.		
Description:	As a guest or registered user, I want to view a list of active jobs with pagination support and search/filter capabilities so that I can browse available opportunities or manage my posted jobs.		
Preconditions:	**PRE-1** The system is connected to the database. **PRE-2** At least one active job exists in the jobs table.		
Post-conditions:	**POST-1** A paginated list of active jobs is displayed. **POST-2** Users can click a job to view its details.		

Normal Flow:	**A. View List Job** 1. User accesses the job list page. 2. The system receives the request with default parameters (page 1, no filters, default sorting by created_at DESC). 3. The system queries the database to retrieve the job list based on role permissions. 4. The system calculates total records and total pages. 5. The system returns the first page of job list with pagination information. 6. User views the job list.
Alternative Flows:	**A.1a Search by Keyword** 1. User enters a keyword in the search bar before submitting the request. 2. The system searches jobs where title or description matches the keyword (case-insensitive). 3. The system displays matching jobs. 4. Return to step A.6. **A.1b Filter by Category** 1. User selects a category from the filter dropdown. 2. The system retrieves jobs matching the selected category. 3. The system displays filtered jobs. 4. Return to step A.6. **A.1c Filter by Budget Range** 1. User sets a minimum budget value. 2. The system retrieves jobs with budget greater than or equal to the value. 3. The system displays filtered jobs. 4. Return to step A.6. **A.1d Filter by Skills** 1. User selects one or more skills. 2. The system retrieves jobs requiring the selected skills. 3. The system displays filtered jobs. 4. Return to step A.6. **A.1e Combine Multiple Filters** 1. User applies multiple filters simultaneously (e.g., search + category + budget). 2. The system applies all filter conditions using AND logic. 3. The system displays combined results. 4. Return to step A.6. **A.5a No Jobs Found** 1. The system finds no jobs matching the entered criteria (zero total records). 2. The system displays the message: “No jobs available. Please adjust your filters or try again later.” 3. Return to step A.1.

	A.6a Navigate Between Pages 1. User clicks a page number, Next, or Previous button. 2. The system retrieves the corresponding page of jobs. 3. The system displays the requested page. 4. Return to step A.6.
Exceptions:	**EX-01 Database Connection Failure** 1. The system cannot connect to the database while retrieving job data. 2. The system displays the message: “Unable to load jobs. Please try again later.” 3. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	**BR-JOB-01:** Only jobs with status = ’open’ or ’in_progress’ shall be displayed to freelancers and guests. **BR-JOB-02:** Clients shall see all their jobs regardless of status. **BR-JOB-03:** Default sorting shall be by created_at in descending order. **BR-JOB-04:** The system shall support pagination with default page size of 10. **BR-JOB-05:** Search shall match keywords in title and description fields (case-insensitive). **BR-JOB-06:** When multiple filters are applied, all conditions shall be combined using AND logic.
Other Information:	Default page size: 10 jobs per page. Filters available: Category, Budget (min), Skills, Status (for clients only).
Assumptions:	Jobs are created by registered clients. Categories and skills are predefined in the system.

2.2.3.2 UC-03.2 View Job Detail

UC ID and Name	UC-03.2 View Job Detail		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Guest, Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user selects a job from the job list by clicking on the job title or "View Details" button.		
Description:	Allows users to view detailed information about a specific job including title, description, budget, deadline, category, required skills, and client information.		

Preconditions:	**PRE-1** The user is viewing the job list (UC-03.1). **PRE-2** The selected job exists and is active.
Post-conditions:	**POST-1** The selected job information is displayed successfully.
Normal Flow:	**A. View Job Detail** 1. User selects a job from the job list (clicks on job title or "View Details" button). 2. The system receives the request with job_id parameter. 3. The system queries the database to retrieve job information by job_id from the jobs table. 4. The system queries the client information associated with the job. 5. The system displays job details: title, description, budget, deadline, category, required skills, status, client name, and client avatar. 6. User views the job detail information.
Alternative Flows:	N/A
Exceptions:	**EX-01 Job Not Found** 1. The system cannot find the selected job (job does not exist or has been deleted). 2. The system displays the message: "Job not found." 3. The use case ends. **EX-02 Job Inactive** 1. The system determines the job is inactive (status = 'closed' or 'deleted') and the requester is not the job owner. 2. The system displays the message: "This job is no longer available." 3. The use case ends. **EX-03 Database Connection Failure** 1. The system cannot retrieve job information due to a database connection failure. 2. The system displays the message: "Unable to load job details. Please try again later." 3. The use case ends.
Priority:	High
Frequency of Use:	High

Business Rules:	**BR-JOB-07:** Job information displayed includes: title, description, budget, deadline, category, required skills, client name, created date, status. **BR-JOB-08:** If the job is inactive (status = 'closed' or 'deleted'), only the job owner can view full details.
Other Information:	N/A
Assumptions:	The job exists and is associated with a valid client account.

2.2.3.3 UC-06.2 Post New Job

UC ID and Name	UC-06.2 Post New Job		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When a client clicks the "Post New Job" or "Create Job" button from the job management page.		
Description:	Allows a registered client to create and publish a new job posting with details including title, description, budget, category, and required skills.		
Preconditions:	**PRE-1** The client is logged in to the system. **PRE-2** The client has an active account.		
Post-conditions:	**POST-1** A new job record is created in the jobs table with status = 'draft'. **POST-2** The job is saved to the database. **POST-3** The client is redirected to the job detail page or job list with a success message.		

Normal Flow:	**A. Post New Job** 1. Client navigates to the job management page. 2. Client clicks the "Post New Job" button. 3. The system displays the job creation form. 4. Client enters the following information: - Job Title (required, max 200 characters) - Description (required, max 5000 characters) - Budget (required, must be > 0) - Category (required, dropdown selection) - Required Skills (optional, tags input) 5. Client clicks the "Save Draft" or "Publish Job" button. 6. The system validates all input fields. 7. The system creates a new job record with: - title, description, budget, category - client_id = current user ID - status = 'draft' (if Save Draft) or 'open' (if Publish Job) - created_at = current timestamp 8. The system saves the skills associated with the job. 9. The system redirects to the job detail page with a success message.
Alternative Flows:	**A.2 Cancel Creation** 1. Client clicks the "Cancel" button. 2. The system discards all input data. 3. The system returns to the job list page. 4. The use case ends. **A.6a Validation Failed** 1. The system validates input fields and finds invalid or incomplete data. 2. The system highlights the errors and displays: "Invalid data. Please check your input." 3. Return to step A.4. **A.6b Budget Invalid** 1. The system finds that budget is less than or equal to 0. 2. The system displays: "Budget must be greater than 0." 3. Return to step A.4. **A.6c Missing Required Fields** 1. The system finds that title or description is empty. 2. The system displays: "Please fill in all required fields." 3. Return to step A.4.
Exceptions:	**EX-01 Database Insert Failure** 1. The system cannot save the new job record due to a database error. 2. The system displays: "Unable to create job. Please try again later." 3. The use case ends.

Priority:	High
Frequency of Use:	Medium
Business Rules:	**BR-JOB-09:** Job title is required and must not exceed 200 characters. **BR-JOB-10:** Job description is required and must not exceed 5000 characters. **BR-JOB-11:** Budget must be a positive number (greater than 0). **BR-JOB-12:** Category must be selected from predefined categories. **BR-JOB-13:** New jobs shall have status = 'draft' by default when saved as draft, or 'open' when published.
Other Information:	N/A
Assumptions:	The client has sufficient permissions to create jobs. Categories and skills are predefined in the system.

2.2.3.4 UC-06.3 Edit Job

UC ID and Name	UC-06.3 Edit Job		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When a client clicks the "Edit" button on a job they own from the job list or job detail page.		
Description:	Allows the client who created a job to modify its details such as title, description, budget, category, skills, and status. Editing is restricted if the job has an active contract.		
Preconditions:	**PRE-1** The client is logged in to the system. **PRE-2** The client is the owner of the job. **PRE-3** The job exists and is not deleted. **PRE-4** The job does not have an active contract.		
Post-conditions:	**POST-1** The job record is updated with new information. **POST-2** The updated job is visible with new details. **POST-3** A log entry is created for the edit action.		

Normal Flow:	**A. Edit Job** 1. Client navigates to the job detail page or job list. 2. Client clicks the "Edit" button on a job they own. 3. The system retrieves the current job data and pre-fills the edit form. 4. Client modifies the following fields as needed: - Job Title - Description - Budget - Category - Required Skills - Status (if applicable) 5. Client clicks the "Save Changes" button. 6. The system validates the updated input. 7. The system updates the job record in the jobs table. 8. The system updates the associated skills. 9. The system logs the edit action with timestamp. 10. The system redirects to the job detail page with a success message.
Alternative Flows:	**A.2 Cancel Edit** 1. Client clicks the "Cancel" button. 2. The system discards all changes. 3. The system returns to the job detail page. 4. The use case ends. **A.6 Validation Failed** 1. The system validates the updated input and finds invalid or incomplete data. 2. The system highlights the errors and displays appropriate error messages. 3. Return to step A.4.
Exceptions:	**EX-01 Job Not Found** 1. The system cannot find the selected job (job does not exist). 2. The system displays: "Job not found." 3. The use case ends. **EX-02 Unauthorized Access** 1. The system determines the client is not the owner of the job. 2. The system displays: "You do not have permission to edit this job." 3. The use case ends. **EX-03 Database Update Failure** 1. The system cannot update the job record due to a database error. 2. The system displays: "Unable to update job. Please try again later." 3. The use case ends.

Priority:	High
Frequency of Use:	Medium
Business Rules:	**BR-JOB-14:** Only the job owner can edit the job. **BR-JOB-15:** Editing is restricted if the job has an active contract. **BR-JOB-16:** All validation rules from UC-06.2 apply to edits. **BR-JOB-17:** Edit actions shall be logged for auditing purposes.
Other Information:	N/A
Assumptions:	The client has sufficient permissions to edit jobs. The job exists and is accessible.

2.2.3.5 UC-06.4 Delete Job

UC ID and Name	UC-06.4 Delete Job		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When a client clicks the "Delete" button on a job they own from the job list or job detail page.		
Description:	Allows the client who created a job to delete it (soft delete). The job is hidden from public view but retained in the database. The system checks for active proposals or contracts before allowing deletion.		
Preconditions:	**PRE-1** The client is logged in to the system. **PRE-2** The client is the owner of the job. **PRE-3** The job exists and is not already deleted. **PRE-4** No active proposals or contracts exist for this job.		
Post-conditions:	**POST-1** The job is soft-deleted (deleted_at field is set). **POST-2** The job is no longer visible to freelancers or guests. **POST-3** A log entry is created for the delete action.		

Normal Flow:	**A. Delete Job** 1. Client navigates to the job detail page or job list. 2. Client clicks the "Delete" button on a job they own. 3. The system displays a confirmation dialog: "Are you sure you want to delete this job? This action cannot be undone." 4. Client confirms by clicking "Yes, Delete". 5. The system checks for active proposals or contracts. 6. The system updates the job record: - status = 'deleted' - deleted_at = current timestamp 7. The system logs the delete action. 8. The system redirects to the job list with a success message: "Job deleted successfully."
Alternative Flows:	**A.3 Cancel Delete** 1. Client clicks the "Cancel" button in the confirmation dialog. 2. The system closes the dialog. 3. The system returns to the job detail page or job list. 4. The use case ends. **A.5 Job Cannot Be Deleted** 1. The system finds an active proposal or contract associated with the job. 2. The system displays: "This job cannot be deleted because it has active proposals/contracts. Please close them first." 3. The use case ends.
Exceptions:	**EX-01 Job Not Found** 1. The system cannot find the selected job (job does not exist). 2. The system displays: "Job not found." 3. The use case ends. **EX-02 Unauthorized Access** 1. The system determines the client is not the owner of the job. 2. The system displays: "You do not have permission to delete this job." 3. The use case ends. **EX-03 Database Update Failure** 1. The system cannot update the job record due to a database error. 2. The system displays: "Unable to delete job. Please try again later." 3. The use case ends.
Priority:	Medium
Frequency of Use:	Low

Business Rules:	**BR-JOB-18:** Only the job owner can delete the job. **BR-JOB-19:** Deletion should be soft delete (set deleted_at timestamp, status = 'deleted'). **BR-JOB-20:** Jobs with active proposals or contracts cannot be deleted. **BR-JOB-21:** Delete actions shall be logged for auditing purposes.
Other Information:	Soft delete: job is hidden from public but retained in database for audit.
Assumptions:	The client has sufficient permissions to delete jobs. The job exists and is accessible.

2.2.3.6 UC-06.5 Change Job Status

UC ID and Name	UC-06.5 Change Job Status		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When a client or system action triggers a status change on a job.		
Description:	Allows the job owner to change the status of a job following the workflow: draft → open → in_progress → completed → closed. Each transition has specific constraints and permissions. The system may also auto-transition statuses based on business events.		
Preconditions:	**PRE-1** The client is logged in to the system (for manual changes). **PRE-2** The client is the job owner. **PRE-3** The job exists and is not deleted.		
Post-conditions:	**POST-1** The job status is updated to the new status. **POST-2** A log entry is created for the status change. **POST-3** Notifications may be sent to relevant users.		
Normal Flow:	**A. Change Job Status** 1. Client navigates to the job detail page. 2. Client clicks the "Change Status" button. 3. The system displays the current status and available next statuses based on workflow rules. 4. Client selects the new status from the allowed options. 5. The system validates the transition is allowed. 6. The system updates the job status. 7. The system logs the status change. 8. The system sends notifications if applicable (e.g., to freelancers when job becomes open). 9. The system displays a success message: "Job status updated successfully."		

Alternative Flows:	**A.2a System Auto-Transition** 1. System event occurs (e.g., contract created, job completed). 2. System automatically updates job status based on business rules. 3. System logs the status change. 4. The use case ends. **A.2b Cancel Status Change** 1. Client clicks the "Cancel" button. 2. The system discards the status change. 3. The system returns to the job detail page. 4. The use case ends. **A.5 Invalid Transition** 1. The system validates and finds that the requested status transition is not allowed. 2. The system displays: "Status transition from [current] to [new] is not allowed." 3. Return to step A.4.
Exceptions:	**EX-01 Job Not Found** 1. The system cannot find the selected job (job does not exist). 2. The system displays: "Job not found." 3. The use case ends. **EX-02 Unauthorized Access** 1. The system determines the client is not the owner of the job. 2. The system displays: "You do not have permission to change job status." 3. The use case ends. **EX-03 Database Update Failure** 1. The system cannot update the job status due to a database error. 2. The system displays: "Unable to update status. Please try again later." 3. The use case ends.
Priority:	High
Frequency of Use:	Medium

Business Rules:	**BR-JOB-22:** Job status workflow: draft → open → in_progress → completed → closed. **BR-JOB-23:** Allowed transitions: - draft → open (client) - open → in_progress (client, when contract created) - open → closed (client, if cancelled) - in_progress → completed (client, when work done) - in_progress → closed (client, if cancelled) - completed → closed (system/auto) - Any status → deleted (soft delete) **BR-JOB-24:** Only the job owner can manually change status. **BR-JOB-25:** Status changes shall be logged for auditing purposes. **BR-JOB-26:** Notifications shall be sent to freelancers when status changes from draft to open.
Other Information:	Status definitions: - draft: Job created but not published - open: Job is visible and accepting proposals - in_progress: Job has been assigned and is being worked on - completed: Work is done, awaiting finalization - closed: Job is finished and closed - deleted: Job is soft-deleted
Assumptions:	The client has sufficient permissions to change job status. The job exists and is accessible.

2.2.3.7 UC-06.6 View List Bookmarked Job

UC ID and Name	UC-06.6 View List Bookmarked Job		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer accesses the "Bookmarked Jobs" page from the navigation menu or profile.		
Description:	Allows a freelancer to view a list of all jobs they have bookmarked. The list displays job title, budget, client name, and bookmark date with pagination support.		
Preconditions:	**PRE-1** The freelancer is logged in to the system. **PRE-2** The system is connected to the database.		
Post-conditions:	**POST-1** A paginated list of bookmarked jobs is displayed. **POST-2** The freelancer can click a job to view its details or remove the bookmark.		

Normal Flow:	**A. View List Bookmarked Job** 1. Freelancer accesses the "Bookmarked Jobs" page. 2. The system receives the request with default parameters (page 1, sorting by bookmark created_at DESC). 3. The system queries the database to retrieve the bookmark list from the bookmarks table for the current freelancer. 4. The system joins with the jobs table to fetch job details. 5. The system calculates total records and total pages. 6. The system returns the first page of bookmarks with pagination information. 7. The system displays: job title, budget, client name, and bookmark date. 8. Freelancer views the bookmark list.
Alternative Flows:	**A.3 No Bookmarks Found** 1. The system queries and finds no bookmarks for the freelancer. 2. The system displays: "You have no bookmarked jobs. Start exploring jobs to save your favorites." 3. The use case ends. **A.8a Navigate Between Pages** 1. Freelancer clicks a page number, Next, or Previous button. 2. The system retrieves the corresponding page of bookmarks. 3. The system displays the requested page. 4. Return to step A.7. **A.8b Sort by Different Criteria** 1. Freelancer selects a sorting option (e.g., by budget, by client name). 2. The system reorders the bookmark list accordingly. 3. The system displays the sorted list. 4. Return to step A.7. **A.8c View Job Detail** 1. Freelancer clicks on a job title in the list. 2. The system navigates to the job detail page. 3. The use case ends. **A.8d Remove Bookmark Directly** 1. Freelancer clicks the "Remove Bookmark" button on a job in the list. 2. The system removes the bookmark and updates the displayed list. 3. Return to step A.7.
Exceptions:	**EX-01 Database Connection Failure** 1. The system cannot retrieve bookmark data due to a database connection error. 2. The system displays: "Unable to load bookmarked jobs. Please try again later." 3. The use case ends.

Priority:	Medium
Frequency of Use:	Medium
Business Rules:	**BR-JOB-27:** Only the freelancer can view their own bookmarks. **BR-JOB-28:** Bookmarks shall be sorted by created_at in descending order by default. **BR-JOB-29:** Displayed information includes: job title, budget, client name, bookmark date. **BR-JOB-30:** The system shall support pagination with default page size of 10.
Other Information:	Default page size: 10 bookmarks per page. Sorting options: by bookmark date (default), by budget, by client name.
Assumptions:	The freelancer is logged in and has a valid account. Bookmarks are created only by freelancers.

2.2.3.8 UC-06.7 Bookmark Job

UC ID and Name	UC-06.7 Bookmark Job		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer clicks the "Bookmark" button on a job detail page or job list.		
Description:	Allows a freelancer to add a job to their bookmark list for easy tracking. The system automatically checks for duplicates and ignores if the job is already bookmarked.		
Preconditions:	**PRE-1** The freelancer is logged in to the system. **PRE-2** The job exists and is active.		
Post-conditions:	**POST-1** The job is added to the freelancer's bookmark list. **POST-2** The bookmark button changes to "Bookmarked". **POST-3** A success message is displayed.		

Normal Flow:	**A. Bookmark Job** 1. Freelancer views a job detail page or job list. 2. Freelancer clicks the "Bookmark" button on the job. 3. The system receives the request with job_id and freelancer_id parameters. 4. The system checks if the bookmark already exists in the bookmarks table. 5. If not exists, the system creates a new bookmark record with: - freelancer_id = current user ID - job_id = selected job ID - created_at = current timestamp 6. The system updates the button state to "Bookmarked". 7. The system displays the message: "Job bookmarked successfully."
Alternative Flows:	**A.4 Bookmark Already Exists** 1. The system detects that a bookmark already exists for this freelancer and job. 2. The system skips creating a new record. 3. The system displays the message: "This job is already bookmarked." 4. The use case ends.
Exceptions:	**EX-01 Job Not Found** 1. The system cannot find the selected job (job does not exist). 2. The system displays: "Job not found." 3. The use case ends. **EX-02 Database Connection Failure** 1. The system cannot create the bookmark due to a database error. 2. The system displays: "Unable to bookmark job. Please try again later." 3. The use case ends.
Priority:	Medium
Frequency of Use:	High
Business Rules:	**BR-JOB-31:** Only freelancers can bookmark jobs. **BR-JOB-32:** Each freelancer can bookmark a job only once (no duplicates). **BR-JOB-33:** If a bookmark already exists, the system silently ignores the duplicate. **BR-JOB-34:** Bookmarks can only be created for active jobs (status = 'open' or 'in_progress').
Other Information:	N/A
Assumptions:	The freelancer is logged in and has a valid account. The job exists and is active.

2.2.3.9 UC-06.8 Remove Bookmark

UC ID and Name	UC-06.8 Remove Bookmark		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer clicks the "Remove Bookmark" or "Unbookmark" button on a job detail page or bookmark list.		
Description:	Allows a freelancer to remove a job from their bookmark list. The system checks if the bookmark exists before deleting and returns a 404 error if not found.		
Preconditions:	**PRE-1** The freelancer is logged in to the system. **PRE-2** The job is bookmarked by this freelancer.		
Post-conditions:	**POST-1** The bookmark is deleted from the bookmarks table. **POST-2** The button state changes to "Bookmark". **POST-3** A success message is displayed.		
Normal Flow:	**A. Remove Bookmark** 1. Freelancer views a job detail page or bookmark list. 2. Freelancer clicks the "Remove Bookmark" button. 3. The system receives the request with job_id and freelancer_id parameters. 4. The system checks if the bookmark exists in the bookmarks table. 5. If exists, the system deletes the bookmark record. 6. The system updates the button state to "Bookmark". 7. The system displays the message: "Bookmark removed successfully."		
Alternative Flows:	**A.1 Remove from Bookmark List** 1. Freelancer clicks the remove button on a bookmark in the bookmark list page. 2. The system performs the same deletion flow as the normal flow. 3. The system updates the bookmark list by removing the deleted item. 4. Return to step A.1.		
Exceptions:	**EX-01 Job Not Found** 1. The system cannot find the selected job (job does not exist). 2. The system displays: "Job not found." 3. The use case ends. **EX-02 Bookmark Not Found** 1. The system cannot find the bookmark record for the given freelancer and job. 2. The system displays: "Bookmark not found." 3. The use case ends.		

	EX-03 Database Connection Failure 1. The system cannot delete the bookmark due to a database error. 2. The system displays: "Unable to remove bookmark. Please try again later." 3. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	**BR-JOB-35:** Only the freelancer can remove their own bookmarks. **BR-JOB-36:** The system must check if the bookmark exists before deletion. **BR-JOB-37:** If the bookmark does not exist, the system shall return a 404 Not Found error. **BR-JOB-38:** Removing a bookmark is a hard delete from the database.
Other Information:	Returns 404 error if bookmark does not exist.
Assumptions:	The freelancer is logged in and has a valid account. Bookmarks are deleted only by their owners.

2.2.7 Manage Contract

2.2.3.1 UC-13.01 Create Contract

UC ID and Name	UC-13.01 Create Contract		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When a Client initiates contract creation from an accepted job proposal or direct hire action.		
Description:	Allows a Client to create and initiate a new contract with a selected Freelancer, specifying contract title, description, milestones, budget amount, and project timeline.		
Preconditions:	**PRE-1** The user is logged into the system as a Client. **PRE-2** The target Freelancer profile is active and verified in the system. **PRE-3** Total contract budget complies with platform minimum and maximum limits.		

Post-conditions:	**POST-1** A new contract record is created in the database with status `PENDING`. **POST-2** Associated milestone records are persisted and linked to the contract. **POST-3** The Freelancer receives a real-time notification to review and sign the contract.
Normal Flow:	**A. Create Contract** 1. The Client accesses the contract creation form from a proposal or direct hire button. 2. The Client enters contract details: Title, Description, Contract Type, Total Budget, and Due Date. 3. The Client configures the project milestones (title, description, amount, due date). 4. The Client clicks "Create Contract". 5. The system validates all required fields, budget allocations, and future milestone deadlines. 6. The system persists the contract and milestone records in the database. 7. The system redirects the Client to the Contract Detail page, displays a success notification, and notifies the Freelancer.
Alternative Flows:	**A.1 Save as Draft** 1. At step 4, the Client clicks "Save Draft". 2. The system saves the entered information with status `DRAFT` without sending notifications to the Freelancer. 3. The Client can resume editing the draft contract at a later time.
Exceptions:	**EX-01: Validation Error** 1. Required fields are empty, or total milestone amounts do not match the total contract budget. 2. The system highlights the invalid input fields with error messages. 3. The use case ends. **EX-02: Server Error** 1. The database transaction fails while persisting contract and milestones. 2. The system rolls back changes and displays "Unable to create contract. Please try again later." 3. The use case ends.
Priority:	High

Frequency of Use:	High
Business Rules:	BR-CONTRACT-01: Only users with the Client role can create new contracts. BR-CONTRACT-02: Total contract budget must be greater than zero and must equal the sum of all defined milestone amounts.
Other Information:	N/A
Assumptions:	The Client and Freelancer have agreed upon the scope and terms prior to formal contract creation.

2.2.3.2 UC-13.02 Update Contract

UC ID and Name	UC-13.02 Update Contract		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When a Client clicks the “Edit Terms” button on a pending Contract Detail page.		
Description:	Allows a Client to update contract terms, scope, milestones, or timeline before the agreement is finalized and signed.		
Preconditions:	**PRE-1** The user is logged in as a Client. **PRE-2** The contract is currently in `PENDING` status (not yet active or completed).		
Post-conditions:	**POST-1** The contract terms are updated in the database. **POST-2** If the Freelancer had previously signed, the signature is reset, requiring re-signing. **POST-3** The Freelancer is notified of the updated contract terms.		

Normal Flow:	**A. Update Contract Terms** 1. The Client navigates to the Contract Detail page of a pending contract. 2. The Client clicks “Edit Terms”. 3. The system displays a form pre-filled with current contract terms and milestones. 4. The Client modifies necessary fields (description, milestones, amounts, or deadlines) and clicks “Save Updates”. 5. The system validates the input data and budget constraints. 6. The system updates the contract and milestone records in the database. 7. The system logs the change, displays a success notification, and notifies the Freelancer to review the revised terms.
Alternative Flows:	**A.1 Cancel Update** 1. While on the edit form, the Client clicks “Cancel”. 2. The system discards all unsaved modifications. 3. Return to step A.1.
Exceptions:	**EX-01: Contract Status Invalid** 1. If the contract is already `ACTIVE` or `COMPLETED`, the system disables editing and displays: “Active contracts cannot be edited directly.” 2. The use case ends. **EX-02: Validation Error** 1. If the Client leaves mandatory fields empty or enters invalid dates, the system highlights the errors. 2. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-CONTRACT-03: Only the Client who created the contract can update its terms. BR-CONTRACT-04: Any update to contract terms resets previous signatures, requiring all parties to review and sign again.
Other Information:	N/A
Assumptions:	The Freelancer will review the updated terms via the standard Sign Contract process (UC-13.06).

2.2.3.3 UC-13.03 Complete Contract

UC ID and Name	UC-13.03 Complete Contract		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the “Complete Contract” button on an active Contract Detail page.		
Description:	Allows a Client or Freelancer to mark an active contract as completed after all milestones are approved and settled, officially concluding the project engagement.		
Preconditions:	**PRE-1** The user is logged into the system as a contract participant. **PRE-2** The contract is currently in `ACTIVE` status. **PRE-3** All milestones associated with the contract have been approved and their escrow funds released.		
Post-conditions:	**POST-1** Contract status is updated to `COMPLETED`. **POST-2** Both parties receive completion notifications and prompts to submit reciprocal reviews and ratings.		
Normal Flow:	**A. Complete Contract** 1. The user navigates to the Contract Detail page of an active contract. 2. The user clicks the “Complete Contract” button. 3. The system displays a confirmation dialog summarizing that all milestones are completed and escrow funds released. 4. The user clicks “Confirm Completion”. 5. The system updates the contract status to `COMPLETED` in the database. 6. The system displays a success toast and redirects both parties to leave ratings and feedback.		
Alternative Flows:	**A.1 Cancel Completion** 1. At the confirmation dialog, the user clicks “Cancel”. 2. The system closes the dialog without changing contract status. 3. Return to step A.1.		

Exceptions:	**EX-01: Pending Milestones** 1. If there are still pending or in-progress milestones, the system displays: “Cannot complete contract. All milestones must be approved first.” 2. The use case ends. **EX-02: System Error** 1. If the database update fails, the system displays: “Failed to complete contract. Please try again.” 2. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-CONTRACT-05: A contract can only transition to `COMPLETED` if its current status is `ACTIVE` and all milestone payments are released. BR-CONTRACT-06: Completed contracts are permanently locked from further deliverable submissions or edits.
Other Information:	N/A
Assumptions:	Both parties agree on the final completion status based on approved deliverables.

2.2.3.4 UC-13.04 View Contract List

UC ID and Name	UC-13.04 View Contract List		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user navigates to the Contracts section from the navigation menu.		
Description:	Allows a Client, Freelancer, or Admin to retrieve a paginated list of contracts with status filters (Pending, Active, Completed, Canceled) and search capabilities.		
Preconditions:	**PRE-1** The user is logged into the system. **PRE-2** The system is connected to the database.		
Post-conditions:	**POST-1** The system renders a paginated list of contracts relevant to the user’s role.		

Normal Flow:	**A. View Contract List** 1. The user navigates to the Contracts page. 2. The system retrieves contracts associated with the user account (Client or Freelancer; Admin retrieves all). 3. The system applies default pagination and sorts by newest created date. 4. The system displays contract cards showing title, counterpart name, budget, progress bar, and status badge.
Alternative Flows:	**A.1 Filter and Sort Contracts** 1. The user selects a status filter tab (e.g., Pending, Active, Completed, Canceled) or sorting option. 2. The system updates the displayed list according to the selected criteria. 3. Return to step A.4.
Exceptions:	**EX-01: No Contracts Found** 1. If the user has no associated contracts matching criteria, the system displays: "No contracts found." 2. The use case ends. **EX-02: Database Error** 1. If the system fails to fetch records, it displays: "Unable to load contracts. Please try again." 2. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-CONTRACT-07: Clients and Freelancers can only view contracts where they are listed as a participant. BR-CONTRACT-08: Administrators can view and monitor all contracts platform-wide.
Other Information:	N/A
Assumptions:	Users understand pagination and status filter conventions.

2.2.3.5 UC-13.05 View Contract Detail

UC ID and Name	UC-13.05 View Contract Detail		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer, Admin	Secondary Actors:	N/A

Trigger:	When the user clicks on a contract item from the Contract List page.
Description:	Allows a Client, Freelancer, or Admin to view full contract details including project scope, participant profiles, escrow financial status, milestones, attached files, and activity logs.
Preconditions:	**PRE-1** The user is authenticated in the system. **PRE-2** The contract exists in the database. **PRE-3** The user is an authorized participant or an Administrator.
Post-conditions:	**POST-1** The system renders the comprehensive details of the selected contract.
Normal Flow:	**A. View Contract Details** 1. The user clicks on a contract row or card from the Contract List. 2. The system verifies that the user is authorized to access the contract. 3. The system loads contract metadata, parties, escrow balances, and milestone items. 4. The system navigates to the Contract Detail page and displays: - Header: Title, Contract ID, Status badge, and Counterpart profile. - Financial Summary: Total budget, Escrow funded balance, and Remaining amount. - Milestones list with progress indicators. - Tab navigation for Milestones, Files, and Activity history.
Alternative Flows:	**A.1 Switch Detail Tabs** 1. The user clicks on the “Files” or “Activities” tab. 2. The system loads and displays the selected section without page reload. 3. Return to step A.4.
Exceptions:	**EX-01: Unauthorized Access** 1. If an unauthorized user attempts direct URL access, the system displays: “Access Denied: You do not have permission to view this contract.” 2. The use case ends. **EX-02: Contract Not Found** 1. If the contract ID is invalid or deleted, the system displays: “Contract not found.” 2. The use case ends.
Priority:	High

Frequency of Use:	High
Business Rules:	BR-CONTRACT-09: Contract financial balances (total budget, escrow balance, released funds) must remain mathematically synchronized with milestone states. BR-CONTRACT-10: Strict data isolation ensures only authorized parties can access contract details.
Other Information:	N/A
Assumptions:	Contracts contain defined deliverables and milestone structures.

2.2.3.6 UC-13.06 Sign Contract

UC ID and Name	UC-13.06 Sign Contract		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer, Client	Secondary Actors:	N/A
Trigger:	When a user clicks the “Sign Contract” button on a pending Contract Detail page.		
Description:	Allows a Freelancer (or Client) to digitally accept and sign a contract, activating the agreement so that project work and escrow deposits can officially commence.		
Preconditions:	**PRE-1** The user is logged into the system. **PRE-2** The contract is currently in `PENDING` status. **PRE-3** The user has reviewed all contract clauses and milestone deliverables.		
Post-conditions:	**POST-1** The contract status transitions to `ACTIVE`. **POST-2** A digital signature audit record with timestamp and user ID is recorded. **POST-3** Both parties receive confirmation notifications that the contract is active.		

Normal Flow:	**A. Sign Contract** 1. The user navigates to the Contract Detail page of a pending contract. 2. The user reviews the terms, deliverables, and milestones. 3. The user clicks “Sign Contract”. 4. The system displays a confirmation dialog for digital agreement acceptance. 5. The user checks the agreement checkbox and clicks “Confirm & Sign”. 6. The system records the signature timestamp and updates contract status to `ACTIVE`. 7. The system renders a success toast, refreshes the contract view, and notifies the other party.
Alternative Flows:	**A.1 Cancel Signature** 1. At the confirmation dialog, the user clicks “Cancel”. 2. The system closes the dialog without updating contract status. 3. Return to step A.1.
Exceptions:	**EX-01: Contract Already Active or Expired** 1. If the contract is no longer in pending status, the system displays: “This contract cannot be signed as it is already active or expired.” 2. The use case ends. **EX-02: Database Transaction Error** 1. If saving the signature record fails, the system displays: “An error occurred while signing the contract. Please try again.” 2. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-CONTRACT-11: A contract transitions to `ACTIVE` only when all required counterpart signatures are recorded. BR-CONTRACT-12: An immutable signature log must be kept for legal transparency and audit compliance.
Other Information:	N/A
Assumptions:	Digital signature denotes authenticated consent to the platform contract terms.

2.2.3.7 UC-13.07 Cancel Contract

UC ID and Name	UC-13.07 Cancel Contract		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks “Cancel Contract” from the contract action menu.		
Description:	Allows a Client or Freelancer to cancel or terminate a contract prior to completion, specifying a cancellation reason and handling pending escrow resolutions.		
Preconditions:	**PRE-1** The user is logged into the system as a contract participant. **PRE-2** The contract is in `PENDING` or `ACTIVE` status (not completed or already canceled).		
Post-conditions:	**POST-1** Contract status is updated to `CANCELED`. **POST-2** Cancellation reason is recorded in the contract history. **POST-3** Unreleased escrow balances are handled according to platform cancellation policies. **POST-4** The other party is notified immediately.		
Normal Flow:	**A. Cancel Contract** 1. The user navigates to the Contract Detail page. 2. The user clicks “Cancel Contract”. 3. The system presents a cancellation confirmation modal requiring a cancellation reason. 4. The user selects a reason category and enters detailed explanation. 5. The user clicks “Confirm Cancellation”. 6. The system updates the contract status to `CANCELED` in the database. 7. The system records the cancellation event, sends an alert to the counterpart, and displays a confirmation toast.		
Alternative Flows:	**A.1 Abort Cancellation** 1. At the confirmation modal, the user clicks “Keep Contract”. 2. The system dismisses the modal without modifying contract state. 3. Return to step A.1.		

Exceptions:	**EX-01: Invalid Status for Cancellation** 1. If the contract is already completed, the system displays: "Completed contracts cannot be canceled." 2. The use case ends. **EX-02: Missing Reason** 1. If the user submits without providing a cancellation reason, the system prompts: "Please provide a cancellation reason." 2. The use case ends.
Priority:	High
Frequency of Use:	Low
Business Rules:	BR-CONTRACT-13: Canceled contracts cannot be reopened or reactivated; a new contract must be created. BR-CONTRACT-14: If work has been submitted on active milestones, cancellation requires dispute resolution or escrow mutual release.
Other Information:	N/A
Assumptions:	Financial escrow refunds adhere to standard platform dispute rules.

2.2.3.8 UC-13.08 View Contract Files

UC ID and Name	UC-13.08 View Contract Files		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user clicks the "Files" tab within the Contract Detail page.		
Description:	Allows contract participants to view and access all files, project briefs, and documents attached to the contract workspace.		
Preconditions:	**PRE-1** The user is authenticated in the system. **PRE-2** The user is a participant of the contract or an Administrator.		
Post-conditions:	**POST-1** The system renders the list of active contract files with download links.		

Normal Flow:	**A. View Contract Files** 1. The user navigates to the Contract Detail page. 2. The user clicks on the “Files” tab. 3. The system queries the database for all active files associated with the contract ID. 4. The system renders the files list displaying file name, extension icon, uploader name, file size, and upload date. 5. The user can click “Download” next to any file to download it from cloud storage.
Alternative Flows:	**A.1 Empty Files List** 1. If no files have been uploaded yet, the system displays an empty state banner with an “Upload File” button. 2. Return to step A.1.
Exceptions:	**EX-01: Storage Retrieval Error** 1. If the system fails to load file records from storage, it displays: “Unable to load contract files. Please try again.” 2. The use case ends.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-CONTRACT-15: Only authorized contract parties (and Admins) can access or download attached files. BR-CONTRACT-16: Soft-deleted files are hidden from regular participants.
Other Information:	N/A
Assumptions:	Files are stored securely in cloud storage with secure download URLs.

2.2.3.9 UC-13.09 Upload Contract File

UC ID and Name	UC-13.09 Upload Contract File		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	Cloud Storage
Trigger:	When a user clicks “Upload File” in the Contract Files section.		
Description:	Allows a Client or Freelancer to upload documentation, technical specifications, or deliverable files to the contract workspace.		

Preconditions:	**PRE-1** The user is logged in as an authorized contract participant. **PRE-2** The contract status is `PENDING` or `ACTIVE`. **PRE-3** The selected file is within the maximum size limit ($\leq$ 25MB) and has a valid file format.
Post-conditions:	**POST-1** The file is securely stored in cloud storage. **POST-2** File metadata (name, URL, public ID, size, uploader) is saved in the database. **POST-3** The file list refreshes and the counterpart is notified.
Normal Flow:	**A. Upload Contract File** 1. The user clicks the "Upload File" button or drags a file into the dropzone. 2. The user selects a file from their local device file chooser. 3. The system validates file size ($\leq$ 25MB) and checks against restricted extensions. 4. The system streams the file to cloud storage. 5. The system records the file metadata in the database linked to the contract ID. 6. The system displays a success toast and appends the new file card to the files list.
Alternative Flows:	**A.1 Cancel Upload** 1. The user cancels the file dialog before choosing a file. 2. The system dismisses the prompt without uploading. 3. Return to step A.1.
Exceptions:	**EX-01: File Size Exceeded** 1. If the selected file exceeds 25MB, the system displays: "File size exceeds 25MB limit. Please upload a smaller file." 2. The use case ends. **EX-02: Unsupported File Format** 1. If the user attempts to upload an executable or restricted file, the system displays: "File format not allowed for security reasons." 2. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-CONTRACT-17: Maximum allowable file size is 25MB per file. BR-CONTRACT-18: Executable file types (.exe, .bat, .sh, .cmd) are strictly blocked to prevent malware.

Other Information:	N/A
Assumptions:	Files are stored on secure cloud object storage.

2.2.3.10 UC-13.10 Delete Contract File

UC ID and Name	UC-13.10 Delete Contract File		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	Cloud Storage
Trigger:	When a user clicks the delete trash icon next to a file card in the Files section.		
Description:	Allows the file uploader or an Administrator to remove an obsolete or incorrect file from the contract workspace.		
Preconditions:	**PRE-1** The user is logged in. **PRE-2** The user is the original uploader of the file or has an Administrator role. **PRE-3** The target file exists in the active files list.		
Post-conditions:	**POST-1** The file asset is removed from cloud storage and soft-deleted in the database. **POST-2** The file card is removed from the files view with a confirmation toast.		
Normal Flow:	**A. Delete File** 1. The user clicks the trash icon next to the target file. 2. The system displays a confirmation dialog: “Are you sure you want to delete this file?” 3. The user clicks “Delete”. 4. The system validates ownership and deletes the file asset from cloud storage. 5. The system marks the database file record with `deletedAt = new Date()`. 6. The dialog closes, the file is removed from the list, and a success toast appears.		
Alternative Flows:	**A.1 Cancel Deletion** 1. At the confirmation dialog, the user clicks “Cancel”. 2. The system dismisses the dialog without deleting the file. 3. Return to step A.1.		

Exceptions:	**EX-01: Permission Denied** 1. If the user is not the uploader and not an Admin, the system displays: “You do not have permission to delete this file.” 2. The use case ends. **EX-02: File Already Deleted** 1. If the file has already been removed, the system displays: “File not found or already deleted.” 2. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-CONTRACT-19: Only the user who uploaded the file or an Administrator can delete it. BR-CONTRACT-20: File deletion operations are recorded in the contract audit logs.
Other Information:	N/A
Assumptions:	Deleted files cannot be retrieved by regular contract participants once deleted.

2.2.8 Manage Forum

2.2.3.1 UC-04.1 View Forum Categories

UC ID and Name	UC-04.1 View Forum Categories		
Created By:	Pham Kha Vy	Date Created:	20/09/2026
Primary Actor:	Guest, Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When any user accesses the Forum section from the navigation menu.		
Description:	As a guest or registered user, I want to view all active forum categories so that I can browse discussion topics and select a category to explore forum posts.		
Preconditions:	**PRE-1** The system is connected to the database. **PRE-2** At least one active (non-deleted) category exists in the ForumCategory table.		

Post-conditions:	**POST-1** The list of active forum categories is displayed with `id`, `name`, `slug`, `description`, `createdAt`, `updatedAt`, and `postCount`. **POST-2** Users can select a category to view its details and related posts.
Normal Flow:	**A. View Forum Categories** 1. The User accesses the Forum page from the navigation menu. 2. The system retrieves all active forum categories from the ForumCategory table. 3. The system displays the category name, description, and post count for each active category. 4. The User selects a category from the list. 5. The system navigates the User to the selected category detail page.
Alternative Flows:	**A.1 No Active Categories Found** 1. The User accesses the Forum page. 2. The system determines that no active categories exist in the ForumCategory table. 3. The system displays the message “No forum categories available.” 4. The User remains on the Forum page. 5. Return to step 1.

Exceptions:	**EX-01: Database Error** 1. The system attempts to retrieve the list of active forum categories from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load forum categories. Please try again later.” 4. The forum category list is not displayed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the forum category data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the current page without any category information being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-FORUM-01: Only categories with `deletedAt = NULL` shall be displayed. BR-FORUM-02: Categories shall be ordered by `name` in ascending order. BR-FORUM-03: `postCount` shall count only posts with `deletedAt = NULL` and `moderationStatus = APPROVED`.
Other Information:	The endpoint is public and does not require authentication.
Assumptions:	Forum categories are created and maintained by administrators.

2.2.3.2 UC-04.2 View Forum Category Detail

UC ID and Name	UC-04.2 View Forum Category Detail		
Created By:	Pham Kha Vy	Date Created:	20/09/2026
Primary Actor:	Guest, Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user selects a forum category from the forum category list.		
Description:	Allows users to view detailed information about a selected forum category including its name, slug, description, and creation date.		

Preconditions:	**PRE-1** The user is viewing the forum category list (UC-04.1). **PRE-2** The selected category exists and has `deletedAt = NULL`.
Post-conditions:	**POST-1** The selected category information is displayed successfully including `id`, `name`, `slug`, `description`, `createdAt`, `updatedAt`, and `postCount`. **POST-2** The user can continue to browse posts belonging to the selected category.
Normal Flow:	**A. View Forum Category Detail** 1. The User selects a forum category from the Forum Categories page. 2. The system validates that the selected category exists and is active. 3. The system retrieves the category information including its name, slug, description, and creation date. 4. The system displays the category details. 5. The system renders the post list page for the selected category.
Alternative Flows:	**A.1 Selected Category Is No Longer Available** 1. The User selects a forum category. 2. The system determines that the selected category has been deleted or is not found. 3. The system displays the message “The selected category is no longer available.” 4. The system redirects the User to the Forum Categories page. 5. Return to step 1.

Exceptions:	**EX-01: Database Error** 1. The system attempts to retrieve the selected forum category from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load category details. Please try again later.” 4. The category details are not displayed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the requested category information. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the current page without any category information being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-FORUM-04: Only categories with `deletedAt = NULL` shall be accessible. BR-FORUM-05: Category information displayed includes `id`, `name`, `slug`, `description`, `createdAt`, `updatedAt`, and `postCount`.
Other Information:	The endpoint is public and does not require authentication.
Assumptions:	Forum categories are created and maintained by administrators.

2.2.3.3 UC-04.3 View Post List

UC ID and Name	UC-04.3 View Post List		
Created By:	Pham Kha Vy	Date Created:	21/09/2026
Primary Actor:	Guest, Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user opens a forum category to browse discussion posts.		
Description:	Allows users to view a paginated list of forum posts within a selected category. The list includes post titles, authors, creation dates, like counts, and comment counts.		

Preconditions:	**PRE-1** The selected category exists and is active (`deletedAt = NULL`). **PRE-2** The system is connected to the database.
Post-conditions:	**POST-1** A paginated list of forum posts is displayed with pagination metadata `{ page, limit, total, totalPages }`. **POST-2** Users can select a post to view its details.
Normal Flow:	**A. View Post List** 1. The User opens a forum category from the Forum Categories page. 2. The system validates that the selected category exists and is active. 3. The system retrieves posts associated with the selected category from the ForumPost table, excluding deleted posts and posts that are not approved. 4. The system optionally filters posts by user or search term, and applies pagination. 5. The system displays the post titles, authors, creation dates, like counts, and comment counts. 6. The User selects a post to view its details.
Alternative Flows:	**A.1 No Posts Found** 1. The User opens a forum category. 2. The system determines that no posts exist in the selected category. 3. The system displays the message “No discussion posts available in this category.” 4. The User remains on the category page. 5. Return to step 1. **A.5 Navigate Between Pages** 1. The User clicks a page number, “Next,” or “Previous” button, or changes the page size. 2. The system retrieves the requested page of posts from the ForumPost table. 3. The system displays the requested page of posts. 4. Return to step 5.

Exceptions:	**EX-01: Database Error** 1. The system attempts to retrieve the list of posts for the selected category from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load forum posts. Please try again later.” 4. The post list is not displayed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the requested post data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the current page without any post information being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-FORUM-06: Only posts with `deletedAt = NULL` and `moderationStatus = APPROVED` shall be displayed. BR-FORUM-07: Posts shall be sorted by `createdAt` in descending order. BR-FORUM-08: The system shall support pagination and configurable page sizes. BR-FORUM-09: `commentCount` shall count only comments with `deletedAt = NULL`.
Other Information:	Default page size is 10. A search bar and “My Posts” toggle (filter by user) are available on the frontend.
Assumptions:	Forum posts are associated with valid categories and authors.

2.2.3.4 UC-04.4 View Post Detail

UC ID and Name	UC-04.4 View Post Detail		
Created By:	Pham Kha Vy	Date Created:	21/09/2026
Primary Actor:	Guest, Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user selects a post from the post list.		

Description:	Allows users to view the complete details of a forum post including its title, content, author, category, and creation date.
Preconditions:	**PRE-1** The selected post exists. **PRE-2** The post has not been soft-deleted and has `moderationStatus = APPROVED`.
Post-conditions:	**POST-1** The complete post information is displayed including nested category and user profile. **POST-2** Users can view related comments and reactions associated with the post.
Normal Flow:	**A. View Post Detail** 1. The User selects a post from the post list. 2. The system validates that the selected post exists, is not deleted, and is approved. 3. The system retrieves the post information, author information, and category information. 4. The system displays the post detail page with the title, content, author, category, and creation date. 5. The system displays the available comments and reaction count.
Alternative Flows:	**A.1a Selected Post Is No Longer Available (soft-deleted)** 1. The User selects a post. 2. The system determines that the selected post has been soft-deleted. 3. The system displays the message “This post is no longer available.” 4. The system redirects the User to the previous post list page. 5. Return to step 1. **A.1b Selected Post Not Found** 1. The User selects a post. 2. The system determines that the selected post does not exist or is not yet approved. 3. The system displays the message “Post not found.” 4. The system redirects the User to the previous post list page. 5. Return to step 1.

Exceptions:	**EX-01: Database Error** 1. The system attempts to retrieve the post, author, and category information from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load post details. Please try again later.” 4. The post details are not displayed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the requested post information. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the current page without any post information being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-FORUM-10: Only posts with `deletedAt = NULL` and `moderationStatus = APPROVED` shall be displayed. BR-FORUM-11: Post details shall include title, content, author, category, and created date.
Other Information:	Comments and reactions are displayed together with the post details.
Assumptions:	Each post is associated with a valid author and category.

2.2.3.5 UC-04.5 Create Post

UC ID and Name	UC-04.5 Create Post		
Created By:	Pham Kha Vy	Date Created:	22/09/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the “Create Post” button in the Forum section.		
Description:	Allows authenticated users to create a new forum post by providing a title, content, and optional category.		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** At least one active forum category exists (optional for assignment).
Post-conditions:	**POST-1** A new post is created in the ForumPost table. **POST-2** The new post appears in the selected category after being approved (if moderation applies).
Normal Flow:	**A. Create Post** 1. The User clicks the “Create Post” button in the Forum section. 2. The system verifies that the User is authenticated. 3. The system displays the Create Post dialog. 4. The User enters the post title and content, and optionally selects a forum category. 5. The User clicks the “Publish” button. 6. The system validates the input data. 7. The system creates a new post in the ForumPost table associated with the authenticated User. 8. If the new post is approved, the system redirects the User to the newly created post detail page; otherwise it shows a message that the post is awaiting admin review.

Alternative Flows:	**A.3 User Not Authenticated** 1. The User clicks the “Create Post” button. 2. The system determines that the User is not authenticated. 3. The system redirects the User to the Login page. 4. The User logs in successfully. 5. The system returns the User to the Create Post dialog. 6. Return to step 3. **A.5 User Cancels Creation** 1. The User clicks the “Cancel” button on the Create Post dialog. 2. The system discards the unsaved data. 3. The system returns the User to the Forum page. 4. The use case ends. **A.4a Required Fields Missing** 1. The User clicks the “Publish” button. 2. The system determines that the title or content is empty. 3. The system displays the message “Please complete all required fields.” 4. Return to step 4. **A.4b Invalid Category Selected** 1. The User submits the post. 2. The system determines that the selected category does not exist. 3. The system displays the message “Invalid category selected.” 4. Return to step 4. **A.8 Post Awaits Moderation** 1. The User submits the post. 2. The post moderation status is PENDING. 3. The system displays the message “Your post has been saved and is awaiting admin review.” 4. The post is not displayed publicly until approved. 5. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to save the new post to the ForumPost table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to create post. Please try again later.” 4. The post is not created. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while submitting the post. 2. The system cannot save the post data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the Create Post dialog without the post being saved. 5. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-FORUM-12: Only authenticated users can create posts. BR-FORUM-13: Every post must belong to exactly one user and may belong to a category. BR-FORUM-14: Title and content are required fields. BR-FORUM-15: A new post shall default to `moderationStatus = PENDING` unless moderation data is provided.
Other Information:	The post creator is automatically assigned from the currently authenticated user account.
Assumptions:	Forum categories are available and maintained by administrators.

2.2.3.6 UC-04.6 Edit Post

UC ID and Name	UC-04.6 Edit Post		
Created By:	Pham Kha Vy	Date Created:	22/09/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When the author selects the “Edit Post” option on their post.		
Description:	Allows the author of a post to modify its title, content, or category.		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected post exists and has `deletedAt = NULL`. **PRE-3** The user is the author of the selected post.
Post-conditions:	**POST-1** The post information is updated successfully, including a regenerated slug when the title changes. **POST-2** The updated post is displayed to users.
Normal Flow:	**A. Edit Post** 1. The User selects the "Edit Post" option on their post. 2. The system verifies that the User is the author of the selected post. 3. The system displays the existing post information in an editable dialog. 4. The User modifies the title, content, or category. 5. The User clicks the "Save Changes" button. 6. The system validates the updated information. 7. The system updates the post record in the ForumPost table, regenerating the slug if the title changed. 8. The system displays the updated post details.
Alternative Flows:	**A.1 User Is Not the Author** 1. The User selects the "Edit Post" option on a post. 2. The system determines that the User is not the author of the selected post. 3. The system displays the message "You are not authorized to edit this post." 4. Return to step 1. **A.5 User Cancels Editing** 1. The User clicks the "Cancel" button on the edit dialog. 2. The system discards the unsaved changes. 3. The system returns the User to the post detail page. 4. The use case ends. **A.4 Required Fields Missing** 1. The User clicks the "Save Changes" button. 2. The system determines that the title or content is empty. 3. The system displays the message "Please complete all required fields." 4. Return to step 4.

Exceptions:	**EX-01: Database Error** 1. The system attempts to update the post record in the ForumPost table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to update post. Please try again later.” 4. The post is not updated. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while saving the changes. 2. The system cannot save the updated post data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the edit dialog without the changes being saved. 5. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-FORUM-16: Only the author of a post can edit that post. BR-FORUM-17: Title and content are required fields. BR-FORUM-18: The original post ID remains unchanged after editing; the slug is regenerated if the title changes.
Other Information:	Editing does not trigger re-moderation.
Assumptions:	The user remains authenticated throughout the editing process.

2.2.3.7 UC-04.7 Delete Post

UC ID and Name	UC-04.7 Delete Post		
Created By:	Pham Kha Vy	Date Created:	23/09/2026
Primary Actor:	Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When the author or an Admin selects the “Delete Post” option on a post.		
Description:	Allows the author of a post or an Admin to remove it from public view by performing a soft delete operation.		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected post exists and has `deletedAt = NULL`. **PRE-3** The user is the author of the selected post or has the Admin role.
Post-conditions:	**POST-1** The post is soft-deleted by setting the `deletedAt` field. **POST-2** The post is no longer displayed in forum post listings.
Normal Flow:	**A. Delete Post** 1. The User selects the "Delete Post" option on a post. 2. The system verifies that the User is the author of the selected post or has the Admin role. 3. The system displays a confirmation message. 4. The User confirms the deletion request. 5. The system performs a soft delete by updating the `deletedAt` field of the selected post. 6. The system removes the post from public forum listings. 7. The system redirects the User to the category post list page.
Alternative Flows:	**A.1a User Not Authorized** 1. The User selects the "Delete Post" option on a post. 2. The system determines that the User is not the author nor an Admin. 3. The system displays the message "You are not authorized to delete this post." 4. Return to step 1. **A.4 User Cancels Deletion** 1. The User selects the "Cancel" option on the confirmation message. 2. The system cancels the deletion process. 3. The system returns the User to the post detail page. 4. The use case ends. **A.1b Selected Post Not Found or Already Deleted** 1. The User confirms the deletion request. 2. The system determines that the selected post does not exist or is already soft-deleted. 3. The system displays the message "Post not found." 4. Return to step 1.

Exceptions:	**EX-01: Database Error** 1. The system attempts to update the `deletedAt` field of the selected post in the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to delete post. Please try again later.” 4. The post is not deleted. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the deletion request. 2. The system cannot complete the deletion of the post. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The post remains unchanged. 5. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-FORUM-19: Only the author of a post or an Admin can delete that post. BR-FORUM-20: Post deletion shall be implemented as a soft delete using the `deletedAt` field. BR-FORUM-21: Soft-deleted posts shall not appear in public forum listings.
Other Information:	Deleted posts remain stored in the database and can be restored by administrators.
Assumptions:	The user remains authenticated during the deletion process.

2.2.3.8 UC-04.8 Like Post

UC ID and Name	UC-04.8 Like Post		
Created By:	Pham Kha Vy	Date Created:	23/09/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the “Like” button on a forum post.		
Description:	Allows authenticated users to express appreciation for a post by adding a like reaction (toggle action).		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected post exists and has `deletedAt = NULL`.
Post-conditions:	**POST-1** A like record is created in the ForumLike table associated with the user and post. **POST-2** The post's reaction count is updated and the "Like" button is highlighted.
Normal Flow:	**A. Like Post** 1. The User clicks the "Like" button on a post. 2. The system verifies that the User is authenticated. 3. The system verifies that the post exists and has not been deleted. 4. The system verifies that the User has not already liked the post. 5. The system creates a new like record associated with the User and the post in the ForumLike table. 6. The system updates the displayed reaction count and highlights the "Like" button.
Alternative Flows:	**A.1a User Not Authenticated** 1. The User clicks the "Like" button. 2. The system determines that the User is not authenticated. 3. The system redirects the User to the Login page. 4. The User logs in successfully. 5. The system returns the User to the selected post. 6. Return to step 1. **A.1b Selected Post Not Found or Deleted** 1. The User clicks the "Like" button on a post. 2. The system determines that the selected post does not exist or is soft-deleted. 3. The system displays the message "Post not found." 4. Return to step 1. **A.4 Post Already Liked** 1. The User clicks the "Like" button on a post they have already liked. 2. The system determines that the User has already liked the post. 3. The system performs the unlike action (see UC-04.9). 4. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to create a new like record in the ForumLike table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to process your request. Please try again later.” 4. The like reaction is not saved. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the like request. 2. The system cannot save the reaction. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The reaction count remains unchanged. 5. The use case ends.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-FORUM-22: Only authenticated users can like posts. BR-FORUM-23: A user can have only one active like reaction per post. BR-FORUM-24: The reaction shall be stored in the ForumLike table.
Other Information:	The like action is a toggle and uses optimistic updates on the frontend.
Assumptions:	The selected post is accessible to the user.

2.2.3.9 UC-04.9 Unlike Post

UC ID and Name	UC-04.9 Unlike Post		
Created By:	Pham Kha Vy	Date Created:	24/09/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks an active “Like” button on a post that they have previously liked.		
Description:	Allows authenticated users to remove their like reaction from a forum post.		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected post exists and has `deletedAt = NULL`. **PRE-3** The user has previously liked the selected post.
Post-conditions:	**POST-1** The corresponding like record is removed from the ForumLike table. **POST-2** The post's reaction count is updated and the "Like" button is set to its inactive state.
Normal Flow:	**A. Unlike Post** 1. The User clicks the active "Like" button on a post. 2. The system verifies that the User is authenticated. 3. The system verifies the existing like relationship between the User and the post. 4. The system removes the corresponding like record from the ForumLike table. 5. The system updates the displayed reaction count and changes the "Like" button to its inactive state.
Alternative Flows:	**A.1a Selected Post Not Found** 1. The User clicks the "Like" button on a post. 2. The system determines that the selected post does not exist or is soft-deleted. 3. The system displays the message "Post not found." 4. Return to step 1. **A.1b User Not Authenticated** 1. The User attempts to remove a reaction from a post. 2. The system determines that the User is not authenticated. 3. The system redirects the User to the Login page. 4. The User logs in successfully. 5. The system returns the User to the selected post. 6. Return to step 1. **A.3 Like Record Not Found** 1. The User clicks the "Like" button on a post. 2. The system determines that no like record exists for the User and the selected post. 3. The system treats the action as a like. 4. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to remove the like record from the ForumLike table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to process your request. Please try again later.” 4. The like reaction is not removed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the unlike request. 2. The system cannot remove the reaction. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The reaction count remains unchanged. 5. The use case ends.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-FORUM-25: Only authenticated users can remove post reactions. BR-FORUM-26: A like reaction can only be removed if a corresponding like record exists. BR-FORUM-27: Removing a reaction shall delete the corresponding record from the ForumLike table.
Other Information:	The like/unlike action is a single toggle endpoint.
Assumptions:	The selected post is accessible to the user.

2.2.3.10 UC-04.10 View Like Post Detail

UC ID and Name	UC-04.10 View Like Post Detail		
Created By:	Pham Kha Vy	Date Created:	24/09/2026
Primary Actor:	Guest, Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user clicks the reaction/like count link on a forum post.		
Description:	Allows users to view the list of people who have liked a specific forum post.		
Preconditions:	**PRE-1** The selected post exists and has `deletedAt = NULL`. **PRE-2** The post detail page is displayed successfully.		

Post-conditions:	**POST-1** The list of likes for the selected post is displayed. **POST-2** Users can view each liker's display name and avatar.
Normal Flow:	**A. View Like Post Detail** 1. The User clicks the reaction count link on the post action bar. 2. The system retrieves all likes for the selected post from the ForumLike table. 3. The system displays a dialog listing all likers with their avatars.
Alternative Flows:	**A.1a No Likes Found** 1. The User clicks the reaction count link. 2. The system determines that no likes exist for the selected post. 3. The system displays the message "No likes yet." 4. The User remains on the post detail page. 5. Return to step 1. **A.1b Selected Post Not Found** 1. The User clicks the reaction count link. 2. The system determines that the selected post does not exist or is soft-deleted. 3. The system displays the message "Post not found." 4. Return to step 1.
Exceptions:	**EX-01: Database Error** 1. The system attempts to retrieve the list of likes for the post from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message "Unable to load reactions. Please try again later." 4. The like list is not displayed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the requested like data. 3. The system displays the message "Cannot connect to the server. Please try again later." 4. The User remains on the current page without like information being displayed. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Medium

Business Rules:	BR-FORUM-28: The like detail shall include the user id and user profile information for each liker.
Other Information:	All likes for a post are returned without pagination.
Assumptions:	The selected post is accessible to the user.

2.2.3.11 UC-04.11 View Post Comments

UC ID and Name	UC-04.11 View Post Comments		
Created By:	Pham Kha Vy	Date Created:	25/09/2026
Primary Actor:	Guest, Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user views a forum post detail page (UC-04.4).		
Description:	Allows users to view all non-deleted comments associated with a selected forum post.		
Preconditions:	**PRE-1** The selected post exists and has `deletedAt = NULL`. **PRE-2** The post detail page is displayed successfully.		
Post-conditions:	**POST-1** A paginated list of comments associated with the selected post is displayed. **POST-2** Users can view comment content, authors, creation dates, and like counts.		
Normal Flow:	**A. View Post Comments** 1. The User opens a post detail page (UC-04.4). 2. The system retrieves comments associated with the selected post from the ForumComment table, excluding deleted comments. 3. The system retrieves author information for each comment. 4. The system sorts the comments by creation date and applies pagination. 5. The system displays the comment content, author information, creation date, and like count for each comment.		

Alternative Flows:	**A.1a No Comments Found** 1. The User opens a post detail page. 2. The system determines that no comments exist for the selected post. 3. The system displays the message “No comments available.” 4. The User remains on the post detail page. 5. Return to step 1. **A.1b Associated Post Is No Longer Available** 1. The User attempts to view comments for a post. 2. The system determines that the associated post does not exist or is soft-deleted. 3. The system displays the message “Post not found.” 4. The system redirects the User to the previous page. 5. Return to step 1.
Exceptions:	**EX-01: Database Error** 1. The system attempts to retrieve the comment list for the selected post from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load comments. Please try again later.” 4. The comment list is not displayed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the requested comment data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the current page without any comment information being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-FORUM-29: Only comments with `deletedAt = NULL` shall be displayed. BR-FORUM-30: Comments shall be sorted by creation date. BR-FORUM-31: For authenticated users, each comment shall indicate whether it has been liked by the current user.

Other Information:	Comments are displayed within the selected post detail page's comment section.
Assumptions:	Each comment is associated with a valid post and author.

2.2.3.12 UC-04.12 Add Comment

UC ID and Name	UC-04.12 Add Comment		
Created By:	Pham Kha Vy	Date Created:	25/09/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user enters comment content and clicks the "Submit Comment" button.		
Description:	Allows authenticated users to add a new comment to a forum post and participate in community discussions.		
Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected post exists and has `deletedAt = NULL`.		
Post-conditions:	**POST-1** A new comment record is inserted into the ForumComment table. **POST-2** The new comment is displayed in the comment list of the post.		
Normal Flow:	**A. Add Comment** 1. The User enters content into the comment input field on a post detail page. 2. The User clicks the "Submit Comment" button. 3. The system validates the comment content. 4. The system creates a new comment record associated with the selected post and the User in the ForumComment table. 5. The system updates the comment list and displays the new comment.		

Alternative Flows:	**A.1a User Not Authenticated** 1. The User attempts to submit a comment. 2. The system determines that the User is not authenticated. 3. The system redirects the User to the Login page. 4. The User logs in successfully. 5. The system returns the User to the selected post. 6. Return to step 1. **A.1b Empty Comment Content** 1. The User attempts to submit a comment. 2. The system determines that the comment content is empty. 3. The system displays the message “Comment content cannot be empty.” 4. Return to step 1. **A.1c Selected Post Not Found** 1. The User clicks the “Submit Comment” button. 2. The system determines that the selected post does not exist or is soft-deleted. 3. The system displays the message “Post not found.” 4. Return to step 1.
Exceptions:	**EX-01: Database Error** 1. The system attempts to save the new comment to the ForumComment table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to submit comment. Please try again later.” 4. The comment is not created. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while submitting the comment. 2. The system cannot save the comment data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the post detail page without the comment being saved. 5. The use case ends.
Priority:	High
Frequency of Use:	High

Business Rules:	BR-FORUM-32: Only authenticated users can create comments. BR-FORUM-33: Comment content cannot be empty. BR-FORUM-34: Each comment shall be associated with a valid post and user.
Other Information:	Comments are displayed in chronological order based on their creation time.
Assumptions:	The selected post remains available during the comment submission process.

2.2.3.13 UC-04.13 Edit Comment

UC ID and Name	UC-04.13 Edit Comment		
Created By:	Pham Kha Vy	Date Created:	26/09/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the “Edit” option on one of their comments.		
Description:	Allows users to modify the content of a comment that they previously created.		
Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected comment exists and has `deletedAt = NULL`. **PRE-3** The user is the owner of the selected comment.		
Post-conditions:	**POST-1** The comment content is updated in the ForumComment table. **POST-2** The updated comment is displayed on the post detail page.		
Normal Flow:	**A. Edit Comment** 1. The User clicks the “Edit” option on their comment. 2. The system verifies that the User is the owner of the selected comment. 3. The system displays the current comment content in an editable form. 4. The User modifies the comment content. 5. The User clicks the “Save Changes” button. 6. The system validates the updated content. 7. The system updates the comment record in the ForumComment table. 8. The system displays the updated comment.		

Alternative Flows:	**A.1a User Not Authorized** 1. The User clicks the “Edit” option on a comment. 2. The system determines that the User is not the owner of the selected comment. 3. The system displays the message “You are not authorized to edit this comment.” 4. Return to step 1. **A.4a Cancel Comment Editing** 1. The User clicks the “Cancel” button on the edit form. 2. The system discards all unsaved changes. 3. The system displays the original comment content. 4. The use case ends. **A.4b Empty Comment Content** 1. The User clicks the “Save Changes” button. 2. The system determines that the updated comment content is empty. 3. The system displays the message “Comment content cannot be empty.” 4. Return to step 4. **A.1b Selected Comment Not Found or Not in Post** 1. The User clicks the “Save Changes” button. 2. The system determines that the selected comment does not exist, is deleted, or does not belong to the post. 3. The system displays the message “Comment not found.” 4. Return to step 1.

Exceptions:	**EX-01: Database Error** 1. The system attempts to update the comment record in the ForumComment table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to update comment. Please try again later.” 4. The comment is not updated. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while saving the changes. 2. The system cannot save the updated comment data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the edit form without the changes being saved. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-FORUM-35: Only the owner of a comment can edit it. BR-FORUM-36: Comment content cannot be empty. BR-FORUM-37: A comment must belong to the specified post to be edited.
Other Information:	The updated comment remains associated with the original post and author.
Assumptions:	The selected comment has not been deleted or hidden by the system.

2.2.3.14 UC-04.14 Delete Comment

UC ID and Name	UC-04.14 Delete Comment		
Created By:	Pham Kha Vy	Date Created:	26/09/2026
Primary Actor:	Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user clicks the “Delete” option on one of their comments.		
Description:	Allows the owner of a comment or an Admin to remove it from public view by performing a soft delete operation.		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected comment exists and has `deletedAt = NULL`. **PRE-3** The user is the owner of the selected comment or has the Admin role.
Post-conditions:	**POST-1** The selected comment is soft-deleted by setting the `deletedAt` field. **POST-2** The comment is no longer displayed to regular users.
Normal Flow:	**A. Delete Comment** 1. The User clicks the "Delete" option on a comment. 2. The system verifies that the User is the owner of the selected comment or has the Admin role. 3. The system displays a confirmation dialog. 4. The User confirms the deletion request. 5. The system updates the `deletedAt` field of the selected comment. 6. The system removes the comment from the visible comment list and refreshes the comment section.
Alternative Flows:	**A.1a User Not Authorized** 1. The User clicks the "Delete" option on a comment. 2. The system determines that the User is not the owner nor an Admin. 3. The system displays the message "You are not authorized to delete this comment." 4. Return to step 1. **A.4 Cancel Deletion** 1. The User clicks the "Cancel" button on the confirmation dialog. 2. The system closes the confirmation dialog. 3. The comment remains unchanged. 4. The use case ends. **A.1b Selected Comment Not Found or Not in Post** 1. The User confirms the deletion request. 2. The system determines that the selected comment does not exist, is already deleted, or does not belong to the post. 3. The system displays the message "Comment not found." 4. Return to step 1.

Exceptions:	**EX-01: Database Error** 1. The system attempts to update the `deletedAt` field of the selected comment in the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to delete comment. Please try again later.” 4. The comment is not deleted. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the deletion request. 2. The system cannot complete the deletion of the comment. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The comment remains unchanged. 5. The use case ends.
Priority:	High
Frequency of Use:	Low
Business Rules:	BR-FORUM-38: Only the owner of a comment or an Admin can delete it. BR-FORUM-39: Comment deletion shall be implemented as a soft delete using the `deletedAt` field. BR-FORUM-40: Soft-deleted comments shall not be displayed in normal comment listings.
Other Information:	Deleted comments may remain available for administrative review and auditing purposes.
Assumptions:	The selected comment has not been permanently removed from the database.

2.2.3.15 UC-04.15 Like Comment

UC ID and Name	UC-04.15 Like Comment		
Created By:	Pham Kha Vy	Date Created:	27/09/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the “Like” button on a comment.		
Description:	Allows authenticated users to express appreciation for a comment by adding a like reaction (toggle action).		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected comment exists and has `deletedAt = NULL`.
Post-conditions:	**POST-1** A like record is created in the ForumLike table associated with the user and comment. **POST-2** The comment's reaction count is updated and the "Like" button is highlighted.
Normal Flow:	**A. Like Comment** 1. The User clicks the "Like" button on a comment. 2. The system verifies that the User is authenticated. 3. The system verifies that the comment exists and has not been deleted. 4. The system verifies that the User has not already liked the comment. 5. The system creates a new like record associated with the User and the comment in the ForumLike table. 6. The system updates the displayed reaction count and highlights the "Like" button.
Alternative Flows:	**A.1a User Not Authenticated** 1. The User clicks the "Like" button on a comment. 2. The system determines that the User is not authenticated. 3. The system redirects the User to the Login page. 4. The User logs in successfully. 5. The system returns the User to the selected post. 6. Return to step 1. **A.1b Selected Comment Not Found** 1. The User clicks the "Like" button on a comment. 2. The system determines that the selected comment does not exist, is deleted, or does not belong to the post. 3. The system displays the message "Comment not found." 4. Return to step 1. **A.4 Comment Already Liked** 1. The User clicks the "Like" button on a comment they have already liked. 2. The system determines that the User has already liked the comment. 3. The system performs the unlike action (see UC-04.16). 4. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to create a new like record in the ForumLike table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to process your request. Please try again later.” 4. The like reaction is not saved. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the like request. 2. The system cannot save the reaction. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The reaction count remains unchanged. 5. The use case ends.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-FORUM-41: Only authenticated users can like comments. BR-FORUM-42: A user can have only one active like reaction per comment. BR-FORUM-43: The reaction shall be stored in the ForumLike table.
Other Information:	The like action is a toggle and uses optimistic updates on the frontend.
Assumptions:	The selected comment is accessible to the user.

2.2.3.16 UC-04.16 Unlike Comment

UC ID and Name	UC-04.16 Unlike Comment		
Created By:	Pham Kha Vy	Date Created:	27/09/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks an active “Like” button on a comment that they have previously liked.		
Description:	Allows authenticated users to remove their like reaction from a forum comment.		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected comment exists and has `deletedAt = NULL`. **PRE-3** The user has previously liked the selected comment.
Post-conditions:	**POST-1** The corresponding like record is removed from the ForumLike table. **POST-2** The comment's reaction count is updated and the "Like" button is set to its inactive state.
Normal Flow:	**A. Unlike Comment** 1. The User clicks the active "Like" button on a comment. 2. The system verifies that the User is authenticated. 3. The system verifies the existing like relationship between the User and the comment. 4. The system removes the corresponding like record from the ForumLike table. 5. The system updates the displayed reaction count and changes the "Like" button to its inactive state.
Alternative Flows:	**A.1a Selected Comment Not Found** 1. The User clicks the "Like" button on a comment. 2. The system determines that the selected comment does not exist or is soft-deleted. 3. The system displays the message "Comment not found." 4. Return to step 1. **A.1b User Not Authenticated** 1. The User attempts to remove a reaction from a comment. 2. The system determines that the User is not authenticated. 3. The system redirects the User to the Login page. 4. The User logs in successfully. 5. The system returns the User to the selected post. 6. Return to step 1. **A.3 Like Record Not Found** 1. The User clicks the "Like" button on a comment. 2. The system determines that no like record exists for the User and the selected comment. 3. The system treats the action as a like. 4. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to remove the like record from the ForumLike table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to process your request. Please try again later.” 4. The like reaction is not removed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the unlike request. 2. The system cannot remove the reaction. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The reaction count remains unchanged. 5. The use case ends.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-FORUM-44: Only authenticated users can remove comment reactions. BR-FORUM-45: A like reaction can only be removed if a corresponding like record exists. BR-FORUM-46: Removing a reaction shall delete the corresponding record from the ForumLike table.
Other Information:	The like/unlike action is a single toggle endpoint.
Assumptions:	The selected comment is accessible to the user.

2.2.3.17 UC-04.17 Report Content

UC ID and Name	UC-04.17 Report Content		
Created By:	Pham Kha Vy	Date Created:	27/09/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	Admin, Moderator
Trigger:	When a user selects the “Report” option for a forum post or comment.		
Description:	Allows authenticated users to report forum posts or comments that violate community guidelines so that moderators can review and take appropriate actions.		

Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The selected post or comment exists and has `deletedAt = NULL`. **PRE-3** The user has not already reported the selected content.
Post-conditions:	**POST-1** A new report record is created in the ForumReport table. **POST-2** The report status is automatically set to PENDING. **POST-3** Administrators and moderators can review the report.
Normal Flow:	**A. Report Content** 1. The User selects the "Report" option on a post or comment. 2. The system verifies that the User is authenticated. 3. The system displays a report dialog. 4. The system checks whether the User has already reported this content. 5. The User enters a report reason in the dialog. 6. The User clicks the "Submit Report" button. 7. The system validates the submitted information. 8. The system creates a new record in the ForumReport table with status set to PENDING. 9. The system displays a confirmation message indicating that the report has been submitted successfully.

Alternative Flows:	**A.1a User Not Authenticated** 1. The User selects the “Report” option on a post or comment. 2. The system determines that the User is not authenticated. 3. The system redirects the User to the Login page. 4. The User logs in successfully. 5. The system returns the User to the selected content. 6. Return to step 1. **A.4 User Already Reported (Duplicate Report)** 1. The User selects the “Report” option on a post or comment. 2. The system determines that the User has already reported the same content. 3. The system displays the message “You have already reported this content.” 4. The system shows a read-only report dialog. 5. The use case ends. **A.5a User Cancels Report** 1. The User closes the report dialog without submitting. 2. The system discards all entered information. 3. The system returns the User to the current post or comment page. 4. The use case ends. **A.5b Reason Not Entered** 1. The User clicks the “Submit Report” button. 2. The system determines that no report reason was entered. 3. The system displays the message “Please enter a report reason.” 4. Return to step 5. **A.1b Content Not Found** 1. The User clicks the “Submit Report” button. 2. The system determines that the selected post or comment no longer exists or is soft-deleted. 3. The system displays the message “The selected content is no longer available.” 4. Return to step 1.

Exceptions:	**EX-01: Database Error** 1. The system attempts to save the new report to the ForumReport table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to submit your report. Please try again later.” 4. The report is not created. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while submitting the report. 2. The system cannot save the report data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the report dialog without the report being saved. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-FORUM-47: Only authenticated users can submit reports. BR-FORUM-48: Reports must be associated with either a post or a comment. BR-FORUM-49: New reports shall be created with status PENDING. BR-FORUM-50: A user may submit only one report for the same content item.
Other Information:	Reports are reviewed by administrators who can list and update report status.
Assumptions:	Moderators regularly review pending reports and take appropriate moderation actions.

2.2.9 Manage Milestone

2.2.3.1 UC-24.01 View Milestone List

UC ID and Name	UC-24.01 View Milestone List		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY

Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user accesses the Contract Details page from the contract list.		
Description:	Allows an authenticated contract participant (Client or Freelancer) to view all project milestones associated with the contract, including their sequence, title, deliverable description, due date, escrow funding status, amount, and work progress status.		
Preconditions:	**PRE-1** The user is authenticated in the system. **PRE-2** The user is a participant of the contract (Client or Freelancer) or an Administrator. **PRE-3** The contract exists in the database.		
Post-conditions:	**POST-1** A list of all milestones is displayed ordered by milestone sequence and creation date. **POST-2** Summary financial statistics (contract total, escrow deposit, remaining balance) are rendered. **POST-3** The user can select a milestone to view its details (UC-24.02) or take role-based actions.		
Normal Flow:	**A. View Milestone List** 1. The User accesses the Contract Details page (**Contracts** → **Contract #ID**). 2. The system queries the database to retrieve all milestones associated with the contract ID where `deletedAt = NULL`. 3. For each milestone, the system retrieves title, description, amount, due date, status, paymentStatus, and submission count. 4. The system calculates the total allocated milestone amount and remaining contract balance. 5. The system displays the milestone cards with financial metrics and role-specific action buttons.		
Alternative Flows:	**A.1 No Milestones Found** 1. The User accesses the Contract Details page. 2. The system determines that no active milestones exist for the contract. 3. The system displays an empty state message “No milestones created yet” and displays the “+ Add Milestone” button for the Client. 4. Return to step 1.		

Exceptions:	**EX-01: Contract Not Found** 1. The system attempts to find the contract by ID, but no record exists. 2. The system displays the message “Contract not found.” and redirects to the 404 page. 3. The use case ends. **EX-02: Unauthorized Access** 1. The authenticated user is neither the client, freelancer, nor admin for this contract. 2. The system denies access with HTTP 403 Forbidden. 3. The use case ends. **EX-03: Database Connection Failure** 1. Database query fails due to network interruption or server timeout. 2. The system displays “Unable to load milestones. Please try again later.” 3. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-MS-01: Only milestones where `deletedAt = NULL` shall be retrieved. BR-MS-02: Milestones shall be sorted chronologically by creation index and due date. BR-MS-03: Milestones in `SUBMITTED`, `CHANGES_REQUESTED`, or `DISPUTED` status shall be auto-expanded upon initial view.
Other Information:	Real-time query invalidation keeps the milestone list synchronized upon escrow deposits or submissions.
Assumptions:	The contract was previously agreed upon and initialized in the system.

2.2.3.2 UC-24.02 View Milestone Detail

UC ID and Name	UC-24.02 View Milestone Detail		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks on a milestone card header or chevron button in the milestone list.		

Description:	Expands the milestone card to display full deliverable scope criteria, attached files, latest deliverable submission notes, external preview URLs, and change request revision history.
Preconditions:	**PRE-1** The user is an authenticated contract participant (Client or Freelancer) or Admin. **PRE-2** The milestone list is loaded on screen.
Post-conditions:	**POST-1** The selected milestone card expands, showing full deliverable criteria and latest submission details. **POST-2** Action buttons corresponding to the user's role and milestone status become interactive.
Normal Flow:	**A. View Milestone Detail** 1. The User clicks the milestone header or expand chevron button. 2. The system retrieves the milestone submissions and deliverable attachments. 3. The system renders the deliverable requirements and acceptance criteria. 4. The system presents the latest submission: submission timestamp, freelancer notes, external links, and downloadable files. 5. If changes were requested, the system displays the client's revision message and deadline. 6. The user inspects the information or triggers available workflow actions.
Alternative Flows:	**A.1 Milestone is Disputed** 1. The milestone has paymentStatus `DISPUTED`. 2. The system renders an alert banner with a "View Dispute" button linking to the arbitration room. **A.2 Collapse Milestone Card** 1. The user clicks the chevron button on an expanded milestone. 2. The card collapses, hiding deliverable scope and submissions.
Exceptions:	**EX-01: Deliverables Failed to Load** 1. The network request to fetch submissions fails. 2. The system displays an error banner with a "Retry" button inside the expanded card. 3. The use case ends.
Priority:	High
Frequency of Use:	High

Business Rules:	BR-MS-04: Deliverable review actions (Approve, Request Changes) shall be rendered only for the Client when status is `SUBMITTED`. BR-MS-05: Work execution actions (Start Work, Submit Work) shall be rendered only for the Freelancer.
Other Information:	Submissions list includes version history of previous revision attempts.
Assumptions:	The milestone ID requested is valid and linked to the active contract.

2.2.3.3 UC-24.03 Create Milestone

UC ID and Name	UC-24.03 Create Milestone		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When the Client clicks the "+ Add Milestone" button on the Contract Details page.		
Description:	Allows the client to create a new milestone by providing a title, deliverable scope description, funding amount, and target due date, validating that the amount does not exceed the remaining contract budget.		
Preconditions:	**PRE-1** The user is logged in as the Client of this contract. **PRE-2** The contract status is `ACTIVE` or `PENDING_SIGN`. **PRE-3** Total existing milestone amounts are less than the contract total amount.		
Post-conditions:	**POST-1** A new milestone record is created in the database with status `PENDING` and paymentStatus `PENDING`. **POST-2** The milestone list updates and the unallocated contract balance is recalculated.		

Normal Flow:	**A. Create Milestone** 1. The Client clicks the "+ Add Milestone" button. 2. The system opens the slide-over creation form with remaining contract budget displayed. 3. The Client enters: Title (required), Description/Deliverable criteria (optional), Amount (required), and Due Date (optional). 4. The Client clicks "Create Milestone". 5. The system validates that title is non-empty, amount > 0, amount $\leq$ remaining balance, and due date is in the future. 6. The system persists the new milestone in the database. 7. The system closes the modal, renders a success notification, and refreshes the milestone list.
Alternative Flows:	**A.1 Cancel Creation** 1. The Client clicks "Cancel" or presses Escape. 2. The system closes the modal without saving data. 3. The use case ends.
Exceptions:	**EX-01: Amount Exceeds Contract Budget** 1. The entered amount exceeds the remaining unallocated balance. 2. The system halts submission and displays "Milestone amount exceeds remaining contract budget." 3. The Client adjusts the amount and resubmits. **EX-02: Due Date In The Past** 1. The selected due date is earlier than the current time. 2. The system displays "Due date must be in the future." 3. Return to step 3. **EX-03: Contract Inactive** 1. Contract has been cancelled or completed. 2. The system rejects creation with "Milestones cannot be added to inactive contracts." 3. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-MS-06: The sum of all non-deleted milestone amounts must not exceed the contract's `totalAmount`. BR-MS-07: Newly created milestones must default to `status = PENDING` and `paymentStatus = PENDING`.

Other Information:	Client can immediately proceed to fund the milestone via Stripe or saved payment card.
Assumptions:	The contract has a defined fixed total price.

2.2.3.4 UC-24.04 Update Milestone

UC ID and Name	UC-24.04 Update Milestone		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When the Client selects “Edit” from the 3-dots dropdown menu on a pending milestone card.		
Description:	Allows the client to modify the title, description, budget amount, or target due date of a milestone that has not yet started work.		
Preconditions:	**PRE-1** The user is the Client of the contract. **PRE-2** The milestone status is `PENDING`.		
Post-conditions:	**POST-1** The milestone record is updated with new specifications. **POST-2** The milestone card re-renders with the updated information.		
Normal Flow:	**A. Update Milestone** 1. The Client clicks the 3-dots option menu on a pending milestone and clicks “Edit”. 2. The system opens the slide-over form pre-populated with current milestone values. 3. The Client modifies title, description, amount, or due date. 4. The Client clicks “Save Changes”. 5. The system verifies that the updated amount does not exceed the remaining contract budget (offsetting the old amount). 6. The system updates the milestone details in the database. 7. The slide-over closes and a success message “Milestone updated successfully” is displayed.		
Alternative Flows:	**A.1 Discard Changes** 1. The Client closes the modal without clicking save. 2. Changes are discarded and the milestone remains unaltered.		

Exceptions:	**EX-01: Milestone In Progress or Completed** 1. The milestone status is no longer `PENDING`. 2. The system rejects the update with "Milestone cannot be edited once work has started." 3. The use case ends. **EX-02: Amount Exceeds Contract Total** 1. The new amount exceeds the available budget threshold. 2. The system displays an error and prompts the client to adjust the value. 3. Return to step 3.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-MS-08: Only milestones in `PENDING` status can be modified. BR-MS-09: Recalculated total amounts must not exceed the contract total amount.
Other Information:	N/A
Assumptions:	The milestone has not been locked by a dispute or escrow release.

2.2.3.5 UC-24.05 Delete Milestone

UC ID and Name	UC-24.05 Delete Milestone		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When the Client selects "Delete" from the 3-dots dropdown menu on a pending milestone card.		
Description:	Allows the client to delete an unstarted and unfunded milestone, restoring its allocated funds back to the contract's unallocated balance.		
Preconditions:	**PRE-1** The user is the Client of the contract. **PRE-2** The milestone status is `PENDING` and paymentStatus is `PENDING`.		
Post-conditions:	**POST-1** The milestone is soft-deleted (`deletedAt = new Date()`) and removed from the active list. **POST-2** Contract unallocated balance is restored.		

Normal Flow:	**A. Delete Milestone** 1. The Client opens the 3-dots menu on a pending milestone and selects “Delete”. 2. The system presents an alert confirmation dialog: “Are you sure you want to delete this milestone?” 3. The Client clicks “Delete”. 4. The system removes the milestone from the contract. 5. The backend sets `deletedAt` timestamp on the milestone record. 6. The dialog closes, the card is removed from the view, and a confirmation toast is displayed.
Alternative Flows:	**A.1 Cancel Deletion** 1. The Client selects “Cancel” in the alert dialog. 2. The dialog closes without deleting the milestone.
Exceptions:	**EX-01: Milestone In Progress or Funded** 1. The milestone has status $\neq$ `PENDING` or paymentStatus $\neq$ `PENDING`. 2. The system rejects deletion with “Milestone cannot be deleted.” 3. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-MS-10: Funded, in-progress, or completed milestones cannot be deleted to protect freelancer rights and escrow funds.
Other Information:	Soft deletion ensures audit history remains traceable in the database.
Assumptions:	The milestone has no pending financial transactions.

2.2.3.6 UC-24.06 Request Milestone Changes

UC ID and Name	UC-24.06 Request Milestone Changes		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When the Client reviews submitted work and clicks “Request Changes”.		

Description:	Allows the client to reject the current submission and ask for modifications, supplying detailed feedback and an optional revision deadline.
Preconditions:	**PRE-1** The user is the Client of the contract. **PRE-2** The milestone status is `SUBMITTED`. **PRE-3** The active submission status is `PENDING_REVIEW`.
Post-conditions:	**POST-1** The submission status transitions to `CHANGES_REQUESTED` with feedback and deadline saved. **POST-2** Milestone status transitions to `CHANGES_REQUESTED`. **POST-3** The Freelancer is notified to revise and resubmit work.
Normal Flow:	**A. Request Changes** 1. The Client clicks "Review Deliverables" on a submitted milestone card. 2. The system opens the Review Deliverable slide-over sheet. 3. The Client clicks "Request Changes". 4. The system presents the feedback message input box and date picker for revision deadline. 5. The Client types specific revision requirements and selects a target date. 6. The Client clicks "Submit Change Request". 7. The system updates the submission status to changes requested and records the feedback message. 8. The database updates the submission status to `CHANGES_REQUESTED` and sets milestone status to `CHANGES_REQUESTED`. 9. The modal closes, a success toast appears, and the milestone card displays an amber revision banner.
Alternative Flows:	**A.1 Cancel Feedback Entry** 1. The Client clicks "Cancel" or "Back to Review". 2. The feedback form closes and the submission remains in `PENDING_REVIEW`.

Exceptions:	**EX-01: Empty Feedback Message** 1. The Client attempts to submit without entering a feedback message. 2. The system displays “Please explain what needs to be changed.” 3. Return to step 5. **EX-02: Submission Already Reviewed** 1. The submission has already been approved or changes requested by another session. 2. The system returns HTTP 400 with “Submission is not pending review.” 3. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-MS-11: A change request must include a non-empty explanation message. BR-MS-12: Requesting changes enables the Freelancer to create a new submission iteration.
Other Information:	Feedback message is archived in submission history to preserve review trail.
Assumptions:	The milestone has a funded escrow deposit.

2.2.3.7 UC-24.07 Approve Milestone

UC ID and Name	UC-24.07 Approve Milestone		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When the Client reviews submitted work and clicks “Approve & Release Payment”.		
Description:	Enables the client to accept completed deliverables, marking the milestone as Completed and releasing escrowed funds to the freelancer’s account.		
Preconditions:	**PRE-1** The user is the Client of the contract. **PRE-2** The milestone is in `SUBMITTED` status and paymentStatus is `FUNDED`. **PRE-3** The submission is in `PENDING_REVIEW` status.		

Post-conditions:	**POST-1** Submission status becomes `APPROVED`. **POST-2** Milestone status transitions to `COMPLETED` and paymentStatus becomes `RELEASED`. **POST-3** Escrow funds are transferred to the freelancer.
Normal Flow:	**A. Approve Milestone** 1. The Client opens the Review Deliverables slide-over sheet. 2. The Client inspects the freelancer notes, links, and files. 3. The Client clicks "Approve & Release ($X,XXX)". 4. The system displays a confirmation dialog informing the client that escrow funds will be immediately released. 5. The Client clicks "Confirm Release". 6. The system initiates the approval and escrow release transaction. 7. The database transaction executes: marks submission `APPROVED`, sets milestone `COMPLETED`, sets paymentStatus `RELEASED`, and debits escrow balance. 8. The sheet closes, a success toast confirms release, and the milestone card displays a completed badge with green styling.
Alternative Flows:	**A.1 Final Contract Milestone** 1. The approved milestone is the last milestone of the contract. 2. The system prompts the Client to complete the contract and leave a rating and review for the freelancer.
Exceptions:	**EX-01: Milestone Not Funded** 1. The milestone has paymentStatus $\neq$ `FUNDED`. 2. The system blocks approval with "Milestone funds must be deposited into escrow prior to approval." 3. The use case ends. **EX-02: Transaction Failure** 1. Escrow fund transfer fails due to banking service timeout. 2. The database rolls back the approval transaction and displays "Payment release failed. Please try again." 3. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-MS-13: Approving a milestone irrevocably releases the associated escrow funds to the freelancer's balance. BR-MS-14: Completed milestones cannot be reopened or edited.

Other Information:	Platform transaction fee is deducted according to platform commission policy.
Assumptions:	Escrow deposit was previously confirmed via Stripe payment gateway.

2.2.3.8 UC-24.08 Update Milestone Progress

UC ID and Name	UC-24.08 Update Milestone Progress		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the Freelancer clicks the "Start Work" button on a funded upcoming milestone card.		
Description:	Transitions a milestone from Pending to In Progress, verifying that funds are secured in escrow and enforcing that only one milestone can be in progress at a time.		
Preconditions:	**PRE-1** The user is the Freelancer assigned to the contract. **PRE-2** The milestone status is `PENDING`. **PRE-3** The milestone paymentStatus is `FUNDED`. **PRE-4** No other milestone in the contract currently has status `IN_PROGRESS`.		
Post-conditions:	**POST-1** Milestone status becomes `IN_PROGRESS`. **POST-2** The milestone card renders the "Submit Work" button for the Freelancer.		
Normal Flow:	**A. Start Work on Milestone** 1. The Freelancer views the Contract Details page. 2. The Freelancer locates an upcoming milestone with status "Pending" and paymentStatus "Funded". 3. The Freelancer clicks "Start Work". 4. The system updates the milestone progress percentage and status. 5. The backend verifies freelancer ownership, funded escrow status, and absence of other active milestones. 6. The database updates the milestone status to `IN_PROGRESS`. 7. The milestone card updates to "In Progress" and the "Submit Work" button becomes enabled.		

Alternative Flows:	**A.1 Milestone Unfunded** 1. The milestone has paymentStatus `PENDING`. 2. The "Start Work" button is replaced by an "Awaiting Escrow Deposit" badge. 3. Work cannot begin until the Client deposits funds. **A.2 Another Milestone In Progress** 1. Another milestone in the contract is already in `IN_PROGRESS` status. 2. The "Start Work" button is disabled with tooltip "Another milestone is already in progress".
Exceptions:	**EX-01: Milestone Already In Progress** 1. Request sent for an already active milestone. 2. System returns HTTP 400 "Milestone is already in progress." 3. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-MS-15: A freelancer shall not start work on a milestone until escrow funds are confirmed (`paymentStatus = FUNDED`). BR-MS-16: Only one milestone per contract can be in `IN_PROGRESS` status at any given time.
Other Information:	Sequential milestone enforcement prevents multitasking disputes and scopes deliverable deadlines.
Assumptions:	The contract is active and signed by both parties.

2.2.10 Manage Milestone File

2.2.3.1 UC-25.01 View Milestone Files

UC ID and Name	UC-25.01 View Milestone Files		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user views the attached deliverables of an expanded milestone or opens the submit work dialog.		
Description:	Retrieves and displays all deliverable files uploaded for a milestone, indicating file name, file type icon, upload date, and secure download link.		

Preconditions:	**PRE-1** The user is an authenticated contract participant (Client or Freelancer) or Admin. **PRE-2** The milestone exists in the contract.
Post-conditions:	**POST-1** The user views all deliverable files attached to the milestone. **POST-2** Files can be downloaded directly from Cloudinary CDN.
Normal Flow:	**A. View Milestone Files** 1. The User expands a milestone card or opens the deliverable modal. 2. The system retrieves all uploaded files and attachments for the milestone. 3. The system filters records where `deletedAt = NULL`. 4. The system renders each file item with its respective icon (PDF, ZIP, Image, Code), file name, and download button. 5. The User clicks "Download" to retrieve the file from Cloudinary CDN.
Alternative Flows:	**A.1 No Files Uploaded** 1. No files are linked to the milestone. 2. The system displays "No files attached".
Exceptions:	**EX-01: Unauthorized User** 1. User is not a participant of the contract. 2. The system rejects request with HTTP 403 Forbidden. 3. The use case ends.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-FILE-01: Only files with `deletedAt = NULL` shall be accessible to participants.
Other Information:	Files are stored securely in Cloudinary under milestone-specific directories.
Assumptions:	The file URLs are valid and accessible via CDN.

2.2.3.2 UC-25.02 Upload Milestone File

UC ID and Name	UC-25.02 Upload Milestone File		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A

Trigger:	When the Freelancer selects a file or drops a file into the upload dropzone in the Submit Milestone dialog.
Description:	Uploads a deliverable file to Cloudinary storage and registers its metadata in the milestone files database, preparing it for inclusion in a submission.
Preconditions:	**PRE-1** The user is the Freelancer (or Client) of the contract. **PRE-2** The file size is $\leq$ 25MB.
Post-conditions:	**POST-1** The file is uploaded to Cloudinary and saved in the `MilestoneFile` table. **POST-2** The uploaded file is appended to the draft submission list.
Normal Flow:	**A. Upload File** 1. The Freelancer selects a file from the device file chooser or drops a file into the dropzone. 2. Client-side validation checks that the file size is within the 25MB limit. 3. The UI shows an upload spinner indicator. 4. The system uploads the file to storage and saves file metadata in the database. 5. The backend stores the file in cloud storage under the milestone asset folder. 6. The backend records `uploaderId`, `fileUrl`, `publicId`, and `fileName` in database. 7. The system returns the created file object and renders it in the attached files list with a success toast.
Alternative Flows:	**A.1 File Size Exceeded** 1. The selected file exceeds 25MB. 2. The system alerts "File size exceeds 25MB limit" and halts upload. 3. Return to step 1.
Exceptions:	**EX-01: Cloudinary Upload Failure** 1. Cloud storage upload fails due to network or service error. 2. The server rolls back and returns HTTP 500 "Upload failed. Please try again." 3. The use case ends.
Priority:	High
Frequency of Use:	High

Business Rules:	BR-FILE-02: Maximum upload file size is capped at 25MB per file. BR-FILE-03: Uploaded files must be scanned and verified for secure MIME-types.
Other Information:	Multiple files can be uploaded sequentially for a single milestone deliverable.
Assumptions:	The user device has an active internet connection.

2.2.3.3 UC-25.03 Delete Milestone File

UC ID and Name	UC-25.03 Delete Milestone File		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the user clicks the trash icon next to an uploaded file in the Submit Milestone dialog.		
Description:	Allows the file uploader to remove a file they uploaded, purging the asset from Cloudinary storage and soft-deleting the database record.		
Preconditions:	**PRE-1** The user is the original uploader of the file (`uploaderId === userId`). **PRE-2** The file exists and has not been deleted.		
Post-conditions:	**POST-1** The file asset is deleted from Cloudinary. **POST-2** The database record is marked as deleted (`deletedAt = new Date()`). **POST-3** The file is removed from the UI attachment list.		
Normal Flow:	**A. Delete Milestone File** 1. The User clicks the trash icon next to the file card in the modal. 2. The system deletes the file from storage and removes its record from the database. 3. The backend confirms ownership (`uploaderId === userId`). 4. The backend calls Cloudinary API to remove the asset using its `publicId`. 5. The backend sets `deletedAt` timestamp on the database record. 6. The file is removed from the UI and a toast confirms deletion.		

Alternative Flows:	**A.1 Network Failure** 1. The server delete request fails. 2. The UI unlinks the file locally from the submission form to allow the freelancer to proceed without delay.
Exceptions:	**EX-01: Unauthorized Deletion** 1. The user attempting deletion is not the file's original uploader. 2. The system rejects request with HTTP 403 Forbidden. 3. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-FILE-04: Only the user who uploaded a file has permission to delete it.
Other Information:	N/A
Assumptions:	Cloudinary credentials have asset removal permissions.

2.2.11 Manage Milestone Submission

2.2.3.1 UC-26.01 Submit Milestone

UC ID and Name	UC-26.01 Submit Milestone		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the Freelancer clicks "Submit Work" (or "Resubmit Work") on a milestone card.		
Description:	Allows the freelancer to submit completed deliverables for client evaluation, bundling a message, external preview links, and uploaded files. The milestone transitions to Submitted status.		
Preconditions:	**PRE-1** The user is the Freelancer of the contract. **PRE-2** The milestone status is `IN_PROGRESS` or `CHANGES_REQUESTED`. **PRE-3** The milestone paymentStatus is `FUNDED`.		
Post-conditions:	**POST-1** A new submission record is created with status `PENDING_REVIEW`. **POST-2** Milestone status updates to `SUBMITTED`. **POST-3** The Client is prompted to review deliverables.		

Normal Flow:	**A. Submit Milestone Deliverables** 1. The Freelancer clicks “Submit Work” on the milestone card. 2. The system opens the Submit Deliverables modal. 3. The Freelancer inputs a message explaining the work completed. 4. The Freelancer inputs external preview URLs and clicks “+ Add Link”. 5. The Freelancer uploads deliverable files (UC-25.02). 6. The Freelancer clicks “Submit Deliverables”. 7. The system validates that at least one deliverable element (message, link, or file) is provided. 8. The system creates the submission record and notifies the client for review. 9. The backend transaction creates the submission, links files, and updates milestone status to `SUBMITTED`. 10. The modal closes, a success toast confirms submission, and the card updates to “Submitted - Awaiting Review”.
Alternative Flows:	**A.1 Resubmission After Revision** 1. The milestone has status `CHANGES_REQUESTED`. 2. The modal displays the client’s previous revision request message and target deadline. 3. The Freelancer addresses feedback and submits a new revision bundle.
Exceptions:	**EX-01: Empty Deliverables** 1. The Freelancer clicks submit without providing message, link, or file. 2. The system halts submission with “Please provide at least a message, a link, or a file.” 3. Return to step 3. **EX-02: Milestone Not In Progress** 1. The milestone status is not `IN_PROGRESS` or `CHANGES_REQUESTED`. 2. The system rejects request with “Milestone is not in progress.” 3. The use case ends.
Priority:	High
Frequency of Use:	High

Business Rules:	BR-SUB-01: Only the contract's assigned freelancer can submit milestone work. BR-SUB-02: Submissions require at least one deliverable artifact (notes, links, or files). BR-SUB-03: Submitting work automatically transitions milestone status to `SUBMITTED`.
Other Information:	Submissions are versioned chronologically to support client audits and revision history.
Assumptions:	The milestone work is ready for client review.

2.2.3.2 UC-26.02 Delete Milestone Submission

UC ID and Name	UC-26.02 Delete Milestone Submission		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When the Freelancer discards an ongoing submission draft before confirming submission in the modal.		
Description:	Enables the freelancer to discard an in-progress submission draft, remove attached files, and close the dialog without altering the milestone status.		
Preconditions:	**PRE-1** The freelancer is in the Submit Deliverables dialog. **PRE-2** The submission has not been finalized.		
Post-conditions:	**POST-1** Draft state is discarded and unlinked files are cleaned up. **POST-2** Milestone status remains `IN_PROGRESS` or `CHANGES_REQUESTED`.		
Normal Flow:	**A. Discard Submission Draft** 1. The Freelancer decides to cancel the submission draft. 2. The Freelancer deletes any attached files (UC-25.03) and clears links. 3. The Freelancer clicks "Cancel" or the close icon on the modal. 4. The system resets all local draft inputs. 5. The milestone card remains in its current active state without any status change.		
Alternative Flows:	**A.1 Resume Editing** 1. The Freelancer cancels modal closing to continue refining deliverables. 2. Return to draft editing.		

Exceptions:	**EX-01: File Deletion Failure** 1. Removal of an uploaded draft file fails over network. 2. An inline notification appears while safely closing the dialog. 3. The use case ends.
Priority:	Low
Frequency of Use:	Low
Business Rules:	BR-SUB-04: Official submissions already submitted to the client cannot be unilaterally deleted to maintain an immutable audit trail for dispute arbitration.
Other Information:	N/A
Assumptions:	Drafting takes place locally in client browser state.

2.2.3.3 UC-26.03 View Milestone Submission List

UC ID and Name	UC-26.03 View Milestone Submission List		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	Freelancer, Admin
Trigger:	When a user expands a milestone card with deliverable activity and toggles the “Revision History” view.		
Description:	Displays the complete chronological list of all deliverable submissions made for a milestone, indicating version number, submission timestamp, review status, and feedback remarks.		
Preconditions:	**PRE-1** The user is an authenticated contract participant (Client or Freelancer) or Admin. **PRE-2** The milestone card is expanded.		
Post-conditions:	**POST-1** The full history log of submissions for the milestone is displayed in chronological order.		

Normal Flow:	**A. View Submission List** 1. The User expands a milestone card in the contract view. 2. The system retrieves the list of submissions associated with the milestone. 3. The latest submission is displayed at the top of the deliverables section. 4. If earlier revisions exist, the user clicks the toggle “Revision History (n previous revisions)”. 5. The system renders the accordion containing all past iterations: version index, submission date, status badge, and client feedback snippet.
Alternative Flows:	**A.1 Single Submission** 1. Only one submission attempt exists. 2. The history accordion is omitted and only the primary submission card is shown.
Exceptions:	**EX-01: Submissions Failed to Load** 1. The server request fails. 2. A retry button prompts the user to reload the submission list. 3. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-SUB-05: Submissions shall be listed in descending order by submission date (newest first).
Other Information:	Both Client and Freelancer see the identical submission history.
Assumptions:	Submissions are linked to the current milestone ID.

2.2.3.4 UC-26.04 View Milestone Submission Detail

UC ID and Name	UC-26.04 View Milestone Submission Detail		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	Freelancer, Admin
Trigger:	When the Client clicks “Review Deliverables” on a submitted milestone, or a participant clicks a submission in the history list.		
Description:	Displays the comprehensive details of a specific milestone submission, including submission timestamp, review status, freelancer notes, preview links, attached downloadable files, and interactive evaluation buttons for the client.		

Preconditions:	**PRE-1** The user is an authenticated contract participant or Admin. **PRE-2** The submission exists for the requested milestone.
Post-conditions:	**POST-1** The complete submission details are displayed in a slide-over sheet. **POST-2** The Client can trigger “Request Changes” (UC-24.06) or “Approve Milestone” (UC-24.07).
Normal Flow:	**A. View Submission Detail** 1. The User clicks “Review Deliverables” or clicks a submission entry in the history log. 2. The system retrieves the detailed submission record, including files and review history. 3. The system renders the Review Deliverables slide-over sheet displaying: - Milestone title, index number, and escrow amount. - Submission date and status badge (`PENDING_REVIEW`, `CHANGES_REQUESTED`, or `APPROVED`). - Freelancer’s submission notes and implementation commentary. - List of external preview URLs with open-in-new-tab links. - Attached deliverable files with file type icons and download links. - If revisions were requested, client feedback message and revision due date. 4. If viewed by the Client when status is `PENDING_REVIEW`, the evaluation action buttons “Request Changes” and “Approve & Release Payment” are rendered.
Alternative Flows:	**A.1 Freelancer Read-Only View** 1. The Freelancer opens the submission detail. 2. The same submission content is presented in read-only mode with an “Awaiting Client Review” badge instead of action buttons.
Exceptions:	**EX-01: Submission Not Found** 1. The requested submission ID does not match the milestone. 2. The system returns HTTP 404 Not Found. 3. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-SUB-06: Review actions (Approve / Request Changes) are strictly restricted to the verified contract Client.
Other Information:	N/A

Assumptions:	The submission data is preserved in database with referential integrity.

2.2.12 Manage Conversation

2.2.3.1 UC-05.1 View Conversations List

UC ID and Name	UC-05.1 View Conversations List		
Created By:	Pham Kha Vy	Date Created:	28/09/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user accesses the Conversations section from the navigation menu or notification bell.		
Description:	Allows an authenticated user to view all active conversations they are participating in, sorted by pinned status and most recent message.		
Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The system is connected to the database.		
Post-conditions:	**POST-1** A list of conversations is displayed with the other user's profile, last message preview, unread count, and pin status. **POST-2** The user can select a conversation to view its details (UC-05.3) or start a new conversation (UC-05.2).		
Normal Flow:	**A. View Conversations List** 1. The User accesses the Conversations page. 2. The system retrieves all active conversations where the User is a participant and has not hidden the conversation. 3. For each conversation, the system loads the other participant's profile, the last non-deleted message, and the unread message count. 4. The system sorts conversations: pinned conversations first (by pinned date descending), then by last message time descending (or conversation creation time if no messages). 5. The system displays the conversations list.		

Alternative Flows:	**A.1 No Conversations Found** 1. The User accesses the Conversations page. 2. The system determines that no active conversations exist for the User. 3. The system displays a message indicating no conversations are available. 4. The User remains on the Conversations page. 5. Return to step 1.
Exceptions:	**EX-01: Database Error** 1. The system attempts to retrieve the conversations list from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load conversations. Please try again later.” 4. The conversations list is not displayed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the conversations data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the current page without any conversation information being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-CONV-01: Only conversations where `deletedAt = NULL` shall be displayed. BR-CONV-02: Conversations hidden by the current user (via `participant1HiddenAt` or `participant2HiddenAt`) shall not be displayed. BR-CONV-03: Pinned conversations shall appear before non-pinned conversations. BR-CONV-04: Unread count shall count only messages from the other participant that are not read and not deleted.
Other Information:	The list is updated in real-time via WebSocket events (`conversation:updated`).

Assumptions:	Each user has at least one other user to communicate with in the system.

2.2.3.2 UC-05.2 Start Conversation

UC ID and Name	UC-05.2 Start Conversation		
Created By:	Pham Kha Vy	Date Created:	29/09/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the “Start Conversation” button or navigates to the new conversation page.		
Description:	Allows an authenticated user to start a new conversation with another user. If a conversation already exists between the two users, it is returned and unhidden instead of creating a duplicate.		
Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The target user exists in the system.		
Post-conditions:	**POST-1** A new conversation is created or an existing conversation is unhidden. **POST-2** The user is navigated to the conversation detail page.		
Normal Flow:	**A. Start Conversation** 1. The User clicks the “Start Conversation” button or navigates to the new conversation page. 2. The User enters or selects the participant ID of the user they want to message. 3. The system verifies that the user is not trying to start a conversation with themselves. 4. The system checks whether an existing active conversation between the two users already exists. 5a. If a conversation already exists, the system unhides the conversation for both participants and returns the existing conversation. 5b. If no conversation exists, the system creates a new conversation record in the Conversation table. 6. The system navigates the User to the conversation detail page.		

Alternative Flows:	**A.2a User Not Authenticated** 1. The User attempts to start a conversation. 2. The system determines that the User is not authenticated. 3. The system redirects the User to the Login page. 4. The User logs in successfully. 5. The system returns the User to the new conversation page. 6. Return to step 2. **A.2b User Attempts to Message Self** 1. The User enters their own user ID as the participant. 2. The system determines that the participant ID matches the current user. 3. The system displays the message “You cannot start a conversation with yourself.” 4. Return to step 2.
Exceptions:	**EX-01: Database Error** 1. The system attempts to create or retrieve the conversation from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to start conversation. Please try again later.” 4. The conversation is not created. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot save the conversation data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the current page without the conversation being created. 5. The use case ends.
Priority:	High
Frequency of Use:	Medium

Business Rules:	BR-CONV-05: A user cannot start a conversation with themselves. BR-CONV-06: Each pair of users may have only one conversation (enforced by the unique constraint on `participant1, participant2`). BR-CONV-07: If an existing conversation is found, it shall be unhidden for both participants rather than creating a duplicate. BR-CONV-08: A new conversation record stores the two participant IDs.
Other Information:	The participant ID is provided via a numeric input or a pre-selected user profile.
Assumptions:	The target user exists and is a valid user in the system.

2.2.3.3 UC-05.3 Conversation Detail

UC ID and Name	UC-05.3 Conversation Detail		
Created By:	Pham Kha Vy	Date Created:	30/09/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user selects a conversation from the conversations list (UC-05.1) or after starting a new conversation (UC-05.2).		
Description:	Allows an authenticated user to view the complete message history of a conversation including all sent and received messages, file attachments, and other participant information.		
Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The conversation exists and has not been deleted. **PRE-3** The user is a participant of the conversation.		
Post-conditions:	**POST-1** The full message history is displayed in chronological order. **POST-2** The user can send messages, send files, and interact with messages. **POST-3** All unread messages from the other participant are automatically marked as read.		

Normal Flow:	**A. Conversation Detail** 1. The User opens a conversation from the list. 2. The system validates that the conversation exists, has not been deleted, and the User is a participant. 3. The system retrieves the other participant's profile information. 4. The system retrieves all non-deleted messages associated with the conversation. 5. The system displays the conversation header with the other participant's name and avatar. 6. The system displays the message list in chronological order with file attachments where applicable. 7. The system automatically marks all unread messages from the other participant as read (UC-05.5).
Alternative Flows:	**A.1a Conversation Not Found or Deleted** 1. The User opens a conversation. 2. The system determines that the conversation does not exist or has been deleted. 3. The system displays the message "Conversation not found." 4. The system redirects the User to the conversations list page. 5. Return to step 1. **A.1b User Is Not a Participant** 1. The User opens a conversation. 2. The system determines that the User is not a participant of the conversation. 3. The system displays the message "You do not have access to this conversation." 4. The system redirects the User to the conversations list page. 5. Return to step 1.

Exceptions:	**EX-01: Database Error** 1. The system attempts to retrieve the conversation and messages from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load conversation. Please try again later.” 4. The conversation details are not displayed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the conversation data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the current page without any conversation information being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-CONV-09: Only conversations where `deletedAt = NULL` shall be accessible. BR-CONV-10: Only participants of the conversation may view its messages. BR-CONV-11: All messages shall be sorted by `createdAt` in ascending order. BR-CONV-12: Soft-deleted messages (`deletedAt` not null) shall not be displayed.
Other Information:	New messages are received in real-time via the `message:new` WebSocket event.
Assumptions:	Each conversation is associated with valid participants.

2.2.3.4 UC-05.4 Send Message

UC ID and Name	UC-05.4 Send Message		
Created By:	Pham Kha Vy	Date Created:	01/10/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A

Trigger:	When a user types a message and clicks the “Send” button or presses Enter in the conversation detail page.
Description:	Allows an authenticated user to send a text message in an active conversation. The message is delivered in real-time to the other participant.
Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The conversation exists, has not been deleted, and the User is a participant. **PRE-3** The user is viewing the conversation detail page.
Post-conditions:	**POST-1** A new message record is created in the DirectMessage table. **POST-2** The message appears in the conversation for both participants. **POST-3** Any hidden conversation status is cleared for both participants.
Normal Flow:	**A. Send Message** 1. The User enters a message in the message input field. 2. The User clicks the “Send” button or presses Enter. 3. The system validates the message content. 4. The system verifies that the User is a participant of the conversation. 5. The system creates a new message record in the DirectMessage table with `isRead = false`. 6. The system clears any hidden conversation status for both participants. 7. The system displays the new message in the conversation. 8. The system sends the message to the other participant in real-time via WebSocket.

Alternative Flows:	**A.1a Message Too Long** 1. The User clicks the “Send” button. 2. The system determines that the message exceeds 2000 characters. 3. The system displays the message “Message must not exceed 2000 characters.” 4. Return to step 1. **A.1b Empty Message Without Attachment** 1. The User clicks the “Send” button. 2. The system determines that the message is empty and no file is attached. 3. The system does not create a message. 4. Return to step 1. **A.1c Conversation Not Found or Not a Participant** 1. The User attempts to send a message. 2. The system determines that the conversation does not exist, has been deleted, or the User is not a participant. 3. The system displays the message “Conversation not found.” 4. Return to step 1.
Exceptions:	**EX-01: Database Error** 1. The system attempts to save the new message to the DirectMessage table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to send message. Please try again later.” 4. The message is not created. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while sending the message. 2. The system cannot save the message data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the conversation detail page without the message being sent. 5. The use case ends.
Priority:	High
Frequency of Use:	High

Business Rules:	BR-CONV-13: Only authenticated users can send messages. BR-CONV-14: The user must be a participant of the conversation to send a message. BR-CONV-15: Message content shall not exceed 2000 characters. BR-CONV-16: A message must contain either text content or a file attachment. BR-CONV-17: New messages shall be created with `isRead = false`. BR-CONV-18: Sending a message shall clear the hidden status for both participants.
Other Information:	Messages are sent primarily via the WebSocket gateway (`message:send`) with REST API as a fallback. The message is broadcast to both participants' rooms in real-time.
Assumptions:	The conversation remains active during the message sending process.

2.2.3.5 UC-05.5 Mark Message as Read

UC ID and Name	UC-05.5 Mark Message as Read		
Created By:	Pham Kha Vy	Date Created:	02/10/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user opens a conversation detail page or when a new message arrives from the other participant.		
Description:	Allows an authenticated user to mark all unread messages from the other participant in a conversation as read, either automatically when opening the conversation or manually via the conversation list menu.		
Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The conversation exists, has not been deleted, and the user is a participant. **PRE-3** There are unread messages from the other participant.		
Post-conditions:	**POST-1** All unread messages from the other participant have their `isRead` field updated to `true`. **POST-2** The unread count for the conversation is updated to zero. **POST-3** Both participants receive a real-time update of the conversation status.		

Normal Flow:	**A. Mark Message as Read** 1. The User opens a conversation detail page. 2. The system verifies that the User is a participant of the conversation. 3. The system marks all unread messages from the other participant as read by updating the `isRead` field to `true`. 4. The system returns the count of messages marked as read. 5. The system notifies both participants of the updated conversation status via WebSocket.
Alternative Flows:	**A.1 Manual Mark as Read from List** 1. The User clicks the "Mark as Read" option from the conversation list menu. 2. The system verifies that the User is a participant of the conversation. 3. The system marks all unread messages from the other participant as read. 4. The system returns the count of messages marked as read. 5. The system notifies both participants of the updated conversation status. **A.2 Conversation Not Found or Not a Participant** 1. The User attempts to mark messages as read. 2. The system determines that the conversation does not exist, has been deleted, or the User is not a participant. 3. The system displays the message "Conversation not found." 4. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to update the `isRead` field in the DirectMessage table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to mark messages as read. Please try again later.” 4. The messages are not updated. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot update the read status. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The unread count remains unchanged. 5. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-CONV-19: Only messages from the other participant (where `senderId != userId`) shall be marked as read. BR-CONV-20: Only unread messages (`isRead = false`) that are not deleted shall be updated. BR-CONV-21: The system shall return the count of messages that were marked as read.
Other Information:	Mark as read is performed automatically when the conversation is opened and when a new incoming message arrives. It can also be triggered manually from the conversation list context menu.
Assumptions:	The user remains authenticated during the mark-as-read process.

2.2.3.6 UC-05.6 Delete/Recall Message

UC ID and Name	UC-05.6 Delete/Recall Message		
Created By:	Pham Kha Vy	Date Created:	03/10/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the “Delete” option on a message they have sent.		

Description:	Allows the sender of a message to recall or delete it from the conversation by performing a soft delete operation. The message is removed from view for both participants.
Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The message exists and has not been deleted. **PRE-3** The user is the sender of the message.
Post-conditions:	**POST-1** The message is soft-deleted by setting the `deletedAt` field. **POST-2** The message is no longer displayed to either participant. **POST-3** Both participants receive a real-time notification of the deletion.
Normal Flow:	**A. Delete/Recall Message** 1. The User clicks the "Delete" option on a message they sent. 2. The system verifies that the User is the sender of the message. 3. The system performs a soft delete by updating the `deletedAt` field of the message. 4. The system removes the message from the conversation view for both participants. 5. The system sends a `message:deleted` event to both participants via WebSocket. 6. The system updates the conversation list for both participants via a `conversation:updated` event.
Alternative Flows:	**A.2 User Is Not the Sender** 1. The User attempts to delete a message they did not send. 2. The system determines that the User is not the sender of the message. 3. The system does not perform the deletion. 4. The message remains unchanged. 5. The use case ends. **A.1 Message Not Found or Already Deleted** 1. The User confirms the deletion request. 2. The system determines that the message does not exist or has already been deleted. 3. The system displays the message "Message not found." 4. Return to step 1.

Exceptions:	**EX-01: Database Error** 1. The system attempts to update the `deletedAt` field of the message in the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to delete message. Please try again later.” 4. The message is not deleted. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the deletion request. 2. The system cannot complete the deletion of the message. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The message remains unchanged. 5. The use case ends.
Priority:	High
Frequency of Use:	Low
Business Rules:	BR-CONV-22: Only the sender of a message can delete it. BR-CONV-23: Message deletion shall be implemented as a soft delete using the `deletedAt` field. BR-CONV-24: Soft-deleted messages shall not be displayed in the conversation view or the last message preview. BR-CONV-25: Soft-deleted messages shall not be counted in unread counts.
Other Information:	Deleted messages remain stored in the database. The delete action is performed via the WebSocket gateway (`message:delete`) with REST API as a fallback.
Assumptions:	The message has not been permanently removed from the database.

2.2.3.7 UC-05.7 Pin/Unpin Conversation

UC ID and Name	UC-05.7 Pin/Unpin Conversation		
Created By:	Pham Kha Vy	Date Created:	04/10/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A

Trigger:	When a user selects the “Pin” or “Unpin” option from the conversation list context menu.
Description:	Allows an authenticated user to pin or unpin a conversation to keep it at the top of their conversation list. Pinning is independent per user.
Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The conversation exists, has not been deleted, and the user is a participant.
Post-conditions:	**POST-1** The conversation’s `participant1PinnedAt` or `participant2PinnedAt` field is updated for the current user’s side. **POST-2** The conversation list is re-sorted with pinned conversations appearing first.
Normal Flow:	**A. Pin Conversation** 1. The User clicks the “Pin” option from the conversation list context menu. 2. The system verifies that the User is a participant of the conversation. 3. The system sets the `participant1PinnedAt` or `participant2PinnedAt` field (depending on which side the user is) to the current timestamp. 4. The system returns the updated pin status. 5. The system re-sorts the conversations list to place the pinned conversation first.

Alternative Flows:	**A.1 Unpin Conversation** 1. The User clicks the “Unpin” option from the conversation list context menu. 2. The system verifies that the User is a participant of the conversation. 3. The system sets the `participant1PinnedAt` or `participant2PinnedAt` field to `null`. 4. The system returns the updated pin status. 5. The system re-sorts the conversations list. **A.2 Conversation Not Found or Not a Participant** 1. The User attempts to pin or unpin a conversation. 2. The system determines that the conversation does not exist, has been deleted, or the User is not a participant. 3. The system displays the message “Conversation not found.” 4. The use case ends.
Exceptions:	**EX-01: Database Error** 1. The system attempts to update the pin status in the Conversation table. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to update pin status. Please try again later.” 4. The pin status is not updated. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot save the pin status. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The pin status remains unchanged. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Medium

Business Rules:	BR-CONV-26: Only participants of a conversation may pin or unpin it. BR-CONV-27: Pinning is per-user; each participant has their own independent pin flag. BR-CONV-28: Pinned conversations shall appear before non-pinned conversations in the list. BR-CONV-29: Pinning or unpinning is performed via REST only (no WebSocket event).
Other Information:	The frontend re-sorts the conversation list client-side on successful pin/unpin.
Assumptions:	The user remains authenticated during the pin/unpin process.

2.2.3.8 UC-05.8 Send File

UC ID and Name	UC-05.8 Send File		
Created By:	Pham Kha Vy	Date Created:	05/10/2026
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a user clicks the file attachment button and selects a file in the conversation detail page (UC-05.3).		
Description:	Allows an authenticated user to send a file attachment in a conversation. The file is uploaded to cloud storage and a message with the attachment metadata is created.		
Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The conversation exists, has not been deleted, and the user is a participant. **PRE-3** The file to be sent meets the size and type requirements.		
Post-conditions:	**POST-1** The file is uploaded to cloud storage. **POST-2** A new message record is created in the DirectMessage table with the file attachment metadata (`fileUrl`, `fileName`, `fileSize`, `fileType`). **POST-3** The message with the file attachment is displayed in the conversation for both participants.		

Normal Flow:	**A. Send File** 1. The User clicks the file attachment button. 2. The User selects a file from their device. 3. The system validates the file size (maximum 25 MB) and file type (allowed MIME types). 4. The system uploads the file to cloud storage. 5. The system creates a new message record in the DirectMessage table with the file attachment metadata. 6. The system clears any hidden conversation status for both participants. 7. The system displays the file message in the conversation. 8. The system sends the message to the other participant in real-time via WebSocket.
Alternative Flows:	**A.2a File Too Large** 1. The User selects a file. 2. The system determines that the file exceeds the maximum size of 25 MB. 3. The system displays the message “File size must not exceed 25 MB.” 4. Return to step 2. **A.2b File Type Not Allowed** 1. The User selects a file. 2. The system determines that the file type is not in the allowed MIME types list. 3. The system displays the message “This file type is not allowed.” 4. Return to step 2. **A.1 Conversation Not Found or Not a Participant** 1. The User attempts to send a file. 2. The system determines that the conversation does not exist, has been deleted, or the User is not a participant. 3. The system displays the message “Conversation not found.” 4. Return to step 1.

Exceptions:	**EX-01: File Upload Failure** 1. The system attempts to upload the file to cloud storage. 2. The upload operation fails due to a storage service error. 3. The system displays the message “Unable to upload file. Please try again later.” 4. The file is not uploaded and the message is not created. 5. The use case ends. **EX-02: Database Error** 1. The system attempts to save the message record with the file attachment to the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to send file. Please try again later.” 4. The message is not created. 5. The use case ends. **EX-03: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while uploading the file. 2. The system cannot complete the file upload or message creation. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The User remains on the conversation detail page without the file being sent. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-CONV-30: Only authenticated users can send files. BR-CONV-31: The user must be a participant of the conversation to send a file. BR-CONV-32: The maximum file size is 25 MB. BR-CONV-33: Only allowed file types may be uploaded (images, PDF, Office documents, text files, code files, and archives). BR-CONV-34: The file shall be uploaded to cloud storage and the message shall store the file URL, name, size, and type. BR-CONV-35: Sending a file shall clear the hidden status for both participants.

Other Information:	Files are stored in cloud storage (Cloudinary). Image files are rendered as previews in the chat; other file types are displayed as downloadable file cards.
Assumptions:	The cloud storage service is available and accessible.

2.2.13 Manage Saved Search

2.2.3.1 UC View Saved Searches

UC ID and Name	UC View Saved Searches		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer accesses the “Saved Searches” page from the navigation menu or profile.		
Description:	Allows a freelancer to retrieve all saved job search filters with their labels and filter criteria, so they can review the criteria they previously saved.		
Preconditions:	**PRE-1** The freelancer is authenticated. **PRE-2** The system is connected to the database.		
Post-conditions:	**POST-1** A list of saved searches is displayed, each showing its label and filter criteria. **POST-2** No data is changed by this use case. **POST-3** The freelancer can select a saved search to view its details (UC View Saved Search Detail).		
Normal Flow:	**A. View Saved Searches** 1. The Freelancer opens the Saved Searches page. 2. The system retrieves the saved searches of the Freelancer. 3. The system displays the list, each item showing its label and filter criteria (keywords, category, skills, budget). The use case ends.		

Alternative Flows:	**A.1 No Saved Searches Found** *(branch at step 2)* 1. The system determines that the freelancer has no saved searches. 2. The system displays a message indicating no saved searches are available. 3. The use case ends. **A.2 View Saved Search Detail** *(branch at step 3)* 1. The Freelancer selects a saved search from the list. 2. Continue with UC View Saved Search Detail.
Exceptions:	**EX-01: System Error While Loading the List** *(branch at step 2)* 1. The system cannot retrieve the saved searches due to a processing or connection error. 2. The system displays the message “Couldn’t load saved searches. Please try again later.” 3. The freelancer remains on the current page. 4. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-SAVE-01: A freelancer can only view their own saved searches. BR-SAVE-02: Saved searches shall be sorted by the time they were saved, newest first. BR-SAVE-03: Each saved search shall display its label and its filter criteria (keywords, category, skills, budget).
Other Information:	The saved search list is returned without pagination.
Assumptions:	A freelancer may have zero or more saved searches over time.

2.2.3.2 UC View Saved Search Detail

UC ID and Name	UC View Saved Search Detail		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer opens the detail of a saved search from the saved searches list or navigates to it directly.		

Description:	Allows a freelancer to view the full details of a saved search, including its label and all filter criteria such as keywords, category, skills, and budget.
Preconditions:	**PRE-1** The freelancer is authenticated.
Post-conditions:	**POST-1** The saved search details are displayed: label and filter criteria. **POST-2** No data is changed by this use case. **POST-3** The freelancer can edit (UC Edit Saved Search), delete (UC Delete Saved Search), or apply the saved search to a job search.
Normal Flow:	**A. View Saved Search Detail** 1. The Freelancer opens a saved search detail. 2. The system retrieves the saved search. 3. The system displays the saved search with its label and filter criteria (keywords, category, skills, budget). The use case ends.
Alternative Flows:	**A.1 Apply Saved Search** *(branch at step 3)* 1. The Freelancer chooses to run the saved search. 2. Continue with UC View List Job. **A.2 Edit Saved Search** *(branch at step 3)* 1. The Freelancer chooses the edit action. 2. Continue with UC Edit Saved Search. **A.3 Delete Saved Search** *(branch at step 3)* 1. The Freelancer chooses the delete action. 2. Continue with UC Delete Saved Search.
Exceptions:	**EX-01: Saved Search Not Found** *(branch at step 2)* 1. The system cannot find the saved search for the freelancer. 2. The system displays the message “Saved search not found.” 3. The use case ends. **EX-02: System Error While Loading the Detail** *(branch at step 2)* 1. The system cannot load the saved search due to a processing or connection error. 2. The system displays the message “Couldn’t load the saved search. Please try again later.” 3. The freelancer remains on the current page. 4. The use case ends.
Priority:	Medium
Frequency of Use:	Medium

Business Rules:	BR-SAVE-04: The detail shall be accessible only by the owner of the saved search. BR-SAVE-05: A saved search is considered not found if it does not exist or belongs to another user.
Other Information:	Filter criteria include keywords, category, skills, and budget.
Assumptions:	The saved search detail is only reachable by the owning freelancer.

2.2.3.3 UC Create Saved Search

UC ID and Name	UC Create Saved Search		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer chooses to save the current search from the job search page or the saved searches feature.		
Description:	Allows a freelancer to save the current search parameters as a named filter for quick reuse later.		
Preconditions:	**PRE-1** The freelancer is authenticated. **PRE-2** The freelancer is on the job search page with a search form or has access to the saved searches feature. **PRE-3** The current search parameters are valid.		
Post-conditions:	**POST-1** A new saved search is created and stored with its label and filter criteria. **POST-2** The saved search appears in the freelancer's saved searches list with its label. **POST-3** The current search is considered saved.		
Normal Flow:	**A. Create Saved Search** 1. The Freelancer chooses to save the current search. 2. The system prepares a save using the current filter criteria and a default name. 3. The Freelancer enters a name for the saved search. 4. The Freelancer confirms the save. 5. The system validates the name. 6. The system saves the search with the current filter criteria. 7. The system confirms the save to the Freelancer. The use case ends.		

Alternative Flows:	**A.1 Cancel Save** *(branch at step 4)* 1. The Freelancer cancels the save. 2. The system takes no action. 3. The use case ends. **A.2 Invalid Name** *(branch at step 5)* 1. The system determines that the name is empty or exceeds 100 characters. 2. The system displays an appropriate error message. 3. Return to step 3 of the main flow.
Exceptions:	**EX-01: System Error While Saving** *(branch at step 6)* 1. The system cannot save the search due to a processing or connection error. 2. The system displays the message “Couldn’t save the search. Please try again later.” 3. The freelancer remains on the current page without the saved search being created. 4. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-SAVE-06: The saved search name is required and must not exceed 100 characters. BR-SAVE-07: A freelancer may create multiple saved searches; there is no uniqueness constraint on the name. BR-SAVE-08: The saved search shall capture the current filter criteria including keywords, category, skills, and budget.
Other Information:	The default name is typically derived from the search criteria (e.g., a keyword or category summary).
Assumptions:	The current search parameters are valid at the time of saving.

2.2.3.4 UC Edit Saved Search

UC ID and Name	UC Edit Saved Search		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer chooses to rename or edit a saved search from the saved searches list or detail page.		

Description:	Allows the owner of a saved search to modify its name or filter criteria. Only the owner can edit a saved search.
Preconditions:	**PRE-1** The freelancer is authenticated.
Post-conditions:	**POST-1** The saved search is updated with the new name and/or filter criteria. **POST-2** The updated label is reflected in the saved searches list.
Normal Flow:	**A. Edit Saved Search** 1. The Freelancer selects the edit option on a saved search. 2. The system loads the current name and filter criteria of the saved search. 3. The Freelancer updates the name and/or the filter criteria. 4. The Freelancer confirms the changes. 5. The system validates the new name. 6. The system updates the saved search. 7. The system confirms the update to the Freelancer. The use case ends.
Alternative Flows:	**A.1 Cancel Edit** *(branch at step 4)* 1. The Freelancer cancels the edit. 2. The system takes no action. 3. The use case ends. **A.2 Invalid Name** *(branch at step 5)* 1. The system determines that the name is empty or exceeds 100 characters. 2. The system displays an appropriate error message. 3. Return to step 3 of the main flow.

Exceptions:	**EX-01: Saved Search Not Found** *(branch at step 6)* 1. The system cannot find the saved search, or the saved search belongs to another user. 2. The system displays the message “Saved search not found.” 3. The system does not modify the saved search. 4. The use case ends. **EX-02: System Error While Updating** *(branch at step 6)* 1. The system cannot update the saved search due to a processing or connection error. 2. The system displays the message “Couldn’t update the saved search. Please try again later.” 3. The freelancer remains on the current page without the changes being saved. 4. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-SAVE-09: Only the owner of a saved search can edit it. BR-SAVE-10: The update shall only affect the saved search belonging to the current user. BR-SAVE-11: The name is required and must not exceed 100 characters.
Other Information:	The API supports updating the name; filter criteria may be edited via the search page and re-saved.
Assumptions:	The freelancer can only edit saved searches they own.

2.2.3.5 UC Delete Saved Search

UC ID and Name	UC Delete Saved Search		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer chooses to delete a saved search from the saved searches list or detail page.		
Description:	Allows the owner of a saved search to permanently delete it from the system.		
Preconditions:	**PRE-1** The freelancer is authenticated.		

Post-conditions:	**POST-1** The saved search is permanently removed. **POST-2** The saved search no longer appears in the freelancer's saved searches list.
Normal Flow:	**A. Delete Saved Search** 1. The Freelancer selects the delete option on a saved search. 2. The system requests confirmation of the deletion. 3. The Freelancer confirms the deletion. 4. The system deletes the saved search. 5. The system removes the saved search from the list. The use case ends.
Alternative Flows:	**A.1 Cancel Delete** *(branch at step 3)* 1. The Freelancer cancels the deletion. 2. The system takes no action. 3. The use case ends.
Exceptions:	**EX-01: Saved Search Not Found** *(branch at step 4)* 1. The system cannot find the saved search, or the saved search belongs to another user. 2. The system displays the message "Saved search not found." 3. The system does not delete the saved search. 4. The use case ends. **EX-02: System Error While Deleting** *(branch at step 4)* 1. The system cannot delete the saved search due to a processing or connection error. 2. The system displays the message "Couldn't delete the saved search. Please try again later." 3. The freelancer remains on the current page without the saved search being deleted. 4. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-SAVE-12: Only the owner of a saved search can delete it. BR-SAVE-13: Deletion is permanent; the saved search cannot be restored. BR-SAVE-14: Deletion shall succeed only if the saved search belongs to the current user.
Other Information:	Deleting a saved search does not affect any existing jobs or search results.
Assumptions:	The freelancer can only delete saved searches they own.

2.2.3.6 UC View Job Alerts

UC ID and Name	UC View Job Alerts		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer accesses the “Job Alerts” page from the navigation menu or profile.		
Description:	Allows a freelancer to view all of their job alerts, each showing its criteria, frequency (daily, weekly, or instant), and notification channel.		
Preconditions:	**PRE-1** The freelancer is authenticated. **PRE-2** The system is connected to the database.		
Post-conditions:	**POST-1** A list of job alerts is displayed, each showing its criteria, frequency, and notification channel. **POST-2** No data is changed by this use case. **POST-3** The freelancer can select a job alert to view its details (UC View Job Alert Detail), create a new one (UC Create Job Alert), or manage existing alerts.		
Normal Flow:	**A. View Job Alerts** 1. The Freelancer opens the Job Alerts page. 2. The system retrieves the job alerts of the Freelancer. 3. The system displays each job alert with its criteria (keywords/skills), frequency (daily, weekly, or instant), and notification channel (e.g., email, in-app). The use case ends.		
Alternative Flows:	**A.1 No Job Alerts Found** *(branch at step 2)* 1. The system determines that the freelancer has no job alerts. 2. The system displays a message indicating no job alerts are available. 3. The use case ends.		
Exceptions:	**EX-01: System Error While Loading the List** *(branch at step 2)* 1. The system cannot retrieve the job alerts due to a processing or connection error. 2. The system displays the message “Couldn’t load job alerts. Please try again later.” 3. The freelancer remains on the current page. 4. The use case ends.		
Priority:	High		

Frequency of Use:	Medium
Business Rules:	BR-ALERT-01: A freelancer can only view their own job alerts. BR-ALERT-02: Each job alert shall display its criteria (keywords/skills), frequency, and notification channel.
Other Information:	The job alerts list is typically sorted by creation date in descending order.
Assumptions:	A freelancer may have zero or more job alerts over time.

2.2.3.7 UC View Job Alert Detail

UC ID and Name	UC View Job Alert Detail		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer opens the detail of a job alert from the job alerts list or navigates to it directly.		
Description:	Allows a freelancer to view the configuration of a job alert, including its filters, frequency, active status, and the time it was last triggered.		
Preconditions:	**PRE-1** The freelancer is authenticated.		
Post-conditions:	**POST-1** The job alert configuration is displayed: filters, frequency, active status, and last triggered time. **POST-2** No data is changed by this use case. **POST-3** The freelancer can edit (UC Edit Job Alert), delete (UC Delete Job Alert), or toggle the alert.		
Normal Flow:	**A. View Job Alert Detail** 1. The Freelancer opens a job alert detail. 2. The system retrieves the job alert. 3. The system displays the filters (keywords/skills), frequency, active status, and the last triggered time. The use case ends.		
Alternative Flows:	**A.1 Edit Job Alert** *(branch at step 3)* 1. The Freelancer chooses the edit action. 2. Continue with UC Edit Job Alert. **A.2 Toggle Active Status** *(branch at step 3)* 1. The Freelancer toggles the alert on or off. 2. The system updates the active status. 3. Return to step 3 of the main flow.		

Exceptions:	**EX-01: Job Alert Not Found** *(branch at step 2)* 1. The system cannot find the job alert for the freelancer. 2. The system displays a “Not Found” page. 3. The use case ends. **EX-02: System Error While Loading the Detail** *(branch at step 2)* 1. The system cannot load the job alert due to a processing or connection error. 2. The system displays the message “Couldn’t load the job alert. Please try again later.” 3. The freelancer remains on the current page. 4. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-ALERT-03: The detail shall be accessible only by the owner of the job alert. BR-ALERT-04: A job alert is considered not found if it does not exist or belongs to another user.
Other Information:	The last triggered time is updated whenever the system sends a notification for the alert.
Assumptions:	The job alert detail page is only reachable by the owning freelancer.

2.2.3.8 UC Create Job Alert

UC ID and Name	UC Create Job Alert		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	System
Trigger:	When a freelancer chooses to create a job alert from the job alerts page or the job search page.		
Description:	Allows a freelancer to create a job alert with criteria, frequency, and delivery channel. The system automatically sends notifications for matching job postings from that point on.		
Preconditions:	**PRE-1** The freelancer is authenticated. **PRE-2** The freelancer provides at least one search criterion (e.g., keywords or skills).		

Post-conditions:	**POST-1** A new job alert is created and stored with its criteria, frequency, and notification channels. **POST-2** The alert is active by default. **POST-3** The system starts sending notifications for future jobs matching the alert criteria.
Normal Flow:	**A. Create Job Alert** 1. The Freelancer chooses to create a job alert. 2. The system prepares an empty alert form. 3. The Freelancer enters the alert criteria: keywords (optional), required skills (optional), frequency (daily, weekly, or instant), and notification channel (email and/or in-app). 4. The Freelancer confirms the creation. 5. The system validates the criteria. 6. The system saves the job alert (active by default). 7. The system confirms the creation to the Freelancer. The use case ends.
Alternative Flows:	**A.1 Cancel Create** *(branch at step 4)* 1. The Freelancer cancels the creation. 2. The system takes no action. 3. The use case ends. **A.2 Missing Criteria** *(branch at step 5)* 1. The system determines that no search criterion is provided (empty keywords and skills). 2. The system displays an appropriate error message. 3. Return to step 3 of the main flow.
Exceptions:	**EX-01: System Error While Creating** *(branch at step 6)* 1. The system cannot create the job alert due to a processing or connection error. 2. The system displays the message “Couldn’t create the job alert. Please try again later.” 3. The freelancer remains on the current page without the job alert being created. 4. The use case ends.
Priority:	High
Frequency of Use:	Medium

Business Rules:	BR-ALERT-05: A job alert shall have at least one search criterion (keywords or skills). BR-ALERT-06: Supported frequencies are daily, weekly, and instant. BR-ALERT-07: Supported notification channels are email and in-app. BR-ALERT-08: A new job alert shall be active by default.
Other Information:	Once created, the system evaluates new job postings against the alert criteria and notifies the freelancer according to the frequency and channel. This notification behavior is handled outside the scope of this use case.
Assumptions:	The freelancer wants to receive automatic notifications for matching jobs.

2.2.3.9 UC Edit Job Alert

UC ID and Name	UC Edit Job Alert		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	System
Trigger:	When a freelancer chooses to edit a job alert from the job alerts list or detail page.		
Description:	Allows the owner of a job alert to update its criteria, frequency, notification channels, and active status (on/off). Only the owner can edit a job alert.		
Preconditions:	**PRE-1** The freelancer is authenticated.		
Post-conditions:	**POST-1** The job alert configuration is updated. **POST-2** Future notifications follow the updated configuration.		
Normal Flow:	**A. Edit Job Alert** 1. The Freelancer selects the edit option on a job alert. 2. The system loads the current configuration of the job alert. 3. The Freelancer updates the criteria, frequency, notification channels, and/or active status. 4. The Freelancer confirms the changes. 5. The system validates the criteria. 6. The system updates the job alert. 7. The system confirms the update to the Freelancer. The use case ends.		

Alternative Flows:	**A.1 Cancel Edit** *(branch at step 4)* 1. The Freelancer cancels the edit. 2. The system takes no action. 3. The use case ends. **A.2 Missing Criteria** *(branch at step 5)* 1. The system determines that no search criterion is provided (empty keywords and skills). 2. The system displays an appropriate error message. 3. Return to step 3 of the main flow.
Exceptions:	**EX-01: Job Alert Not Found** *(branch at step 6)* 1. The system cannot find the job alert, or the job alert belongs to another user. 2. The system displays a “Not Found” page. 3. The system does not modify the job alert. 4. The use case ends. **EX-02: System Error While Updating** *(branch at step 6)* 1. The system cannot update the job alert due to a processing or connection error. 2. The system displays the message “Couldn’t update the job alert. Please try again later.” 3. The freelancer remains on the current page without the changes being saved. 4. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-ALERT-09: Only the owner of a job alert can edit it. BR-ALERT-10: The update shall only affect the job alert belonging to the current user. BR-ALERT-11: Turning an alert off stops future notifications until it is turned on again.
Other Information:	Editing does not affect already-sent notifications.
Assumptions:	The freelancer can only edit job alerts they own.

2.2.3.10 UC Delete Job Alert

UC ID and Name	UC Delete Job Alert		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY

Primary Actor:	Freelancer	Secondary Actors:	System
Trigger:	When a freelancer chooses to delete a job alert from the job alerts list or detail page.		
Description:	Allows the owner of a job alert to permanently delete it. Deleting an alert stops all future notifications for that alert.		
Preconditions:	**PRE-1** The freelancer is authenticated.		
Post-conditions:	**POST-1** The job alert is permanently removed. **POST-2** No further notifications are sent for the deleted alert.		
Normal Flow:	**A. Delete Job Alert** 1. The Freelancer selects the delete option on a job alert. 2. The system requests confirmation of the deletion. 3. The Freelancer confirms the deletion. 4. The system deletes the job alert. 5. The system removes the job alert from the list. The use case ends.		
Alternative Flows:	**A.1 Cancel Delete** *(branch at step 3)* 1. The Freelancer cancels the deletion. 2. The system takes no action. 3. The use case ends.		
Exceptions:	**EX-01: Job Alert Not Found** *(branch at step 4)* 1. The system cannot find the job alert, or the job alert belongs to another user. 2. The system displays a “Not Found” page. 3. The system does not delete the job alert. 4. The use case ends. **EX-02: System Error While Deleting** *(branch at step 4)* 1. The system cannot delete the job alert due to a processing or connection error. 2. The system displays the message “Couldn’t delete the job alert. Please try again later.” 3. The freelancer remains on the current page without the job alert being deleted. 4. The use case ends.		
Priority:	Medium		
Frequency of Use:	Low		

Business Rules:	BR-ALERT-12: Only the owner of a job alert can delete it. BR-ALERT-13: Deletion is permanent; the job alert cannot be restored. BR-ALERT-14: Deletion shall succeed only if the job alert belongs to the current user.
Other Information:	Deleting a job alert immediately stops all future notifications. Already-sent notifications are not retracted.
Assumptions:	The freelancer can only delete job alerts they own.

2.2.14 Manage Permission

2.2.3.1 UC-15.01 View Permission List

UC ID and Name	UC-15.01 View Permission List		
Created By:	ThuanNT	Date Created:	28/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin navigates to the Permission Management section from the admin dashboard.		
Description:	Retrieve all permissions (id, name, path, method, module) for admin to assign to roles. Supports pagination, keyword search, filtering by HTTP method and module, and sorting.		
Preconditions:	**PRE-1** The user is authenticated as an Admin with a valid JWT token. **PRE-2** The caller's role is permitted to access `GET /api/permissions`. **PRE-3** The system is connected to the database.		
Post-conditions:	**POST-1** A paginated list of active permissions is displayed with `id`, `name`, `path`, `method`, `module`, and `createdAt`. **POST-2** The distinct module list is displayed in the filter dropdown. **POST-3** Total permission count and pagination controls are displayed.		

Normal Flow:	**A. View Permission List** 1. Admin clicks “Permissions” in the admin navigation menu. 2. The frontend validates query parameters (`page`, `limit`, `search`, `method`, `module`, `sortBy`, `order`). 3. The frontend sends `GET /api/permissions` with query parameters. 4. The system validates the JWT token and verifies Admin access rights. 5. The PermissionsController calls `PermissionsService.getPermissions()`. 6. The PermissionsService invokes `PermissionsRepository.findAll(filter)`. 7. The repository executes a Prisma transaction querying permissions where `deletedAt = null`, counting total records, and fetching distinct module names. 8. The database returns the matching permission records, total count, and module list. 9. The system returns HTTP 200 with permissions list, pagination metadata, and modules. 10. The frontend renders the permission table, method badges, module badges, and filter options.
Alternative Flows:	**A.2 Keyword Search** 1. Admin enters a keyword in the search box. 2. The frontend re-queries permissions matching `name`, `path`, or `module` (case-insensitive). 3. The system refreshes the displayed list. 4. Return to step A.10. **A.3 Filter by Method or Module** 1. Admin selects an HTTP method or a module from the filter dropdown. 2. The system executes the query with the selected filters. 3. The system refreshes the displayed list. 4. Return to step A.10. **A.4 No Permissions Found** 1. No active permissions match the current search or filter criteria. 2. The system returns an empty array with `total = 0`. 3. The frontend displays “No permissions match your filters.” 4. Return to step A.10.
Exceptions:	**EX-01: Unauthorized / Forbidden** 1. The JWT token is missing, expired, or user is banned/deleted. 2. The system returns HTTP 401 Unauthorized or 403 Forbidden. 3. The frontend displays an authentication error message and redirects if needed. 4. The use case ends.

	EX-02: Database Query Failure 1. An internal database error occurs while fetching permissions. 2. The system throws `FailedToLoadPermissionsException` (HTTP 500). 3. The frontend displays "Couldn't load permissions. Try again." 4. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-PERM-01: Only permissions with `deletedAt = null` shall be returned. BR-PERM-02: Search shall match `name`, `path`, or `module` case-insensitively. BR-PERM-03: Method filter shall match exact HTTP verbs (GET, POST, PUT, PATCH, DELETE). BR-PERM-04: Module filter shall match exact module name case-insensitively. BR-PERM-05: Pagination supports `page >= 1` and `limit` between 1 and 100.
Other Information:	The distinct module list is returned alongside pagination data to populate filter dropdowns dynamically.
Assumptions:	Permission definitions are seeded and maintained in the system database.

2.2.3.2 UC-15.02 View Permission Detail

UC ID and Name	UC-15.02 View Permission Detail		
Created By:	ThuanNT	Date Created:	28/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks "View detail" on a permission row or navigates directly to a permission detail URL.		
Description:	View detailed information of a specific permission (id, name, path, method, module, createdAt).		
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The caller's role is permitted to access `GET /api/permissions/:id`. **PRE-3** The selected permission exists in the database.		
Post-conditions:	**POST-1** Detailed permission information (id, name, path, method, module, createdAt) is displayed. **POST-2** Admin can navigate back to the permission list.		

Normal Flow:	**A. View Permission Detail** 1. Admin clicks "View detail" on a permission row or accesses `/admin/permissions/:id`. 2. Frontend checks permission ID parameter and ensures it is a positive integer. 3. Frontend sends `GET /api/permissions/:id`. 4. The system validates the JWT token and verifies Admin access rights. 5. The PermissionsController passes the parsed ID to `PermissionsService.getPermissionById(id)`. 6. The PermissionsService calls `PermissionsRepository.findById(id)`. 7. The repository queries `prisma.permission.findFirst` where `id = id` and `deletedAt = null`. 8. Database returns the permission record. 9. System returns HTTP 200 with permission detail. 10. Frontend displays permission name, HTTP method badge, route path, module, ID, and creation date.
Alternative Flows:	**A.2 Invalid Permission ID** 1. Admin enters a non-numeric or non-positive permission ID in the URL. 2. The frontend or NestJS ParseIntPipe rejects the request. 3. The frontend displays the Not Found or Bad Request page. 4. The use case ends.
Exceptions:	**EX-01: Permission Not Found** 1. No active permission exists with the specified ID or it has been soft-deleted. 2. The repository throws `PermissionNotFoundException` (HTTP 404). 3. The frontend displays "Permission not found." 4. The use case ends. **EX-02: Database Query Failure** 1. A database error occurs during the lookup. 2. The system throws `FailedToLoadPermissionDetailException` (HTTP 500). 3. The frontend displays "Failed to load permission detail." 4. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-PERM-06: Only permissions with `deletedAt = null` shall be accessible. BR-PERM-07: The detail payload includes `id`, `name`, `path`, `method`, `module`, and `createdAt`. BR-PERM-08: Access requires valid Admin credentials and permissions.

Other Information:	Detail information can be viewed in a modal dialog or dedicated detail route.
Assumptions:	The permission ID provided is valid within the system.

2.2.15 Session Management

2.2.3.1 UC-16.01 View Session List

UC ID and Name	UC-16.01 View Session List		
Created By:	ThuanNT	Date Created:	28/09/2026
Primary Actor:	Client, Expert, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When an authenticated user accesses the active sessions section in their account settings.		
Description:	View a list of active login sessions for the current user's account, showing device, IP address, creation time, expiration time, and marking the currently used session.		
Preconditions:	**PRE-1** The user is authenticated with a valid JWT access token. **PRE-2** The session ID stored in the access token exists in the Session table.		
Post-conditions:	**POST-1** A paginated list of sessions for the user is displayed with device, IP, timestamps, and `isCurrent` badge. **POST-2** User can view details (UC-16.02) or revoke remote sessions (UC-16.03).		

Normal Flow:	**A. View Session List** 1. User clicks “Active sessions” in account settings. 2. Frontend prepares query parameters (`page`, `limit`, `search`, `sortBy`, `order`). 3. Frontend sends `GET /api/sessions` with Bearer token. 4. System verifies JWT and extracts `userId` and `sessionId`. 5. SessionsController invokes `SessionsService.getMySessions(userId, query, sessionId)`. 6. SessionsService delegates to `SessionsRepository.findAllByUserId(userId, filter, sessionId)`. 7. The repository executes a Prisma transaction querying sessions for `userId` with optional search on `ipAddress` or `deviceInfo`, and counts total records. 8. The repository maps sessions and computes `isCurrent = (session.id === sessionId)`. 9. System returns HTTP 200 with sessions array and pagination metadata. 10. Frontend renders the session list, highlighting the current session with a “Current” badge.
Alternative Flows:	**A.2 Search Sessions** 1. User enters IP address or device name in search box. 2. System re-queries sessions matching `ipAddress` or `deviceInfo` (case-insensitive). 3. System refreshes displayed sessions. 4. Return to step A.10. **A.3 No Sessions Match** 1. No sessions match the search query. 2. System returns empty list with `total = 0`. 3. Frontend displays “No sessions match your search.” 4. Return to step A.10.
Exceptions:	**EX-01: Unauthorized / Session Revoked** 1. JWT access token is expired, invalid, or current session was revoked. 2. System returns HTTP 401 Unauthorized. 3. Frontend redirects user to login page. 4. The use case ends. **EX-02: Database Query Failure** 1. Database error occurs while querying user sessions. 2. System throws `FailedToLoadSessionsException` (HTTP 500). 3. Frontend displays “Failed to load sessions.” 4. The use case ends.
Priority:	High

Frequency of Use:	Medium
Business Rules:	BR-SES-01: Only sessions belonging to the authenticated `userId` shall be returned. BR-SES-02: `isCurrent` shall be true only when `session.id` matches the token's `sessionId`. BR-SES-03: Keyword search matches `ipAddress` or `deviceInfo` case-insensitively. BR-SES-04: Session active status is derived from `expiresAt > now`. BR-SES-05: Pagination supports `page >= 1` and `limit` between 1 and 100.
Other Information:	The current active session does not display a revoke button for self-revocation from the list to prevent accidental lockout.
Assumptions:	Each session record represents a valid login event.

2.2.3.2 UC-16.02 View Session Detail

UC ID and Name	UC-16.02 View Session Detail		
Created By:	ThuanNT	Date Created:	28/09/2026
Primary Actor:	Client, Expert, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user clicks "View detail" on a session item.		
Description:	View login session information (device, IP address, login time, approximate location/status, and whether it is the current session).		
Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The session ID exists and belongs to the authenticated user.		
Post-conditions:	**POST-1** Session details (deviceInfo, ipAddress, expiresAt, createdAt, isCurrent) are displayed. **POST-2** User can trigger revoke session (UC-16.03) from the detail view.		

Normal Flow:	**A. View Session Detail** 1. User clicks “View detail” on a session item. 2. Frontend checks session ID parameter. 3. Frontend sends `GET /api/sessions/:id`. 4. System verifies JWT and extracts `userId` and `sessionId`. 5. SessionsController invokes `SessionsService.getMySessionById(id, userId, sessionId)`. 6. SessionsService delegates to `SessionsRepository.findByIdForUser(id, userId, sessionId)`. 7. The repository queries `prisma.session.findFirst` where `id = id` and `userId = userId`. 8. Database returns session; system computes `isCurrent = (id === sessionId)`. 9. System returns HTTP 200 with session detail. 10. Frontend displays device information, IP address, creation date, expiration date, active/expired status, and current session indicator.
Alternative Flows:	**A.2 Invalid Session ID** 1. User specifies a non-integer session ID in URL. 2. NestJS ParseIntPipe rejects request with HTTP 400 Bad Request. 3. Frontend displays error message. 4. The use case ends.
Exceptions:	**EX-01: Session Not Found or Not Owned** 1. Session does not exist or belongs to another user account. 2. System throws `SessionNotFoundException` (HTTP 404). 3. Frontend displays “Session not found.” 4. The use case ends. **EX-02: Database Query Failure** 1. Database error occurs while loading session detail. 2. System throws `FailedToLoadSessionDetailException` (HTTP 500). 3. Frontend displays “Failed to load session details.” 4. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-SES-06: A session detail is strictly accessible only by its owner (`userId`). BR-SES-07: The payload includes `id`, `userId`, `deviceInfo`, `ipAddress`, `expiresAt`, `createdAt`, and `isCurrent`.
Other Information:	Session details are presented in a modal dialog or drawer for quick inspection.

Assumptions:	The authenticated user owns the session.

2.2.3.3 UC-16.03 Revoke Session

UC ID and Name	UC-16.03 Revoke Session		
Created By:	ThuanNT	Date Created:	28/09/2026
Primary Actor:	Client, Expert, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When an authenticated user clicks “Revoke” on an active session item.		
Description:	Log out of a specific device remotely by deleting the session record. The refresh token associated with that session is immediately invalidated.		
Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The targeted session belongs to the current user. **PRE-3** The session is active and has not expired (`expiresAt > now`).		
Post-conditions:	**POST-1** The session row is permanently deleted from the Session table. **POST-2** The refresh token for that session can no longer refresh access tokens. **POST-3** If the revoked session was current device, user is logged out; otherwise, session list refreshes.		

Normal Flow:	**A. Revoke Session** 1. User clicks “Revoke” on an active session item. 2. Frontend displays confirmation dialog informing that the remote device will be signed out. 3. User confirms the revocation action. 4. Frontend sends `DELETE /api/sessions/:id`. 5. System verifies JWT and extracts `userId` and `sessionId`. 6. SessionsController invokes `SessionsService.revokeMySession(id, userId, sessionId)`. 7. SessionsService invokes `SessionsRepository.deleteActiveByIdForUser(id, userId)`. 8. The repository queries the session to ensure it belongs to `userId` and verifies `expiresAt > now`. 9. The repository deletes the session record (`prisma.session.delete`). 10. SessionsService determines `loggedOut = (sessionId === id)`. 11. System returns HTTP 200 with `{ message: 'Session revoked successfully', loggedOut }`. 12. Frontend displays success message and updates the session list (or logs out and redirects to login if `loggedOut = true`).
Alternative Flows:	**A.2 Revoke Current Active Session** 1. User revokes the session matching their current device. 2. System sets `loggedOut = true`. 3. Frontend clears local credentials and redirects to `/login`. 4. The use case ends. **A.3 Session Already Expired** 1. The targeted session has already expired (`expiresAt <= now`). 2. System throws `SessionAlreadyExpiredException` (HTTP 400). 3. Frontend displays “Session has already expired.” 4. The use case ends. **A.4 User Cancels Revocation** 1. User clicks “Cancel” in confirmation dialog. 2. Dialog closes without sending deletion request. 3. The use case ends.
Exceptions:	**EX-01: Session Not Found** 1. Session does not exist or does not belong to the user. 2. System throws `SessionNotFoundException` (HTTP 404). 3. Frontend displays “Session not found.” 4. The use case ends.

	EX-02: Deletion Failure 1. Database error occurs during deletion. 2. System throws `FailedToRevokeSessionException` (HTTP 500). 3. Frontend displays "Failed to revoke session." 4. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-SES-08: Only sessions belonging to `userId` can be revoked. BR-SES-09: Expired sessions cannot be revoked (returns HTTP 400). BR-SES-10: Deleting the session row immediately revokes refresh token capabilities. BR-SES-11: If current session is revoked, `loggedOut` is returned as true.
Other Information:	Revoking a session invalidates the device immediately on the next token refresh attempt.
Assumptions:	The user has confirmed the intent to revoke the session.

2.2.16 Forum Category Management

2.2.3.1 UC-17.01 Create Forum Category

UC ID and Name	UC-17.01 Create Forum Category		
Created By:	ThuanNT	Date Created:	28/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks "Create Category" on the Forum Category Management page.		
Description:	Allows admins to create a new forum category with name and optional description. An automated slug is generated from the name.		
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The user has permission to access `POST /api/forums/admin/categories`. **PRE-3** The system is connected to the database.		
Post-conditions:	**POST-1** A new record is created in ForumCategory with name, unique slug, description, and `deletedAt = null`. **POST-2** The new category appears in the forum category list with `postCount = 0`.		

Normal Flow:	**A. Create Forum Category** 1. Admin clicks “Create Category” button on `/admin/categories`. 2. Frontend opens Create Category modal with Name (required) and Description (optional). 3. Admin enters category name and optional description, then clicks “Save”. 4. Frontend validates inputs (name 1–100 characters; description up to 500 characters). 5. Frontend sends `POST /api/forums/admin/categories` with request body. 6. System verifies JWT token and Admin role. 7. ForumAdminController invokes `ForumAdminService.createAdminCategory(roleName, body)`. 8. ForumAdminService delegates to `ForumAdminRepo.createAdminCategory(data)`. 9. The repository verifies that no active category (`deletedAt = null`) exists with the same name (case-insensitive). 10. The repository generates a unique slug via `slugify(name)`, appending a numeric counter if colliding. 11. The repository creates the category in the database (`prisma.forumCategory.create`). 12. System returns HTTP 201 with created category detail and `postCount = 0`. 13. Frontend displays “Category created successfully!” and refreshes category list.
Alternative Flows:	**A.2 Duplicate Category Name** 1. An active category with the same name already exists. 2. System throws `ForumCategoryAlreadyExistsException` (HTTP 409 Conflict). 3. Frontend displays “Category name already exists.” 4. Return to step A.3. **A.3 Form Validation Error** 1. Name is empty or exceeds 100 characters, or description exceeds 500 characters. 2. Frontend validation prevents submission, or backend DTO validation returns HTTP 400. 3. Frontend displays validation error message. 4. Return to step A.3. **A.4 Admin Cancels Creation** 1. Admin clicks “Cancel” in modal dialog. 2. Modal closes and unsaved input is discarded. 3. The use case ends.

Exceptions:	**EX-01: Forbidden** 1. User lacks Admin role. 2. System returns HTTP 403 Forbidden. 3. Frontend displays “Forbidden: Account inactive or unauthorized.” 4. The use case ends. **EX-02: Database Query Failure** 1. Database error occurs during category insertion. 2. System throws `FailedToCreateForumCategoryException` (HTTP 500). 3. Frontend displays “Failed to create category.” 4. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-FCAT-01: Only users with the Admin role can create forum categories. BR-FCAT-02: Category name is required, max 100 characters, unique among active categories. BR-FCAT-03: Description is optional, max 500 characters. BR-FCAT-04: Slug is generated automatically from name and made unique with numeric suffix when necessary. BR-FCAT-05: Newly created categories start with `postCount = 0`.
Other Information:	The slug is used for clean, SEO-friendly forum URLs on the client side.
Assumptions:	The admin has authority to create categories.

2.2.3.2 UC-17.02 Update Forum Category

UC ID and Name	UC-17.02 Update Forum Category		
Created By:	ThuanNT	Date Created:	28/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks “Edit” on a category row or detail page.		
Description:	Allows admins to update an existing forum category’s name and description. If name changes, the slug is regenerated automatically.		
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The targeted category exists and has `deletedAt = null`.		

Post-conditions:	**POST-1** The category's name, description, and slug (if name changed) are updated in the database. **POST-2** The updated category is reflected in the category list.
Normal Flow:	**A. Update Forum Category** 1. Admin clicks "Edit" on a category row or detail page. 2. Frontend opens Edit Category modal pre-filled with current name and description. 3. Admin edits name and/or description, then clicks "Save Changes". 4. Frontend validates inputs. 5. Frontend sends `PATCH /api/forums/admin/categories/:id` with updated data. 6. System verifies JWT token and Admin role. 7. ForumAdminController invokes `ForumAdminService.updateAdminCategory(roleName, id, body)`. 8. ForumAdminService delegates to `ForumAdminRepo.updateAdminCategory(id, body)`. 9. The repository verifies the category exists with `deletedAt = null`. 10. If name changed, repository checks uniqueness against other active categories and generates a new unique slug. 11. The repository updates the record (`prisma.forumCategory.update`). 12. System returns HTTP 200 with updated category details. 13. Frontend displays "Category updated successfully!" and refreshes table.
Alternative Flows:	**A.2 Duplicate Category Name** 1. The updated name conflicts with another active category. 2. System throws `ForumCategoryAlreadyExistsException` (HTTP 409 Conflict). 3. Frontend displays "Category name already exists." 4. Return to step A.3. **A.3 Admin Cancels Editing** 1. Admin clicks "Cancel" in Edit Category modal. 2. Modal closes without saving changes. 3. The use case ends.
Exceptions:	**EX-01: Category Not Found** 1. Category does not exist or has already been soft-deleted. 2. System throws `ForumCategoryNotFoundException` (HTTP 404). 3. Frontend displays "Category not found." 4. The use case ends.

	EX-02: Database Query Failure 1. Database error occurs during category update. 2. System throws `FailedToUpdateForumCategoryException` (HTTP 500). 3. Frontend displays "Failed to update category." 4. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-FCAT-06: Only Admin users can update forum categories. BR-FCAT-07: Partial updates are supported (name or description can be updated independently). BR-FCAT-08: If name changes, it must be unique among active categories and slug is regenerated. BR-FCAT-09: Soft-deleted categories cannot be updated.
Other Information:	Existing posts retain their category association after category name or description update.
Assumptions:	The category ID is valid.

2.2.3.3 UC-17.03 Delete Forum Category

UC ID and Name	UC-17.03 Delete Forum Category		
Created By:	ThuanNT	Date Created:	28/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks "Delete" on a category row or detail page.		
Description:	Allows admins to soft-delete a forum category. In a database transaction, active posts in this category are detached to Uncategorized (categoryId = null) and the category is soft-deleted by setting deletedAt.		
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The targeted category exists and is active (`deletedAt = null`).		
Post-conditions:	**POST-1** Posts belonging to this category have `categoryId = null` (moved to Uncategorized). **POST-2** The category's `deletedAt` field is set to current timestamp. **POST-3** The category no longer appears in the active category list.		

Normal Flow:	**A. Delete Forum Category** 1. Admin clicks “Delete” on a category row or detail page. 2. Frontend displays confirmation dialog: “Are you sure you want to delete this category? Active posts will become Uncategorized and the category can be restored later.” 3. Admin confirms deletion. 4. Frontend sends `DELETE /api/forums/admin/categories/:id`. 5. System verifies JWT token and Admin role. 6. ForumAdminController invokes `ForumAdminService.deleteAdminCategory(roleName, id)`. 7. ForumAdminService delegates to `ForumAdminRepo.deleteAdminCategory(id)`. 8. The repository verifies the category exists with `deletedAt = null`. 9. The repository executes a Prisma transaction: a. Updates active posts in this category setting `categoryId = null`. b. Updates category record setting `deletedAt = new Date()`. 10. System returns HTTP 200 with `{ message: 'Forum category deleted successfully' }`. 11. Frontend displays “Category deleted (soft delete)” and refreshes list.
Alternative Flows:	**A.2 Admin Cancels Deletion** 1. Admin clicks “Cancel” in confirmation dialog. 2. Dialog closes without sending deletion request. 3. The use case ends.
Exceptions:	**EX-01: Category Not Found** 1. Category does not exist or has already been deleted. 2. System throws `ForumCategoryNotFoundException` (HTTP 404). 3. Frontend displays “Category not found.” 4. The use case ends. **EX-02: Transaction Failure** 1. Database error occurs during the transaction. 2. System throws `FailedToDeleteForumCategoryException` (HTTP 500). 3. Frontend displays “Delete category failed.” 4. The use case ends.
Priority:	Medium
Frequency of Use:	Low

Business Rules:	BR-FCAT-10: Only Admin users can delete forum categories. BR-FCAT-11: Deletion is a soft delete implemented via `deletedAt`. BR-FCAT-12: In a transaction, active posts under the category are moved to Uncategorized (`categoryId = null`) to preserve discussion threads. BR-FCAT-13: Soft-deleted categories are hidden from the active list but can be restored.
Other Information:	Soft-deleted categories can be viewed in the deleted filter view and restored via UC-17.06.
Assumptions:	The admin has confirmed the deletion.

2.2.3.4 UC-14.01 View Forum Category List

UC ID and Name	UC-14.01 View Forum Category List		
Created By:	ThuanNT	Date Created:	28/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin accesses the Forum Categories section from the admin sidebar.		
Description:	Allows admins to view the list of forum categories with id, name, slug, description, creation/update dates, deleted status, and post count. Supports pagination, keyword search, sorting, and deleted status filtering.		
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The user has permission to access `GET /api/forums/admin/categories`. **PRE-3** The system is connected to the database.		
Post-conditions:	**POST-1** A paginated list of forum categories is displayed with metadata and post counts. **POST-2** Admin can navigate to create (UC-17.01), edit (UC-17.02), delete (UC-17.03), view detail (UC-14.02), or restore (UC-17.06).		

Normal Flow:	**A. View Forum Category List** 1. Admin clicks “Forum Categories” in the admin navigation. 2. Frontend prepares query parameters (`page`, `limit`, `search`, `deleted`, `sortBy`, `sortOrder`). 3. Frontend sends `GET /api/forums/admin/categories` with query parameters. 4. System verifies JWT token and Admin role. 5. ForumAdminController invokes `ForumAdminService.getAdminCategoryLists(roleName, ...)`. 6. ForumAdminService delegates to `ForumAdminRepo.getAdminCategoryLists(...)`. 7. The repository queries `prisma.forumCategory.findMany` with filters on `deletedAt` and `name`, includes active `_count.posts`, and counts total matching categories. 8. Database returns categories and total count. 9. System returns HTTP 200 with `{ categories, total }`. 10. Frontend renders the category list, post counts, status badges, pagination, and action buttons.
Alternative Flows:	**A.2 Search by Name** 1. Admin types in the search input. 2. System queries categories where `name` contains search keyword (case-insensitive). 3. Table refreshes with matching categories. 4. Return to step A.10. **A.3 Filter Deleted Categories** 1. Admin toggles the “Deleted” filter to true. 2. System queries categories where `deletedAt != null`. 3. Table displays soft-deleted categories with a “Restore” action button. 4. Return to step A.10. **A.4 No Categories Found** 1. No categories match search or filter parameters. 2. System returns empty categories array with `total = 0`. 3. Frontend displays “No categories found.” 4. Return to step A.10.
Exceptions:	**EX-01: Unauthorized / Forbidden** 1. JWT token is missing, expired, or user is not Admin. 2. System returns HTTP 401 Unauthorized or 403 Forbidden. 3. Frontend displays error message. 4. The use case ends.

	EX-02: Database Query Failure 1. Database error occurs while querying categories. 2. System throws `FailedToLoadForumCategoriesException` (HTTP 500). 3. Frontend displays "Failed to load forum categories." 4. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-FCAT-14: Only Admin role users can access admin category list. BR-FCAT-15: When `deleted` query is omitted or false, only active categories (`deletedAt = null`) are returned. BR-FCAT-16: When `deleted = true`, only soft-deleted categories (`deletedAt != null`) are returned. BR-FCAT-17: Post count is computed strictly for active posts (`deletedAt = null`). BR-FCAT-18: Sorting is supported on `id`, `name`, and `createdAt`.
Other Information:	Admins can switch between active and trash views to manage or restore deleted categories.
Assumptions:	Categories and posts are stored in the database.

2.2.3.5 UC-14.02 View Forum Category Detail

UC ID and Name	UC-14.02 View Forum Category Detail		
Created By:	ThuanNT	Date Created:	28/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks "View detail" on a category row or accesses /admin/categories/:id.		
Description:	Allows admins to view the details of a specific forum category including id, name, slug, description, creation/update dates, and post count.		
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The targeted category exists and has `deletedAt = null`.		
Post-conditions:	**POST-1** Detailed category information and post statistics are displayed. **POST-2** Admin can trigger Edit (UC-17.02), Delete (UC-17.03), or view associated posts.		

Normal Flow:	**A. View Forum Category Detail** 1. Admin clicks "View detail" on a category row or accesses `/admin/categories/:id`. 2. Frontend checks category ID parameter. 3. Frontend sends `GET /api/forums/admin/categories/:id`. 4. System verifies JWT token and Admin role. 5. ForumAdminController invokes `ForumAdminService.getAdminCategoryById(roleName, id)`. 6. ForumAdminService delegates to `ForumAdminRepo.getAdminCategoryById(id)`. 7. The repository queries `prisma.forumCategory.findFirst` where `id = id` and `deletedAt = null`, including active post count. 8. Database returns the category record. 9. System returns HTTP 200 with category detail payload. 10. Frontend displays category ID, Name, Slug, Description, Post Count, Created Date, and Updated Date with management actions.
Alternative Flows:	**A.2 Invalid Category ID** 1. Admin enters a non-numeric category ID. 2. NestJS ParseIntPipe rejects request with HTTP 400 Bad Request. 3. Frontend displays error page. 4. The use case ends.
Exceptions:	**EX-01: Category Not Found** 1. Category does not exist or has been deleted. 2. System throws `ForumCategoryNotFoundException` (HTTP 404). 3. Frontend displays "Category not found." 4. The use case ends. **EX-02: Database Query Failure** 1. Database error occurs during category lookup. 2. System throws `FailedToLoadCategoryDetailException` (HTTP 500). 3. Frontend displays "Failed to load category details." 4. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-FCAT-19: Only categories with `deletedAt = null` are accessible in detail view. BR-FCAT-20: Detail payload includes `id`, `name`, `slug`, `description`, `createdAt`, `updatedAt`, and `postCount`. BR-FCAT-21: Only users with the Admin role can access this endpoint.

Other Information:	Admins can click “View Posts” to browse all active forum discussions under this category.
Assumptions:	The category exists in the system.

2.2.3.6 UC-17.06 Restore Forum Category

UC ID and Name	UC-17.06 Restore Forum Category		
Created By:	ThuanNT	Date Created:	28/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks “Restore” on a soft-deleted category in the deleted categories list.		
Description:	Restore a deleted Category forum by setting deletedAt back to null. Verifies that no active category currently exists with the same name before restoring.		
Preconditions:	**PRE-1** The user is authenticated as an Admin. **PRE-2** The targeted category exists and is currently soft-deleted (`deletedAt != null`). **PRE-3** No active category exists with the same name.		
Post-conditions:	**POST-1** The category’s `deletedAt` field is set back to `null` in the database. **POST-2** The category reappears in the active forum category list.		

Normal Flow:	**A. Restore Forum Category** 1. Admin clicks “Restore” on a deleted category item in the trash/deleted view. 2. Frontend displays confirmation dialog: “Are you sure you want to restore this category?” 3. Admin confirms restoration. 4. Frontend sends `PATCH /api/forums/admin/categories/:id/restore`. 5. System verifies JWT token and Admin role. 6. ForumAdminController invokes `ForumAdminService.restoreAdminCategory(roleName, id)`. 7. ForumAdminService delegates to `ForumAdminRepo.restoreAdminCategory(id)`. 8. The repository finds the category where `id = id` and `deletedAt != null`. 9. The repository checks if an active category (`deletedAt = null`) already exists with the same name. 10. The repository updates the category setting `deletedAt = null` (`prisma.forumCategory.updateMany`). 11. System returns HTTP 200 with restored category details. 12. Frontend displays “Category restored successfully” and refreshes the category list.
Alternative Flows:	**A.2 Conflicting Active Category Name** 1. An active category with the same name was created after this category was deleted. 2. System throws `ForumCategoryAlreadyExistsException` (HTTP 409 Conflict). 3. Frontend displays “Cannot restore category: an active category with the same name already exists.” 4. The use case ends. **A.3 Admin Cancels Restoration** 1. Admin clicks “Cancel” in confirmation dialog. 2. Dialog closes without sending restore request. 3. The use case ends.
Exceptions:	**EX-01: Category Not Found or Not Deleted** 1. Category does not exist or has already been restored (`deletedAt = null`). 2. System throws `ForumCategoryNotFoundException` (HTTP 404). 3. Frontend displays “Category not found or not deleted.” 4. The use case ends.

	EX-02: Restore Operation Failure 1. Database error occurs during restore update. 2. System throws `FailedToRestoreForumCategoryException` (HTTP 500). 3. Frontend displays "Restore category failed." 4. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-FCAT-22: Only Admin role users can restore forum categories. BR-FCAT-23: Category must have `deletedAt != null` to be restored. BR-FCAT-24: Category name must be unique among active categories; conflict throws HTTP 409 Conflict. BR-FCAT-25: Restoring resets `deletedAt = null`; previously detached posts remain in Uncategorized.
Other Information:	Restoring makes the category immediately available for new forum discussions.
Assumptions:	The admin has authority to restore soft-deleted categories.

2.2.17 Manage Dispute

2.2.3.1 UC-33.01 Open Dispute

UC ID and Name	UC-33.01 Open Dispute		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	Admin
Trigger:	When an authorized contract participant clicks the "Open Dispute" button on an active, funded milestone.		
Description:	Allows a Client or Freelancer to initiate a formal dispute over milestone deliverables, payment issues, or unresponsive behavior, freezing the milestone escrow funds and inviting the other party to provide counter-evidence.		

Preconditions:	**PRE-1** The user is authenticated in the system. **PRE-2** The user is an active participant (Client or Freelancer) of the contract. **PRE-3** The target milestone is in `FUNDED` payment status and has not yet been marked `COMPLETED`. **PRE-4** No existing active dispute is currently associated with this milestone.
Post-conditions:	**POST-1** A new dispute record is persisted in the database with status `OPEN` or `WAITING_RESPONSE`. **POST-2** Milestone status is frozen, preventing premature completion, submission, or manual escrow release. **POST-3** Arbitration fee invoices with a 24-hour payment deadline are established for the involved parties. **POST-4** The respondent receives a notification alerting them of the dispute with a 48-hour response deadline.
Normal Flow:	**A. Open Dispute** 1. The user expands the disputed milestone in the contract view and clicks “Open Dispute”. 2. The system renders the Open Dispute modal with fee schedule and arbitration rules. 3. The user selects a dispute reason category (e.g., Deliverables Unsatisfactory, Unresponsive Counterpart, Scope Discrepancy). 4. The user enters a detailed explanation outlining the issue, chronological events, and requested outcome. 5. The user selects and uploads supporting evidence files (up to 5 documents, images, or chat archives). 6. The user agrees to the arbitration fee policy and terms of resolution. 7. The user clicks “Submit Dispute”. 8. The system validates the input, locks milestone escrow funds, creates the dispute record, and schedules notification deadlines. 9. The modal closes, a success notification appears, and the user is redirected to the Dispute Detail page.
Alternative Flows:	**A.1 Cancel Dispute Filing** 1. At any step prior to submission, the user clicks “Cancel”. 2. The system closes the modal without creating a dispute. 3. The milestone remains in its active status.

Exceptions:	**EX-01: Milestone Ineligible for Dispute** 1. If the milestone has not been funded into escrow or is already completed, the system displays: “Disputes can only be opened on funded, incomplete milestones.” 2. The use case ends. **EX-02: Existing Active Dispute** 1. If a dispute already exists for this milestone, the system displays: “A dispute is already active for this milestone.” 2. The use case ends. **EX-03: Evidence File Limit Exceeded** 1. If more than 5 evidence files are attached, the system displays: “You may attach a maximum of 5 evidence files.” 2. The use case ends.
Priority:	High
Frequency of Use:	Low
Business Rules:	BR-DISPUTE-01: Only direct contract parties (Client or Freelancer) can open a dispute. BR-DISPUTE-02: Filing a dispute immediately freezes milestone escrow balance from release, cancellation, or refund until resolution is achieved. BR-DISPUTE-03: A maximum of 5 evidence files can be uploaded during the initial dispute claim.
Other Information:	N/A
Assumptions:	Parties have attempted direct communication before escalating to formal dispute.

2.2.3.2 UC-33.02 View Dispute List

UC ID and Name	UC-33.02 View Dispute List		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user navigates to the “Disputes” section from the main navigation menu.		
Description:	Allows a Client, Freelancer, or Administrator to retrieve a paginated overview of disputes relevant to their account, with status filters and deadline indicators.		

Preconditions:	**PRE-1** The user is authenticated in the system. **PRE-2** The database connection is active.
Post-conditions:	**POST-1** A paginated list of dispute records matching user role and filter criteria is displayed.
Normal Flow:	**A. View Dispute List** 1. The user navigates to the Disputes page. 2. The system queries disputes where the user is either the opener or the respondent (or all platform disputes if Admin). 3. The system sorts disputes chronologically with urgent cases (pending response or unpaid fees) prioritized at the top. 4. The system renders dispute summary cards displaying: Dispute ID, Contract Title, Disputed Amount, Opener, Respondent, Status Badge, and Response Deadline.
Alternative Flows:	**A.1 Filter by Status** 1. The user selects a status tab (All, Open, Waiting Response, Under Review, Decision Made, Finalized). 2. The system retrieves and renders matching records. 3. Return to step A.4.
Exceptions:	**EX-01: No Disputes Found** 1. If no records match the criteria, the system renders an empty state: “No disputes found.” 2. The use case ends. **EX-02: Database Query Failure** 1. If data retrieval fails, the system displays: “Unable to load dispute list. Please try again.” 2. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-DISPUTE-04: Non-admin users can only view disputes in which they are an opener or respondent. BR-DISPUTE-05: Administrators have global visibility into all platform dispute records.
Other Information:	N/A
Assumptions:	Dispute listings are paginated to maintain optimal rendering performance.

2.2.3.3 UC-33.03 View Dispute Detail

UC ID and Name	UC-33.03 View Dispute Detail		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user clicks on a dispute card from the list or follows a dispute notification link.		
Description:	Allows an involved party or Administrator to view full case records, including claims, evidence files, fee payment status, resolution proposals, decision reviews, and audit timeline.		
Preconditions:	**PRE-1** The user is authenticated in the system. **PRE-2** The user is an authorized participant (opener or respondent) or an Administrator. **PRE-3** The dispute record exists in the database.		
Post-conditions:	**POST-1** The system renders the comprehensive dispute details and checks for any overdue fee conditions.		
Normal Flow:	**A. View Dispute Detail** 1. The user clicks on a dispute entry. 2. The system verifies user authorization to view the case. 3. The system checks fee deadlines: if 24 hours have elapsed without payment, the fee status is updated to `OVERDUE`. 4. The system renders the Dispute Detail page displaying: - Case Header: Dispute ID, Contract Name, Milestone Name, Disputed Amount, Status badge. - Opener Claim: Reason, description narrative, submission timestamp, and evidence attachments. - Respondent Rebuttal: Counter-statement, submission date, and evidence attachments (if submitted). - Arbitration Fee Section: Fee status per party and payment button (if pending). - Resolution Proposal: Arbitrator findings and proposed split amounts (if decision made). - Timeline: Chronological audit history of dispute milestones.		
Alternative Flows:	**A.1 Download Evidence File** 1. The user clicks on an attached evidence file card. 2. The system initiates the secure download from cloud storage. 3. Return to step A.4.		

Exceptions:	**EX-01: Unauthorized Access** 1. An unauthorized user attempts direct URL access. 2. The system displays: “Access Denied: You do not have permission to view this dispute.” 3. The use case ends. **EX-02: Dispute Not Found** 1. The specified dispute ID does not exist in the database. 2. The system displays: “Dispute not found.” 3. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-DISPUTE-06: Case evidence and claims are strictly accessible only to case participants and designated administrators. BR-DISPUTE-07: Unpaid arbitration fees exceeding the 24-hour deadline automatically transition to `OVERDUE`.
Other Information:	N/A
Assumptions:	All participants have access to a modern browser capable of rendering document and image evidence.

2.2.3.4 UC-33.04 Respond to Dispute

UC ID and Name	UC-33.04 Respond to Dispute		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer (Respondent)	Secondary Actors:	Admin
Trigger:	When the designated respondent clicks “Submit Response” on an open dispute.		
Description:	Allows the respondent party to submit their official rebuttal statement and counter-evidence within the 48-hour response window, ensuring equal hearing during arbitration.		
Preconditions:	**PRE-1** The user is authenticated as the designated respondent (`respondentId = userId`). **PRE-2** The dispute status is `OPEN` or `WAITING_RESPONSE`. **PRE-3** The respondent has not previously submitted a response (single submission allowed). **PRE-4** The submission occurs before the 48-hour response deadline.		

Post-conditions:	**POST-1** The counter-response description and evidence files are persisted in the database. **POST-2** Dispute status transitions to `UNDER_REVIEW`. **POST-3** The opener and administrator are notified that the rebuttal has been submitted.
Normal Flow:	**A. Respond to Dispute** 1. The respondent navigates to the Dispute Detail page. 2. The respondent reviews the opener's claims and clicks "Submit Response". 3. The system displays the rebuttal submission form with the remaining deadline countdown. 4. The respondent enters a detailed counter-explanation addressing the claims. 5. The respondent uploads supporting evidence files (up to 5 documents or screenshots). 6. The respondent clicks "Confirm & Submit Response". 7. The system validates the input, records the rebuttal, updates dispute status to `UNDER_REVIEW`, and logs the event. 8. The system displays a success toast and refreshes the dispute view with the rebuttal displayed.
Alternative Flows:	**A.1 Cancel Response Draft** 1. The respondent clicks "Cancel" before submitting. 2. The system closes the form without saving; the case remains in waiting status until the deadline expires. 3. Return to step A.1.

Exceptions:	**EX-01: Response Deadline Expired** 1. If the 48-hour response deadline has elapsed, the system displays: "The response window has closed. The case will be reviewed based on available evidence." 2. The use case ends. **EX-02: Response Already Submitted** 1. If the respondent has already submitted their rebuttal, the system displays: "You have already submitted your response for this dispute." 2. The use case ends. **EX-03: Evidence Limit Exceeded** 1. If more than 5 evidence files are selected, the system displays: "Maximum 5 evidence files allowed." 2. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-DISPUTE-08: Only the designated respondent can submit a formal response. BR-DISPUTE-09: A respondent is allowed exactly one rebuttal submission per dispute case. BR-DISPUTE-10: Submitting a valid response moves the dispute status to `UNDER_REVIEW`.
Other Information:	N/A
Assumptions:	The respondent provides factual and verifiable evidence to substantiate their position.

2.2.3.5 UC-33.05 Respond to Dispute Resolution

UC ID and Name	UC-33.05 Respond to Dispute Resolution		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	Admin
Trigger:	When a participant clicks "Accept Decision" or "Reject Decision" upon reviewing an arbitrator's proposed resolution.		
Description:	Allows the Client and Freelancer to independently accept or reject a proposed resolution. If both accept, the decision is finalized and funds are disbursed; if either party rejects, the dispute escalates to final review.		

Preconditions:	**PRE-1** The user is authenticated as the Client or Freelancer of the disputed contract. **PRE-2** The dispute status is currently `DECISION_MADE`. **PRE-3** The user has not yet submitted their decision review for this round.
Post-conditions:	**POST-1** The user's review choice (`ACCEPT` or `REJECT`) is recorded in the database. **POST-2** If both parties submit `ACCEPT`, the dispute transitions to `FINALIZED`, milestone escrow funds are automatically disbursed according to split amounts, and milestone status is resolved. **POST-3** If either party submits `REJECT`, the dispute transitions to `REVIEW_REQUESTED` for final determination.
Normal Flow:	**A. Respond to Resolution Proposal** 1. The participant opens the Dispute Detail page. 2. The system presents the proposed resolution: Client Split Amount, Freelancer Split Amount, and Arbitrator Findings. 3. The user selects either "Accept Decision" or "Reject Decision". 4. If rejecting, the user enters a mandatory objection explanation. 5. The user clicks "Submit Decision Response". 6. The system records the response in the decision review repository. 7. If both parties have now accepted: - System executes escrow payout transaction allocating split balances. - Dispute status updates to `FINALIZED`. - Milestone status updates to `COMPLETED` and paymentStatus to `RELEASED`. 8. If any party rejected: - Dispute status updates to `REVIEW_REQUESTED`. - An escalation alert is dispatched to the administrator. 9. The system updates the UI and confirms submission with a notification.
Alternative Flows:	**A.1 Unilateral Acceptance Waiting** 1. One party accepts while the counterpart's response remains pending. 2. The system confirms the user's acceptance and shows: "Waiting for counterpart's response." 3. Dispute status remains `DECISION_MADE`.

Exceptions:	**EX-01: Response Already Submitted** 1. If the user attempts to vote again, the system displays: "You have already submitted your response to this decision." 2. The use case ends. **EX-02: Dispute Not Pending Review** 1. If the dispute is not in `DECISION_MADE` status, the system displays: "This dispute is not awaiting decision reviews." 2. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-DISPUTE-11: Mutual acceptance by both parties triggers immediate, automated escrow disbursement. BR-DISPUTE-12: A single rejection automatically escalates the dispute to `REVIEW_REQUESTED`. BR-DISPUTE-13: Each participant can only submit one response per proposed decision round.
Other Information:	N/A
Assumptions:	Disbursement transactions are executed atomically in database transactions.

2.2.3.6 UC-33.06 Propose Dispute Resolution

UC ID and Name	UC-33.06 Propose Dispute Resolution		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Admin	Secondary Actors:	Client, Freelancer
Trigger:	When an Administrator opens a dispute under review and clicks "Make Decision" or "Propose Resolution".		
Description:	Allows an Administrator/Arbitrator to examine claims and evidence from both sides, determine a fair financial split of milestone escrow funds, record formal justification findings, and issue the proposed resolution for participant review.		
Preconditions:	**PRE-1** The user is authenticated and holds the Admin role. **PRE-2** The dispute status is `OPEN`, `WAITING_RESPONSE`, or `UNDER_REVIEW`. **PRE-3** The total milestone escrow amount remains secured in escrow.		

Post-conditions:	**POST-1** Dispute status transitions to `DECISION_MADE`. **POST-2** Split amounts (Client Amount, Freelancer Amount) and formal findings are persisted in the database. **POST-3** Participant decision review records are initialized in `PENDING` status. **POST-4** Both Client and Freelancer are notified of the arbitrator's decision.
Normal Flow:	**A. Propose Resolution** 1. The Admin accesses the Dispute Detail page in the Admin Dashboard. 2. The Admin analyzes the contract deliverables, opener's claims, respondent's rebuttal, and all evidence files. 3. The Admin clicks "Propose Resolution". 4. The system presents the Decision Form displaying total milestone escrow budget. 5. The Admin enters the allocation amounts: - Client Amount ($) - Freelancer Amount ($) - Written Decision Justification (reasoning and findings) 6. The system validates that `Client Amount + Freelancer Amount` equals exactly the total milestone amount. 7. The Admin clicks "Issue Decision". 8. The system executes the decision transaction: updates dispute status to `DECISION_MADE`, initializes participant review tracking, and logs the administrative action. 9. The system displays a confirmation notification and dispatches decision alerts to both parties.
Alternative Flows:	**A.1 Split Evenly (50/50)** 1. The Admin clicks the "50/50 Split" shortcut button. 2. The system automatically populates equal halves of the milestone amount into Client and Freelancer fields. 3. Return to step A.5.

Exceptions:	**EX-01: Split Amount Mismatch** 1. If the sum of Client Amount and Freelancer Amount does not equal the milestone amount, the system displays: “Total split amount must exactly equal the milestone escrow amount.” 2. The use case ends. **EX-02: Unauthorized Role** 1. A non-admin user attempts to issue an arbitration decision. 2. The system rejects the action with “Access Denied.” 3. The use case ends.
Priority:	High
Frequency of Use:	Low
Business Rules:	BR-DISPUTE-14: Only users with the Admin role can formulate and propose formal dispute resolutions. BR-DISPUTE-15: The sum of awarded amounts must mathematically equal 100% of the disputed milestone escrow funds. BR-DISPUTE-16: The decision proposal must include explicit written justification for legal and platform transparency.
Other Information:	N/A
Assumptions:	The Administrator acts as an impartial arbitrator adhering to platform terms of service.

2.2.18 Manage Moderation

2.2.3.1 UC-06.1 View Moderation Queue

UC ID and Name	UC-06.1 View Moderation Queue		
Created By:	Pham Kha Vy	Date Created:	06/10/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin accesses the Moderation section from the admin navigation menu.		
Description:	Allows an administrator to view all forum posts with a PENDING moderation status, review their content, AI moderation scores, and categories, then approve or reject them.		
Preconditions:	**PRE-1** The user is authenticated and has the Admin role. **PRE-2** There are forum posts with a PENDING moderation status in the system.		

Post-conditions:	**POST-1** A paginated list of PENDING forum posts is displayed. **POST-2** The Admin can select a post to view its full details. **POST-3** The Admin can approve or reject each post.
Normal Flow:	**A. View Moderation Queue** 1. The Admin navigates to the Moderation page from the admin sidebar. 2. The system retrieves all forum posts where `moderationStatus = PENDING` and `deletedAt = NULL`, ordered by `createdAt ASC`. 3. For each post, the system loads the author's profile (display name, avatar), the moderation score, moderation categories, and creation date. 4. The system displays the moderation queue as a table with columns: ID, Title, Score (color-coded), Categories (as badges), Author, and Date. 5. The Admin can view the AI moderation score: green if less than 0.3, amber if between 0.3 and 0.8, red if 0.8 or above.
Alternative Flows:	**A.2 No Pending Posts Found** 1. The Admin navigates to the Moderation page. 2. The system determines that no posts have a PENDING moderation status. 3. The system displays a message indicating the moderation queue is empty. 4. The use case ends.

Exceptions:	**EX-01: Unauthorized Access** 1. A user without the Admin role attempts to access the Moderation page. 2. The system determines that the user does not have the Admin role. 3. The system displays the message “You do not have permission to access this page.” 4. The use case ends. **EX-02: Database Error** 1. The system attempts to retrieve the pending posts from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load moderation queue. Please try again later.” 4. The moderation queue is not displayed. 5. The use case ends. **EX-03: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the moderation data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The Admin remains on the current page without any moderation data being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-MOD-01: Only posts with `moderationStatus = PENDING` and `deletedAt = NULL` shall appear in the moderation queue. BR-MOD-02: The AI moderation score is assigned by the SensitiveAI proxy during post creation and ranges from 0.0 to 1.0. BR-MOD-03: Posts with a score less than 0.3 are considered low risk (green), 0.3–0.8 medium risk (amber), 0.8 or above high risk (red). BR-MOD-04: Only users with the Admin role may access the moderation queue.

Other Information:	The moderation score and categories are populated by the SensitiveAI proxy at post creation time. If the proxy does not provide a score, the post defaults to PENDING.
Assumptions:	The SensitiveAI proxy is operational and assigns scores to new posts.

2.2.3.2 UC-06.2 Moderation Item Detail

UC ID and Name	UC-06.2 Moderation Item Detail		
Created By:	Pham Kha Vy	Date Created:	06/10/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks the “View” action on a post in the moderation queue.		
Description:	Allows an administrator to view the complete content and metadata of a pending forum post before deciding whether to approve or reject it.		
Preconditions:	**PRE-1** The user is authenticated and has the Admin role. **PRE-2** The post exists with a PENDING moderation status.		
Post-conditions:	**POST-1** The full post content, author information, moderation score, categories, and creation date are displayed. **POST-2** The Admin can approve the post or reject it.		
Normal Flow:	**A. View Moderation Detail** 1. The Admin clicks the “View” action on a post in the moderation queue. 2. The system retrieves the full post content including title, content body, author profile, moderation score, moderation categories, and creation date. 3. The system displays a detail dialog showing all post information. 4. The system displays the moderation score and categories with visual indicators. 5. The system displays Approve and Reject action buttons.		

Alternative Flows:	**A.1 Post Not Found or No Longer Pending** 1. The Admin clicks the “View” action on a post. 2. The system determines that the post does not exist, has been deleted, or is no longer in PENDING status. 3. The system displays the message “Post not found or no longer pending.” 4. Return to step 1.
Exceptions:	**EX-01: Unauthorized Access** 1. A user without the Admin role attempts to view a moderation item detail. 2. The system determines that the user does not have the Admin role. 3. The system displays the message “You do not have permission to access this page.” 4. The use case ends. **EX-02: Database Error** 1. The system attempts to retrieve the post detail from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load post detail. Please try again later.” 4. The detail view is not displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-MOD-05: Only posts with `moderationStatus = PENDING` may be viewed through the moderation detail. BR-MOD-06: The detail view shall display the AI moderation score and categories as visual indicators. BR-MOD-07: The Admin must be able to approve or reject the post from the detail view.
Other Information:	The detail is shown in a modal dialog on the frontend, not a separate page.
Assumptions:	The post data is consistent and not modified during the review process.

2.2.3.3 UC-06.3 Resolve Moderation

UC ID and Name	UC-06.3 Resolve Moderation		
Created By:	Pham Kha Vy	Date Created:	07/10/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks the “Approve” or “Reject” button on a moderation item detail.		
Description:	Allows an administrator to resolve a moderation item by approving or rejecting the post. Approval makes the post publicly visible; rejection soft-deletes the post and moves it to the trash.		
Preconditions:	**PRE-1** The user is authenticated and has the Admin role. **PRE-2** The post exists with a PENDING moderation status.		
Post-conditions:	**POST-1** If approved, the post’s `moderationStatus` is set to APPROVED and it becomes publicly visible. **POST-2** If rejected, the post’s `moderationStatus` is set to REJECTED, it is soft-deleted, and moved to the trash. **POST-3** The post is removed from the moderation queue.		
Normal Flow:	**A. Approve Post** 1. The Admin clicks the “Approve” button on the moderation item detail. 2. The system verifies that the user has the Admin role. 3. The system updates the post: sets `moderationStatus = APPROVED`, records the admin’s ID in `reviewedById`, and sets `reviewedAt` to the current timestamp. 4. The system returns the updated post. 5. The post is removed from the moderation queue. 6. The post becomes publicly visible on the forum.		

Alternative Flows:	**A.1 Reject Post** 1. The Admin clicks the “Reject” button on the moderation item detail. 2. The system verifies that the user has the Admin role. 3. The system updates the post: sets `moderationStatus = REJECTED`, records the admin’s ID in `reviewedById`, sets `reviewedAt` to the current timestamp, and sets `deletedAt` to the current timestamp. 4. The system returns the updated post. 5. The post is removed from the moderation queue and moved to the trash. **A.3 Post Not Found or No Longer Pending** 1. The Admin confirms the approve or reject action. 2. The system determines that the post does not exist, has been deleted, or is no longer in PENDING status. 3. The system displays the message “Post not found or no longer pending.” 4. Return to the moderation queue.

Exceptions:	**EX-01: Unauthorized Access** 1. A user without the Admin role attempts to approve or reject a post. 2. The system determines that the user does not have the Admin role. 3. The system returns a 403 Forbidden error. 4. The use case ends. **EX-02: Database Error** 1. The system attempts to update the post's moderation status in the database. 2. The database operation fails due to an internal database error. 3. The system displays the message "Failed to update forum post." 4. The post remains in PENDING status. 5. The use case ends. **EX-03: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the approval or rejection. 2. The system cannot save the updated moderation status. 3. The system displays the message "Cannot connect to the server. Please try again later." 4. The post remains in PENDING status. 5. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-MOD-08: Only the Admin role may approve or reject a post. BR-MOD-09: Approval shall set `moderationStatus` to APPROVED and make the post publicly visible. BR-MOD-10: Rejection shall set `moderationStatus` to REJECTED, soft-delete the post, and move it to the trash. BR-MOD-11: The `reviewedById` and `reviewedAt` fields shall record which admin reviewed the post and when. BR-MOD-12: A rejected post may later be restored from the trash, which returns it to PENDING status (UC-07.3). BR-MOD-13: Once a post is approved or rejected, it cannot be re-moderated unless it is restored from trash.

Other Information:	Both approve and reject use the same repository update with a WHERE clause of `moderationStatus = PENDING AND deletedAt = NULL`. No WebSocket events are emitted for moderation actions.
Assumptions:	The post has not been modified by the author during the review process.

2.2.3.4 UC-06.4 View Reports Queue

UC ID and Name	UC-06.4 View Reports Queue		
Created By:	Pham Kha Vy	Date Created:	08/10/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin accesses the Reports section from the admin navigation menu.		
Description:	Allows an administrator to view all user-submitted reports on forum posts and comments, filter by status, search by reason, and update report statuses.		
Preconditions:	**PRE-1** The user is authenticated and has the Admin role. **PRE-2** There are forum reports in the system.		
Post-conditions:	**POST-1** A paginated list of reports is displayed with reporter info, target (post or comment), reason, status, and date. **POST-2** The Admin can filter reports by status and search by reason. **POST-3** The Admin can view report details and update the report status.		

Normal Flow:	**A. View Reports Queue** 1. The Admin navigates to the Reports page from the admin sidebar. 2. The system retrieves all forum reports with pagination, optionally filtered by status and search term. 3. For each report, the system loads the reporter's profile, the target post or comment, and the admin who handled it (if any). 4. The system displays the reports as a table with columns: ID, Reporter, Reason (truncated to 40 characters), Target (post title or comment content), Status (as a badge), and Date. 5. The Admin can filter by status using tabs: All, Pending, Reviewed, Resolved, Dismissed. 6. The Admin can search reports by reason using the search bar.
Alternative Flows:	**A.2 No Reports Found** 1. The Admin navigates to the Reports page. 2. The system determines that no reports match the current filter or search criteria. 3. The system displays a message indicating no reports were found. 4. The use case ends.

Exceptions:	**EX-01: Unauthorized Access** 1. A user without the Admin role attempts to access the Reports page. 2. The system determines that the user does not have the Admin role. 3. The system displays the message “You do not have permission to access this page.” 4. The use case ends. **EX-02: Database Error** 1. The system attempts to retrieve the reports from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load reports. Please try again later.” 4. The reports list is not displayed. 5. The use case ends. **EX-03: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the reports data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The Admin remains on the current page without any reports data being displayed. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-MOD-14: Only users with the Admin role may access the reports queue. BR-MOD-15: Reports may have one of four statuses: PENDING, REVIEWED, RESOLVED, or DISMISSED. BR-MOD-16: The search filter shall perform a case-insensitive match on the report reason. BR-MOD-17: Reports are ordered by `createdAt DESC`.
Other Information:	The reports queue is separate from the moderation queue. The moderation queue handles new post approval, while the reports queue handles user-flagged content.
Assumptions:	At least one report exists in the system.

2.2.3.5 UC-06.5 Update Report Status

UC ID and Name	UC-06.5 Update Report Status		
Created By:	Pham Kha Vy	Date Created:	08/10/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin changes the status of a report from the reports queue using the inline status dropdown.		
Description:	Allows an administrator to update the status of a user-submitted report (e.g., from PENDING to REVIEWED, RESOLVED, or DISMISSED).		
Preconditions:	**PRE-1** The user is authenticated and has the Admin role. **PRE-2** The report exists in the system.		
Post-conditions:	**POST-1** The report's `status` is updated to the new value. **POST-2** The admin's ID is recorded in the `adminId` field.		
Normal Flow:	**A. Update Report Status** 1. The Admin selects a new status from the status dropdown on a report row. 2. The system verifies that the user has the Admin role. 3. The system retrieves the report by ID. 4. The system verifies that the report is associated with a post (postId is not null). 5. The system updates the report's `status` and records the admin's ID in `adminId`. 6. The system returns the updated report. 7. The reports list refreshes to reflect the new status.		
Alternative Flows:	**A.2 Report Not Found** 1. The Admin attempts to update the status of a report. 2. The system determines that the report does not exist. 3. The system displays the message "Report not found." 4. Return to the reports queue. **A.4 Report Not Associated With a Post** 1. The Admin attempts to update the status of a report. 2. The system determines that the report is not associated with a post (postId is null). 3. The system displays the message "Report not found." 4. Return to the reports queue.		

Exceptions:	**EX-01: Unauthorized Access** 1. A user without the Admin role attempts to update a report status. 2. The system determines that the user does not have the Admin role. 3. The system returns a 403 Forbidden error. 4. The use case ends. **EX-02: Database Error** 1. The system attempts to update the report status in the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Failed to update report status.” 4. The report status is not updated. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-MOD-18: Only the Admin role may update a report status. BR-MOD-19: Valid status transitions are: PENDING to REVIEWED, REVIEWED to RESOLVED or DISMISSED, RESOLVED to REVIEWED, DISMISSED to REVIEWED. BR-MOD-20: The `adminId` field shall record which admin updated the status. BR-MOD-21: Report status updates are performed via REST only (no WebSocket events).
Other Information:	The status update is performed inline in the reports table without a detail page.
Assumptions:	The report data is consistent and not modified during the update process.

2.2.19 Manage ID Verification

2.2.3.1 UC-06.6 View ID Verification Requests

UC ID and Name	UC-06.6 View ID Verification Requests		
Created By:	Pham Kha Vy	Date Created:	09/10/2026
Primary Actor:	Admin	Secondary Actors:	N/A

Trigger:	When an admin accesses the Identity Verification section from the admin navigation menu.
Description:	Allows an administrator to view all identity verification requests submitted by users, filter by status, and search by user name or email.
Preconditions:	**PRE-1** The user is authenticated and has the Admin role. **PRE-2** There are identity verification requests in the system.
Post-conditions:	**POST-1** A paginated list of verification requests is displayed with user info, document type, status, and submission date. **POST-2** The Admin can select a request to view its full details. **POST-3** The Admin can approve or reject each request.
Normal Flow:	**A. View Verification Requests** 1. The Admin navigates to the Identity Verification page from the admin sidebar. 2. The system retrieves all identity verification requests where `deletedAt = NULL`, optionally filtered by status and search term. 3. For each request, the system loads the user's profile (avatar, display name, email), document type, status, and creation date. 4. The system displays the requests as a table with columns: ID, User (avatar + name + email), Document Type, Status (as a color-coded badge), and Submitted Date. 5. The Admin can filter by status using tabs: All, Pending, Approved, Rejected. 6. The Admin can search by user name or email using the search bar.
Alternative Flows:	**A.2 No Requests Found** 1. The Admin navigates to the Identity Verification page. 2. The system determines that no verification requests match the current filter or search criteria. 3. The system displays a message indicating no requests were found. 4. The use case ends.

Exceptions:	**EX-01: Unauthorized Access** 1. A user without the Admin role attempts to access the Identity Verification page. 2. The system determines that the user does not have the Admin role. 3. The system displays the message “You do not have permission to access this page.” 4. The use case ends. **EX-02: Database Error** 1. The system attempts to retrieve the verification requests from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load verification requests. Please try again later.” 4. The requests list is not displayed. 5. The use case ends. **EX-03: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the verification data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The Admin remains on the current page without any verification data being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-IDV-01: Only non-deleted (`deletedAt = NULL`) verification requests shall be displayed. BR-IDV-02: Document types include PASSPORT, ID_CARD, DRIVER_LICENSE, RESIDENCE_PERMIT, and OTHER. BR-IDV-03: Verification statuses include PENDING, APPROVED, and REJECTED. BR-IDV-04: The search filter shall perform a case-insensitive match on the user’s email and display name. BR-IDV-05: Requests are ordered by `createdAt DESC`.

Other Information:	File URLs are transformed for the admin UI: local files use a proxy endpoint (`/api/backend/admin/identity-verifications/:id/file`), external URLs (e.g., Cloudinary) are returned as-is.
Assumptions:	At least one verification request exists in the system.

2.2.3.2 UC-06.7 ID Verification Detail

UC ID and Name	UC-06.7 ID Verification Detail		
Created By:	Pham Kha Vy	Date Created:	09/10/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks the "View" action on a verification request in the list.		
Description:	Allows an administrator to view the complete details of an identity verification request, including the uploaded document image, user profile information, and existing review notes.		
Preconditions:	**PRE-1** The user is authenticated and has the Admin role. **PRE-2** The verification request exists and has not been deleted.		
Post-conditions:	**POST-1** The full verification detail is displayed including document image, user info, document type, status, review notes, and submission date. **POST-2** If the request is PENDING, the Admin can approve or reject it.		
Normal Flow:	**A. View Verification Detail** 1. The Admin clicks the "View" action on a verification request. 2. The system retrieves the full verification detail including: user info (id, email, createdAt, profile with display name, avatar, bio, and freelancer profile with title and idVerified status), admin info (if reviewed), document type, file URL, status, review notes, creation date, and review date. 3. The system displays a detail dialog showing all verification information. 4. The system displays the document image using the file URL. 5. The system displays the user's freelancer profile verification status. 6. If the request is PENDING, the system displays a textarea for review notes along with Approve and Reject buttons.		

Alternative Flows:	**A.2 Request Not Found** 1. The Admin clicks the “View” action on a verification request. 2. The system determines that the request does not exist or has been deleted. 3. The system displays the message “Identity verification not found.” 4. Return to the verification requests list.
Exceptions:	**EX-01: Unauthorized Access** 1. A user without the Admin role attempts to view a verification detail. 2. The system determines that the user does not have the Admin role. 3. The system displays the message “You do not have permission to access this page.” 4. The use case ends. **EX-02: Database Error** 1. The system attempts to retrieve the verification detail from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load verification detail. Please try again later.” 4. The detail view is not displayed. 5. The use case ends. **EX-03: Document File Not Available** 1. The system attempts to retrieve the document file. 2. The file is not available or the path is invalid. 3. The system displays a placeholder or error message for the document image. 4. The Admin can still view other detail information and perform approve/reject actions.
Priority:	High
Frequency of Use:	Medium

Business Rules:	BR-IDV-06: The detail view shall display the user's freelancer profile `idVerified` status. BR-IDV-07: Local document files shall be served through a secure proxy endpoint with path traversal protection. BR-IDV-08: The review notes field shall accept up to 1000 characters. BR-IDV-09: Only PENDING requests shall display the Approve and Reject buttons.
Other Information:	The detail is shown in a modal dialog on the frontend. The document file proxy endpoint validates the file path to prevent path traversal attacks.
Assumptions:	The uploaded document file is accessible and not corrupted.

2.2.3.3 UC-06.8 Approve ID Verification

UC ID and Name	UC-06.8 Approve ID Verification		
Created By:	Pham Kha Vy	Date Created:	10/10/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks the "Approve" button on the identity verification detail.		
Description:	Allows an administrator to approve an identity verification request, setting the user's freelancer profile `idVerified` flag to true.		
Preconditions:	**PRE-1** The user is authenticated and has the Admin role. **PRE-2** The verification request exists with a PENDING status.		
Post-conditions:	**POST-1** The verification request's `status` is set to APPROVED. **POST-2** The reviewing admin's ID and review timestamp are recorded. **POST-3** The user's freelancer profile `idVerified` is set to true. **POST-4** The request is removed from the pending queue.		

Normal Flow:	**A. Approve Verification** 1. The Admin clicks the “Approve” button on the verification detail. 2. The Admin optionally enters review notes (up to 1000 characters). 3. The system verifies that the verification request exists with PENDING status. 4. The system starts a database transaction. 5. The system updates the verification request: sets `status = APPROVED`, records the admin’s ID in `adminId`, sets `reviewedAt` to the current timestamp, and saves the review notes. 6. The system looks up the user’s freelancer profile. 7. If a freelancer profile exists, the system sets `idVerified = true`. 8. The system commits the transaction. 9. The system returns the updated verification detail. 10. The request is no longer visible in the PENDING queue.
Alternative Flows:	**A.3a Request Not Found** 1. The Admin confirms the approval. 2. The system determines that the verification request does not exist or has been deleted. 3. The system displays the message “Identity verification not found.” 4. Return to the verification requests list. **A.3b Request Already Reviewed** 1. The Admin confirms the approval. 2. The system determines that the verification request has already been reviewed (status is not PENDING). 3. The system displays the message “Identity verification has already been reviewed.” 4. Return to the verification detail.

Exceptions:	**EX-01: Unauthorized Access** 1. A user without the Admin role attempts to approve a verification request. 2. The system determines that the user does not have the Admin role. 3. The system returns a 403 Forbidden error. 4. The use case ends. **EX-02: Database Error** 1. The system attempts to update the verification request and freelancer profile in the database. 2. The database operation fails due to an internal database error. 3. The system rolls back the transaction. 4. The system displays the message “Failed to update identity verification.” 5. The verification status remains PENDING. 6. The use case ends. **EX-03: Transaction Timeout** 1. The database transaction exceeds the 15-second timeout. 2. The system rolls back the transaction. 3. The system displays the message “Failed to update identity verification. Please try again later.” 4. The verification status remains PENDING. 5. The use case ends.
Priority:	High
Frequency of Use:	Low
Business Rules:	BR-IDV-10: Only the Admin role may approve a verification request. BR-IDV-11: Only requests with `status = PENDING` may be approved. BR-IDV-12: Approval shall set the user’s freelancer profile `idVerified = true`. BR-IDV-13: The approve and reject operations shall be performed within a database transaction to ensure atomicity. BR-IDV-14: The transaction timeout is 15 seconds with a max wait of 10 seconds. BR-IDV-15: If the user does not have a freelancer profile, only the verification status is updated (no idVerified change).

Other Information:	The approve operation uses a Prisma interactive transaction to update both the verification document and the freelancer profile atomically.
Assumptions:	The user's freelancer profile exists and is accessible during the approval process.

2.2.3.4 UC-06.9 Reject ID Verification

UC ID and Name	UC-06.9 Reject ID Verification		
Created By:	Pham Kha Vy	Date Created:	11/10/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks the "Reject" button on the identity verification detail.		
Description:	Allows an administrator to reject an identity verification request, setting the user's freelancer profile `idVerified` flag to false.		
Preconditions:	**PRE-1** The user is authenticated and has the Admin role. **PRE-2** The verification request exists with a PENDING status.		
Post-conditions:	**POST-1** The verification request's `status` is set to REJECTED. **POST-2** The reviewing admin's ID and review timestamp are recorded. **POST-3** The user's freelancer profile `idVerified` is set to false. **POST-4** The request is removed from the pending queue.		

Normal Flow:	**A. Reject Verification** 1. The Admin clicks the “Reject” button on the verification detail. 2. The Admin optionally enters review notes (up to 1000 characters). 3. The system verifies that the verification request exists with PENDING status. 4. The system starts a database transaction. 5. The system updates the verification request: sets `status = REJECTED`, records the admin’s ID in `adminId`, sets `reviewedAt` to the current timestamp, and saves the review notes. 6. The system looks up the user’s freelancer profile. 7. If a freelancer profile exists, the system sets `idVerified = false`. 8. The system commits the transaction. 9. The system returns the updated verification detail. 10. The request is no longer visible in the PENDING queue.
Alternative Flows:	**A.3a Request Not Found** 1. The Admin confirms the rejection. 2. The system determines that the verification request does not exist or has been deleted. 3. The system displays the message “Identity verification not found.” 4. Return to the verification requests list. **A.3b Request Already Reviewed** 1. The Admin confirms the rejection. 2. The system determines that the verification request has already been reviewed (status is not PENDING). 3. The system displays the message “Identity verification has already been reviewed.” 4. Return to the verification detail.

Exceptions:	**EX-01: Unauthorized Access** 1. A user without the Admin role attempts to reject a verification request. 2. The system determines that the user does not have the Admin role. 3. The system returns a 403 Forbidden error. 4. The use case ends. **EX-02: Database Error** 1. The system attempts to update the verification request and freelancer profile in the database. 2. The database operation fails due to an internal database error. 3. The system rolls back the transaction. 4. The system displays the message “Failed to update identity verification.” 5. The verification status remains PENDING. 6. The use case ends. **EX-03: Transaction Timeout** 1. The database transaction exceeds the 15-second timeout. 2. The system rolls back the transaction. 3. The system displays the message “Failed to update identity verification. Please try again later.” 4. The verification status remains PENDING. 5. The use case ends.
Priority:	High
Frequency of Use:	Low
Business Rules:	BR-IDV-16: Only the Admin role may reject a verification request. BR-IDV-17: Only requests with `status = PENDING` may be rejected. BR-IDV-18: Rejection shall set the user’s freelancer profile `idVerified = false`. BR-IDV-19: The reject operation shall be performed within a database transaction to ensure atomicity. BR-IDV-20: If the user does not have a freelancer profile, only the verification status is updated (no idVerified change). BR-IDV-21: A user may re-submit a verification request after rejection (the system does not prevent re-submission).

Other Information:	The reject operation uses the same Prisma interactive transaction pattern as approve. Rejection explicitly sets `idVerified = false`, unlike approve which sets it to true.
Assumptions:	The user's freelancer profile exists and is accessible during the rejection process.

2.2.20 Manage Forum Trash

2.2.3.1 UC-06.10 View Deleted Posts / Comments List

UC ID and Name	UC-06.10 View Deleted Posts / Comments List		
Created By:	Pham Kha Vy	Date Created:	12/10/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin accesses the Trash section from the admin navigation menu.		
Description:	Allows an administrator to view all soft-deleted forum posts and comments, including posts that were rejected by moderators. The admin can toggle between the Posts and Comments tabs.		
Preconditions:	**PRE-1** The user is authenticated and has the Admin role. **PRE-2** There are soft-deleted posts or comments in the system.		
Post-conditions:	**POST-1** A paginated list of deleted posts is displayed with title, status, author, and deletion date. **POST-2** A paginated list of deleted comments is displayed with content, post title, author, and deletion date. **POST-3** The Admin can select an item to view its full details. **POST-4** The Admin can restore items from trash.		

Normal Flow:	**A. View Deleted Posts List** 1. The Admin navigates to the Trash page from the admin sidebar. 2. The system retrieves all forum posts where `deletedAt IS NOT NULL` or `moderationStatus = REJECTED`, ordered by `deletedAt DESC`. 3. For each post, the system loads the author's profile and the moderation status. 4. The system displays the deleted posts as a table with columns: ID, Title, Status badge (REJECTED, PENDING, or APPROVED), Author (avatar + name), and Deleted On date. 5. The Admin can click the "Comments" tab to view deleted comments.
Alternative Flows:	**A.1 View Deleted Comments List** 1. The Admin clicks the "Comments" tab on the Trash page. 2. The system retrieves all forum comments where `deletedAt IS NOT NULL`, ordered by `deletedAt DESC`. 3. For each comment, the system loads the author's profile and the associated post. 4. The system displays the deleted comments as a table with columns: ID, Content (truncated), Post Title, Author (avatar + name), and Deleted On date. **A.2 No Deleted Items Found** 1. The Admin navigates to the Trash page. 2. The system determines that no deleted posts or comments exist. 3. The system displays a message indicating the trash is empty. 4. The use case ends.

Exceptions:	**EX-01: Unauthorized Access** 1. A user without the Admin role attempts to access the Trash page. 2. The system determines that the user does not have the Admin role. 3. The system displays the message “You do not have permission to access this page.” 4. The use case ends. **EX-02: Database Error** 1. The system attempts to retrieve the deleted items from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load trash items. Please try again later.” 4. The trash list is not displayed. 5. The use case ends. **EX-03: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the trash data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The Admin remains on the current page without any trash data being displayed. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-TRASH-01: The posts trash shall include both user-deleted posts (`deletedAt IS NOT NULL`) and admin-rejected posts (`moderationStatus = REJECTED`). BR-TRASH-02: The comments trash shall include only user-deleted comments (`deletedAt IS NOT NULL`). BR-TRASH-03: Deleted items are ordered by `deletedAt DESC` (most recently deleted first). BR-TRASH-04: Posts in the trash may have their original moderation status displayed as a badge. BR-TRASH-05: Only users with the Admin role may access the trash.

Other Information:	The trash page uses server-side rendering for the initial data load. The Posts and Comments tabs are managed client-side.
Assumptions:	At least one deleted post or comment exists in the system.

2.2.3.2 UC-06.11 View Deleted Posts / Comments Detail

UC ID and Name	UC-06.11 View Deleted Posts / Comments Detail		
Created By:	Pham Kha Vy	Date Created:	12/10/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks the “View” action on a deleted post or comment in the trash list.		
Description:	Allows an administrator to view the complete content and metadata of a deleted post or comment before deciding whether to restore it.		
Preconditions:	**PRE-1** The user is authenticated and has the Admin role. **PRE-2** The deleted post or comment exists in the trash.		
Post-conditions:	**POST-1** The full content of the deleted item is displayed. **POST-2** The Admin can restore the item.		
Normal Flow:	**A. View Deleted Post Detail** 1. The Admin clicks the “View” action on a deleted post in the trash. 2. The system retrieves the full post content including title, content body, author profile, moderation status, moderation score, moderation categories, creation date, and deletion date. 3. The system displays a detail dialog showing all post information. 4. The system displays the Restore button. 5. If the post was rejected, the system displays a note: “Restoring this post will return it to the moderation queue for review.”		

Alternative Flows:	**A.1 View Deleted Comment Detail** 1. The Admin clicks the “View” action on a deleted comment in the trash. 2. The system retrieves the full comment content including content body, author profile, associated post, creation date, and deletion date. 3. The system displays a detail dialog showing all comment information. 4. The system displays the Restore button. **A.2 Item Not Found** 1. The Admin clicks the “View” action on a deleted item. 2. The system determines that the item does not exist or is no longer in the trash. 3. The system displays the message “Item not found in trash.” 4. Return to the trash list.
Exceptions:	**EX-01: Unauthorized Access** 1. A user without the Admin role attempts to view a deleted item detail. 2. The system determines that the user does not have the Admin role. 3. The system displays the message “You do not have permission to access this page.” 4. The use case ends. **EX-02: Database Error** 1. The system attempts to retrieve the deleted item detail from the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Unable to load item detail. Please try again later.” 4. The detail view is not displayed. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Low

Business Rules:	BR-TRASH-06: The detail view shall display the original moderation status for posts. BR-TRASH-07: Posts with `moderationStatus = REJECTED` shall display a note indicating restoration returns them to the moderation queue. BR-TRASH-08: The Admin must be able to restore the item from the detail view.
Other Information:	The detail is shown in a modal dialog on the frontend.
Assumptions:	The deleted item data is consistent and not modified during the view process.

2.2.3.3 UC-06.12 Restore Deleted Posts / Comments

UC ID and Name	UC-06.12 Restore Deleted Posts / Comments		
Created By:	Pham Kha Vy	Date Created:	13/10/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks the “Restore” button on a deleted post or comment detail.		
Description:	Allows an administrator to restore a soft-deleted post or comment from the trash. Restored posts are either made immediately public (if previously approved) or returned to the moderation queue (if previously rejected).		
Preconditions:	**PRE-1** The user is authenticated and has the Admin role. **PRE-2** The post or comment exists in the trash (has been soft-deleted or rejected).		
Post-conditions:	**POST-1** The item’s `deletedAt` field is set to null. **POST-2** If the post was REJECTED, its `moderationStatus` is reset to PENDING and the `reviewedById` and `reviewedAt` fields are cleared. **POST-3** If the post was previously APPROVED, it becomes publicly visible immediately. **POST-4** The item is removed from the trash list.		

Normal Flow:	**A. Restore Post** 1. The Admin clicks the “Restore” button on the deleted post detail. 2. The system verifies that the post exists in the trash (`deletedAt IS NOT NULL` or `moderationStatus = REJECTED`). 3. The system sets `deletedAt = null`. 4. If the original `moderationStatus` was REJECTED, the system also sets `moderationStatus = PENDING`, `reviewedById = null`, and `reviewedAt = null`. 5. The system returns the restored post. 6. If the post was previously APPROVED, it becomes publicly visible immediately. 7. If the post was REJECTED, it re-appears in the moderation queue (UC-06.1).
Alternative Flows:	**A.1 Restore Comment** 1. The Admin clicks the “Restore” button on the deleted comment detail. 2. The system verifies that the comment exists in the trash (`deletedAt IS NOT NULL`). 3. The system sets `deletedAt = null`. 4. The system returns the restored comment. 5. The comment becomes visible again in the associated post. **A.2 Item Not Found in Trash** 1. The Admin confirms the restore action. 2. The system determines that the item does not exist in the trash or has already been restored. 3. The system displays the message “Item not found in trash.” 4. Return to the trash list.

Exceptions:	**EX-01: Unauthorized Access** 1. A user without the Admin role attempts to restore an item from trash. 2. The system determines that the user does not have the Admin role. 3. The system returns a 403 Forbidden error. 4. The use case ends. **EX-02: Database Error** 1. The system attempts to update the `deletedAt` field in the database. 2. The database operation fails due to an internal database error. 3. The system displays the message “Failed to update forum post.” or “Failed to update forum comment.” 4. The item remains in the trash. 5. The use case ends. **EX-03: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the restore request. 2. The system cannot save the restored item. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The item remains in the trash. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-TRASH-09: Only the Admin role may restore items from trash. BR-TRASH-10: Restoring a previously APPROVED post (soft-deleted by user) shall make it immediately public. BR-TRASH-11: Restoring a REJECTED post shall reset its `moderationStatus` to PENDING and clear the review metadata, returning it to the moderation queue. BR-TRASH-12: Restoring a comment shall make it visible again in the associated post. BR-TRASH-13: There is no permanent deletion endpoint – items in trash can only be restored.

Other Information:	The restore operation for posts uses a conditional logic: if the post was REJECTED, it goes back to PENDING for re-review. If it was APPROVED and soft-deleted by the user, clearing `deletedAt` makes it public immediately. Comments always become visible upon restore.
Assumptions:	The item has not been permanently removed from the database.

2.2.21 Manage Proposal

2.2.3.1 UC View My Proposals

UC ID and Name	UC View My Proposals		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer accesses the “My Proposals” page from the navigation menu or profile.		
Description:	Allows a freelancer to retrieve the list of all proposals they have submitted across jobs, with their status and the associated job.		
Preconditions:	**PRE-1** The freelancer is authenticated. **PRE-2** The system is connected to the database.		
Post-conditions:	**POST-1** A paginated list of the freelancer’s proposals is displayed, each showing the associated job and its status. **POST-2** No data is changed by this use case. **POST-3** The freelancer can select a proposal to view its detail (UC View My Proposal Detail) or navigate between pages.		
Normal Flow:	**A. View My Proposals** 1. The Freelancer opens the My Proposals page. 2. The system retrieves the proposals submitted by the Freelancer. 3. The system displays the proposal list, each item showing the associated job and its status (pending, accepted, rejected, withdrawn). The use case ends.		

Alternative Flows:	**A.1 Filter by Status** *(branch at step 3)* 1. The Freelancer selects a status filter (pending, accepted, rejected, withdrawn). 2. The system refreshes the displayed list with only the proposals of the selected status. 3. Return to step 3 of the main flow. **A.2 No Proposals Found** *(branch at step 2)* 1. The system determines that the freelancer has no submitted proposals. 2. The system displays a message indicating no proposals are available. 3. The use case ends.
Exceptions:	**EX-01: System Error While Loading the List** *(branch at step 2)* 1. The system cannot retrieve the proposals due to a processing or connection error. 2. The system displays the message "Couldn't load your proposals. Please try again later." 3. The freelancer remains on the current page. 4. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-PROP-01: A freelancer can only view their own proposals. BR-PROP-02: Deleted proposals shall not be displayed. BR-PROP-03: The list shall support pagination with a default page size of 10.
Other Information:	The list may be sorted by submission time, newest first.
Assumptions:	A freelancer may have zero or more submitted proposals over time.

2.2.3.2 UC View My Proposal Detail

UC ID and Name	UC View My Proposal Detail		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer opens the detail of their proposal from the My Proposals list or navigates to it directly.		

Description:	Allows a freelancer to view the full details of a submitted proposal, including cover letter, bid amount, estimated duration, attachments, job information, and current status.
Preconditions:	**PRE-1** The freelancer is authenticated.
Post-conditions:	**POST-1** The proposal details are displayed: cover letter, bid amount, estimated duration, attachments, job information, and status. **POST-2** No data is changed by this use case. **POST-3** Depending on the status, the freelancer can edit (UC Edit Proposal) or withdraw (UC Withdraw Proposal) the proposal.
Normal Flow:	**A. View My Proposal Detail** 1. The Freelancer opens a proposal detail. 2. The system retrieves the proposal together with its job information. 3. The system displays the proposal with its cover letter, bid amount, estimated duration, attachments, job information, and current status. The use case ends.
Alternative Flows:	**A.1 Edit Proposal** *(branch at step 3)* 1. The Freelancer chooses the edit action. 2. Continue with UC Edit Proposal. **A.2 Withdraw Proposal** *(branch at step 3)* 1. The Freelancer chooses the withdraw action. 2. Continue with UC Withdraw Proposal.
Exceptions:	**EX-01: Proposal Not Found** *(branch at step 2)* 1. The system cannot find the proposal for the freelancer. 2. The system displays a “Not Found” page. 3. The use case ends. **EX-02: System Error While Loading the Detail** *(branch at step 2)* 1. The system cannot load the proposal due to a processing or connection error. 2. The system displays the message “Couldn’t load the proposal. Please try again later.” 3. The freelancer remains on the current page. 4. The use case ends.
Priority:	Medium
Frequency of Use:	Medium

Business Rules:	BR-PROP-04: The detail shall be accessible only by the owner of the proposal. BR-PROP-05: A proposal is considered not found if it does not exist or belongs to another user.
Other Information:	The detail also shows the job title, budget range, deadline, and client display name.
Assumptions:	The proposal detail is only reachable by the owning freelancer.

2.2.3.3 UC Submit Proposal

UC ID and Name	UC Submit Proposal		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer chooses to respond to a job that is open for proposals.		
Description:	Allows a freelancer to submit a proposal for a job, providing a cover letter, bid amount, estimated duration, and optional attachments.		
Preconditions:	**PRE-1** The freelancer is authenticated. **PRE-2** The freelancer is on a job detail page that is open for proposals.		
Post-conditions:	**POST-1** A new proposal is created with status pending. **POST-2** The proposal appears in the freelancer's My Proposals list. **POST-3** The submitted proposal becomes visible to the client.		
Normal Flow:	**A. Submit Proposal** 1. The Freelancer chooses to submit a proposal for a job. 2. The Freelancer enters the cover letter, the bid amount, and the estimated duration, and optionally attaches files. 3. The Freelancer confirms the submission. 4. The system validates the entered data. 5. The system creates the proposal with status pending. 6. The system confirms the submission to the Freelancer. The use case ends.		

Alternative Flows:	**A.1 Cancel Submit** *(branch at step 3)* 1. The Freelancer cancels the submission. 2. The system takes no action. 3. The use case ends. **A.2 Validation Failed** *(branch at step 4)* 1. The system determines that the entered data is invalid or incomplete. 2. The system displays an appropriate error message. 3. Return to step 2 of the main flow.
Exceptions:	**EX-01: Duplicate Proposal** *(branch at step 5)* 1. The system detects that the freelancer already has an active proposal for this job. 2. The system displays a message indicating that a proposal for this job already exists. 3. The use case ends. **EX-02: Job Not Open** *(branch at step 5)* 1. The system determines that the job no longer exists or is no longer open for proposals. 2. The system displays an appropriate error message. 3. The use case ends. **EX-03: System Error While Submitting** *(branch at step 5)* 1. The system cannot create the proposal due to a processing or connection error. 2. The system displays the message “Couldn’t submit the proposal. Please try again later.” 3. The freelancer remains on the current page without the proposal being created. 4. The use case ends.
Priority:	High
Frequency of Use:	High

Business Rules:	BR-PROP-06: Proposal status is pending by default upon submission. BR-PROP-07: Cover letter is required and must not exceed 5000 characters. BR-PROP-08: Bid amount is required and must be a positive number. BR-PROP-09: Estimated duration is required and must be a positive integer (days). BR-PROP-10: A freelancer can submit only one active proposal per job.
Other Information:	Attachments are optional and are uploaded together with the proposal.
Assumptions:	The freelancer is a verified registered freelancer with an active account.

2.2.3.4 UC Edit Proposal

UC ID and Name	UC Edit Proposal		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer chooses to edit their proposal from the My Proposals list or detail page.		
Description:	Allows the owner of a proposal to update its details (cover letter, bid amount, estimated duration) only while the proposal status is pending.		
Preconditions:	**PRE-1** The freelancer is authenticated.		
Post-conditions:	**POST-1** The proposal details are updated. **POST-2** The updated proposal is reflected in the freelancer's My Proposals list.		

Normal Flow:	**A. Edit Proposal** 1. The Freelancer selects the edit option on a pending proposal. 2. The system presents the edit form pre-filled with the current values. 3. The Freelancer updates the cover letter, bid amount, and/or estimated duration. 4. The Freelancer confirms the changes. 5. The system validates the updated data. 6. The system updates the proposal. 7. The system confirms the update to the Freelancer. The use case ends.
Alternative Flows:	**A.1 Cancel Edit** *(branch at step 4)* 1. The Freelancer cancels the edit. 2. The system takes no action. 3. The use case ends. **A.2 Validation Failed** *(branch at step 5)* 1. The system determines that the updated data is invalid or incomplete. 2. The system displays an appropriate error message. 3. Return to step 3 of the main flow.

Exceptions:	**EX-01: Proposal Not Found** *(branch at step 6)* 1. The system cannot find the proposal for the freelancer. 2. The system displays a “Not Found” page. 3. The system does not modify the proposal. 4. The use case ends. **EX-02: Proposal Not Editable** *(branch at step 6)* 1. The system determines that the proposal status is no longer pending (accepted, rejected, or withdrawn). 2. The system displays a message indicating the proposal can no longer be edited. 3. The use case ends. **EX-03: System Error While Updating** *(branch at step 6)* 1. The system cannot update the proposal due to a processing or connection error. 2. The system displays the message “Couldn’t update the proposal. Please try again later.” 3. The freelancer remains on the current page without the changes being saved. 4. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-PROP-11: Only the owner of a proposal can edit it. BR-PROP-12: A proposal can only be edited while its status is pending. BR-PROP-13: All validation rules of UC Submit Proposal apply to edits.
Other Information:	Editing is not allowed once the proposal has been accepted, rejected, or withdrawn.
Assumptions:	The freelancer can only edit proposals they own.

2.2.3.5 UC Withdraw Proposal

UC ID and Name	UC Withdraw Proposal		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	N/A
Trigger:	When a freelancer chooses to withdraw their proposal from the My Proposals list or detail page.		

Description:	Allows the owner of a proposal to withdraw it, changing its status to withdrawn. A proposal that has already been accepted cannot be withdrawn.
Preconditions:	**PRE-1** The freelancer is authenticated.
Post-conditions:	**POST-1** The proposal status is set to withdrawn. **POST-2** The proposal is no longer active, so the freelancer may submit a new proposal for the same job.
Normal Flow:	**A. Withdraw Proposal** 1. The Freelancer selects the withdraw option on a proposal that is not accepted. 2. The system asks for confirmation of the withdrawal. 3. The Freelancer confirms the withdrawal. 4. The system sets the proposal status to withdrawn. 5. The system confirms the withdrawal to the Freelancer. The use case ends.
Alternative Flows:	**A.1 Cancel Withdraw** *(branch at step 3)* 1. The Freelancer cancels the withdrawal. 2. The system takes no action. 3. The use case ends.
Exceptions:	**EX-01: Proposal Not Found** *(branch at step 4)* 1. The system cannot find the proposal for the freelancer. 2. The system displays a “Not Found” page. 3. The system does not modify the proposal. 4. The use case ends. **EX-02: Proposal Cannot Be Withdrawn** *(branch at step 4)* 1. The system determines that the proposal has already been accepted. 2. The system displays a message indicating an accepted proposal cannot be withdrawn. 3. The use case ends. **EX-03: System Error While Withdrawing** *(branch at step 4)* 1. The system cannot update the proposal due to a processing or connection error. 2. The system displays the message “Couldn’t withdraw the proposal. Please try again later.” 3. The freelancer remains on the current page without the proposal being withdrawn. 4. The use case ends.

Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-PROP-14: Only the owner of a proposal can withdraw it. BR-PROP-15: A proposal cannot be withdrawn once it has been accepted. BR-PROP-16: Withdrawal changes the proposal status to withdrawn.
Other Information:	A withdrawn proposal frees the freelancer to submit a new proposal for the same job if it is still open.
Assumptions:	The freelancer can only withdraw proposals they own.

2.2.3.6 UC View Job Proposals

UC ID and Name	UC View Job Proposals		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	N/A
Trigger:	When a client opens the proposals of a job they own from the job detail page or job management page.		
Description:	Allows a client to retrieve all proposals submitted for a specific job they own, including freelancer information, bid amount, cover letter, and status.		
Preconditions:	**PRE-1** The client is authenticated. **PRE-2** The system is connected to the database.		
Post-conditions:	**POST-1** A paginated list of proposals submitted for the job is displayed. **POST-2** No data is changed by this use case. **POST-3** The client can select a proposal to view its detail (UC View Proposal Detail), accept (UC Accept Proposal), or reject (UC Reject Proposal) it.		
Normal Flow:	**A. View Job Proposals** 1. The Client opens the proposals of a job they own. 2. The system retrieves the proposals submitted for the job. 3. The system displays the list, each item showing the freelancer information, bid amount, cover letter, and status (pending, accepted, rejected, withdrawn). The use case ends.		

Alternative Flows:	**A.1 Filter by Status** *(branch at step 3)* 1. The Client selects a status filter (pending, accepted, rejected, withdrawn). 2. The system refreshes the displayed list with only the proposals of the selected status. 3. Return to step 3 of the main flow. **A.2 No Proposals Found** *(branch at step 2)* 1. The system determines that no proposals have been submitted for the job. 2. The system displays a message indicating no proposals are available. 3. The use case ends.
Exceptions:	**EX-01: Job Not Found** *(branch at step 2)* 1. The system cannot find the job. 2. The system displays the message “The job could not be found.” 3. The use case ends. **EX-02: Job Deleted or Unavailable** *(branch at step 2)* 1. The system determines that the job is deleted or unavailable. 2. The system displays an appropriate error message. 3. The use case ends. **EX-03: Forbidden Access** *(branch at step 2)* 1. The system determines that the job belongs to another client. 2. The system displays a “Not Found” page. 3. The use case ends. **EX-04: System Error While Loading the List** *(branch at step 2)* 1. The system cannot retrieve the proposals due to a processing or connection error. 2. The system displays the message “Couldn’t load the proposals. Please try again later.” 3. The client remains on the current page. 4. The use case ends.
Priority:	High
Frequency of Use:	High

Business Rules:	BR-PROP-17: Only the client who owns the job can view its proposals. BR-PROP-18: Deleted proposals shall not be displayed. BR-PROP-19: The list shall support pagination with a default page size of 10.
Other Information:	The list includes each proposal's freelancer profile, bid amount, cover letter, and status.
Assumptions:	Only submitted proposals are listed.

2.2.3.7 UC View Proposal Detail

UC ID and Name	UC View Proposal Detail		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	N/A
Trigger:	When a client selects a proposal from a job's proposal list, or a freelancer selects one of their own proposals from the My Proposals list.		
Description:	Allows the job owner or the proposal owner to view the full detail of a proposal: cover letter, bid amount, estimated duration, freelancer profile, and attachments.		
Preconditions:	**PRE-1** The user is authenticated.		
Post-conditions:	**POST-1** The proposal detail is displayed: cover letter, bid amount, estimated duration, freelancer profile, and attachments. **POST-2** No data is changed by this use case. **POST-3** On the client side, the client can accept (UC Accept Proposal) or reject (UC Reject Proposal) the proposal; on the freelancer side, the freelancer can edit or withdraw (UC Edit Proposal / UC Withdraw Proposal) it.		
Normal Flow:	**A. View Proposal Detail** 1. The User selects a proposal from the relevant list. 2. The system retrieves the proposal together with the job and the associated freelancer/client profile. 3. The system verifies that the user is the job owner or the proposal owner. 4. The system displays the full proposal detail: cover letter, bid amount, estimated duration, freelancer profile, and attachments. The use case ends.		

Alternative Flows:	**A.1 Accept Proposal (Client)** *(branch at step 4)* 1. The Client chooses the accept action. 2. Continue with UC Accept Proposal. **A.2 Reject Proposal (Client)** *(branch at step 4)* 1. The Client chooses the reject action. 2. Continue with UC Reject Proposal.
Exceptions:	**EX-01: Proposal Not Found** *(branch at step 2)* 1. The system cannot find the proposal. 2. The system displays the message “Proposal not found.” 3. The use case ends. **EX-02: Job Unavailable** *(branch at step 2)* 1. The system determines that the associated job is deleted or unavailable. 2. The system displays an appropriate error message. 3. The use case ends. **EX-03: Forbidden Access** *(branch at step 3)* 1. The system determines that the user is neither the job owner nor the proposal owner. 2. The system displays a “Not Found” page. 3. The use case ends. **EX-04: System Error While Loading the Detail** *(branch at step 2)* 1. The system cannot load the proposal due to a processing or connection error. 2. The system displays the message “Couldn’t load the proposal. Please try again later.” 3. The user remains on the current page. 4. The use case ends.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-PROP-20: Access to a proposal is limited to the job owner (Client) or the proposal owner (Freelancer). BR-PROP-21: A proposal is considered not found if it does not exist or belongs to neither the user nor their job.
Other Information:	The freelancer view and the client job-owner view present the same proposal contents.
Assumptions:	Each proposal is associated with exactly one job and one freelancer.

2.2.3.8 UC Accept Proposal

UC ID and Name	UC Accept Proposal		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	Freelancer, System
Trigger:	When a client chooses to accept a pending proposal from a job's proposal list or detail page.		
Description:	Allows a client to accept a freelancer's proposal, changing its status to accepted. The system also rejects the other pending proposals of the job, moves the job to in progress, and creates a contract.		
Preconditions:	**PRE-1** The client is authenticated.		
Post-conditions:	**POST-1** The proposal status is set to accepted. **POST-2** All other pending proposals of the job are rejected. **POST-3** The job status is updated to in progress. **POST-4** A new contract is created for the job and the accepted proposal. **POST-5** A notification is sent to the freelancer.		
Normal Flow:	**A. Accept Proposal** 1. The Client opens the detail of a pending proposal of a job they own. 2. The Client chooses the accept action. 3. The system asks for confirmation of the acceptance. 4. The Client confirms the acceptance. 5. The system validates the proposal status and that the job belongs to the current client. 6. The system sets the proposal status to accepted. 7. The system rejects the other pending proposals of the job. 8. The system updates the job status to in progress. 9. The system creates a contract for the job and the accepted proposal. 10. The system notifies the Freelancer and confirms the acceptance to the Client. The use case ends.		
Alternative Flows:	**A.1 Cancel Accept** *(branch at step 4)* 1. The Client cancels the acceptance. 2. The system takes no action. 3. The use case ends.		

Exceptions:	**EX-01: Proposal Not Found** *(branch at step 5)* 1. The system cannot find the proposal. 2. The system displays the message “Proposal not found.” 3. The use case ends. **EX-02: Job Unavailable** *(branch at step 5)* 1. The system determines that the job is deleted or no longer open. 2. The system displays an appropriate error message. 3. The use case ends. **EX-03: Forbidden Access** *(branch at step 5)* 1. The system determines that the job belongs to another client. 2. The system displays a “Not Found” page. 3. The use case ends. **EX-04: Proposal Not Pending** *(branch at step 5)* 1. The system determines that the proposal status is no longer pending. 2. The system displays a message indicating the proposal can no longer be accepted. 3. The use case ends. **EX-05: Proposal Incomplete** *(branch at step 5)* 1. The system determines that the proposal has no valid bid amount. 2. The system displays a message that the proposal is incomplete. 3. The use case ends. **EX-06: System Error While Accepting** *(branch at step 6)* 1. The system cannot complete the acceptance and contract creation due to a processing or connection error. 2. The system displays the message “Couldn’t accept the proposal. Please try again later.” 3. The client remains on the current page and the proposal is not affected. 4. The use case ends.
Priority:	High
Frequency of Use:	Medium

Business Rules:	BR-PROP-22: Only the client who owns the job can accept a proposal. BR-PROP-23: Only a pending proposal can be accepted. BR-PROP-24: Accepting a proposal rejects all other pending proposals of the same job. BR-PROP-25: Accepting a proposal moves the job status to in progress. BR-PROP-26: Accepting a proposal creates a contract whose amount equals the proposal bid amount.
Other Information:	Acceptance and contract creation are performed together as a single unit; the freelancer is notified to review the contract.
Assumptions:	The proposal carries a valid bid amount required for contract creation.

2.2.3.9 UC Reject Proposal

UC ID and Name	UC Reject Proposal		
Created By:	Tran Phat Tai	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	Freelancer, System
Trigger:	When a client chooses to reject a pending proposal from a job's proposal list or detail page.		
Description:	Allows a client to reject a freelancer's proposal, changing its status to rejected. The client may optionally provide a rejection reason.		
Preconditions:	**PRE-1** The client is authenticated.		
Post-conditions:	**POST-1** The proposal status is set to rejected. **POST-2** The proposal is no longer actionable by the client.		

Normal Flow:	**A. Reject Proposal** 1. The Client opens the detail of a pending proposal of a job they own. 2. The Client chooses the reject action. 3. The system asks for confirmation of the rejection and allows an optional rejection reason. 4. The Client enters an optional rejection reason and confirms. 5. The system validates the proposal status and that the job belongs to the current client. 6. The system sets the proposal status to rejected. 7. The system confirms the rejection to the Client. The use case ends.
Alternative Flows:	**A.1 Cancel Reject** *(branch at step 4)* 1. The Client cancels the rejection. 2. The system takes no action. 3. The use case ends.

Exceptions:	**EX-01: Proposal Not Found** *(branch at step 5)* 1. The system cannot find the proposal. 2. The system displays the message “Proposal not found.” 3. The use case ends. **EX-02: Job Unavailable** *(branch at step 5)* 1. The system determines that the job is deleted or unavailable. 2. The system displays an appropriate error message. 3. The use case ends. **EX-03: Forbidden Access** *(branch at step 5)* 1. The system determines that the job belongs to another client. 2. The system displays a “Not Found” page. 3. The use case ends. **EX-04: Proposal Not Pending** *(branch at step 5)* 1. The system determines that the proposal status is no longer pending. 2. The system displays a message indicating the proposal can no longer be rejected. 3. The use case ends. **EX-05: System Error While Rejecting** *(branch at step 6)* 1. The system cannot update the proposal due to a processing or connection error. 2. The system displays the message “Couldn’t reject the proposal. Please try again later.” 3. The client remains on the current page and the proposal is not affected. 4. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-PROP-27: Only the client who owns the job can reject a proposal. BR-PROP-28: Only a pending proposal can be rejected. BR-PROP-29: A rejection reason is optional and stored with the proposal.
Other Information:	Rejecting a proposal does not affect the job status or other proposals.
Assumptions:	The client can only reject proposals on jobs they own.

2.2.22 Manage User

2.2.3.1 UC-17.1 View User List

UC ID and Name	UC-17.1 View User List		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	The Admin navigates to the `/admin/users` page or refreshes the user list view.		
Description:	The Admin retrieves a paginated, filterable, and sortable list of all active (non-soft-deleted) users in the system. The list displays user ID, email, display name, creation date, ban status, and assigned roles.		
Preconditions:	**PRE-1** The Admin is authenticated with a valid access token and an active session. **PRE-2** The Admin's role includes a permission record matching the `GET` method on the `/api/users` path.		
Post-conditions:	**POST-1** A paginated list of active users is returned and rendered in the `UsersTable` component. **POST-2** The filter bar (`UsersFilterBar`) reflects the currently applied search, role filter, sort column, and sort order.		
Normal Flow:	**A. Retrieve User List** 1. The Admin opens the `/admin/users` page. 2. The frontend sends `GET /api/users` with default query parameters: `page=1`, `limit=10`, `sortBy=id`, `sortOrder=desc`. 3. The backend verifies the access token via the global `AuthGuard` and checks permission via the global `PermissionGuard`. 4. The backend queries the database for users where `deletedAt = null`, ordered by the requested sort column and direction. 5. The backend returns a response containing `users` (array of user objects with id, email, isBanned, createdAt, updatedAt, displayName, avatarUrl, roles) and `pagination` (page, limit, total, totalPages). 6. The frontend renders the user list in `UsersTable` and the pagination controls.		

Alternative Flows:	**A.1 Search Users** 1. The Admin enters a search term in the search bar. 2. The frontend sends `GET /api/users?search={term}`. 3. The backend performs a case-insensitive search on the `email` and `displayName` fields of active users. 4. The filtered, paginated result set is returned and rendered. Return to step 6. **A.2 Filter by Role** 1. The Admin selects a role from the role filter dropdown (CLIENT, FREELANCER, ADMIN, CUSTOM, or a custom role name). 2. The frontend sends `GET /api/users?role={roleName}`. 3. The backend filters active users whose assigned roles include the specified role name (case-insensitive). Selecting `all` or `ALL` applies no role filter. 4. The filtered result is returned and rendered. Return to step 6. **A.3 Change Sort Order** 1. The Admin clicks a column header to sort. 2. The frontend sends `GET /api/users?sortBy={column}&sortOrder={asc|desc}`. 3. The backend re-queries with the new sort parameters. 4. The re-sorted result is returned and rendered. Return to step 6. **A.4 Navigate Pages** 1. The Admin clicks a pagination control (next, previous, or page number). 2. The frontend sends `GET /api/users?page={n}&limit={limit}`. 3. The backend returns the requested page of results. 4. The new page is rendered.

Exceptions:	**EX-01: Server Error** 1. The backend encounters an unexpected error while querying the database. 2. The backend returns an error response with code `Error.FailedToGetUsers` and HTTP status 500. 3. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	**BR-USER-01:** Only active users (`deletedAt = null`) are included in the list. Soft-deleted users are excluded. **BR-USER-02:** The default sort order is by `id` in descending order (newest users first). **BR-USER-03:** The maximum page size is 100; if `limit` exceeds 100, it is capped at 100. **BR-USER-04:** Search on `email` and `displayName` is case-insensitive. **BR-USER-05:** Role filtering is case-insensitive and supports both built-in role names and custom role names.
Other Information:	Frontend route: `/admin/users`. Components: `UsersTable`, `UsersFilterBar`. Guards: global `AuthGuard` + `PermissionGuard`.
Assumptions:	The Admin has the necessary permission records in the database to access this endpoint. The database is available and responsive.

2.2.3.2 UC-17.2 View User Detail

UC ID and Name	UC-17.2 View User Detail		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	The Admin selects a user row from the user list or navigates directly to `/admin/users/[id]`.		
Description:	The Admin retrieves the full account information for a specific user, including general profile data, assigned roles with permissions, social links, activity statistics, and role-specific profile tabs (Client Profile, Freelancer Profile, Custom Role Profiles).		

Preconditions:	**PRE-1** The Admin is authenticated with a valid access token and an active session. **PRE-2** The Admin's role includes a permission record matching the `GET` method on the `/api/users/:id` path.
Post-conditions:	**POST-1** The user's general info is displayed in `UserGeneralInfo` and `UserDetailHeader`. **POST-2** Role-specific tabs (`UserRoleTabs`) are rendered showing Client Profile, Freelancer Profile, and Admin & Custom Roles sections.
Normal Flow:	**A. Retrieve User Detail** 1. The Admin navigates to `/admin/users/[id]` for the target user. 2. The frontend sends `GET /api/users/:id`. 3. The backend verifies the access token and permission. 4. The backend retrieves the user record including: general fields (id, email, isBanned, createdAt, updatedAt, displayName, avatarUrl, coverUrl, bio, onlineStatus, availabilityStatus, profileCompletionPercent), assigned roles (with id, name, description, isPrimary), social links, activity statistics (jobsPosted, contractsAsClient, contractsAsFreelancer, proposals, reviewsReceived, idVerificationDocuments), client profile (if present), freelancer profile (if present), and custom role profiles. 5. The backend returns the complete user detail object. 6. The frontend renders `UserDetailHeader`, `UserGeneralInfo`, and `UserRoleTabs` with each available tab populated from the response data.

Alternative Flows:	**A.1 View Client Profile Tab** 1. The Admin clicks the “Client Profile” tab. 2. The frontend displays the `clientProfile` data (companyName, companyDescription, website, createdAt, updatedAt) from the already-retrieved response. **A.2 View Freelancer Profile Tab** 1. The Admin clicks the “Freelancer Profile” tab. 2. The frontend displays the `freelancerProfile` data (title, education, certifications, languages, idVerified, skills with proficiency levels, portfolioItems) from the already-retrieved response. **A.3 View Admin & Custom Roles Tab** 1. The Admin clicks the “Admin & Custom Roles” tab. 2. The frontend displays the `customRoleProfiles` data (roleName, description, isPrimary, permissions list with name, path, method, module).
Exceptions:	**EX-01: User Not Found** 1. The backend cannot find a user with the specified ID (or the user is soft-deleted). 2. The backend returns an error response with code `Error.UserNotFound` and HTTP status 404. 3. The use case ends. **EX-02: Server Error** 1. The backend encounters an unexpected error while retrieving the user detail. 2. The backend returns an error response with code `Error.FailedToGetUser` and HTTP status 500. 3. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	**BR-USER-06:** The `clientProfile` and `freelancerProfile` fields may be `null` if the user does not have the corresponding role or has not yet created that profile. **BR-USER-07:** The `customRoleProfiles` array contains only roles that are neither the built-in Client, Freelancer, nor Admin role. **BR-USER-08:** Each role in the response includes an `isPrimary` flag indicating whether it is the user’s primary role.

Other Information:	Frontend route: `/admin/users/[id]`. Components: `UserDetailHeader`, `UserGeneralInfo`, `UserRoleTabs`. Guards: global `AuthGuard` + `PermissionGuard`.
Assumptions:	The user ID is valid and corresponds to an existing, non-soft-deleted user. The Admin has the necessary permission records to access this endpoint.

2.2.3.3 UC-17.3 Create User

UC ID and Name	UC-17.3 Create User		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	The Admin opens the "Create User" dialog (`CreateUserDialog`) from the user management page and submits the form.		
Description:	The Admin creates a new user account by providing a full name, email address, password, and selecting an initial role (Client, Freelancer, or a custom role). The system creates the user record, a profile with the display name, and assigns the selected primary role.		
Preconditions:	**PRE-1** The Admin is authenticated with a valid access token and an active session. **PRE-2** The Admin's role includes a permission record matching the `POST` method on the `/api/users` path. **PRE-3** The Admin has navigated to the `/admin/users` page.		
Post-conditions:	**POST-1** A new user record is created in the database with the provided email (lowercased), and a Profile record is created with `displayName` set to the provided full name. **POST-2** A primary UserRole record linking the user to the selected role is created. **POST-3** The new user's summary (id, email, displayName, roles) is returned and the user list is refreshed.		

Normal Flow:	**A. Create New User** 1. The Admin clicks the “Create User” button on the user list page. 2. The `CreateUserDialog` opens with fields: fullName, email, password, confirmPassword, and roleId (Select dropdown grouped by “Built-in” for Client/Freelancer and “Custom roles”, excluding Admin). 3. The Admin fills in all required fields and selects a role. 4. The Admin submits the form. 5. The frontend sends `POST /api/users` with body: `email` (lowercased, max 254 chars), `fullName` (required, max 100 chars), `password` (8–32 chars, must contain uppercase letter and number), `confirmPassword` (must match password), `roleId` (positive integer). 6. The backend validates the input fields. 7. The backend checks that the email is unique among non-deleted users. 8. The backend verifies the specified role exists and is active. 9. The backend verifies the role is not the Admin role. 10. The backend creates a User record, a Profile record (`displayName = fullName`), and a primary UserRole record. 11. The backend returns the new user’s id, email, displayName, and roles. 12. The frontend closes the dialog and refreshes the user list.

Alternative Flows:	**A.1 Email Already Exists** 1. At step 7, the backend detects the email already exists among non-deleted users. 2. The backend returns an error response with code `Error.EmailAlreadyExists` and HTTP status 409. 3. The frontend displays an error message indicating the email is already in use. 4. The Admin corrects the email and resubmits, returning to step 4. **A.2 Role Not Found** 1. At step 8, the backend cannot find the specified role or the role is inactive. 2. The backend returns an error response with code `Error.CreateUserRoleNotFound` and HTTP status 404. 3. The frontend displays an error message and the Admin selects a valid role. Return to step 4. **A.3 Attempt to Assign Admin Role** 1. At step 9, the backend detects the selected role is the Admin role. 2. The backend returns an error response with code `Error.CannotAssignAdminRole` and HTTP status 403. 3. The frontend displays an error message indicating Admin role assignment is not permitted. 4. The Admin selects a different role and resubmits, returning to step 4.

Exceptions:	**EX-01: Password Mismatch** 1. At step 6, the backend detects that `confirmPassword` does not match `password`. 2. A validation error is returned. 3. The use case ends (the form remains open for correction). **EX-02: Invalid Password Format** 1. At step 6, the backend detects the password does not meet the 8–32 character requirement or lacks an uppercase letter or number. 2. A validation error is returned. 3. The use case ends. **EX-03: Server Error** 1. The backend encounters an unexpected error during user creation. 2. The backend returns an error response with code `Error.FailedToCreateUser` and HTTP status 500. 3. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	**BR-USER-09:** The email is automatically lowercased before storage and must be at most 254 characters. **BR-USER-10:** Email uniqueness is checked among all non-soft-deleted users (`deletedAt = null`). **BR-USER-11:** The password must be between 8 and 32 characters and must contain at least one uppercase letter and one number. **BR-USER-12:** The Admin role cannot be assigned during user creation; only Client, Freelancer, or custom roles are permitted. **BR-USER-13:** The selected role must exist in the role table and be active. **BR-USER-14:** The assigned role is marked as the user's primary role (`isPrimary = true`).
Other Information:	Frontend component: `CreateUserDialog`. Guards: global `AuthGuard` + `PermissionGuard`. A Profile record is created alongside the User record to store the display name.
Assumptions:	At least one non-Admin role (Client, Freelancer, or a custom role) exists and is active in the system. The Admin has the necessary permission records to access this endpoint.

2.2.3.4 UC-17.4 Edit User

UC ID and Name	UC-17.4 Edit User		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	The Admin opens the "Edit User" dialog (`EditUserDialog`) for a specific user and submits changes to the general account fields.		
Description:	The Admin edits the general account information of an existing user, including email, full name (display name), and ban status. At least one field must be changed. The system validates the changes, updates the user and profile records, and returns the updated summary.		
Preconditions:	**PRE-1** The Admin is authenticated with a valid access token and an active session. **PRE-2** The Admin's role includes a permission record matching the `PATCH` method on the `/api/users/:id` path. **PRE-3** The target user exists and is not soft-deleted.		
Post-conditions:	**POST-1** The user's email, display name, and/or ban status are updated in the database. **POST-2** If `fullName` is provided, the Profile record's `displayName` is upserted. **POST-3** The updated user summary (id, email, displayName, isBanned, roles) is returned.		

Normal Flow:	**A. Update User General Info** 1. The Admin clicks the “Edit” action on a user row or from the user detail page. 2. The `EditUserDialog` opens, pre-filled with the user’s current email, fullName, and isBanned status. 3. The Admin modifies one or more fields and submits the form. 4. The frontend sends `PATCH /api/users/:id` with body containing at least one of: `email` (optional, lowercase), `fullName` (optional, nullable), `isBanned` (optional boolean). 5. The backend verifies the access token and permission. 6. The backend finds the user by ID (must not be soft-deleted). 7. The backend validates that at least one updatable field is provided. 8. If `email` is provided, the backend checks uniqueness among non-deleted users excluding the target user. 9. If `isBanned = true` and the target user ID matches the actor’s ID, the backend rejects the request. 10. The backend updates the User record fields. 11. If `fullName` is provided, the backend upserts the Profile record with `displayName = fullName`. 12. The backend returns the updated user summary. 13. The frontend closes the dialog and refreshes the user data.
Alternative Flows:	**A.1 Ban Status Toggled** 1. At step 3, the Admin toggles the isBanned switch. 2. The frontend includes `isBanned: true` or `isBanned: false` in the request body. 3. The backend updates the `isBanned` field on the User record. Return to step 12. **A.2 Email Changed** 1. At step 3, the Admin modifies the email field. 2. At step 8, the backend confirms the new email is unique. 3. The backend lowercases and updates the email field. Return to step 10. **A.3 Display Name Changed** 1. At step 3, the Admin modifies the fullName field. 2. At step 11, the backend upserts the Profile record with the new displayName. Return to step 12.

Exceptions:	**EX-01: Attempt to Ban Self** 1. At step 9, the Admin attempts to set `isBanned = true` on their own account. 2. The backend returns an error response with code `Error.CannotBanSelf` and HTTP status 403. 3. The use case ends. **EX-02: Duplicate Email** 1. At step 8, the backend detects the new email already exists among non-deleted users (excluding the target user). 2. The backend returns an error response with code `Error.EmailAlreadyExists` and HTTP status 409. 3. The use case ends. **EX-03: No Changes Provided** 1. At step 7, the backend detects that none of the updatable fields (`email`, `fullName`, `isBanned`) are present in the request body. 2. The backend returns an error response with code `Error.NothingToUpdate` and HTTP status 400. 3. The use case ends. **EX-04: User Not Found** 1. At step 6, the backend cannot find a user with the specified ID. 2. The backend returns an error response with code `Error.UserNotFound` and HTTP status 404. 3. The use case ends. **EX-05: Server Error** 1. The backend encounters an unexpected error during the update. 2. The backend returns an error response with code `Error.FailedToUpdateUser` and HTTP status 500. 3. The use case ends.
Priority:	High
Frequency of Use:	Medium

Business Rules:	**BR-USER-15:** An Admin cannot ban their own account (`isBanned = true` is rejected when the target user ID equals the actor's ID). **BR-USER-16:** Email uniqueness is enforced among non-soft-deleted users, excluding the target user itself. **BR-USER-17:** At least one updatable field must be provided; an empty update request is rejected. **BR-USER-18:** The email is automatically lowercased before storage. **BR-USER-19:** The `fullName` field is nullable; setting it to `null` clears the display name.
Other Information:	Frontend component: `EditUserDialog`. Guards: global `AuthGuard` + `PermissionGuard`. The `UserActive` decorator extracts the actor's userId from the token for the self-ban check.
Assumptions:	The target user exists and is not soft-deleted. The Admin has the necessary permission records to access this endpoint.

2.2.3.5 UC-17.5 Edit Client Profile

UC ID and Name	UC-17.5 Edit Client Profile		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	The Admin clicks the "Edit Client Profile" button (`EditClientProfileButton`) on the Client Profile tab of a user's detail page and submits changes.		
Description:	The Admin edits the Client profile of a user who has the Client role, modifying the company name, company description, and/or website URL. The system validates the changes and upserts the client profile record.		
Preconditions:	**PRE-1** The Admin is authenticated with a valid access token and an active session. **PRE-2** The Admin's role includes a permission record matching the `PATCH` method on the `/api/users/:id/client-profile` path. **PRE-3** The target user exists, is not soft-deleted, and either has the Client role assigned or already has an existing client profile record.		

Post-conditions:	**POST-1** The client profile record is created (if absent) or updated with the provided field values. **POST-2** Empty string values are converted to `null`, clearing the field content. **POST-3** The updated client profile object is returned.
Normal Flow:	**A. Update Client Profile** 1. The Admin navigates to the user detail page and clicks the "Client Profile" tab. 2. The Admin clicks the "Edit Client Profile" button. 3. The `EditClientProfileDialog` opens, pre-filled with current values (companyName, companyDescription, website). 4. The Admin modifies one or more fields and submits the form. 5. The frontend sends `PATCH /api/users/:id/client-profile` with body containing at least one of: `companyName` (max 255 chars, nullable), `companyDescription` (max 5000 chars, nullable), `website` (URL format, nullable). 6. The backend verifies the access token and permission. 7. The backend verifies the target user exists and is not soft-deleted. 8. The backend checks that the user has the Client role or already has an existing client profile record. 9. The backend converts any empty string values to `null`. 10. The backend upserts the client profile record with the provided fields. 11. The backend returns the updated client profile object (id, profileId, userId, companyName, companyDescription, website, createdAt, updatedAt). 12. The frontend closes the dialog and refreshes the Client Profile tab.

Alternative Flows:	**A.1 Clear Company Name** 1. At step 4, the Admin clears the companyName field (sets it to empty string or null). 2. At step 9, the backend converts the empty string to `null`. 3. The companyName is set to `null` in the database. Return to step 11. **A.2 Clear Website** 1. At step 4, the Admin clears the website field. 2. At step 9, the backend converts the empty string to `null`. 3. The website is set to `null` in the database. Return to step 11.
Exceptions:	**EX-01: User Not Found** 1. At step 7, the backend cannot find a user with the specified ID. 2. The backend returns an error response with code `Error.UserNotFound` and HTTP status 404. 3. The use case ends. **EX-02: User Has No Client Role** 1. At step 8, the backend determines the user does not have the Client role and does not have an existing client profile. 2. The backend returns an error response with code `Error.UserHasNoClientRole` and HTTP status 403. 3. The use case ends. **EX-03: Server Error** 1. The backend encounters an unexpected error during the update. 2. The backend returns an error response with code `Error.FailedToUpdateClientProfile` and HTTP status 500. 3. The use case ends.
Priority:	Medium
Frequency of Use:	Low

Business Rules:	**BR-CLT-01:** The target user must have the Client role or an existing client profile record; otherwise the update is rejected. **BR-CLT-02:** The `companyName` field is limited to 255 characters. **BR-CLT-03:** The `companyDescription` field is limited to 5000 characters. **BR-CLT-04:** The `website` field must be a valid URL format. **BR-CLT-05:** Empty string values are automatically converted to `null` to clear field content. **BR-CLT-06:** At least one field must be provided in the request body.
Other Information:	Frontend components: `EditClientProfileButton`, `EditClientProfileDialog`. Guards: global `AuthGuard` + `PermissionGuard`. The client profile is upserted, creating it if it does not yet exist.
Assumptions:	The Admin is editing a user that has been assigned the Client role or has an existing client profile. The Admin has the necessary permission records to access this endpoint.

2.2.3.6 UC-17.6 Edit Freelancer Basics

UC ID and Name	UC-17.6 Edit Freelancer Basics		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	The Admin clicks the “Edit Freelancer Profile” button (`EditFreelancerProfileButton`) on the Freelancer Profile tab of a user’s detail page and submits changes to the basic profile fields.		
Description:	The Admin edits the Freelancer profile’s basic fields (title, bio, languages, education, certifications) for a user who has the Freelancer role. The system validates the inputs and upserts the freelancer profile record.		

Preconditions:	**PRE-1** The Admin is authenticated with a valid access token and an active session. **PRE-2** The Admin's role includes a permission record matching the `PATCH` method on the `/api/users/:id/freelancer-profile` path. **PRE-3** The target user exists, is not soft-deleted, and either has the Freelancer role assigned or already has an existing freelancer profile record.
Post-conditions:	**POST-1** The freelancer profile record is created (if absent via upsert) or updated with the provided field values. **POST-2** Empty strings are converted to `null`; empty or null arrays are converted to `[]`. **POST-3** A success message "Freelancer profile updated successfully" is returned.
Normal Flow:	**A. Update Freelancer Basic Info** 1. The Admin navigates to the user detail page and clicks the "Freelancer Profile" tab. 2. The Admin clicks the "Edit Freelancer Profile" button. 3. The `EditFreelancerProfileDialog` opens, pre-filled with current values (title, bio, languages, education, certifications). 4. The Admin modifies one or more fields and submits the form. 5. The frontend sends `PATCH /api/users/:id/freelancer-profile` with body containing at least one of: `title` (max 255 chars, nullable), `bio` (max 5000 chars, nullable), `languages` (string array, max 100 items, each max 255 chars, nullable), `education` (same constraints), `certifications` (same constraints). 6. The backend verifies the access token and permission. 7. The backend verifies the target user exists and is not soft-deleted. 8. The backend checks that the user has the Freelancer role or an existing freelancer profile. 9. The backend converts empty string values to `null` and empty/null array values to `[]`. 10. The backend upserts the freelancer profile record. 11. The backend returns the success message. 12. The frontend closes the dialog and refreshes the Freelancer Profile tab.

Alternative Flows:	**A.1 Clear Bio** 1. At step 4, the Admin clears the bio field. 2. At step 9, the backend converts the empty string to `null`. 3. The bio is set to `null` in the database. Return to step 11. **A.2 Clear Languages List** 1. At step 4, the Admin removes all entries from the languages array. 2. At step 9, the backend converts the empty array to `[]`. 3. The languages array is set to `[]` in the database. Return to step 11.
Exceptions:	**EX-01: User Not Found** 1. At step 7, the backend cannot find a user with the specified ID. 2. The backend returns an error response with code `Error.UserNotFound` and HTTP status 404. 3. The use case ends. **EX-02: User Has No Freelancer Role** 1. At step 8, the backend determines the user does not have the Freelancer role and does not have an existing freelancer profile. 2. The backend returns an error response with code `Error.UserHasNoFreelancerRole` and HTTP status 403. 3. The use case ends. **EX-03: Validation Error** 1. At step 9, the backend detects a credential item exceeds 255 characters or the array exceeds 100 items. 2. A validation error is returned. 3. The use case ends. **EX-04: Server Error** 1. The backend encounters an unexpected error during the update. 2. The backend returns an error response with code `Error.FailedToUpdateFreelancerProfile` and HTTP status 500. 3. The use case ends.
Priority:	Medium
Frequency of Use:	Low

Business Rules:	**BR-FLR-01:** The target user must have the Freelancer role or an existing freelancer profile; otherwise the update is rejected. **BR-FLR-02:** The `title` field is limited to 255 characters. **BR-FLR-03:** The `bio` field is limited to 5000 characters. **BR-FLR-04:** The `languages`, `education`, and `certifications` arrays may contain at most 100 items each, with each item limited to 255 characters. **BR-FLR-05:** Empty string values are converted to `null`; empty or null array values are converted to `[]`. **BR-FLR-06:** At least one field must be provided in the request body.
Other Information:	Frontend components: `EditFreelancerProfileButton`, `EditFreelancerProfileDialog`. Guards: global `AuthGuard` + `PermissionGuard`. The freelancer profile is upserted, creating it and the parent Profile if they do not yet exist.
Assumptions:	The Admin is editing a user that has been assigned the Freelancer role or has an existing freelancer profile. The Admin has the necessary permission records to access this endpoint.

2.2.3.7 UC-17.7 Edit Freelancer Skills

UC ID and Name	UC-17.7 Edit Freelancer Skills		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	The Admin clicks the “Manage Freelancer Skills” button (`ManageFreelancerSkillsButton`) on the Freelancer Profile tab of a user’s detail page and submits the new skill set.		
Description:	The Admin replaces the entire skill set of a freelancer by selecting skills from the skill catalog with proficiency levels (1–10). The system deletes all existing skills and creates new ones from the provided list.		

Preconditions:	**PRE-1** The Admin is authenticated with a valid access token and an active session. **PRE-2** The Admin's role includes a permission record matching the `PUT` method on the `/api/users/:id/freelancer-profile/skills` path. **PRE-3** The target user exists, is not soft-deleted, and has the Freelancer role or an existing freelancer profile.	
Post-conditions:	**POST-1** All existing FreelancerSkill records for the freelancer are deleted. **POST-2** New FreelancerSkill records are created for each skill in the submitted list, linked to catalog skills (found or created via case-insensitive match). **POST-3** A success message "Freelancer skills updated successfully" is returned.	

Normal Flow:	**A. Replace Freelancer Skills** 1. The Admin navigates to the Freelancer Profile tab and clicks "Manage Freelancer Skills." 2. The skill management dialog opens, showing the current skill list with a "Select" dropdown populated from the skill catalog (`GET /api/users/skills-catalog` returning active skills as `[{id, name}]`) and a proficiency level input (1–10) per row. 3. The Admin adds skill rows by selecting a skill from the catalog dropdown and setting a proficiency level. The Admin may also remove rows. 4. The Admin submits the form. 5. The frontend sends `PUT /api/users/:id/freelancer-profile/skills` with body: `skills[]` (max 100 items), each containing `skillName` (max 100 chars) and `proficiencyLevel` (integer 1–10). 6. The backend verifies the access token and permission. 7. The backend ensures the freelancer profile exists (auto-creates if needed). 8. The backend deletes ALL existing FreelancerSkill records for this freelancer profile. 9. For each skill in the submitted array, the backend performs a case-insensitive lookup in the skill catalog; if no match is found, a new catalog skill is created. 10. The backend creates a FreelancerSkill record linking the freelancer profile to the catalog skill with the specified proficiency level. 11. The backend returns the success message. 12. The frontend closes the dialog and refreshes the Freelancer Profile tab.

Alternative Flows:	**A.1 Clear All Skills** 1. At step 3, the Admin removes all skill rows (empty array). 2. At step 8, the backend deletes all existing FreelancerSkill records. 3. No new skills are created (empty `skills[]` array). Return to step 11. **A.2 Add New Catalog Skill** 1. At step 3, the Admin selects a skill name not currently in the catalog. 2. At step 9, the backend does not find a match in the catalog (case-insensitive). 3. The backend creates a new catalog skill record with the provided name. 4. The backend creates a FreelancerSkill linking to the new catalog skill. Return to step 10.
Exceptions:	**EX-01: Invalid Proficiency Level** 1. At step 9, the backend detects a `proficiencyLevel` outside the 1–10 range. 2. The backend returns a validation error with code `SkillLevelInvalid`. 3. The use case ends. **EX-02: Too Many Skills** 1. At step 9, the backend detects the `skills[]` array contains more than 100 items. 2. The backend returns a validation error with code `TooManySkills`. 3. The use case ends. **EX-03: Server Error** 1. The backend encounters an unexpected error during skill replacement. 2. The backend returns an error response with code `Error.FailedToSaveSkills` and HTTP status 500. 3. The use case ends.
Priority:	Medium
Frequency of Use:	Low

Business Rules:	**BR-FLR-07:** This is a full-replacement operation: all existing skills are deleted and replaced with the submitted list. **BR-FLR-08:** The `proficiencyLevel` must be an integer between 1 and 10 inclusive. **BR-FLR-09:** The maximum number of skills per freelancer is 100. **BR-FLR-10:** Skill catalog lookup is case-insensitive; if a matching skill does not exist, a new catalog entry is created. **BR-FLR-11:** The `skillName` field is limited to 100 characters.
Other Information:	Frontend components: `ManageFreelancerSkillsButton`, skill management dialog with Select dropdown and level input per row. Guards: global `AuthGuard` + `PermissionGuard`. Uses `PUT` (not PATCH) for full replacement semantics.
Assumptions:	The skill catalog is populated with active skills. The target user has been assigned the Freelancer role or has an existing freelancer profile. The Admin has the necessary permission records to access this endpoint.

2.2.3.8 UC-17.8 Edit Freelancer Portfolio

UC ID and Name	UC-17.8 Edit Freelancer Portfolio		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	The Admin clicks the “Manage Freelancer Portfolio” button (`ManageFreelancerPortfolioButton`) on the Freelancer Profile tab of a user’s detail page and performs create, edit, or delete operations on portfolio items.		
Description:	The Admin manages the portfolio items of a freelancer by creating new items, editing existing items, or soft-deleting items. Each portfolio item contains a title, description, technologies list, and optional project URL.		

Preconditions:	**PRE-1** The Admin is authenticated with a valid access token and an active session. **PRE-2** The Admin's role includes permission records matching: `POST` on `/api/users/:id/freelancer-profile/portfolio-items` (create), `PATCH` on `/api/users/:id/freelancer-profile/portfolio-items/:itemId` (edit), and `DELETE` on `/api/users/:id/freelancer-profile/portfolio-items/:itemId` (delete). **PRE-3** The target user exists, is not soft-deleted, has the Freelancer role or an existing freelancer profile.
Post-conditions:	**POST-1 (Create)** A new portfolio item record is created in the database. **POST-2 (Edit)** The specified portfolio item's fields are updated. **POST-3 (Delete)** The specified portfolio item is soft-deleted (`deletedAt` is set). **POST-4** A success message is returned for each operation.
Normal Flow:	**A. Create Portfolio Item** 1. The Admin navigates to the Freelancer Profile tab and clicks "Manage Freelancer Portfolio." 2. The portfolio management dialog opens, listing existing portfolio items with their title, description, technologies, and project URL. 3. The Admin clicks the "Add Portfolio Item" action. 4. The Admin fills in: `title` (required, max 255 chars), `description` (max 5000 chars, nullable), `technologies` (string array, max 50 items, each max 100 chars, default []), `projectUrl` (URL format, nullable). 5. The Admin submits the form. 6. The frontend sends `POST` `/api/users/:id/freelancer-profile/portfolio-items` with the item data. 7. The backend verifies the access token and permission. 8. The backend creates the portfolio item record. 9. The backend returns a success message "Portfolio item created successfully." 10. The dialog refreshes to show the new item in the list.

Alternative Flows:	**A.1 Edit Portfolio Item** 1. From the portfolio list dialog (step 2), the Admin clicks the "Edit" action on an existing item. 2. The Admin modifies one or more fields (title, description, technologies, projectUrl) and submits. 3. The frontend sends `PATCH /api/users/:id/freelancer-profile/portfolio-items/:itemId` with the updated fields. 4. The backend verifies the item exists, is not soft-deleted, and belongs to this freelancer profile. 5. The backend updates the item fields. 6. The backend returns a success message "Portfolio item updated successfully." 7. The dialog refreshes to reflect the changes. **A.2 Delete Portfolio Item** 1. From the portfolio list dialog (step 2), the Admin clicks the "Delete" action on an existing item. 2. The Admin confirms the deletion. 3. The frontend sends `DELETE /api/users/:id/freelancer-profile/portfolio-items/:itemId`. 4. The backend verifies the item exists, is not soft-deleted, and belongs to this freelancer profile. 5. The backend performs a soft delete by setting `deletedAt` on the portfolio item record. 6. The backend returns a success message "Portfolio item deleted successfully." 7. The dialog refreshes and the deleted item is no longer listed.

Exceptions:	**EX-01: Portfolio Item Not Found** 1. During edit or delete, the backend cannot find the specified item, the item is soft-deleted, or it does not belong to the target freelancer profile. 2. The backend returns an error response with code `Error.PortfolioItemNotFound` and HTTP status 404. 3. The use case ends. **EX-02: Server Error on Create** 1. The backend encounters an unexpected error while creating the portfolio item. 2. The backend returns an error response with code `Error.FailedToCreatePortfolioItem` and HTTP status 500. 3. The use case ends. **EX-03: Server Error on Edit** 1. The backend encounters an unexpected error while updating the portfolio item. 2. The backend returns an error response with code `Error.FailedToUpdatePortfolioItem` and HTTP status 500. 3. The use case ends. **EX-04: Server Error on Delete** 1. The backend encounters an unexpected error while soft-deleting the portfolio item. 2. The backend returns an error response with code `Error.FailedToDeletePortfolioItem` and HTTP status 500. 3. The use case ends.
Priority:	Medium
Frequency of Use:	Low

Business Rules:	**BR-FLR-12:** Portfolio items are soft-deleted (setting `deletedAt`); they are not permanently removed from the database. **BR-FLR-13:** The `title` field is required and limited to 255 characters. **BR-FLR-14:** The `description` field is limited to 5000 characters. **BR-FLR-15:** The `technologies` array may contain at most 50 items, each limited to 100 characters. **BR-FLR-16:** The `projectUrl` field must be a valid URL format. **BR-FLR-17:** Edit and delete operations verify item ownership: the item must belong to the specified freelancer profile. **BR-FLR-18:** Soft-deleted portfolio items are excluded from queries and cannot be edited or deleted again.
Other Information:	Frontend component: `ManageFreelancerPortfolioButton`, portfolio management dialog listing items with add/edit/delete actions. Guards: global `AuthGuard` + `PermissionGuard`. Three separate endpoints are used: `POST` (create), `PATCH` (edit), and `DELETE` (soft-delete).
Assumptions:	The target user has been assigned the Freelancer role or has an existing freelancer profile. The Admin has the necessary permission records for all three portfolio endpoints.

2.2.23 Manage Payment

2.2.3.1 UC-43.01 Connect Stripe Account

UC ID and Name	UC-43.01 Connect Stripe Account		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer	Secondary Actors:	Stripe Connect
Trigger:	When a Freelancer clicks “Set up Payouts” or “Connect Stripe” in their Billing and Payout settings.		
Description:	Allows a Freelancer to connect their bank or debit account via Stripe Connect Express onboarding so that they can receive automated milestone releases and dispute disbursements.		

Preconditions:	**PRE-1** The user is authenticated in the system with the Freelancer role. **PRE-2** The Freelancer account is active and verified.
Post-conditions:	**POST-1** A Stripe Express account ID is created and associated with the Freelancer record. **POST-2** The Freelancer is redirected through Stripe hosted onboarding to verify banking details. **POST-3** Payout capabilities (`payouts_enabled`) are registered upon successful verification.
Normal Flow:	**A. Connect Stripe Account** 1. The Freelancer navigates to the Payout Settings page. 2. The system checks connection status; if unlinked, it displays a "Connect with Stripe" button. 3. The Freelancer clicks "Connect with Stripe". 4. The system requests an onboarding link from the Stripe Connect API with appropriate return URLs. 5. The system redirects the Freelancer to the Stripe-hosted onboarding flow. 6. The Freelancer provides legal entity, identification, and bank account or routing details to Stripe. 7. Stripe redirects the Freelancer back to the platform return URL. 8. The system queries Stripe to verify that `payouts_enabled` and `charges_enabled` are active. 9. The system updates the Freelancer payout status to "Connected" and displays a green verified badge.
Alternative Flows:	**A.1 Update Existing Stripe Account** 1. An already connected Freelancer clicks "Manage Payout Account". 2. The system generates a Stripe Express dashboard login link. 3. The Freelancer is directed to the Stripe Express portal to modify bank accounts or tax information.

Exceptions:	**EX-01: Onboarding Link Generation Failure** 1. If the payment gateway connection fails, the system displays: "Unable to initiate Stripe setup. Please try again later." 2. The use case ends. **EX-02: Verification Incomplete** 1. If the Freelancer exits the onboarding before providing required tax or identity details, Stripe marks requirements pending. 2. The system marks status as "Action Required" and prompts the Freelancer to resume verification. 3. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-PAYMENT-01: Freelancers must have a fully verified Stripe Connect account with payouts enabled to receive contract payments. BR-PAYMENT-02: Sensitive bank account numbers are stored exclusively within Stripe; the platform only persists the Stripe Account ID.
Other Information:	N/A
Assumptions:	Freelancers reside in countries supported by Stripe Connect.

2.2.3.2 UC-43.02 View Payout History

UC ID and Name	UC-43.02 View Payout History		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user clicks on the "Payouts" tab in the financial dashboard.		
Description:	Allows a Freelancer (or Admin) to view a paginated list of all payout transfers sent to their connected bank account, showing transfer dates, amounts, status, and destination account snippets.		
Preconditions:	**PRE-1** The user is authenticated in the system. **PRE-2** The user is an authorized Freelancer or an Administrator.		
Post-conditions:	**POST-1** A paginated listing of payout transactions is displayed.		

Normal Flow:	**A. View Payout History** 1. The Freelancer navigates to the Payout History page. 2. The system queries payout transfer records associated with the user's account (Admin retrieves all platform payouts). 3. The system sorts payouts chronologically (newest first). 4. The system renders the payout table displaying: Payout ID, Initiated Date, Estimated Arrival Date, Destination Bank (last 4 digits), Amount, and Status badge (`PAID`, `PENDING`, or `FAILED`).
Alternative Flows:	**A.1 Empty Payout History** 1. If no payouts have been executed yet, the system renders: "No payouts found. Complete and release milestones to receive earnings." 2. The use case ends.
Exceptions:	**EX-01: Retrieval Failure** 1. If the database or Stripe service query times out, the system displays: "Failed to retrieve payout history. Please refresh the page." 2. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-PAYMENT-03: Freelancers can only view their own payout transfers. BR-PAYMENT-04: Failed payouts display a warning badge and prompt the user to inspect their bank connection.
Other Information:	N/A
Assumptions:	Payout processing times follow standard banking network schedules (typically 1-3 business days).

2.2.3.3 UC-43.03 View Payout Detail

UC ID and Name	UC-43.03 View Payout Detail		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user clicks on a payout row in the Payout History table.		

Description:	Displays detailed financial breakdown of a specific payout transfer, including related milestone, gross amount, platform commission deductions, net transferred amount, arrival estimate, and bank reference code.
Preconditions:	**PRE-1** The user is authenticated. **PRE-2** The payout record exists. **PRE-3** The user is the owner of the payout or an Administrator.
Post-conditions:	**POST-1** The system renders the comprehensive payout details modal or page.
Normal Flow:	**A. View Payout Detail** 1. The user clicks on a specific payout entry. 2. The system checks authorization and loads the payout record and linked milestone contract data. 3. The system renders the detail view showing: - Transfer Header: Transfer Reference ID, Status, and Date. - Financial Ledger: Milestone gross revenue, platform service fee, net payout amount. - Banking Information: Receiving financial institution, account type, masked account number. - Associated Contract: Contract title and milestone identifier.
Alternative Flows:	N/A
Exceptions:	**EX-01: Unauthorized Payout Inspection** 1. A user attempts to view a payout belonging to another user. 2. The system returns: “Access Denied: You cannot view this payout.” 3. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-PAYMENT-05: Payout ledgers must maintain itemized transparency between gross earnings, platform fees, and net disbursements.
Other Information:	N/A
Assumptions:	Bank statement descriptors match the registered platform business name.

2.2.3.4 UC-43.04 Fund Milestone

UC ID and Name	UC-43.04 Fund Milestone		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY

Primary Actor:	Client	Secondary Actors:	Stripe Payment Gateway
Trigger:	When a Client clicks “Deposit Funds” or “Fund Milestone” on a pending contract milestone.		
Description:	Allows a Client to securely deposit the required milestone amount into platform escrow using credit/debit cards or saved payment methods via Stripe, enabling the freelancer to begin work with guaranteed funding.		
Preconditions:	**PRE-1** The user is logged in as the contract Client. **PRE-2** The milestone paymentStatus is currently `PENDING`. **PRE-3** The contract platform fee has been paid or bundled.		
Post-conditions:	**POST-1** The payment is authorized and captured into platform escrow through Stripe. **POST-2** Milestone paymentStatus transitions to `FUNDED`. **POST-3** A transaction deposit record is logged and the Freelancer is notified that work can begin.		
Normal Flow:	**A. Fund Milestone via Payment Gateway** 1. The Client navigates to the milestone and clicks “Deposit into Escrow ($X,XXX)”. 2. The system presents the payment checkout modal with saved card options and new payment method inputs. 3. The Client selects a payment method (saved card or enters card credentials). 4. The Client clicks “Pay & Fund Milestone”. 5. The system initiates a Stripe Payment Intent or Checkout Session. 6. Stripe authenticates the payment (including 3D Secure verification if required by the issuing bank). 7. Upon successful payment confirmation, the system captures funds into escrow, sets milestone paymentStatus to `FUNDED`, and updates contract escrow balances. 8. The modal closes, a receipt notification is displayed, and the Freelancer receives an alert to commence deliverables.		
Alternative Flows:	**A.1 Use Saved Payment Method** 1. At step 3, the Client selects a pre-saved credit card. 2. The system charges the saved customer payment method directly without re-entering card numbers. 3. Proceed to step 7.		

Exceptions:	**EX-01: Card Declined** 1. If the issuing bank declines the card (insufficient funds, expired card, fraud check), Stripe returns an error. 2. The system displays: "Payment failed: The card was declined. Please try another payment method." 3. The milestone remains unfunded and the use case ends. **EX-02: Milestone Already Funded** 1. If the milestone is already funded by a concurrent session, the system prevents duplicate charges. 2. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-PAYMENT-06: Freelancers are not required or encouraged to begin work until the milestone paymentStatus is verified as `FUNDED`. BR-PAYMENT-07: Deposited escrow funds are held securely by the platform and cannot be unilaterally withdrawn without milestone completion, mutual cancellation, or dispute resolution.
Other Information:	N/A
Assumptions:	Client payment methods support online transactions in USD.

2.2.3.5 UC-43.05 Refund Payment

UC ID and Name	UC-43.05 Refund Payment		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Admin	Secondary Actors:	Stripe Payment Gateway
Trigger:	When a contract cancellation is approved or an Administrator issues an escrow refund.		
Description:	Allows the system or an Administrator to refund escrowed milestone funds back to the Client's original payment method via Stripe, adjusting escrow balances accordingly.		
Preconditions:	**PRE-1** The user is logged in as an authorized Client or Administrator. **PRE-2** The milestone has funds deposited (`paymentStatus = FUNDED`). **PRE-3** The contract cancellation or refund request satisfies platform refund eligibility rules.		

Post-conditions:	**POST-1** A refund request is executed via the Stripe Refund API against the original charge ID. **POST-2** Milestone paymentStatus transitions to `REFUNDED`. **POST-3** A refund transaction record is generated and both parties receive formal notifications.
Normal Flow:	**A. Execute Escrow Refund** 1. An Administrator (or mutual contract cancellation workflow) selects "Refund Milestone". 2. The system displays the refund dialog indicating the full refundable amount and original payment method. 3. The user enters a refund justification and clicks "Confirm Refund". 4. The system sends a refund request to Stripe targeting the original payment intent. 5. Stripe processes the refund back to the customer's funding source. 6. The system records the refund transaction, deducts the escrow balance, and sets milestone paymentStatus to `REFUNDED`. 7. The system sends an email receipt to the Client and updates the contract status.
Alternative Flows:	N/A
Exceptions:	**EX-01: Refund Already Processed** 1. If the charge has already been refunded, the system blocks the duplicate request. 2. The use case ends. **EX-02: Payment Gateway Refund Error** 1. If Stripe returns an error (e.g., charge disputed or card closed), the system alerts the administrator for manual bank wire resolution. 2. The use case ends.
Priority:	High
Frequency of Use:	Low
Business Rules:	BR-PAYMENT-08: Refunds must strictly return to the original payment method used for the escrow deposit. BR-PAYMENT-09: Platform fees are non-refundable unless authorized by platform dispute policy.
Other Information:	N/A

Assumptions:	Bank processing of card refunds takes 5-10 business days depending on the customer's financial institution.

2.2.3.6 UC-43.06 Pay Platform Fee

UC ID and Name	UC-43.06 Pay Platform Fee		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	Stripe Payment Gateway
Trigger:	When a Client hires a Freelancer and activates a contract requiring platform service fees.		
Description:	Allows a Client to pay the one-time contract initiation or platform service fee via Stripe Payment Intent, Checkout Session, or saved credit card.		
Preconditions:	**PRE-1** The user is logged in as a Client. **PRE-2** The contract has been accepted and is awaiting platform fee settlement.		
Post-conditions:	**POST-1** The platform fee charge is captured successfully. **POST-2** The contract record marks platform fee as paid. **POST-3** A platform fee invoice receipt is generated for the Client.		
Normal Flow:	**A. Pay Platform Fee** 1. The Client views the contract activation checkout summary. 2. The system details the calculated platform service fee (e.g., standard platform commission). 3. The Client selects payment method (saved card or new card). 4. The Client clicks "Pay Platform Fee". 5. The system initiates and executes the Stripe charge. 6. The system records the platform fee transaction in database ledger. 7. The system advances contract state, unlocks escrow funding steps, and displays a success confirmation.		
Alternative Flows:	**A.1 Combined Fee and Milestone Payment** 1. The Client opts to pay the platform fee simultaneously with the first milestone funding. 2. The system bundles both charges into a single checkout session.		

Exceptions:	**EX-01: Payment Rejection** 1. If the charge is declined, the system displays: "Platform fee payment failed. Please check your payment details." 2. The contract remains in pending status.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-PAYMENT-10: Contract activation requires successful platform fee settlement.
Other Information:	N/A
Assumptions:	Platform fee percentage is determined by global platform business rules.

2.2.3.7 UC-43.07 Release Payment

UC ID and Name	UC-43.07 Release Payment		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client	Secondary Actors:	Stripe Connect Transfer
Trigger:	When a Client approves submitted milestone deliverables and confirms escrow release.		
Description:	Allows a Client to authorize the immediate release of escrowed milestone funds to the Freelancer, triggering a Stripe Connect transfer to the Freelancer's connected account.		
Preconditions:	**PRE-1** The user is logged in as the contract Client. **PRE-2** The milestone is in `FUNDED` status. **PRE-3** The Freelancer has an active, verified Stripe Connect payout account.		
Post-conditions:	**POST-1** A Stripe transfer is dispatched transferring net milestone funds to the Freelancer's account. **POST-2** Milestone status transitions to `COMPLETED` and paymentStatus to `RELEASED`. **POST-3** Both parties receive immediate settlement confirmations.		

Normal Flow:	**A. Release Milestone Escrow Payment** 1. The Client reviews submitted deliverables and clicks “Approve & Release Funds”. 2. The system renders the release confirmation modal detailing the amount released to the Freelancer. 3. The Client clicks “Confirm Release”. 4. The system executes a database transaction marking submission `APPROVED` and milestone paymentStatus `RELEASED`. 5. The system initiates a Stripe Transfer to the Freelancer’s Stripe Connect destination account. 6. The system debits the contract escrow balance and logs the release transaction. 7. The system closes the modal, renders a completed badge, and sends an alert to the Freelancer.
Alternative Flows:	**A.1 Final Contract Release** 1. The released milestone is the final milestone in the contract. 2. Upon release, the system prompts the Client to complete the contract and leave a rating review.
Exceptions:	**EX-01: Freelancer Stripe Account Missing or Restricted** 1. If the Freelancer’s Stripe account is unlinked or has payouts disabled, the system displays: “Freelancer must complete payout onboarding before funds can be released.” 2. Escrow funds remain secured in the platform until payout requirements are fulfilled. 3. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-PAYMENT-11: Escrow releases are irreversible once the Stripe transfer has been executed. BR-PAYMENT-12: The transferred amount equals the agreed milestone amount minus any applicable platform fee share.
Other Information:	N/A
Assumptions:	Stripe Connect transfers settle instantly into the freelancer’s platform balance.

2.2.3.8 UC-43.08 Pay Dispute Fee

UC ID and Name	UC-43.08 Pay Dispute Fee

Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer	Secondary Actors:	Stripe Payment Gateway
Trigger:	When a dispute party clicks “Pay Arbitration Fee” on their pending dispute case.		
Description:	Allows a Client or Freelancer involved in an active dispute to pay their mandatory arbitration fee within the 24-hour deadline via Stripe.		
Preconditions:	**PRE-1** The user is authenticated as an involved dispute participant. **PRE-2** The dispute fee status is `PENDING`. **PRE-3** The current time is within the 24-hour fee payment window.		
Post-conditions:	**POST-1** The arbitration fee charge is processed via Stripe. **POST-2** Fee status transitions to `PAID` with recorded payment reference. **POST-3** The case proceeds toward arbitration review.		
Normal Flow:	**A. Pay Dispute Arbitration Fee** 1. The user navigates to the Dispute Detail page. 2. The user views the fee invoice banner with the remaining countdown timer. 3. The user clicks “Pay Arbitration Fee ($XX.XX)”. 4. The system presents the payment interface (Payment Intent or Checkout Session). 5. The user enters or selects payment card details and authorizes the payment. 6. Stripe confirms successful capture of the fee. 7. The system updates the fee record status to `PAID` with a timestamp. 8. The page refreshes showing a green “Fee Paid” badge and payment receipt.		
Alternative Flows:	N/A		

Exceptions:	**EX-01: Fee Deadline Expired** 1. If the 24-hour deadline has elapsed, the fee status becomes `OVERDUE`. 2. The system informs the user: "The fee payment deadline has expired. This may impact arbitrator review." 3. The use case ends. **EX-02: Fee Already Paid** 1. If the user attempts duplicate payment, the system displays: "Fee has already been paid." 2. The use case ends.
Priority:	High
Frequency of Use:	Low
Business Rules:	BR-PAYMENT-13: Both disputing parties must pay an equal arbitration fee to ensure fair mutual commitment. BR-PAYMENT-14: Arbitration fees are refunded to the prevailing party or split based on final dispute resolution findings.
Other Information:	N/A
Assumptions:	Arbitration fee amount is fixed platform-wide (e.g., $25.00).

2.2.3.9 UC-43.09 View Payment History

UC ID and Name	UC-43.09 View Payment History		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user clicks on the "Transactions" or "Billing History" tab in their account settings.		
Description:	Allows users to view a comprehensive, paginated transaction ledger of all platform financial activities, including milestone deposits, escrow releases, refunds, platform fees, and dispute fee payments.		
Preconditions:	**PRE-1** The user is authenticated in the system. **PRE-2** Database connection is operational.		
Post-conditions:	**POST-1** A paginated list of transaction records is displayed with date, type, amount, status, and related contract/milestone.		

Normal Flow:	**A. View Transaction History** 1. The user navigates to the Billing & Transactions page. 2. The system queries transactions matching the user ID (Client, Freelancer, or all for Admin). 3. The system sorts records chronologically with newest transactions first. 4. The system renders the transaction table displaying: Transaction ID, Timestamp, Type (Deposit, Release, Fee, Refund), Amount, Status, and Action buttons.
Alternative Flows:	**A.1 Filter by Transaction Type** 1. The user selects a filter dropdown (Deposits, Releases, Refunds, Fees). 2. The system updates the list to display only records matching the chosen category. 3. Return to step A.4.
Exceptions:	**EX-01: No Transactions Found** 1. If the user has no transaction history, the system displays: "No transactions found." 2. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-PAYMENT-15: Non-admin users are strictly restricted to transactions where their account is the payer or payee. BR-PAYMENT-16: Financial transaction histories are immutable and cannot be deleted or purged by users.
Other Information:	N/A
Assumptions:	Transactions are logged concurrently with payment gateway confirmations.

2.2.3.10 UC-43.10 View Payment Detail

UC ID and Name	UC-43.10 View Payment Detail		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, Freelancer, Admin	Secondary Actors:	N/A
Trigger:	When a user clicks on a transaction row in the Payment History table.		

Description:	Displays detailed transaction receipt information, including payment gateway references, itemized fee breakdowns, currency conversion, and downloadable PDF receipt.
Preconditions:	**PRE-1** The user is logged in. **PRE-2** The transaction record exists in the system. **PRE-3** The user is an authorized party to the transaction or an Administrator.
Post-conditions:	**POST-1** Detailed receipt view is presented on screen with an option to download a PDF receipt.
Normal Flow:	**A. View Payment Detail** 1. The user clicks a transaction item. 2. The system validates authorization and queries transaction details from the database. 3. The system renders the transaction receipt sheet displaying: - Receipt Number and Transaction Reference ID. - Associated Contract and Milestone details. - Payer and Payee profiles. - Itemized Breakdown: Base amount, Platform fee, Processing fee, Total charged/transferred. - Payment Method snippet (e.g., Visa ending in 4242). - Timestamp and Payment Gateway status code. 4. The user clicks “Download Receipt” to download a formal PDF invoice.
Alternative Flows:	N/A
Exceptions:	**EX-01: Unauthorized Inspection** 1. An unauthorized user attempts to access a transaction by URL. 2. The system denies access with “Permission Denied.” 3. The use case ends.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-PAYMENT-17: Receipts must provide full tax, fee, and identification details to comply with e-commerce accounting standards.
Other Information:	N/A
Assumptions:	PDF receipts are generated dynamically from verified transaction records.

2.2.3.11 UC-43.11 Execute Dispute Resolution Payment

UC ID and Name	UC-43.11 Execute Dispute Resolution Payment

Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Admin, System	Secondary Actors:	Stripe Payment Gateway
Trigger:	When both dispute parties accept a proposed resolution or an Administrator issues a final arbitration settlement.		
Description:	Executes the automated financial settlement of a disputed milestone by refunding the awarded client portion to the Client's card and transferring the awarded freelancer portion to the Freelancer's Stripe Connect account.		
Preconditions:	**PRE-1** The dispute has reached a finalized decision state. **PRE-2** The awarded split amounts (`clientAmount` and `freelancerAmount`) sum up to the total milestone escrow balance. **PRE-3** Escrow funds remain secured and intact in platform custody.		
Post-conditions:	**POST-1** Awarded client amount is refunded via Stripe Refund to the Client. **POST-2** Awarded freelancer amount is transferred via Stripe Transfer to the Freelancer's connected account. **POST-3** The dispute status transitions to `FINALIZED` and milestone status is resolved. **POST-4** Immutable audit ledger entries are created for both financial legs.		

Normal Flow:	**A. Execute Settlement Payouts** 1. The system triggers the dispute settlement workflow upon mutual agreement or administrative finalization. 2. The database transaction opens: - Validates that `clientAmount + freelancerAmount == milestoneAmount`. - Sets dispute status to `FINALIZED`. 3. If `clientAmount > 0`: - The system invokes the Stripe Refund API to return the allocated funds to the Client's payment source. 4. If `freelancerAmount > 0`: - The system invokes the Stripe Transfer API to transfer the allocated funds to the Freelancer's Stripe Connect account. 5. The system records transaction ledger records for both legs with Stripe transfer IDs. 6. The system commits the database transaction and updates milestone paymentStatus to `RELEASED`. 7. Settlement notifications and breakdown receipts are transmitted to both Client and Freelancer.
Alternative Flows:	**A.1 100% Allocation to One Party** 1. If the entire escrow is awarded to the Client: only the Stripe refund is executed. 2. If the entire escrow is awarded to the Freelancer: only the Stripe transfer is executed.
Exceptions:	**EX-01: Transfer Failure to Freelancer** 1. If the Stripe transfer fails (e.g., freelancer bank restriction), the client refund is held or logged, and an emergency alert is escalated to platform financial ops. 2. The use case ends. **EX-02: Amount Inconsistency** 1. If split amounts do not reconcile with the escrow balance, the settlement is aborted. 2. The use case ends.
Priority:	High
Frequency of Use:	Low
Business Rules:	BR-PAYMENT-18: Dispute settlements must execute atomically; either both legs succeed or the failure is flagged for manual administrative intervention. BR-PAYMENT-19: Dispute resolution payouts are legally final and cannot be reopened on the platform.
Other Information:	N/A

Assumptions:	Stripe API operates with high availability and idempotency keys to prevent duplicate payout execution.

2.2.3.12 UC-43.12 Handle Stripe Webhook Event

UC ID and Name	UC-43.12 Handle Stripe Webhook Event		
Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Stripe Webhook Service	Secondary Actors:	System
Trigger:	When Stripe transmits an asynchronous HTTP POST webhook event to `/api/payments/webhook`.		
Description:	Listens for, verifies, and processes asynchronous Stripe events (e.g., `payment_intent.succeeded`, `checkout.session.completed`, `transfer.created`, `account.updated`), reconciling database state with payment gateway events.		
Preconditions:	**PRE-1** The incoming request contains a valid `stripe-signature` header. **PRE-2** The platform webhook signing secret is configured.		
Post-conditions:	**POST-1** The event signature is cryptographically verified. **POST-2** Associated milestone, payment, or account records are updated in the database. **POST-3** HTTP 200 OK is returned to Stripe to acknowledge event receipt.		

Normal Flow:	**A. Process Stripe Webhook** 1. Stripe dispatches a webhook event payload with its signature header. 2. The system verifies the cryptographic signature against the endpoint signing secret. 3. The system parses the event type: - If `payment_intent.succeeded` or `checkout.session.completed`: updates milestone paymentStatus to `FUNDED` and records transaction. - If `account.updated`: refreshes the Freelancer's Stripe Connect status (`payouts_enabled`). - If `charge.refunded`: verifies that milestone refund records match the refund ID. 4. The system commits database updates. 5. The system responds with HTTP status 200 to Stripe within 3 seconds.
Alternative Flows:	**A.1 Unhandled Event Type** 1. Stripe dispatches an informational event not subscribed to by platform logic. 2. The system logs the event type and returns HTTP 200 without making state changes.
Exceptions:	**EX-01: Invalid Webhook Signature** 1. If the signature is missing or fails HMAC verification, the system rejects the request with HTTP 400 Bad Request. 2. No database changes are made. 3. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-PAYMENT-20: Webhook processing must be idempotent to prevent duplicate database mutations from retried Stripe events. BR-PAYMENT-21: Requests failing signature verification must be immediately dropped.
Other Information:	N/A
Assumptions:	Stripe automatically retries unacknowledged webhook events over 72 hours.

2.2.3.13 UC-43.13 Sync Payment Intent Status

UC ID and Name	UC-43.13 Sync Payment Intent Status

Created By:	Vo Nhut Tin	Date Created:	DD/MM/YYYY
Primary Actor:	Client, System	Secondary Actors:	Stripe API
Trigger:	When a user completes a Stripe Checkout redirect or when client-side confirms a Payment Intent and requests immediate state verification.		
Description:	Allows the client application or system to actively poll and synchronize the status of a specific Stripe Payment Intent or Checkout Session, updating local database records without waiting solely for asynchronous webhooks.		
Preconditions:	**PRE-1** The user is authenticated in the system. **PRE-2** A valid Payment Intent ID or Checkout Session ID is provided.		
Post-conditions:	**POST-1** The platform queries the Stripe API in real time. **POST-2** Local payment, milestone, and contract records are synchronized to match the Stripe authoritative state. **POST-3** The updated status is returned to the client UI to immediately unlock funded views.		
Normal Flow:	**A. Synchronize Payment Status** 1. Following payment authorization or return from Stripe Checkout, the frontend issues a sync request with the Payment Intent ID or Session ID. 2. The system checks user authorization to inspect the transaction. 3. The system queries the Stripe API to fetch the authoritative status of the Payment Intent. 4. If Stripe reports `status = succeeded`: - The system verifies whether the milestone is already marked `FUNDED`. - If still pending, the system updates milestone paymentStatus to `FUNDED` and records the deposit transaction. 5. The system returns the synchronized status and updated milestone entity to the frontend. 6. The UI immediately transitions to show the milestone as funded and dismisses the loading state.		

Alternative Flows:	**A.1 Payment Still Pending or Processing** 1. If Stripe reports the payment is still processing (e.g., bank transfer or 3DS pending), the system retains the pending status and informs the client. 2. The frontend displays a waiting indicator.
Exceptions:	**EX-01: Payment Intent Not Found** 1. If the provided payment identifier does not exist on Stripe, the system displays: "Invalid payment reference." 2. The use case ends. **EX-02: Payment Canceled or Failed** 1. If Stripe indicates the payment was canceled or failed, the system records the failure reason and updates local status to `FAILED`. 2. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-PAYMENT-22: Active sync operations serve as a fail-safe complement to asynchronous Stripe webhooks, eliminating latency for the end user. BR-PAYMENT-23: Idempotent database operations ensure that concurrent sync and webhook updates never create duplicate financial transactions.
Other Information:	N/A
Assumptions:	Stripe API responds within normal low-latency parameters (< 500ms).

2.2.24 Manage Advertisement Banner

2.2.3.1 UC-18.1 Advertisement Banners List

UC ID and Name	UC-18.1 Advertisement Banners List		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin navigates to the Banners Management section from the admin dashboard.		

Description:	Allows an administrator to retrieve the list of advertisement banners with their id, title, image, link, position, active period, and status, with support for search, filtering by position and deleted status, sorting, and pagination.
Preconditions:	**PRE-1** The user is authenticated as an Admin whose role is allowed to access the banners endpoints (guarded by `AuthGuard` and `PermissionGuard`). **PRE-2** The system is connected to the database.
Post-conditions:	**POST-1** A paginated list of banners is displayed with `id`, `title`, `imageUrl`, `linkUrl`, `position`, `startDate`, `endDate`, `isActive`, `deletedAt`, `createdAt`, and `updatedAt`. **POST-2** The admin can create (UC-18.3), edit (UC-18.4), delete (UC-18.5), or open a banner detail (UC-18.2).
Normal Flow:	**A. View Advertisement Banners List** 1. The Admin navigates to the Banners Management page (`/admin/banners`). 2. The system sends `GET /api/admin/banners` with `page` (default 1), `limit` (default 10, max 100), and optionally `search`, `deleted`, `position`, `sortBy`, and `sortOrder`. 3. The system queries the AdvertisementBanner table applying the keyword search on `title` (case-insensitive), the `position` filter, and the `deleted` filter (`true`, `false`, or absent). 4. The system sorts the result by `id` or `createdAt` in ascending or descending order (default `desc`). 5. The system returns the `banners` array and the pagination metadata `{ page, limit, total, totalPages }`. 6. The system displays each banner row (image, title, position, start/end dates, status) in the BannersTable together with the BannersFilterBar. 7. The Admin can search by title, filter by position or status (All/Active/Deleted), sort, or paginate.

Alternative Flows:	**A.1 Search Banners by Title** 1. The Admin enters a keyword in the search box. 2. The system re-queries banners whose `title` contains the keyword (case-insensitive). 3. The system refreshes the displayed list. 4. Return to step 6. **A.2 Filter Banners by Position** 1. The Admin selects a position in the position dropdown (`GLOBAL_HEADER`, `HOME_HERO`, `HOME_BODY`, `SEARCH_RESULTS`, `FOOTER`). 2. The system re-queries banners whose `position` equals the selected value. 3. The system refreshes the displayed list. 4. Return to step 6. **A.3 Filter Banners by Status** 1. The Admin selects `All`, `Active`, or `Deleted` in the status filter. 2. The system re-queries banners with the corresponding `deleted` flag (absent for All, false for Active, true for Deleted). 3. The system refreshes the displayed list. 4. Return to step 6. **A.4 No Banners Found** 1. The system determines that no banners match the current search or filters. 2. The system displays an empty table row. 3. The Admin remains on the page. 4. Return to step 6.

Exceptions:	**EX-01: Database Error** 1. The system attempts to load the banners from the database. 2. The database operation fails due to an internal database error. 3. The system returns the message “Error.FailedToLoadBanners”. 4. The frontend displays the message “Couldn’t load banners. Try again.” 5. The banner list is not displayed. 6. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the banner data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The Admin remains on the current page without any banner information being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	High
Business Rules:	**BR-BAN-01:** The endpoint `GET /api/admin/banners` is guarded by `AuthGuard` and `PermissionGuard`. **BR-BAN-02:** Search shall match the banner `title` in a case-insensitive manner. **BR-BAN-03:** The `position` filter accepts only `GLOBAL_HEADER`, `HOME_HERO`, `HOME_BODY`, `SEARCH_RESULTS`, and `FOOTER`. **BR-BAN-04:** The `deleted` filter returns only active banners when `false` or absent, and only soft-deleted banners when `true`. **BR-BAN-05:** Sorting is restricted to the fields `id` and `createdAt`; `sortOrder` is `asc` or `desc` (default `desc`). **BR-BAN-06:** Pagination shall support `page >= 1` and `limit` between 1 and 100.
Other Information:	The list page provides Create, Edit, Delete, and View actions to start UC-18.3, UC-18.4, UC-18.5, and UC-18.2.
Assumptions:	Banner records are stored in the AdvertisementBanner table of the system database.

2.2.3.2 UC-18.2 Banner Detail

UC ID and Name	UC-18.2 Banner Detail		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin selects a banner from the banner list (UC-18.1) or navigates directly to a banner detail URL.		
Description:	Allows an administrator to view the detailed information of a single advertisement banner, including its id, title, image, link, position, active period, status, and timestamps. The detail remains accessible even when the banner is soft-deleted.		
Preconditions:	**PRE-1** The user is authenticated as an Admin whose role is allowed to access `GET /api/admin/banners/:id` (guarded by `AuthGuard` and `PermissionGuard`). **PRE-2** The selected banner exists in the database.		
Post-conditions:	**POST-1** The detailed information of the selected banner is displayed, including the `deletedAt` field. **POST-2** The admin can edit (UC-18.4) or delete (UC-18.5) the displayed banner.		
Normal Flow:	**A. View Banner Detail** 1. The Admin clicks the view action on a banner row (UC-18.1) or navigates directly to `/admin/banners/:id`. 2. The system sends `GET /api/admin/banners/:id`. 3. The system queries the banner with the matching `id`. 4. The system verifies that the banner exists. 5. The system returns the banner object with `id`, `title`, `imageUrl`, `linkUrl`, `position`, `startDate`, `endDate`, `isActive`, `deletedAt`, `createdAt`, and `updatedAt`. 6. The system displays the banner details together with Edit and Delete actions.		

Alternative Flows:	**A.1 Banner Not Found** 1. The Admin opens a banner detail page. 2. The system determines that no banner exists with the given `id`. 3. The system returns the message “Error.BannerNotFound” and the frontend displays “Failed to load banner detail. The banner may not exist.” 4. The Admin returns to the banner list. 5. Return to step 1. **A.2 Invalid Banner ID** 1. The Admin navigates to a banner detail URL with a non-positive or non-integer `id`. 2. The system displays the Not Found page. 3. The use case ends.
Exceptions:	**EX-01: Database Error** 1. The system attempts to load the banner detail from the database. 2. The database operation fails due to an internal database error. 3. The system returns the message “Error.FailedToLoadBannerDetail”. 4. The banner details are not displayed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the banner detail data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The Admin remains on the current page without banner information being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	**BR-BAN-07:** A banner detail shall be accessible only when a banner with the given `id` exists. **BR-BAN-08:** The detail shall include `id`, `title`, `imageUrl`, `linkUrl`, `position`, `startDate`, `endDate`, `isActive`, `deletedAt`, `createdAt`, and `updatedAt`; the `deletedAt` value is returned even when the banner is soft-deleted.

Other Information:	A soft-deleted banner can still be opened in the detail page, which is used to reach the Edit (UC-18.4) and Delete (UC-18.5) flows.
Assumptions:	The banner id is a positive integer.

2.2.3.3 UC-18.3 Create Banner

UC ID and Name	UC-18.3 Create Banner		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks the “Create” button on the Banners Management page (UC-18.1).		
Description:	Allows an administrator to create a new advertisement banner by providing a title (required), an image, an optional link, a position, optional start/end dates, and an active status.		
Preconditions:	**PRE-1** The user is authenticated as an Admin whose role is allowed to access `POST /api/admin/banners` (guarded by `AuthGuard` and `PermissionGuard`). **PRE-2** The system is connected to the database.		
Post-conditions:	**POST-1** A new record is created in the AdvertisementBanner table with the provided fields and `isActive` (default `true`). **POST-2** The new banner appears in the banner list (UC-18.1).		

Normal Flow:	**A. Create Banner** 1. The Admin clicks the "Create" button on the Banners Management page (`/admin/banners`). 2. The system opens the BannerFormDialog with fields: `title` (required), `imageUrl` (with an upload control), `linkUrl`, `position` select (default `GLOBAL_HEADER`), `startDate`/`endDate` date pickers, and an `isActive` toggle (default `true`). 3. The Admin fills in the fields; if an image is selected, the system sends `POST /api/admin/banners/upload-image` (multipart, max 10MB, allowed types `image/jpeg`, `image/png`, `image/gif`, `image/webp`, `image/bmp`, `image/tiff`) and stores the returned `imageUrl`. 4. The Admin clicks the Save button. 5. The system validates the request body. 6. The system sends `POST /api/admin/banners` with `title`, `imageUrl`, `linkUrl`, `position`, `startDate`, `endDate`, and `isActive`. 7. The system checks that no other active banner (`deletedAt = NULL`) occupies the selected `position`. 8. The system checks that `startDate` is not later than `endDate` when both are present. 9. The system creates the banner record and returns the created banner detail. 10. The system displays the message "Banner created successfully!" and refreshes the banner list.

Alternative Flows:	**A.1 Missing Required Title** 1. The Admin submits the form. 2. The system determines that `title` is empty. 3. The frontend disables submission until a title is provided; the backend returns “Error.BannerTitleRequired” if submitted empty. 4. Return to step 3. **A.2 Title Too Long** 1. The Admin submits the form. 2. The system determines that `title` exceeds 255 characters. 3. The system returns the message “Error.BannerTitleTooLong”. 4. Return to step 3. **A.3 Image URL or Link URL Too Long** 1. The Admin submits the form. 2. The system determines that `imageUrl` or `linkUrl` exceeds 500 characters. 3. The system returns “Error.BannerImageUrlTooLong” or “Error.BannerLinkUrlTooLong”. 4. Return to step 3. **A.4 Position Already Taken** 1. The Admin submits the form. 2. The system determines that another active banner (`deletedAt = NULL`) already occupies the selected `position`. 3. The system returns the message “Error.BannerPositionTaken” (409). 4. Return to step 3. **A.5 Invalid Date Range** 1. The Admin submits the form. 2. The system determines that `startDate` is later than `endDate`. 3. The system returns the message “Error.BannerInvalidDateRange”. 4. Return to step 3. **A.6 No Image Selected** 1. The Admin attempts to upload without a file. 2. The system returns the message “Error.BannerImageRequired” (400). 3. Return to step 3. **A.7 Invalid Image Type** 1. The Admin uploads a file whose type is not supported. 2. The system returns the message “Error.BannerImageInvalidType” (400). 3. Return to step 3. **A.8 Image Too Large**

Exceptions:	**EX-01: Database Error** 1. The system attempts to save the banner record in the database. 2. The database operation fails due to an internal database error. 3. The system returns the message “Error.FailedToCreateBanner”. 4. The frontend displays “Failed to create banner. Try again.” 5. The banner is not created. 6. The use case ends. **EX-02: Image Upload Failure** 1. The system attempts to upload the image. 2. The Cloudinary (or local storage) operation fails. 3. The system returns the message “Error.FailedToUploadBannerImage”. 4. The banner is not created. 5. The use case ends. **EX-03: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while creating the banner. 2. The system cannot save the banner data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The Admin remains on the dialog without the banner being saved. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Low

Business Rules:	**BR-BAN-09:** The banner `title` is required and shall not exceed 255 characters. **BR-BAN-10:** `imageUrl` and `linkUrl` are optional, must be valid URLs when provided, and shall not exceed 500 characters. **BR-BAN-11:** `position` defaults to `GLOBAL_HEADER` and shall be one of the banner position enum values. **BR-BAN-12:** Only one active banner (`deletedAt = NULL`) is allowed per `position`; otherwise `Error.BannerPositionTaken` (409) is returned. **BR-BAN-13:** When both `startDate` and `endDate` are present, `startDate` must be earlier than or equal to `endDate`; otherwise `Error.BannerInvalidDateRange`. **BR-BAN-14:** `isActive` defaults to `true` when not provided. **BR-BAN-15:** Uploaded images must be at most 10MB and of type `image/jpeg`, `image/png`, `image/gif`, `image/webp`, `image/bmp`, or `image/tiff`.
Other Information:	The image is uploaded through `POST /api/admin/banners/upload-image`; it is uploaded to Cloudinary when configured, otherwise it is saved locally, and `{ imageUrl, publicId }` is returned.
Assumptions:	The banner image is provided either by upload or by an image URL.

2.2.3.4 UC-18.4 Edit Banner

UC ID and Name	UC-18.4 Edit Banner		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks the edit action on a banner row (UC-18.1) or on the banner detail page (UC-18.2).		
Description:	Allows an administrator to update an existing advertisement banner's title, image, link, position, active period, or active status. All fields are optional in the update request.		
Preconditions:	**PRE-1** The user is authenticated as an Admin whose role is allowed to access `PATCH /api/admin/banners/:id` (guarded by `AuthGuard` and `PermissionGuard`). **PRE-2** The selected banner exists in the database.		

Post-conditions:	**POST-1** The banner's `title`, `imageUrl`, `linkUrl`, `position`, `startDate`, `endDate`, and/or `isActive` are updated in the AdvertisementBanner table. **POST-2** The updated banner is displayed in the banner list (UC-18.1).
Normal Flow:	**A. Edit Banner** 1. The Admin clicks the edit action on a banner row (UC-18.1) or on the banner detail page (UC-18.2). 2. The system opens the BannerFormDialog pre-filled with the current banner values (`title`, `imageUrl`, `linkUrl`, `position`, `startDate`, `endDate`, `isActive`). 3. The Admin modifies the desired fields; the image can be replaced through `POST /api/admin/banners/upload-image`. 4. The Admin clicks the Save Changes button. 5. The system sends `PATCH /api/admin/banners/:id` with only the changed fields. 6. The system verifies that the banner exists. 7. If `position` is changed, the system checks that the new position is not taken by another active banner (`deletedAt = NULL`), excluding the banner itself. 8. The system applies the same date range rule as creation when both `startDate` and `endDate` are present. 9. The system updates the banner record and returns the updated banner detail. 10. The system displays the message "Banner updated successfully!" and refreshes the page.

Alternative Flows:	**A.1 Banner Not Found** 1. The Admin submits the update request. 2. The system determines that the banner does not exist. 3. The system returns the message “Error.BannerNotFound” (404). 4. The Admin returns to the banner list. 5. Return to step 1. **A.2 Position Already Taken** 1. The Admin submits the update request. 2. The system determines that another active banner already occupies the new `position`. 3. The system returns the message “Error.BannerPositionTaken” (409). 4. Return to step 3. **A.3 Invalid Date Range** 1. The Admin submits the update request. 2. The system determines that `startDate` is later than `endDate`. 3. The system returns the message “Error.BannerInvalidDateRange”. 4. Return to step 3. **A.4 Validation Errors** 1. The Admin submits the update request. 2. The system determines that `title` is empty, exceeds 255 characters, or that `imageUrl`/`linkUrl` exceed 500 characters. 3. The system returns the corresponding validation error (`BannerTitleRequired`, `BannerTitleTooLong`, `BannerImageUrlTooLong`, `BannerLinkUrlTooLong`). 4. Return to step 3. **A.5 Admin Cancels Editing** 1. The Admin clicks the “Cancel” button in the BannerFormDialog. 2. The system discards the unsaved changes and closes the dialog. 3. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to update the banner record in the database. 2. The database operation fails due to an internal database error. 3. The system returns the message "Error.FailedToUpdateBanner". 4. The frontend displays "Failed to update banner." 5. The banner is not updated. 6. The use case ends. **EX-02: Image Upload Failure** 1. The system attempts to upload a replacement image. 2. The image upload operation fails. 3. The system returns the message "Error.FailedToUploadBannerImage". 4. The banner is not updated. 5. The use case ends. **EX-03: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while saving the changes. 2. The system cannot save the updated banner data. 3. The system displays the message "Cannot connect to the server. Please try again later." 4. The Admin remains on the dialog without the changes being saved. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Low

Business Rules:	**BR-BAN-16:** All fields (`title`, `imageUrl`, `linkUrl`, `position`, `startDate`, `endDate`, `isActive`) are optional in the update request (partial update). **BR-BAN-17:** When `position` is changed, the new position must not be taken by another active banner, excluding the banner itself; otherwise `Error.BannerPositionTaken` (409). **BR-BAN-18:** The date range rule (`startDate` earlier than or equal to `endDate` when both are present) also applies on update. **BR-BAN-19:** An update shall succeed only when the banner with the given `id` exists; otherwise `Error.BannerNotFound` (404). **BR-BAN-20:** The original banner `id`, `createdAt`, and `deletedAt` remain unchanged after the update.
Other Information:	When the position is unchanged, the position uniqueness check is skipped.
Assumptions:	The admin remains authenticated throughout the editing process.

2.2.3.5 UC-18.5 Delete Banner

UC ID and Name	UC-18.5 Delete Banner		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks the delete action on a banner row (UC-18.1) or on the banner detail page (UC-18.2).		
Description:	Allows an administrator to soft-delete an existing advertisement banner by setting its `deletedAt` field to the current timestamp.		
Preconditions:	**PRE-1** The user is authenticated as an Admin whose role is allowed to access `DELETE /api/admin/banners/:id` (guarded by `AuthGuard` and `PermissionGuard`). **PRE-2** The selected banner exists in the database.		
Post-conditions:	**POST-1** The banner is soft-deleted by setting its `deletedAt` field to the current timestamp. **POST-2** The deleted banner no longer appears in the active banner list (UC-18.1) unless the Deleted filter is applied.		

Normal Flow:	**A. Delete Banner** 1. The Admin clicks the delete action on a banner row (UC-18.1) or on the banner detail page (UC-18.2). 2. The system opens the DeleteBannerDialog with a confirmation message. 3. The Admin confirms the deletion request. 4. The system sends `DELETE /api/admin/banners/:id`. 5. The system verifies that the banner exists. 6. The system sets the `deletedAt` field of the banner to the current timestamp. 7. The system returns the message “Banner deleted successfully.” and refreshes the banner list.
Alternative Flows:	**A.1 Banner Not Found** 1. The Admin confirms the deletion request. 2. The system determines that the banner does not exist. 3. The system returns the message “Error.BannerNotFound” (404). 4. The Admin returns to the banner list. 5. Return to step 1. **A.2 Admin Cancels Deletion** 1. The Admin clicks the “Cancel” button in the DeleteBannerDialog. 2. The system closes the dialog. 3. The banner remains unchanged. 4. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to update the `deletedAt` field of the banner in the database. 2. The database operation fails due to an internal database error. 3. The system returns the message "Error.FailedToDeleteBanner". 4. The frontend displays "Failed to delete banner." 5. The banner is not deleted. 6. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the deletion request. 2. The system cannot complete the deletion of the banner. 3. The system displays the message "Cannot connect to the server. Please try again later." 4. The banner remains unchanged. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	**BR-BAN-21:** Banner deletion shall be implemented as a soft delete by setting the `deletedAt` field to the current timestamp. **BR-BAN-22:** A deletion shall succeed only when the banner with the given `id` exists; otherwise `Error.BannerNotFound` (404). **BR-BAN-23:** A soft-deleted banner is excluded from the active banner list by default, and the position occupied by it becomes available for a new active banner.
Other Information:	Deleted banners remain stored in the database and are still accessible through the banner detail endpoint (`GET /api/admin/banners/:id`).
Assumptions:	The admin remains authenticated during the deletion process.

2.2.25 Manage Skill

2.2.3.1 UC-19.1 View List Skill

UC ID and Name	UC-19.1 View List Skill		
Created By:	ThuanNT	Date Created:	14/09/2026

Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin navigates to the Skills Management section from the admin dashboard.		
Description:	Allows an administrator to retrieve the list of skills with their id, name, slug, description, status, creation date, and the number of jobs using each skill, with support for search, filtering by active status, sorting, and pagination.		
Preconditions:	**PRE-1** The user is authenticated as an Admin whose role is allowed to access the skills endpoints (guarded by `AuthGuard` and `PermissionGuard`). **PRE-2** The system is connected to the database.		
Post-conditions:	**POST-1** A paginated list of skills is displayed with `id`, `name`, `slug`, `description`, `deletedAt`, `createdAt`, `updatedAt`, and `jobCount`. **POST-2** The admin can create (UC-19.3), edit (UC-19.4), delete (UC-19.5), restore (UC-19.6), or open a skill detail (UC-19.2).		
Normal Flow:	**A. View List Skill** 1. The Admin navigates to the Skills Management page (`/admin/skills`). 2. The system sends `GET /api/admin/skills` with `page` (default 1), `limit` (default 10, max 100), and optionally `search`, `deleted`, `sortBy`, and `sortOrder`. 3. The system queries the Skill table applying the keyword search on `name`, `slug`, or `description` (case-insensitive) and the `deleted` filter (`true`, `false`, or absent). 4. The system sorts the result by `id` or `createdAt` in ascending or descending order (default `desc`). 5. The system computes `jobCount` for each skill (count of JobSkill records where `job.deletedAt` is `null`). 6. The system returns the `skills` array and the pagination metadata `{ page, limit, total, totalPages }`. 7. The system displays the SkillsTable columns: ID, Name, Slug, Status (Active or Deleted badge), Jobs using it, Created, and Actions (edit/delete for active skills, restore for deleted skills, view details).		

Alternative Flows:	**A.1 Search Skills** 1. The Admin enters a keyword in the search box. 2. The system re-queries skills whose `name`, `slug`, or `description` contains the keyword (case-insensitive). 3. The system refreshes the displayed list. 4. Return to step 7. **A.2 Filter Skills by Status** 1. The Admin selects `All`, `Active`, or `Deleted` in the Filter Status dropdown. 2. The system re-queries skills with the corresponding `deleted` flag (absent for All, false for Active, true for Deleted). 3. The system refreshes the displayed list. 4. Return to step 7. **A.3 No Skills Found** 1. The system determines that no skills match the current search or filters. 2. The system displays an empty table row. 3. The Admin remains on the page. 4. Return to step 7.
Exceptions:	**EX-01: Database Error** 1. The system attempts to load the skills from the database. 2. The database operation fails due to an internal database error. 3. The system returns the message "Error.FailedToLoadSkillList". 4. The frontend displays the message "Couldn't load skills. Try again." 5. The skill list is not displayed. 6. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the skill data. 3. The system displays the message "Cannot connect to the server. Please try again later." 4. The Admin remains on the current page without any skill information being displayed. 5. The use case ends.
Priority:	High

Frequency of Use:	High
Business Rules:	**BR-SKL-01:** The endpoint `GET /api/admin/skills` is guarded by `AuthGuard` and `PermissionGuard`. **BR-SKL-02:** Search shall match the skill `name`, `slug`, or `description` in a case-insensitive manner. **BR-SKL-03:** The `deleted` filter returns only active skills when `false` or absent, and only soft-deleted skills when `true`. **BR-SKL-04:** Sorting is restricted to the fields `id` and `createdAt`; `sortOrder` is `asc` or `desc` (default `desc`). **BR-SKL-05:** Pagination shall support `page >= 1` and `limit` between 1 and 100. **BR-SKL-06:** `jobCount` counts only JobSkill records whose associated `job.deletedAt` is `null`.
Other Information:	The list page provides the Actions column to start UC-19.2, UC-19.3, UC-19.4, UC-19.5, and UC-19.6.
Assumptions:	Skill records are stored in the Skill table of the system database.

2.2.3.2 UC-19.2 View Detail Skill

UC ID and Name	UC-19.2 View Detail Skill		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks the view action on a skill row (UC-19.1) or navigates directly to a skill detail URL.		
Description:	Allows an administrator to view the detailed information of a single skill, including its id, name, slug, description, timestamps, deletion status, and the number of jobs currently using the skill.		
Preconditions:	**PRE-1** The user is authenticated as an Admin whose role is allowed to access `GET /api/admin/skills/:id` (guarded by `AuthGuard` and `PermissionGuard`). **PRE-2** The selected skill exists in the database.		
Post-conditions:	**POST-1** The detailed information of the selected skill is displayed together with its `jobCount`. **POST-2** The admin can edit (UC-19.4), delete (UC-19.5), or restore (UC-19.6) the displayed skill.		

Normal Flow:	**A. View Detail Skill** 1. The Admin clicks the view action on a skill row (UC-19.1) or navigates directly to `/admin/skills/:id`. 2. The system sends `GET /api/admin/skills/:id`. 3. The system queries the skill with the matching `id`. 4. The system verifies that the skill exists. 5. The system computes the `jobCount` (count of JobSkill records where `job.deletedAt` is `null`). 6. The system returns `{ id, name, slug, description, deletedAt, createdAt, updatedAt, jobCount }`. 7. The system displays the skill details together with Edit, Delete, and Restore actions as applicable.
Alternative Flows:	**A.1 Skill Not Found** 1. The Admin opens a skill detail page. 2. The system determines that no skill exists with the given `id`. 3. The system returns the message “Error.SkillNotFound” and the frontend displays “Failed to load skill detail. The skill may not exist.” 4. The Admin returns to the skill list. 5. Return to step 1. **A.2 Invalid Skill ID** 1. The Admin navigates to a skill detail URL with a non-positive or non-integer `id`. 2. The system displays the Not Found page. 3. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to load the skill detail from the database. 2. The database operation fails due to an internal database error. 3. The system returns the message "Error.FailedToLoadSkillDetail". 4. The skill details are not displayed. 5. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the request. 2. The system cannot retrieve the skill detail data. 3. The system displays the message "Cannot connect to the server. Please try again later." 4. The Admin remains on the current page without skill information being displayed. 5. The use case ends.
Priority:	High
Frequency of Use:	Medium
Business Rules:	**BR-SKL-07:** A skill detail shall be accessible only when a skill with the given `id` exists; otherwise `Error.SkillNotFound` (404). **BR-SKL-08:** The detail shall include `id`, `name`, `slug`, `description`, `deletedAt`, `createdAt`, `updatedAt`, and `jobCount`. **BR-SKL-09:** `jobCount` counts only JobSkill records whose associated `job.deletedAt` is `null`.
Other Information:	The detail page shows the appropriate action buttons depending on the skill status: Edit/Delete for active skills and Restore for deleted skills.
Assumptions:	The skill id is a positive integer.

2.2.3.3 UC-19.3 Create New Skill

UC ID and Name	UC-19.3 Create New Skill		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A

Trigger:	When an admin clicks the “Create Skill” button on the Skills Management page (UC-19.1).
Description:	Allows an administrator to create a new skill by providing a name (required) and an optional description. A unique slug is generated automatically from the name.
Preconditions:	**PRE-1** The user is authenticated as an Admin whose role is allowed to access `POST /api/admin/skills` (guarded by `AuthGuard` and `PermissionGuard`). **PRE-2** The system is connected to the database.
Post-conditions:	**POST-1** A new record is created in the Skill table with `name`, an auto-generated `slug`, and `description`. **POST-2** The new skill appears in the skill list (UC-19.1) with `jobCount` equal to 0.
Normal Flow:	**A. Create New Skill** 1. The Admin clicks the “Create Skill” button on the Skills Management page (`/admin/skills`). 2. The system opens the CreateSkillDialog with fields: `name` (required) and `description` (optional). 3. The Admin enters the skill name and an optional description. 4. The Admin clicks the “Create” button. 5. The system validates the request body. 6. The system sends `POST /api/admin/skills` with `name` and `description`. 7. The system checks that no active skill (`deletedAt = NULL`) has the same name (case-insensitive). 8. The system generates the slug from the name, appending a numeric suffix (e.g. `-2`, `-3`) if the base slug is already taken among active skills. 9. The system creates the skill record and returns the skill detail with `jobCount`. 10. The system displays the message “Skill created successfully!” and refreshes the skill list.

Alternative Flows:	**A.1 Missing Required Name** 1. The Admin submits the form. 2. The system determines that `name` is empty. 3. The frontend disables submission until a name is provided; the backend returns “Error.SkillNameRequired” if submitted empty. 4. Return to step 3. **A.2 Name Too Long** 1. The Admin submits the form. 2. The system determines that `name` exceeds 100 characters. 3. The system returns the message “Error.SkillNameTooLong”. 4. Return to step 3. **A.3 Description Too Long** 1. The Admin submits the form. 2. The system determines that `description` exceeds 500 characters. 3. The system returns the message “Error.SkillDescriptionTooLong”. 4. Return to step 3. **A.4 Duplicate Skill Name** 1. The Admin submits the form. 2. The system determines that an active skill (`deletedAt = NULL`) with the same name already exists (case-insensitive). 3. The system returns the message “Error.SkillNameAlreadyExists” (409). 4. Return to step 3. **A.5 Admin Cancels Creation** 1. The Admin clicks the “Cancel” button in the CreateSkillDialog. 2. The system discards the unsaved data and closes the dialog. 3. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to save the skill record in the database. 2. The database operation fails due to an internal database error. 3. The system returns the message “Error.FailedToCreateSkill”. 4. The frontend displays “Failed to create skill. Try again.” 5. The skill is not created. 6. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while creating the skill. 2. The system cannot save the skill data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The Admin remains on the dialog without the skill being saved. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	**BR-SKL-10:** The skill `name` is required and shall not exceed 100 characters. **BR-SKL-11:** The skill `description` is optional, is stored as `NULL` when empty, and shall not exceed 500 characters. **BR-SKL-12:** The skill name must be unique among active skills (`deletedAt = NULL`) in a case-insensitive manner; otherwise `Error.SkillNameAlreadyExists` (409). **BR-SKL-13:** The `slug` is generated automatically from the name; a numeric suffix is appended when the base slug is already taken among active skills.
Other Information:	The returned skill detail includes `jobCount`, which is 0 for a newly created skill.
Assumptions:	The admin remains authenticated during the creation process.

2.2.3.4 UC-19.4 Edit Skill

UC ID and Name	UC-19.4 Edit Skill		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A

Trigger:	When an admin clicks the edit action on a skill row (UC-19.1) or on the skill detail page (UC-19.2).
Description:	Allows an administrator to update an existing skill's name and/or description. When the name changes, the slug is regenerated automatically.
Preconditions:	**PRE-1** The user is authenticated as an Admin whose role is allowed to access `PATCH /api/admin/skills/:id` (guarded by `AuthGuard` and `PermissionGuard`). **PRE-2** The selected skill exists in the database.
Post-conditions:	**POST-1** The skill's `name` and/or `description` are updated in the Skill table. **POST-2** The skill's `slug` is regenerated when the name changes. **POST-3** The updated skill is displayed in the skill list (UC-19.1).
Normal Flow:	**A. Edit Skill** 1. The Admin clicks the edit action on a skill row (UC-19.1) or on the skill detail page (UC-19.2). 2. The system opens the UpdateSkillDialog pre-filled with the current skill name and description. 3. The Admin modifies the name and/or description. 4. The Admin clicks the Save Changes button. 5. The system sends `PATCH /api/admin/skills/:id` with only the changed fields. 6. The system verifies that the skill exists. 7. If `name` is changed, the system checks uniqueness against other active skills (`deletedAt = NULL`), excluding the skill itself. 8. If `name` is changed, the system regenerates a unique slug from the new name. 9. The system updates the skill record and returns the skill detail with `jobCount`. 10. The system displays the message "Skill updated successfully!" and refreshes the page.

Alternative Flows:	**A.1 Skill Not Found** 1. The Admin submits the update request. 2. The system determines that the skill does not exist. 3. The system returns the message “Error.SkillNotFound” (404). 4. The Admin returns to the skill list. 5. Return to step 1. **A.2 Duplicate Skill Name** 1. The Admin submits the updated name. 2. The system determines that another active skill has the same name (case-insensitive). 3. The system returns the message “Error.SkillNameAlreadyExists” (409). 4. Return to step 3. **A.3 Validation Errors** 1. The Admin submits the update request. 2. The system determines that `name` is empty, exceeds 100 characters, or that `description` exceeds 500 characters. 3. The system returns the corresponding validation error (`SkillNameRequired`, `SkillNameTooLong`, `SkillDescriptionTooLong`). 4. Return to step 3. **A.4 Admin Cancels Editing** 1. The Admin clicks the “Cancel” button in the UpdateSkillDialog. 2. The system discards the unsaved changes and closes the dialog. 3. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to update the skill record in the database. 2. The database operation fails due to an internal database error. 3. The system returns the message “Error.FailedToUpdateSkill”. 4. The frontend displays “Failed to update skill.” 5. The skill is not updated. 6. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while saving the changes. 2. The system cannot save the updated skill data. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The Admin remains on the dialog without the changes being saved. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	**BR-SKL-14:** The `name` and `description` are optional in the update request (partial update); `description` may be set to `NULL`. **BR-SKL-15:** When `name` is changed, the new name must be unique among other active skills (`deletedAt = NULL`) in a case-insensitive manner; otherwise `Error.SkillNameAlreadyExists` (409). **BR-SKL-16:** When `name` is changed, the `slug` is regenerated from the new name and a numeric suffix is appended when the slug is already taken. **BR-SKL-17:** An update shall succeed only when the skill with the given `id` exists; otherwise `Error.SkillNotFound` (404). **BR-SKL-18:** The original skill `id`, `createdAt`, and `deletedAt` remain unchanged after the update.
Other Information:	When the name is unchanged, the uniqueness check and slug regeneration are skipped.
Assumptions:	The admin remains authenticated throughout the editing process.

2.2.3.5 UC-19.5 Delete Skill

UC ID and Name	UC-19.5 Delete Skill

Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A
Trigger:	When an admin clicks the delete action on an active skill row (UC-19.1) or on the skill detail page (UC-19.2).		
Description:	Allows an administrator to soft-delete an existing active skill by setting its `deletedAt` field. Deletion is blocked when the skill is still used by active jobs.		
Preconditions:	**PRE-1** The user is authenticated as an Admin whose role is allowed to access `DELETE /api/admin/skills/:id` (guarded by `AuthGuard` and `PermissionGuard`). **PRE-2** The selected skill exists in the database. **PRE-3** The selected skill is not used by any active jobs.		
Post-conditions:	**POST-1** The skill is soft-deleted by setting its `deletedAt` field to the current timestamp. **POST-2** The deleted skill no longer appears in the active skill list (UC-19.1) unless the Deleted filter is applied, and its actions change from Edit/Delete to Restore.		
Normal Flow:	**A. Delete Skill** 1. The Admin clicks the delete action on an active skill row (UC-19.1) or on the skill detail page (UC-19.2). 2. The system opens the DeleteSkillDialog with a confirmation message. 3. The Admin confirms the deletion request. 4. The system sends `DELETE /api/admin/skills/:id`. 5. The system verifies that the skill exists. 6. The system checks that no active jobs (JobSkill records where `job.deletedAt` is `null`) use this skill. 7. The system sets the `deletedAt` field of the skill to the current timestamp. 8. The system returns the message “Skill deleted successfully.” and refreshes the skill list.		

Alternative Flows:	**A.1 Skill In Use** 1. The Admin confirms the deletion request. 2. The system determines that one or more active jobs (`job.deletedAt` is `null`) are linked to this skill through JobSkill. 3. The system blocks the deletion and returns the message "Error.SkillInUse" (409). 4. The frontend displays the message "Cannot delete skill. It is currently used by jobs." 5. The use case ends. **A.2 Skill Not Found** 1. The Admin confirms the deletion request. 2. The system determines that the skill does not exist or has already been deleted. 3. The system returns the message "Error.SkillNotFound" (404). 4. The Admin returns to the skill list. 5. Return to step 1. **A.3 Admin Cancels Deletion** 1. The Admin clicks the "Cancel" button in the DeleteSkillDialog. 2. The system closes the dialog. 3. The skill remains unchanged. 4. The use case ends.

Exceptions:	**EX-01: Database Error** 1. The system attempts to update the `deletedAt` field of the skill in the database. 2. The database operation fails due to an internal database error. 3. The system returns the message “Error.FailedToDeleteSkill”. 4. The frontend displays “Failed to delete skill.” 5. The skill is not deleted. 6. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the deletion request. 2. The system cannot complete the deletion of the skill. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The skill remains unchanged. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	**BR-SKL-19:** Skill deletion shall be implemented as a soft delete by setting the `deletedAt` field to the current timestamp. **BR-SKL-20:** A deletion shall succeed only when the skill with the given `id` exists; otherwise `Error.SkillNotFound` (404). **BR-SKL-21:** A skill that is used by at least one active job (JobSkill record where `job.deletedAt` is `null`) shall not be deleted; the system returns `Error.SkillInUse` (409). **BR-SKL-22:** Only active skills (`deletedAt = NULL`) shall be deletable.
Other Information:	Deleted skills remain stored in the database and can be restored through UC-19.6.
Assumptions:	The admin remains authenticated during the deletion process.

2.2.3.6 UC-19.6 Restore Skill

UC ID and Name	UC-19.6 Restore Skill		
Created By:	ThuanNT	Date Created:	14/09/2026
Primary Actor:	Admin	Secondary Actors:	N/A

Trigger:	When an admin clicks the restore action on a deleted skill row (UC-19.1) or on the skill detail page (UC-19.2).
Description:	Allows an administrator to restore a soft-deleted skill by setting its `deletedAt` field to `null`, making it active again.
Preconditions:	**PRE-1** The user is authenticated as an Admin whose role is allowed to access `PATCH /api/admin/skills/:id/restore` (guarded by `AuthGuard` and `PermissionGuard`). **PRE-2** The selected skill exists and is currently soft-deleted (`deletedAt` is not `null`).
Post-conditions:	**POST-1** The skill's `deletedAt` field is set to `null`. **POST-2** The restored skill appears again in the active skill list (UC-19.1) with its Edit/Delete actions.
Normal Flow:	**A. Restore Skill** 1. The Admin clicks the restore action on a deleted skill row (UC-19.1) or on the skill detail page (UC-19.2). 2. The system opens the RestoreSkillDialog with a confirmation message. 3. The Admin confirms the restoration request. 4. The system sends `PATCH /api/admin/skills/:id/restore`. 5. The system verifies that the skill exists and is currently soft-deleted. 6. The system sets the `deletedAt` field of the skill to `null`. 7. The system returns the skill detail with `jobCount`. 8. The system displays the message "Skill restored successfully." and refreshes the skill list.

Alternative Flows:	**A.1 Skill Not Soft-Deleted** 1. The Admin submits the restore request. 2. The system determines that the skill does not exist or is not soft-deleted (`deletedAt` is `null`). 3. The system returns the message “Error.SkillNotFound” (404). 4. The Admin returns to the skill list. 5. Return to step 1. **A.2 Admin Cancels Restoration** 1. The Admin clicks the “Cancel” button in the RestoreSkillDialog. 2. The system closes the dialog. 3. The skill remains soft-deleted. 4. The use case ends.
Exceptions:	**EX-01: Database Error** 1. The system attempts to set the `deletedAt` field of the skill to `null` in the database. 2. The database operation fails due to an internal database error. 3. The system returns the message “Error.FailedToRestoreSkill”. 4. The frontend displays “Failed to restore skill.” 5. The skill is not restored. 6. The use case ends. **EX-02: Network or Server Connection Failure** 1. The connection between the application and the server is interrupted while processing the restore request. 2. The system cannot complete the restoration of the skill. 3. The system displays the message “Cannot connect to the server. Please try again later.” 4. The skill remains soft-deleted. 5. The use case ends.
Priority:	Medium
Frequency of Use:	Low

Business Rules:	**BR-SKL-23:** A skill shall be restored only when it is soft-deleted (`deletedAt` is not `null`); otherwise `Error.SkillNotFound` (404). **BR-SKL-24:** Restoration sets the skill's `deletedAt` field to `null`. **BR-SKL-25:** The original skill `id`, `name`, `slug`, `description`, `createdAt`, and `updatedAt` values remain unchanged after restoration.
Other Information:	The restore action is shown only for soft-deleted skills in the list and detail pages; it is the counterpart of the Delete action (UC-19.5).
Assumptions:	The restored skill remains linked to the same JobSkill and freelancerSkill relations as before deletion.

3 Business Rule

Table 163: Business Rules

ID	Rule Definition
BR-PROF-01	Both basic details (from profiles) and professional details (from freelancer_profiles) must be presented together.
BR-PROF-02	Hidden or private fields should not be displayed to unauthorized users (e.g., clients vs. the profile owner).
BR-PROF-03	Freelancers can only update their own profile.
BR-PROF-04	Mandatory fields (e.g., display_name, title) cannot be left empty.
BR-SKILL-01	All skills belonging to the freelancer must be displayed along with their proficiency level.
BR-SKILL-02	A freelancer cannot add duplicate skills.
BR-SKILL-03	Both skill name and proficiency level are required.
BR-SKILL-04	Skill deletion must be a hard delete operation; the record is permanently removed from the mapping table.

ID	Rule Definition
BR-SKILL-05	Only the owning freelancer or an administrator can delete a skill from the profile.
BR-PORT-01	Only portfolios with deleted_at = NULL shall be displayed.
BR-PORT-02	Portfolio title is a required field.
BR-PORT-03	Media uploads must conform to system-defined size and type restrictions.
BR-PORT-04	Only portfolios with deleted_at = NULL can be accessed.
BR-PORT-05	Only the portfolio owner can edit it.
BR-PORT-06	Portfolio title remains a required field.
BR-PORT-07	Portfolio deletion must be a soft delete (updating the deleted_at column).
BR-PORT-08	Soft-deleted portfolios must not be visible to clients or guests.
BR-FORUM-01	Only categories with is_active = TRUE shall be displayed.
BR-FORUM-02	Category information displayed includes name and description.
BR-FORUM-03	Only categories with is_active = TRUE shall be accessible.
BR-FORUM-04	Category information displayed includes name and description.
BR-FORUM-05	Only posts with deleted_at = NULL shall be displayed.
BR-FORUM-06	Posts with is_pinned = TRUE shall appear before regular posts.
BR-FORUM-07	Posts shall be sorted by created_at in descending order within each group.
BR-FORUM-08	The system shall support pagination for post listings.
BR-FORUM-09	Only posts with deleted_at = NULL shall be displayed.
BR-FORUM-10	Post details shall include title, content, author, category, and created_at information.
BR-FORUM-11	Only comments with deleted_at = NULL shall be displayed.
BR-FORUM-12	Comment information shall include content, author, and created_at.

ID	Rule Definition
BR-FORUM-13	Only comments with deleted_at = NULL shall be displayed.
BR-FORUM-14	Comment details shall include content, author, and created_at information.
BR-FORUM-15	Only authenticated users can create posts.
BR-FORUM-16	Every post must belong to exactly one category.
BR-FORUM-17	Title and content are required fields.
BR-FORUM-18	Only the author of a post can edit that post.
BR-FORUM-19	Every post must belong to a valid category.
BR-FORUM-20	Title and content are required fields.
BR-FORUM-21	Only the author of a post can delete that post.
BR-FORUM-22	Post deletion shall be implemented as a soft delete using the deleted_at field.
BR-FORUM-23	Soft-deleted posts shall not appear in public forum listings.
BR-FORUM-24	Only authenticated users can like posts.
BR-FORUM-25	A user can have only one active like reaction per post.
BR-FORUM-26	The reaction shall be stored in the forum_likes table.
BR-FORUM-27	Only authenticated users can remove post reactions.
BR-FORUM-28	A like reaction can only be removed if a corresponding like record exists.
BR-FORUM-29	Removing a reaction shall delete the corresponding record from the forum_likes table.
BR-FORUM-30	Only authenticated users can create comments.
BR-FORUM-31	Comment content cannot be empty.
BR-FORUM-32	Comment content must contain at least 2 characters.
BR-FORUM-33	Each comment shall be associated with a valid post_id and user_id.

ID	Rule Definition
BR-FORUM-34	Only the owner of a comment can edit it.
BR-FORUM-35	Comment content cannot be empty.
BR-FORUM-36	Comment content must contain at least 2 characters.
BR-FORUM-37	The system shall record the updated_at timestamp when a comment is modified.
BR-FORUM-38	Only the owner of a comment can delete it.
BR-FORUM-39	Comment deletion shall be implemented as a soft delete using the deleted_at field.
BR-FORUM-40	Soft-deleted comments shall not be displayed in normal comment listings.
BR-FORUM-41	Only authenticated users can like comments.
BR-FORUM-42	A user can have only one active like reaction per comment.
BR-FORUM-43	The reaction shall be stored in the forum_likes table using comment_id.
BR-FORUM-44	Only authenticated users can remove comment reactions.
BR-FORUM-45	A like reaction can only be removed if a corresponding like record exists.
BR-FORUM-46	Removing a reaction shall delete the corresponding record from the forum_likes table using comment_id.
BR-FORUM-47	Only authenticated users can submit reports.
BR-FORUM-48	Reports must be associated with either a post_id or comment_id.
BR-FORUM-49	New reports shall be created with status = PENDING.
BR-FORUM-50	A user may submit only one active report for the same content item.
BR-JOB-01	Only jobs with status = 'open' or 'in_progress' shall be displayed to freelancers and guests.
BR-JOB-02	Clients shall see all their jobs regardless of status.
BR-JOB-03	Default sorting shall be by created_at in descending order.

ID	Rule Definition
BR-JOB-04	The system shall support pagination with default page size of 10.
BR-JOB-05	Search shall match keywords in title and description fields (case-insensitive).
BR-JOB-06	When multiple filters are applied, all conditions shall be combined using AND logic.
BR-JOB-07	Job information displayed includes: title, description, budget, deadline, category, required skills, client name, created date, status.
BR-JOB-08	If the job is inactive (status = 'closed' or 'deleted'), only the job owner can view full details.
BR-JOB-09	Job title is required and must not exceed 200 characters.
BR-JOB-10	Job description is required and must not exceed 5000 characters.
BR-JOB-11	Budget must be a positive number (greater than 0).
BR-JOB-12	Category must be selected from predefined categories.
BR-JOB-13	New jobs shall have status = 'draft' by default when saved as draft, or 'open' when published.
BR-JOB-14	Only the job owner can edit the job.
BR-JOB-15	Editing is restricted if the job has an active contract.
BR-JOB-16	All validation rules from UC-06.2 apply to edits.
BR-JOB-17	Edit actions shall be logged for auditing purposes.
BR-JOB-18	Only the job owner can delete the job.
BR-JOB-19	Deletion should be soft delete (set deleted_at timestamp, status = 'deleted').
BR-JOB-20	Jobs with active proposals or contracts cannot be deleted.
BR-JOB-21	Delete actions shall be logged for auditing purposes.
BR-JOB-22	Job status workflow: draft $\rightarrow$ open $\rightarrow$ in_progress $\rightarrow$ completed $\rightarrow$ closed.

ID	Rule Definition
BR-JOB-23	Allowed transitions: draft → open (client); open → in_progress (client, when contract created); open → closed (client, if cancelled); in_progress → completed (client, when work done); in_progress → closed (client, if cancelled); completed → closed (system/auto); Any status → deleted (soft delete).
BR-JOB-24	Only the job owner can manually change status.
BR-JOB-25	Status changes shall be logged for auditing purposes.
BR-JOB-26	Notifications shall be sent to freelancers when status changes from draft to open.
BR-JOB-27	Only the freelancer can view their own bookmarks.
BR-JOB-28	Bookmarks shall be sorted by created_at in descending order by default.
BR-JOB-29	Displayed information includes: job title, budget, client name, bookmark date.
BR-JOB-30	The system shall support pagination with default page size of 10.
BR-JOB-31	Only freelancers can bookmark jobs.
BR-JOB-32	Bookmark is a toggle action. Clicking the button toggles between adding and removing.
BR-JOB-33	If the bookmark does not exist, the system creates it and changes the button to "Bookmarked".
BR-JOB-34	If the bookmark exists, the system deletes it and changes the button to "Bookmark".
BR-JOB-35	Bookmarks can only be created for active jobs (status = 'open' or 'in_progress').
BR-JOB-36	Only the freelancer can remove their own bookmarks.
BR-JOB-37	The system must check if the bookmark exists before deletion.
BR-JOB-38	If the bookmark does not exist, the system shall return a 404 Not Found error.
BR-JOB-39	Removing a bookmark is a hard delete from the database.

4 Functional Requirement

4.1 System Functional Overview

4.1.1 Screen Flow

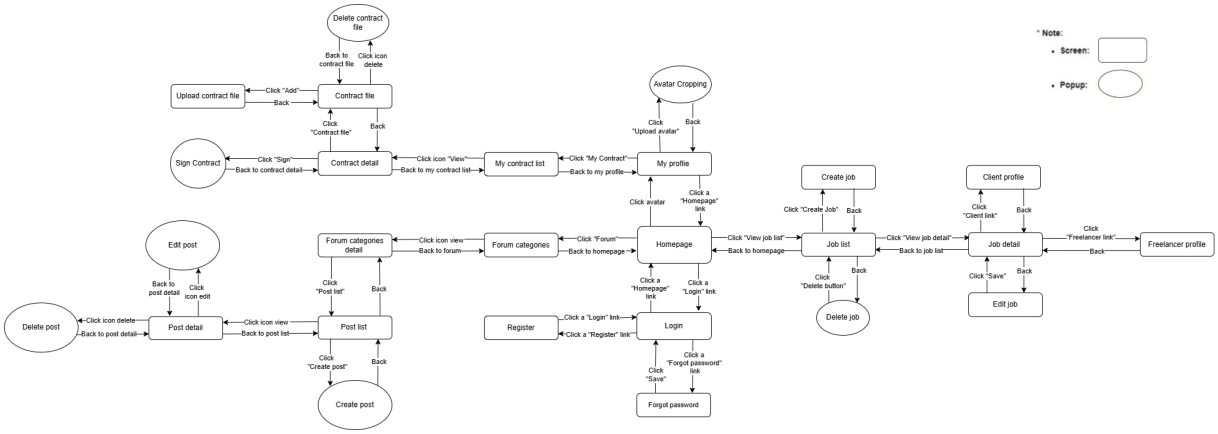


Figure 9: Screen Flow of Client

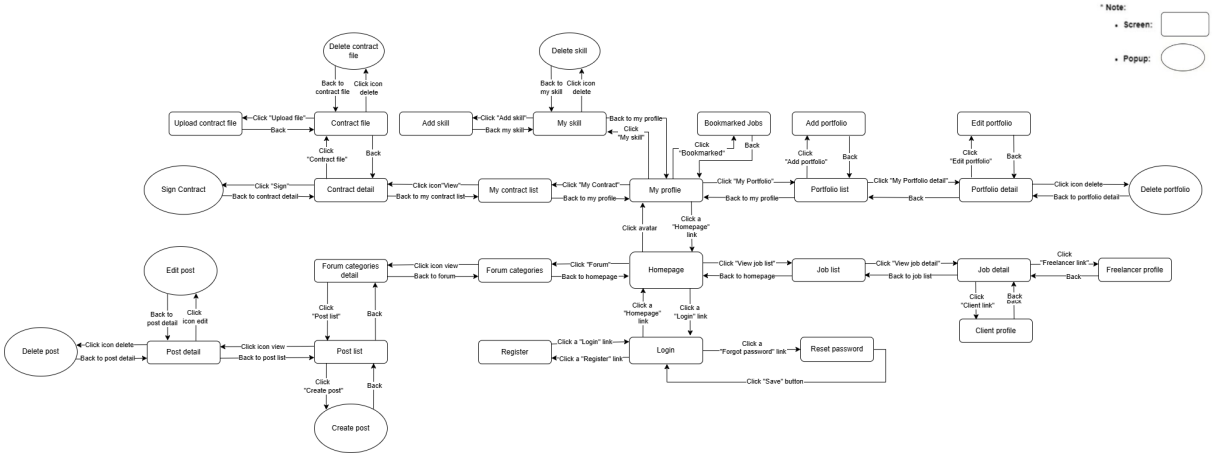


Figure 10: Screen Flow of Freelancer

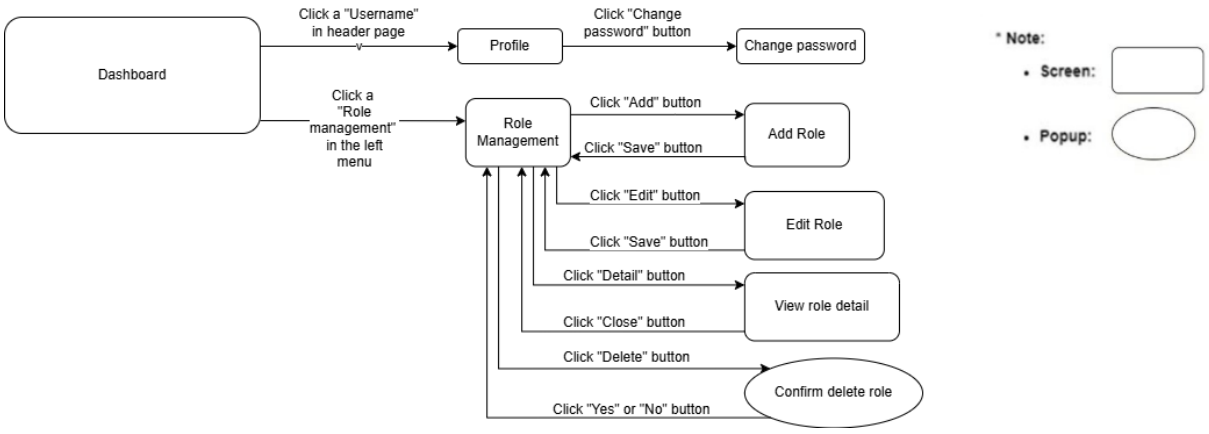


Figure 11: Screen Flow of Admin

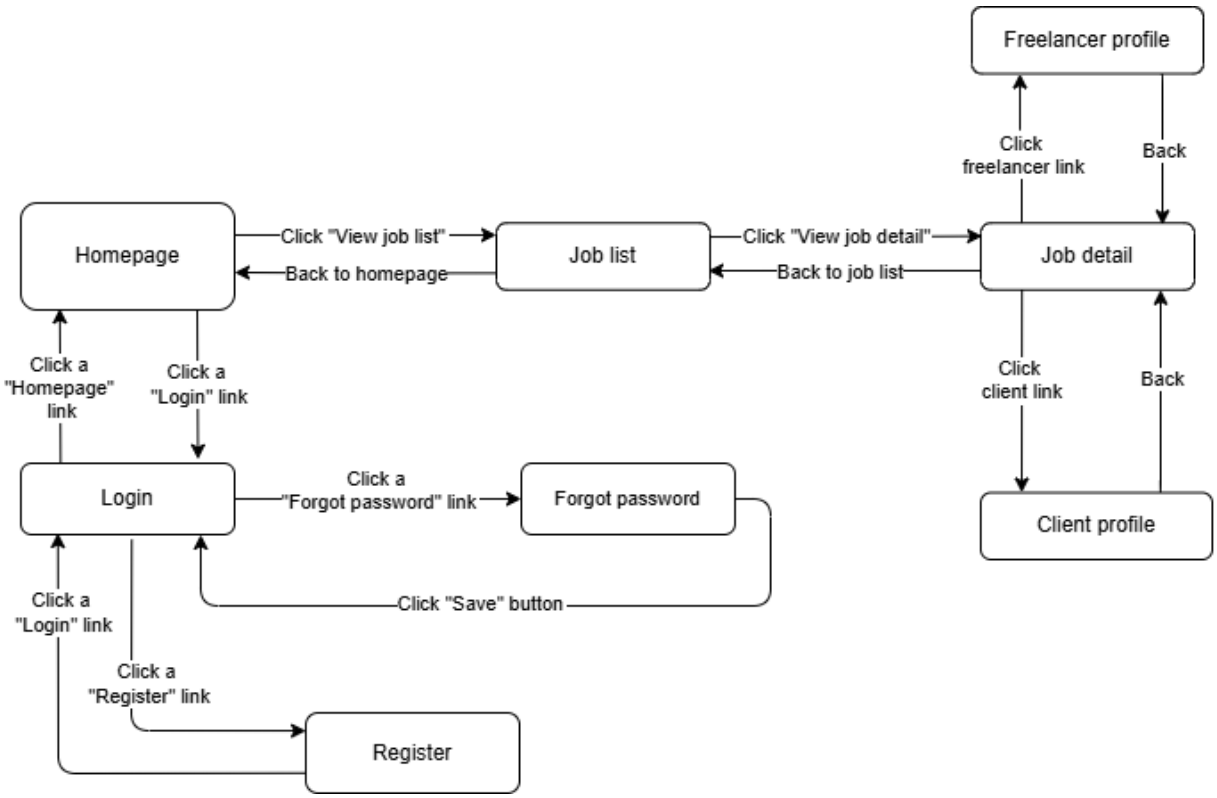


Figure 12: Screen Flow of Guest

4.1.2 Screen Details

No	Actor	Screen Name	Description
1	Guest	Homepage	The main landing page introducing the platform and its features.
2		Login	Allows users to log in using email/password and JWT authentication.
3		Forgot Password	Allows users to request a password reset via email verification link.
4		Register	Allows users to create new accounts using email and password.
5		Job List	Displays a list of available jobs with search and filter capabilities.
6		Job Detail	Displays full details of a specific job posting.
7		Freelancer Profile	Displays a freelancer's public profile including skills and portfolio.
8		Client Profile	Displays a client's public profile including posted jobs and reviews.
9	Freelancer	Google Login	Allows users to login/register using Google OAuth.
10		Login	Allows users to log in using email/password and JWT authentication.
11		Logout	Allows users to logout and terminate authentication session.
12		Register	Allows users to create new accounts using email and password.
13		Forgot Password	Allows users to request password reset via email verification link.
14		My Profile	Main dashboard for freelancer to manage their skills, contracts, and portfolio.

No	Actor	Screen Name	Description
15		My Skill	Displays the list of skills added by the freelancer.
16		Add Skill	Form screen to add a new skill to the profile.
17		Delete Skill	Popup dialog to confirm the deletion of a skill.
18		Bookmarked Jobs	Displays a list of jobs that the freelancer has bookmarked or saved.
19		My Contract List	Displays all active and past contracts associated with the freelancer.
20		Contract Detail	Displays detailed information and terms of a specific contract.
21		Sign Contract	Popup/Action screen allowing the freelancer to digitally sign a contract.
22		Contract File	Displays documents and files attached to a specific contract.
23		Upload Contract File	Form screen to upload new files or documents to a contract.
24		Delete Contract File	Popup dialog to confirm the deletion of a contract file.
25		Portfolio List	Displays a list of projects and works in the freelancer's portfolio.
26		Add Portfolio	Form screen to create and add a new project to the portfolio.
27		Portfolio Detail	Displays full details, descriptions, and media of a specific portfolio project.
28		Edit Portfolio	Form screen to update or modify an existing portfolio project.
29		Delete Portfolio	Popup dialog to confirm the deletion of a portfolio project.

No	Actor	Screen Name	Description
30		Forum Categories	Displays a list of available discussion categories in the forum.
31		Forum Categories Detail	Displays sub-topics and general information of a selected forum category.
32		Post List	Displays a list of discussion threads or posts within a category.
33		Create Post	Form screen allowing the freelancer to create a new forum post.
34		Post Detail	Displays the full content of a forum post along with its replies.
35		Edit Post	Form/Popup to edit the content of an existing forum post.
36		Delete Post	Popup dialog to confirm the deletion of a forum post.
37	Client	Google Login	Allows users to login/register using Google OAuth.
38		Login	Allows users to log in using email/password and JWT authentication.
39		Logout	Allows users to logout and terminate authentication session.
40		Register	Allows users to create new accounts using email and password.
41		Forgot Password	Allows users to request password reset via email verification link.
42		Client Dashboard	Main dashboard for authenticated clients, showing active jobs, contract summaries, and notifications.
43		Create Job Form	Form screen allowing clients to create and publish a new job posting with details (title, description, budget, category, skills).

No	Actor	Screen Name	Description
44		Edit Job	Form screen allowing clients to modify details of an existing job posting.
45		Delete Job	Popup dialog to confirm the soft deletion of a job posting.
46		Change Job Status	Form/Dropdown action to change the status of a job posting (Open, In Progress, Closed, Cancelled).
47		My Contract List	Displays all active and past contracts associated with the client.
48		Contract Detail	Work interface for client to manage contract terms, milestones, and release escrow payments.
49		Create Contract	Form screen allowing the client to initiate and create a new contract with a freelancer.
50		Update Contract Terms	Form/Edit screen to update terms, milestones, and payment settings of a pending contract.
51		Sign Contract	Popup/Action screen allowing the client to digitally sign and fund the contract escrow (Web3).
52		Cancel Contract	Popup/Action screen allowing the client to request contract cancellation and handle escrow refunds.
53		Upload Contract File	Form screen allowing the client to upload files or deliverables to a contract.
54		Delete Contract File	Popup dialog to confirm the deletion of a contract file.
55		View Progress Updates	Screen displaying progress updates and attached proof of work submitted by the freelancer.

No	Actor	Screen Name	Description
56		Hired Freelancers	Screen displaying the list of freelancers currently hired by the client.
57	Admin	Dashboard	Main admin screen displaying system overview and key statistics.
58		Profile	Allows admin to view and update personal account information.
59		Change Password	Allows admin to change their account password.
60		Role Management	Displays a list of all roles along with their assigned permissions.
61		Add Role	Form screen for creating a new role with name, description, and permissions.
62		Edit Role	Form screen for modifying an existing role's name, description, and permissions.
63		View Role Detail	Displays detailed information of a specific role including its permissions.
64		Confirm Delete Role	Popup dialog prompting admin to confirm before permanently deleting a role.

4.1.3 Screen Authorization

No	Screen / Page	Guest	Client	Freelancer	Admin
1	Forum Categories	✓	✓	✓	✓
2	Forum Category Detail	✓	✓	✓	✓
3	Post List	✓	✓	✓	✓
4	Post Detail	✓	✓	✓	✓
5	View Comments	✓	✓	✓	✓
6	View Comment Detail	✓	✓	✓	✓

No	Screen / Page	Guest	Client	Freelancer	Admin
7	Create Post		✓	✓	
8	Edit Post		✓	✓	
9	Delete Post		✓	✓	✓
10	Like Post		✓	✓	
11	Unlike Post		✓	✓	
12	Add Comment		✓	✓	
13	Edit Comment		✓	✓	
14	Delete Comment		✓	✓	✓
15	Like Comment		✓	✓	
16	Unlike Comment		✓	✓	
17	Report Content		✓	✓	
18	View Detailed Freelancer Profile	✓	✓	✓	
19	Update Detailed Freelancer Profile			✓	
20	Get List Freelancer Skill	✓	✓	✓	
21	Add Freelancer Skill			✓	
22	Delete Freelancer Skill			✓	
23	Get Portfolio List			✓	
24	Add New Portfolio			✓	
25	Get Portfolio Detail			✓	
26	Update Portfolio			✓	
27	Delete Portfolio			✓	
28	Create Contract		✓		
29	Update Contract Terms		✓		
30	Complete Contract		✓	✓	
31	Get Contract List		✓	✓	✓
32	Get Contract Detail		✓	✓	✓
33	Sign Contract		✓	✓	

No	Screen / Page	Guest	Client	Freelancer	Admin
34	Cancel Contract		✓	✓	✓
35	Contract Activities		✓	✓	✓
36	Contract Files		✓	✓	✓
37	Upload Contract File		✓	✓	
38	Delete Contract File		✓	✓	✓
39	Progress Update			✓	
40	View Progress Updates		✓	✓	✓
41	Google Login	✓			
42	Login	✓			
43	Logout		✓	✓	✓
44	Register	✓			
45	Forgot Password	✓	✓	✓	✓
46	View Role List				✓
47	View Role Detail				✓
48	Create New Role				✓
49	Update Role				✓
50	Delete Role				✓
51	View List Job	✓	✓	✓	
52	View Job Detail	✓	✓	✓	
53	Post New Job		✓		
54	Edit Job		✓		
55	Delete Job		✓		
56	Change Job Status		✓		
57	View List Bookmarked Job			✓	
58	Bookmark Job			✓	
59	Remove Bookmark			✓	

4.1.4 Non-Screen Functions

#	Feature	System Function	Description
1	**Manage Authentication**	[POST] /api/auth/register [POST] /api/auth/otp [POST] /api/auth/login [POST] /api/auth/refresh-token [POST] /api/auth/logout [POST] /api/auth/forgot-password [GET] /api/auth/google-link [GET] /api/auth/google/callback [GET] /api/auth/me [POST] /api/auth/switch-role [POST] /api/auth/join-role	The Authentication feature handles user identity verification and session establishment. It supports account registration with OTP verification, email/password login with JWT access and refresh tokens, token renewal, logout, and password reset through an email verification link. Users can also sign in with Google OAuth, retrieve the currently authenticated account, and switch or join a role (Client or Freelancer) on the same account.
2	**Manage Account Profile**	[GET] /api/account-profile [PUT] /api/account-profile [PUT] /api/account-profile/password [POST] /api/account-profile/avatar [GET] /api/account-profile/avatar/{userId}/file [GET] /api/clients/{userId} [PUT] /api/clients/me/profile [GET] /api/social-links [POST] /api/social-links [DELETE] /api/social-links/{id} [GET] /api/favorites/freelancers [POST] /api/favorites/freelancers/{freelancerId} [DELETE] /api/favorites/freelancers/{freelancerId} [GET] /api/following/freelancers/discover [GET] /api/following/freelancers [POST] /api/following/freelancers/{freelancerId} [DELETE] /api/following/freelancers/{freelancerId}	The Account Profile feature lets authenticated users manage their account-level information and social network. Users can view and update their account profile, change their password, upload and retrieve their avatar, and manage social links. Clients can view another client's public profile and update their own client profile. Users can also save favorite freelancers, discover and follow freelancers, and unfollow or remove them from the favorites list.

#	Feature	System Function	Description
3	**Manage Freelancer Profile**	[GET] /api/profiles/{id} [PUT] /api/profiles/{id} [POST] /api/profiles/{id}/cv [GET] /api/profiles/{id}/cv/file [POST] /api/profiles/{id}/cv/analyze [DELETE] /api/profiles/{id}/cv [GET] /api/expert-profile/me [PUT] /api/expert-profile/me [PATCH] /api/users/{id}/expert-profile [GET] /api/experts [GET] /api/experts/{id}	The Freelancer Profile feature enables freelancers to build and maintain their professional identity, including bio, professional title, education, availability status and profile strength. Freelancers can upload, download, delete and AI-analyze their CV. The feature also exposes the expert (verified freelancer) profile endpoints, the public expert directory, and allows a freelancer to view and update their own expert profile while administrators can update it on behalf of a user.
4	**Manage Freelancer Skills**	[GET] /api/profiles/{id}/skills [GET] /api/profiles/skills/suggestions [POST] /api/profiles/{id}/skills [DELETE] /api/profiles/skills/{id}	The Freelancer Skills feature allows freelancers to define and showcase their professional capabilities. Users can view the skills attached to a profile, search the skill catalog for suggestions while typing, add a skill together with its proficiency level, and remove a skill that is no longer relevant. This keeps each freelancer's skill set accurate for the client's talent evaluation process.

#	Feature	System Function	Description
5	**Manage Freelancer Portfolio**	[GET] /api/profiles/{id}/portfolios [POST] /api/profiles/{id}/portfolios [GET] /api/profiles/portfolios/{id} [PUT] /api/profiles/portfolios/{id} [DELETE] /api/profiles/portfolios/{id}	The Freelancer Portfolio feature enables freelancers to demonstrate their past work visually. Freelancers can list every portfolio item of a profile, create a new item with project details and media, view the detail of a specific item, update its content or media, and soft-delete an item so that it is no longer displayed publicly. A well-organized portfolio helps freelancers build credibility with prospective clients.
6	**Manage Job**	[GET] /api/jobs [GET] /api/jobs/{slug} [GET] /api/job-categories [GET] /api/job-categories/{slug} [GET] /api/manage-jobs/skills [GET] /api/manage-jobs [GET] /api/manage-jobs/{id} [POST] /api/manage-jobs [PATCH] /api/manage-jobs/{id} [DELETE] /api/manage-jobs/{id} [PATCH] /api/manage-jobs/{id}/status [GET] /api/manage-jobs/bookmarks [POST] /api/manage-jobs/jobs/{slug}/bookmark [GET] /api/manage-jobs/jobs/{slug}/bookmark [DELETE] /api/manage-jobs/bookmarks/{slug}	The Job Management feature is the core marketplace functionality that connects clients with freelancers. Guests and freelancers can browse and search the paginated job list, view a job detail by slug, and browse job categories. Clients can create a job posting with description, budget, required skills and timeline, list their own jobs, view a job detail, update or delete a job, and change its status (Open, In Progress, Closed, Cancelled); they can also look up skills while composing a job. Freelancers can list, add and remove bookmarked jobs and check whether a job is bookmarked.

#	Feature	System Function	Description
7	**Manage Job Categories (Admin)**	[GET] /api/admin/job-categories [GET] /api/admin/job-categories/{id} [POST] /api/admin/job-categories [PATCH] /api/admin/job-categories/{id} [PATCH] /api/admin/job-categories/{id}/ restore [DELETE] /api/admin/job-categories/{id}	The Job Category Management feature allows administrators to maintain the taxonomy used to classify job postings. Administrators can list all categories, view the detail of a category, create a new category, update its name or description, soft-delete a category, and restore a previously deleted category when it is needed again.
8	**Manage Saved Searches**	[GET] /api/saved-searches [GET] /api/saved-searches/{id} [POST] /api/saved-searches [PATCH] /api/saved-searches/{id} [DELETE] /api/saved-searches/{id}	The Saved Search feature allows authenticated users to store job search criteria for later reuse. Users can list their saved searches, view the detail of one saved search, create a new saved search from the current filters, update its name or filter conditions, and delete a saved search that is no longer needed.

#	Feature	System Function	Description
9	**Manage Proposals**	[GET] /api/proposals/my [GET] /api/proposals/jobs/{jobId}/mine [GET] /api/proposals/jobs/{jobId}/submitted [GET] /api/jobs/{jobId}/proposals [GET] /api/proposals/{id} [GET] /api/proposals/{id}/client [POST] /api/proposals/jobs/{jobId} [POST] /api/proposals/jobs/{jobId}/drafts [PATCH] /api/proposals/{id}/draft [PATCH] /api/proposals/{id}/submit [PATCH] /api/proposals/{id}/withdraw [PATCH] /api/proposals/{id}/interview [PATCH] /api/proposals/{id}/reject [PATCH] /api/proposals/{id}/accept	The Proposal Management feature supports the bidding process between freelancers and clients. Freelancers can submit a proposal to a job, save it as a draft, update and submit a draft, withdraw a submitted proposal, and view their own proposals for a job. Clients can list all proposals submitted to their job, view proposal details, and move a proposal through interview, acceptance or rejection, while acceptance can later be used to create a contract.
10	**Manage Contract**	[GET] /api/contracts [GET] /api/contracts/{id} [POST] /api/contracts [PATCH] /api/contracts/{id} [PATCH] /api/contracts/{id}/sign [PATCH] /api/contracts/{id}/complete [PATCH] /api/contracts/{id}/cancel [GET] /api/contracts/{contractId}/files [POST] /api/contracts/{contractId}/files [DELETE] /api/contracts/{contractId}/files/{fileId}	The Contract Management feature manages the agreement between a client and a freelancer throughout the project lifecycle. Users can create a contract, list their contracts, view contract details, update contract terms, sign the contract to confirm participation, complete the contract and request cancellation. Contract participants can also list, upload and delete contract-level files that document the agreement.

#	Feature	System Function	Description
11	**Manage Milestone**	[GET] /api/contracts/{contractId}/milestones [GET] /api/contracts/{contractId}/milestones/{id} [POST] /api/contracts/{contractId}/milestones [PATCH] /api/contracts/{contractId}/milestones/{id} [PATCH] /api/contracts/{contractId}/milestones/{id}/progress [DELETE] /api/contracts/{contractId}/milestones/{id}	The Milestone Management feature breaks a contract into payable deliverables. Contract participants can list and view the milestones of a contract, create a new milestone with its amount and due date, update milestone information, report progress on a milestone, and delete a milestone that is no longer part of the agreement.
12	**Manage Milestone Files**	[GET] /api/contracts/{contractId}/milestones/{milestoneId}/files [POST] /api/contracts/{contractId}/milestones/{milestoneId}/files [DELETE] /api/contracts/{contractId}/milestones/{milestoneId}/files/{fileId}	The Milestone File feature allows contract participants to attach supporting documents and media to a specific milestone. Users can list every file of a milestone, upload a new file, and delete a file that was uploaded by mistake or is no longer relevant.
13	**Manage Milestone Submissions**	[POST] /api/contracts/{contractId}/milestones/{milestoneId}/submissions [GET] /api/contracts/{contractId}/milestones/{milestoneId}/submissions [GET] /api/contracts/{contractId}/milestones/{milestoneId}/submissions/{id} [PATCH] /api/contracts/{contractId}/milestones/{milestoneId}/submissions/{id}/request-changes [PATCH] /api/contracts/{contractId}/milestones/{milestoneId}/submissions/{id}/approve	The Milestone Submission feature supports the deliverable review workflow. A freelancer can submit work for a milestone, and both parties can list the submissions of a milestone and view the detail of one submission. The client can request changes so that the freelancer revises and resubmits the work, or approve the submission to complete the milestone and release the corresponding payment.

#	Feature	System Function	Description
14	**Manage Payments**	[POST] /api/payments/connect/onboarding-link [GET] /api/payments/connect/status [GET] /api/payments/payment-methods [POST] /api/payments/customer-portal [POST] /api/payments/contracts/{contractId}/platform-fee/intent [POST] /api/payments/contracts/{contractId}/platform-fee/checkout-session [POST] /api/payments/contracts/{contractId}/platform-fee/saved-card [POST] /api/payments/contracts/{contractId}/milestones/{milestoneId}/fund/intent [POST] /api/payments/contracts/{contractId}/milestones/{milestoneId}/fund/checkout-session [POST] /api/payments/contracts/{contractId}/milestones/{milestoneId}/fund/saved-card [POST] /api/payments/contracts/{contractId}/milestones/{milestoneId}/release [POST] /api/payments/contracts/{contractId}/milestones/{milestoneId}/refund [POST] /api/payments/disputes/{disputeId}/fee/intent [POST] /api/payments/disputes/{disputeId}/fee/checkout-session [POST] /api/payments/disputes/{disputeId}/settle [GET] /api/payments/transactions [POST] /api/payments/intents/{paymentIntentId}/sync [POST] /api/payments/checkout-sessions/{sessionId}/sync [POST] /api/payments/webhook	The Payment Management feature integrates Stripe to provide secure escrow-style transactions. Users can connect a Stripe account and check its onboarding status, list saved payment methods, open the Stripe customer portal, and pay the platform fee or fund a milestone either through a PaymentIntent, a checkout session, or a saved card. Clients can release or refund milestone funds, pay a dispute fee and settle a dispute, and view their transaction history. The system also synchronizes PaymentIntent and CheckoutSession status and receives asynchronous Stripe webhook events.

#	Feature	System Function	Description
15	**Manage Dispute**	[GET] /api/disputes [POST] /api/disputes [GET] /api/disputes/{id} [POST] /api/disputes/{id}/fee [POST] /api/disputes/{id}/response [GET] /api/disputes/milestone/{milestoneId} [POST] /api/disputes/{id}/review [GET] /api/admin/disputes [GET] /api/admin/disputes/{id} [POST] /api/admin/disputes/{id}/decision [POST] /api/admin/disputes/{id}/ final-decision	The Dispute Management feature handles conflicts that arise during a milestone. Participants can open a dispute for a milestone, list and view their disputes, pay the dispute fee, submit a response to the other party's claim, and review the proposed resolution. The dispute of a specific milestone can be looked up. Administrators can list all disputes, view dispute details, issue a decision, and record the final decision that settles the conflict.
16	**Manage Review**	[GET] /api/reviews/contracts/{contractId} [POST] /api/reviews/contracts/{contractId} [GET] /api/reviews/{reviewId} [PATCH] /api/reviews/{reviewId} [DELETE] /api/reviews/{reviewId} [POST] /api/reviews/{reviewId}/response [PATCH] /api/reviews/responses/{responseId} [DELETE] /api/reviews/responses/ {responseId}	The Review Management feature lets contract participants rate and comment on each other after a contract is completed. Users can list the reviews of a contract, create a review, view, update and delete their own review, and respond to a review they received. Responses can also be updated or deleted by their author, helping both parties maintain a trustworthy reputation on the platform.

#	Feature	System Function	Description
17	**Manage Forum**	[GET] /api/forums/categories [GET] /api/forums/categories/top [GET] /api/forums/categories/{id} [GET] /api/forums/categories/by-slug/{slug} [GET] /api/forums/posts [GET] /api/forums/posts/top [GET] /api/forums/posts/{id} [GET] /api/forums/posts/by-slug/{slug} [GET] /api/forums/users/top [POST] /api/forums/posts [DELETE] /api/forums/posts/{id} [GET] /api/forums/posts/{postId}/comments [POST] /api/forums/posts/{postId}/comments [PATCH] /api/forums/posts/{postId}/comments/{id} [DELETE] /api/forums/posts/{postId}/comments/{id} [POST] /api/forums/posts/{postId}/comments/{commentId}/like [POST] /api/forums/posts/{id}/like [GET] /api/forums/posts/{id}/like [POST] /api/forums/posts/{postId}/reports [POST] /api/forums/posts/{postId}/comments/{commentId}/reports [GET] /api/forums/posts/{postId}/is-reported [GET] /api/forums/posts/{postId}/comments/{commentId}/is-reported	The Forum Management feature enables community discussion and knowledge sharing within the platform. Guests and authenticated users can browse forum categories (including the most active categories), browse and search posts (including the most interacted posts and the most active users), and open a post by id or slug. Registered users can create a post, delete their own post, and manage comments (list, add, edit and delete). Posts and comments support a like/unlike toggle and a liker list, and users can report inappropriate posts or comments and check whether they have already reported them.

#	Feature	System Function	Description
18	**Manage Forum Moderation**	[GET] /api/forums/admin/stats [GET] /api/forums/admin/categories [GET] /api/forums/admin/categories/{id} [POST] /api/forums/admin/categories [PATCH] /api/forums/admin/categories/{id} [DELETE] /api/forums/admin/categories/{id} [PATCH] /api/forums/admin/categories/{id}/restore [GET] /api/forums/admin/comments [GET] /api/forums/admin/pending-posts [GET] /api/forums/admin/trash/posts [GET] /api/forums/admin/trash/comments [PATCH] /api/forums/admin/trash/posts/{id}/restore [PATCH] /api/forums/admin/trash/comments/{id}/restore [PATCH] /api/forums/admin/posts/{id}/approve [PATCH] /api/forums/admin/posts/{id}/reject [GET] /api/forums/reports [PATCH] /api/forums/reports/{reportId}/status	The Forum Moderation feature gives administrators the tools to keep the community healthy. Administrators can view forum statistics, list, view, create, update, soft-delete and restore forum categories, and list all comments. New posts can be reviewed through the pending-post queue and either approved or rejected, while the trash lists allow deleted posts and comments to be reviewed and restored. The report queue lists content reported by users, and each report's status can be updated once it has been handled.
19	**Manage Messaging**	[GET] /api/conversations [POST] /api/conversations [GET] /api/conversations/{id}/messages [POST] /api/conversations/{id}/messages [POST] /api/conversations/{id}/files [DELETE] /api/conversations/{id}/messages/{messageId} [PATCH] /api/conversations/{id}/read [PATCH] /api/conversations/{id}/hide [PATCH] /api/conversations/{id}/pin	The Messaging feature provides real-time communication between clients and freelancers. Users can list their conversations, start a new conversation, list and send messages, and send files within a conversation. A message can be deleted or recalled, a conversation can be marked as read to reset its unread counter, hidden from the conversation list, and pinned or unpinned. Messages are delivered in real time through a WebSocket gateway.

#	Feature	System Function	Description
20	**Manage Notification**	[GET] /api/notifications [PATCH] /api/notifications/read-all [PATCH] /api/notifications/{id}/read [DELETE] /api/notifications/{id}	The Notification feature delivers in-app notifications about relevant events such as proposals, contracts, milestones, payments and forum moderation. Users can list their notifications with their read state, mark a single notification or all notifications as read, and delete a notification they no longer want to keep.
21	**Manage Roles**	[GET] /api/roles [GET] /api/roles/{id} [POST] /api/roles [PATCH] /api/roles/{id} [PUT] /api/roles/{id}/permissions [DELETE] /api/roles/{id}	The Role Management feature allows administrators to define and control user permissions. Administrators can list all roles, view the detail of a role together with its permissions, create a new role, update its name or description, replace the permission set assigned to a role, and delete a role that is no longer needed. This provides fine-grained access control and proper segregation of duties across user types.
22	**Manage Permissions**	[GET] /api/permissions [GET] /api/permissions/{id}	The Permission Management feature exposes the catalog of permissions used by the role-based access control system. Administrators can list every available permission and view the details of a single permission so that permissions can be assigned to roles accurately.

#	Feature	System Function	Description
23	**Manage Sessions**	[GET] /api/sessions [GET] /api/sessions/{id} [DELETE] /api/sessions/{id}	The Session Management feature gives authenticated users visibility and control over their active login sessions. Users can list all sessions of their account, view the details of a specific session (such as device and login time), and revoke a session so that the corresponding refresh token can no longer be used.
24	**Manage Users (Admin)**	[POST] /api/users [GET] /api/users [GET] /api/users/{id} [PATCH] /api/users/{id} [PATCH] /api/users/{id}/client-profile [PATCH] /api/users/{id}/freelancer-profile [PUT] /api/users/{id}/freelancer-profile/skills [POST] /api/users/{id}/freelancer-profile/portfolio-items [PATCH] /api/users/{id}/freelancer-profile/portfolio-items/{itemId} [DELETE] /api/users/{id}/freelancer-profile/portfolio-items/{itemId} [GET] /api/users/skills-catalog	The User Management feature allows administrators to manage accounts from the administration side. Administrators can list users, view user details, create a new user, and update general user information. They can also edit a client profile, edit a freelancer's basic profile and replace the freelancer's skills, manage the freelancer's portfolio items, and read the skill catalog when performing these updates.

#	Feature	System Function	Description
25	**Manage Identity Verification**	[POST] /api/identity-verifications/documents [POST] /api/identity-verifications/ai-check [POST] /api/identity-verifications/liveness-pose [GET] /api/identity-verifications/status [GET] /api/identity-verifications/documents/{id}/file [GET] /api/admin/identity-verifications [GET] /api/admin/identity-verifications/{id} [GET] /api/admin/identity-verifications/{id}/file [PATCH] /api/admin/identity-verifications/{id}/approve [PATCH] /api/admin/identity-verifications/{id}/reject	The Identity Verification feature supports the eKYC process for user accounts. Users can upload identity documents, run an AI check on the submitted documents, capture liveness-pose images, check their verification status, and download an uploaded document file. Administrators can list verification requests, view request details and the attached file, then approve or reject the verification with a reason.
26	**Manage Profile Revisions**	[GET] /api/profile-revisions/me [GET] /api/admin/profile-revisions [GET] /api/admin/profile-revisions/{id} [PATCH] /api/admin/profile-revisions/{id}/approve [PATCH] /api/admin/profile-revisions/{id}/reject	The Profile Revision feature tracks changes made to freelancer profiles so that they can be moderated. A freelancer can view the latest revision of their own profile, while administrators can list all pending revisions, view the detail of a revision, and approve or reject it before the changes become publicly visible.

#	Feature	System Function	Description
27	**Manage Banners**	[GET] /api/banners [GET] /api/banners/image/{filename} [GET] /api/admin/banners [GET] /api/admin/banners/{id} [POST] /api/admin/banners [POST] /api/admin/banners/upload-image [PATCH] /api/admin/banners/{id} [DELETE] /api/admin/banners/{id}	The Advertisement Banner feature manages promotional banners displayed on the platform. The public endpoints return the active banners for a given position and serve the banner image files. Administrators can list and view all banners, create a new banner, upload its image, update its content, target position or schedule, and delete a banner.
28	**Manage Skills (Admin)**	[GET] /api/admin/skills [GET] /api/admin/skills/{id} [POST] /api/admin/skills [PATCH] /api/admin/skills/{id} [PATCH] /api/admin/skills/{id}/restore [DELETE] /api/admin/skills/{id}	The Skill Management feature allows administrators to maintain the master skill catalog used by freelancer profiles and job postings. Administrators can list skills, view the detail of a skill, create a new skill, update its name or category, soft-delete a skill, and restore a previously deleted skill.

4.1.5 Entity Relationship Diagram

4.1.5.1 Overall Entity Relationship Diagram

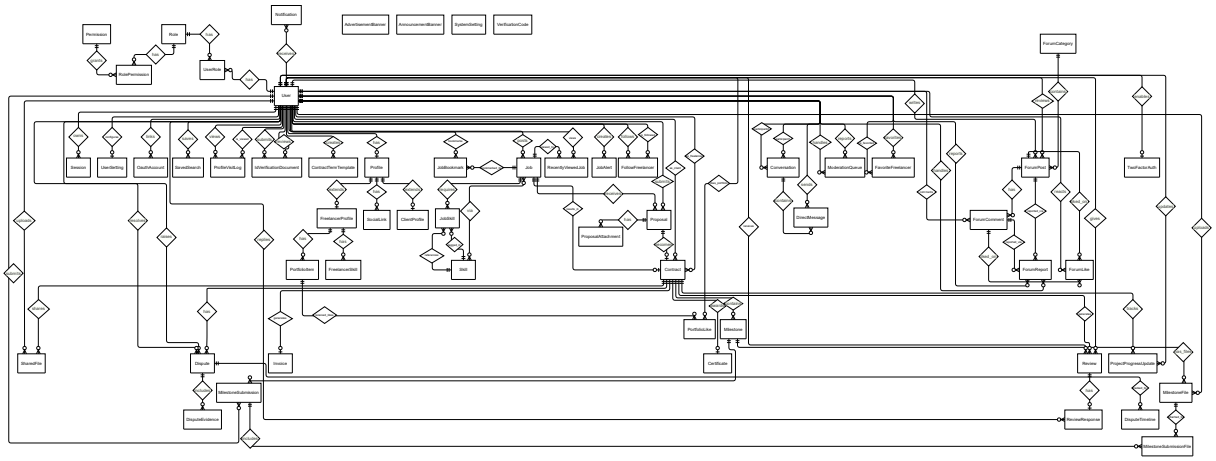


Figure 13: Overall Entity Relationship Diagram

4.1.5.2 Authentication & Sessions

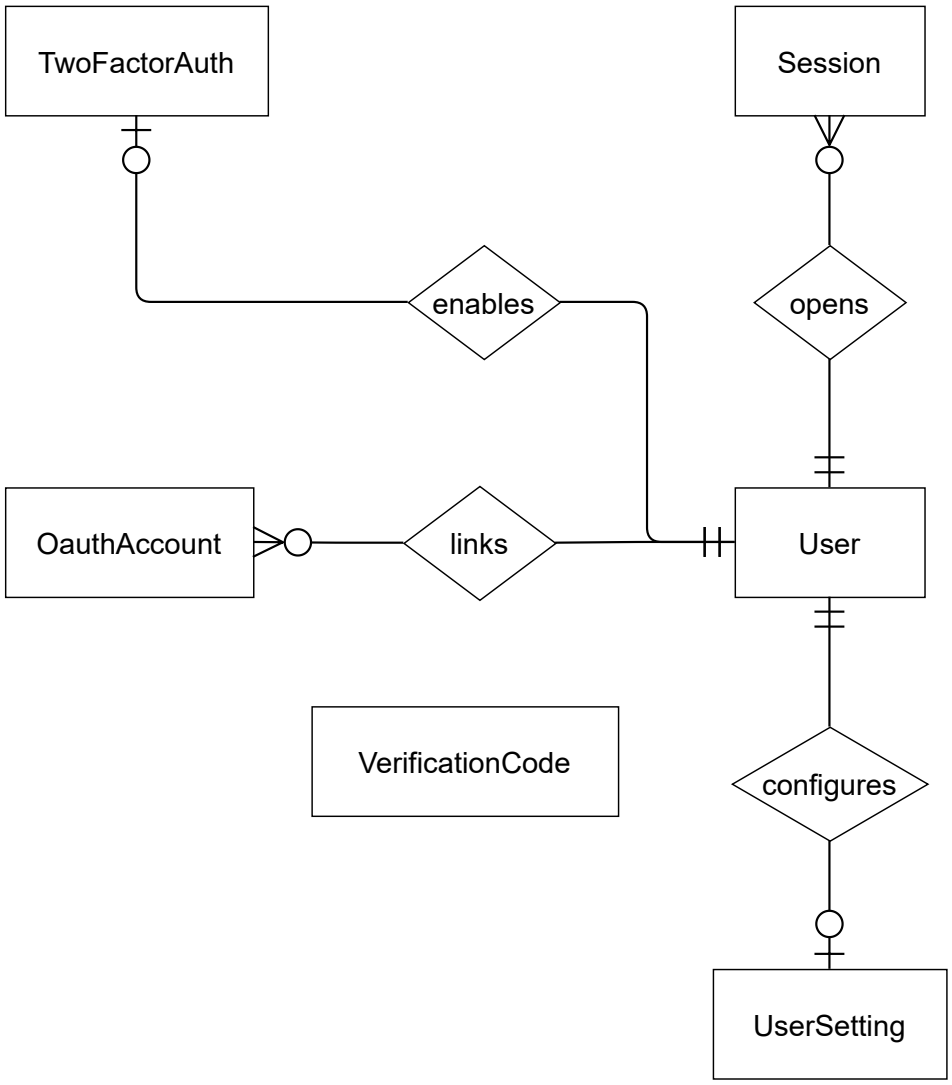


Figure 14: Authentication & Sessions ERD

4.1.5.3 Role-Based Access Control

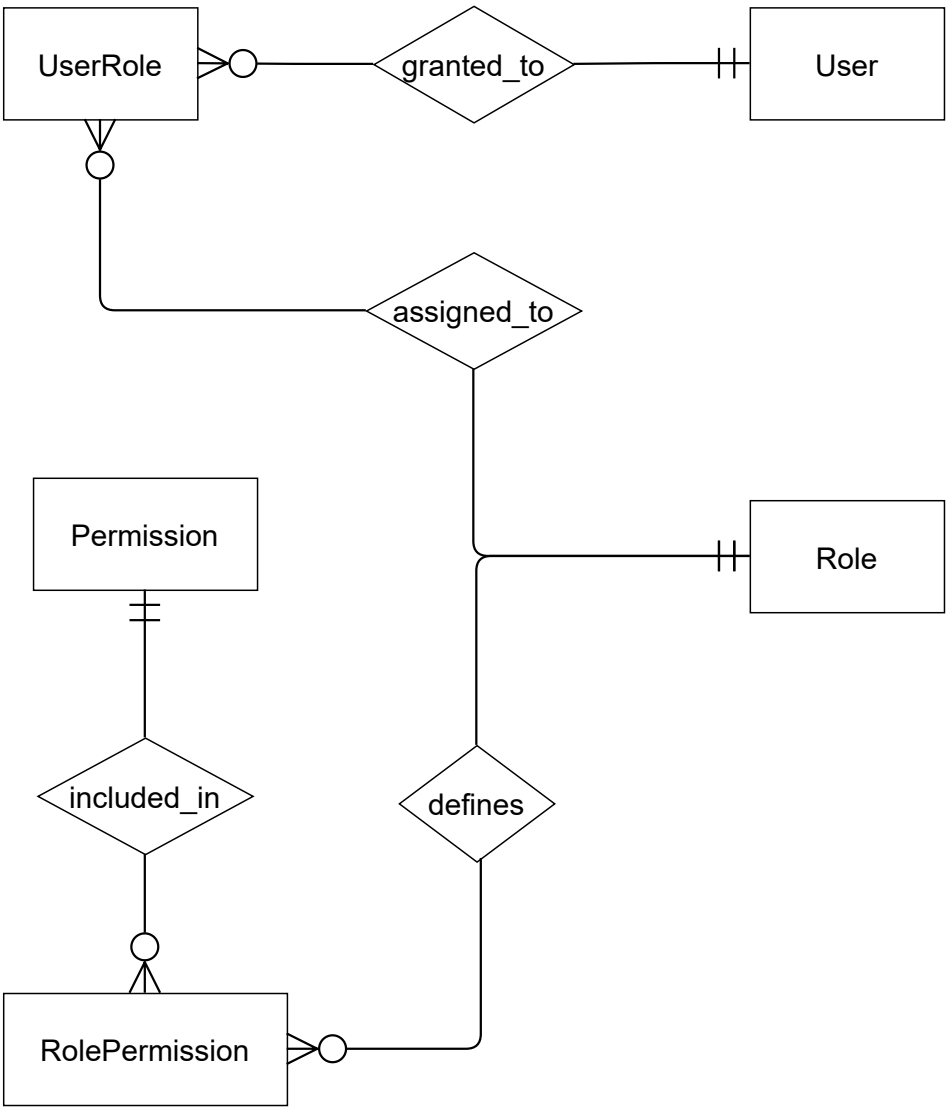


Figure 15: Role-Based Access Control ERD

4.1.5.4 Profile Core

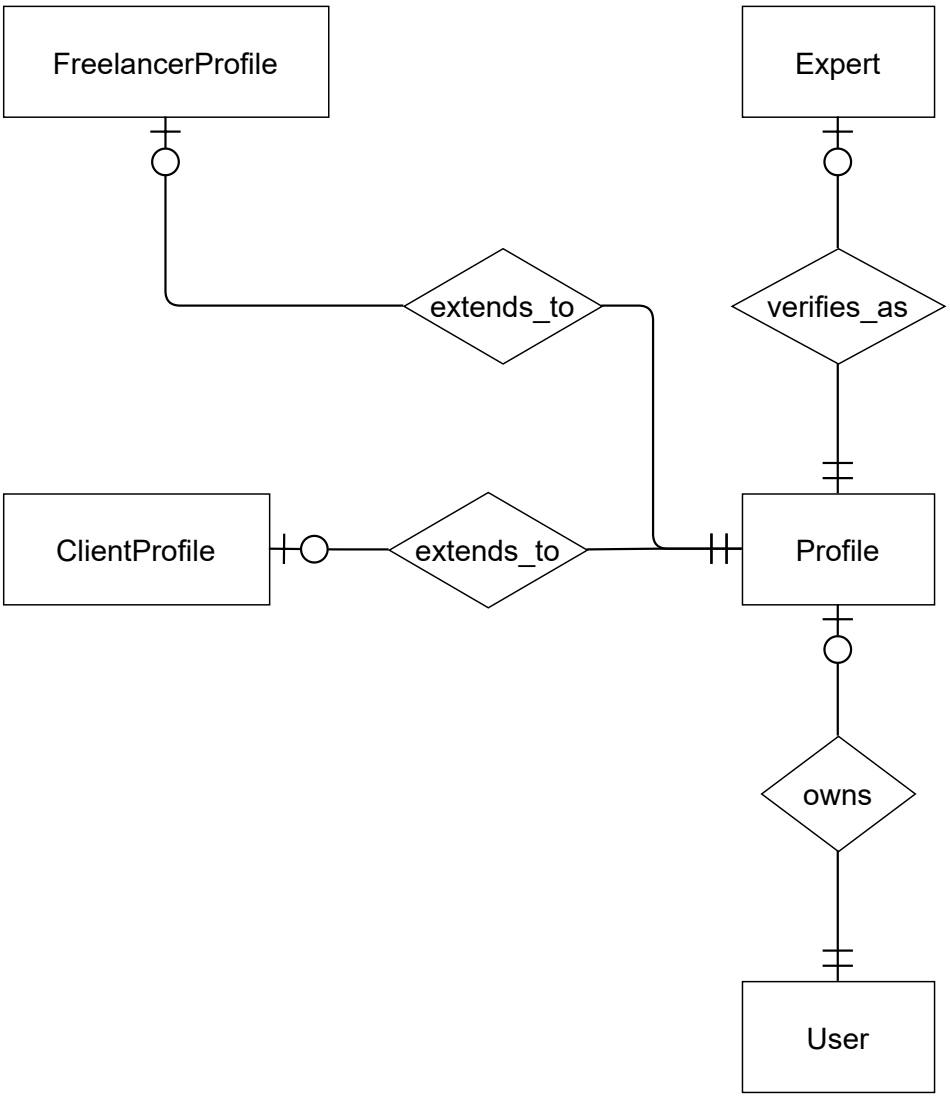


Figure 16: Profile Core ERD

4.1.5.5 Profile History & Social Links

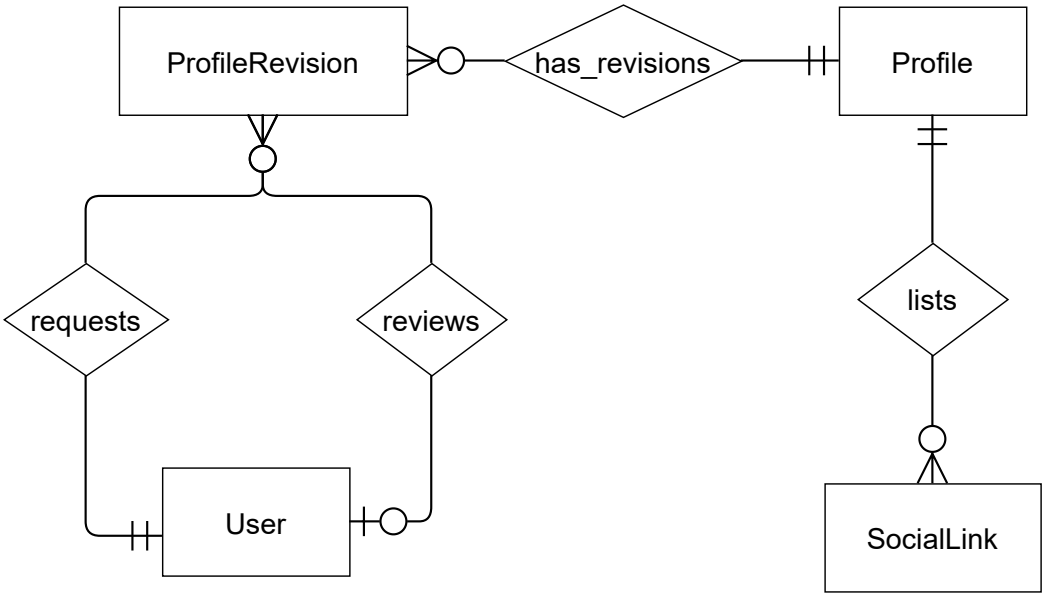


Figure 17: Profile History & Social Links ERD

4.1.5.6 Identity Verification & Profile Visits

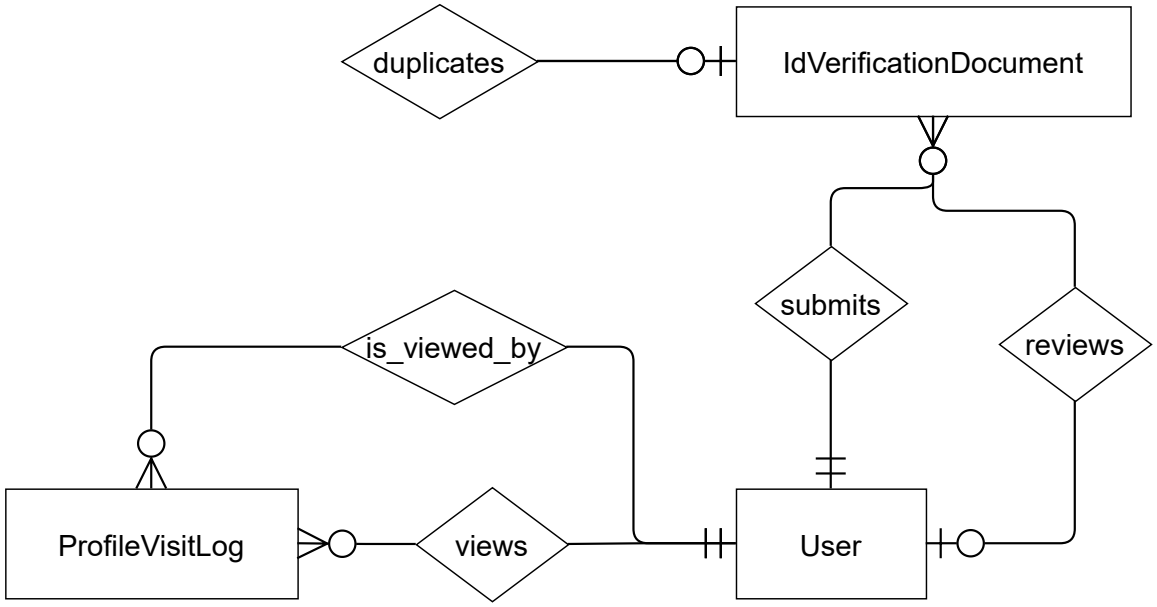


Figure 18: Identity Verification & Profile Visits ERD

4.1.5.7 Portfolio & Skills

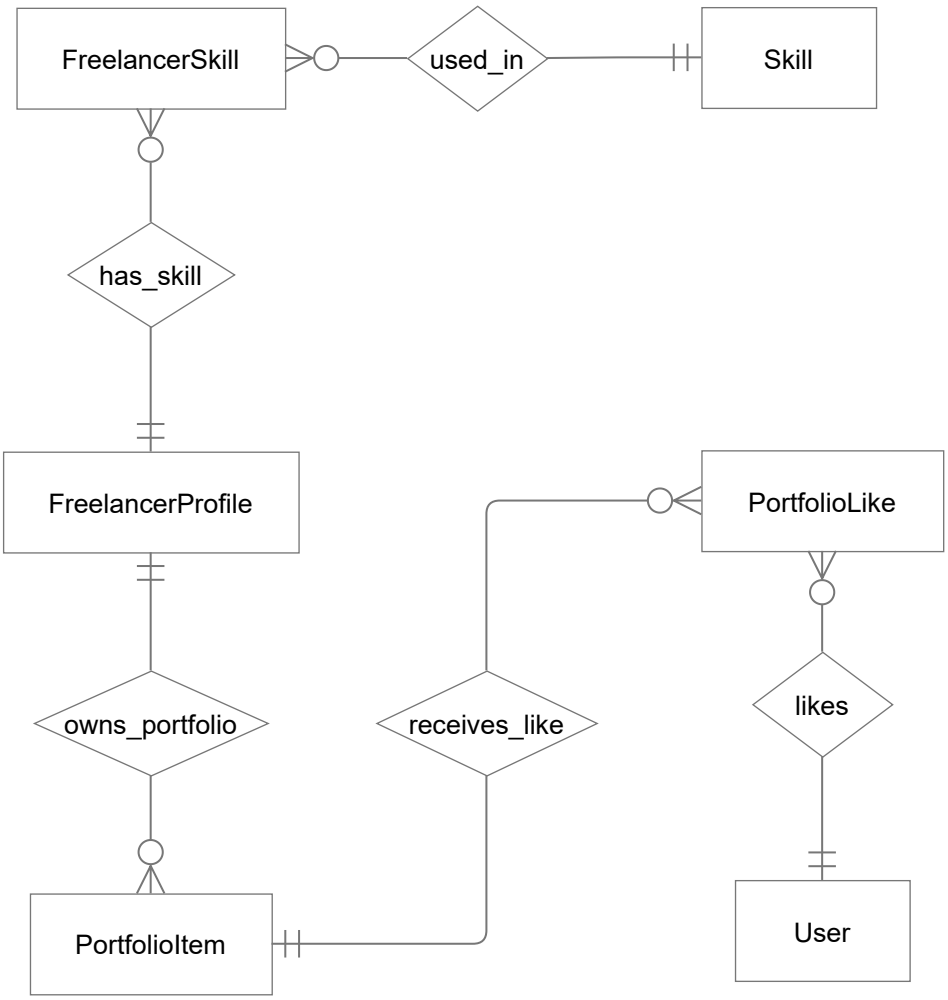


Figure 19: Portfolio & Skills ERD

4.1.5.8 Favorites & Follows

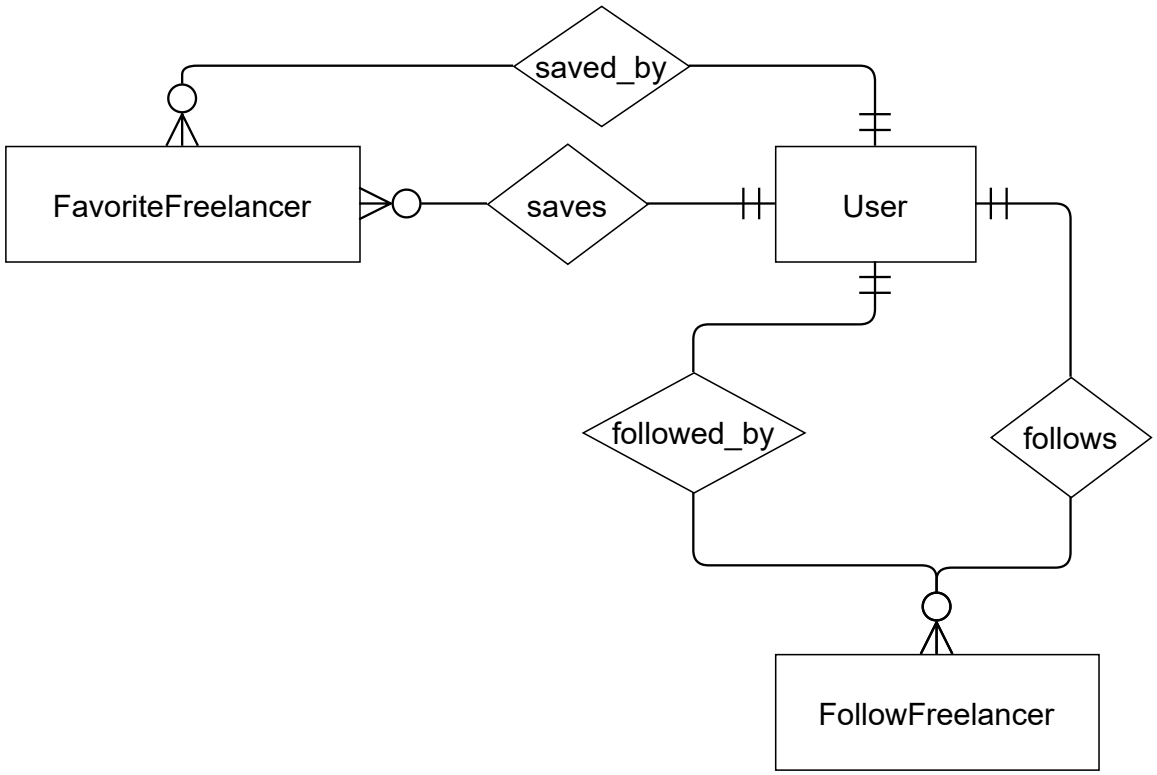


Figure 20: Favorites & Follows ERD

4.1.5.9 Job Posting

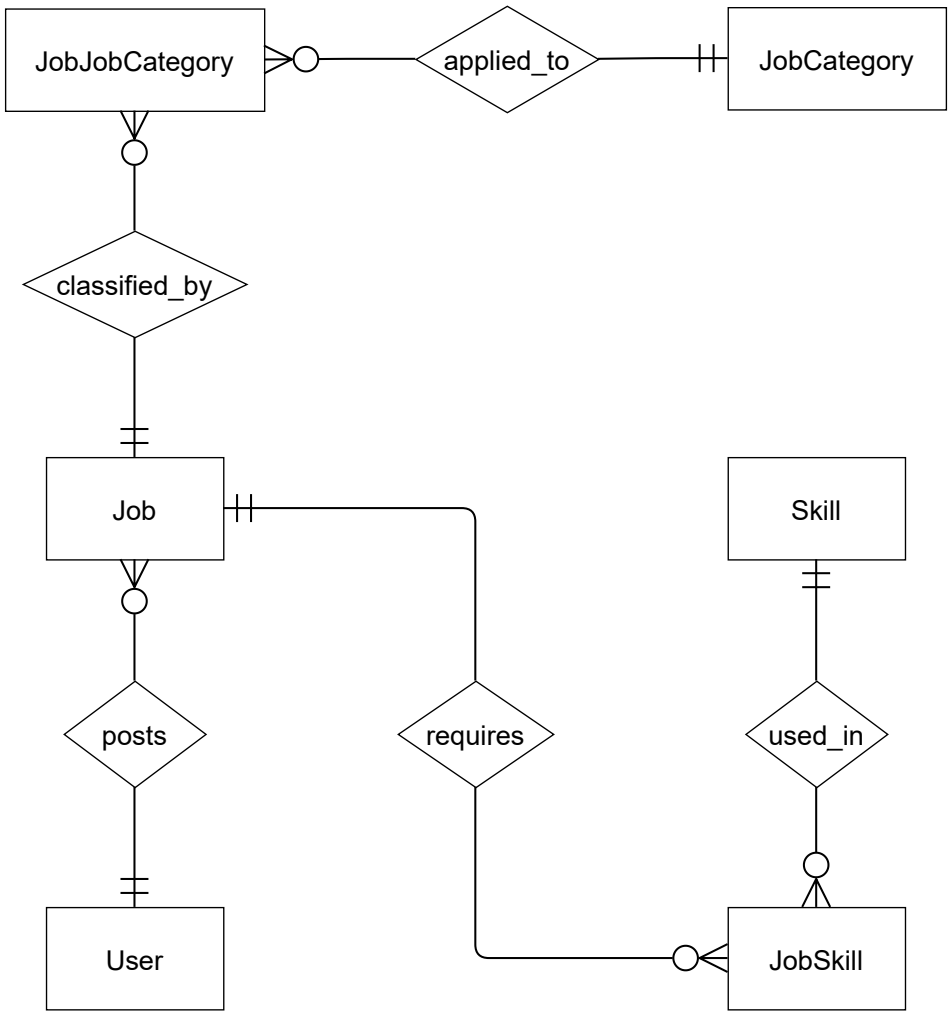


Figure 21: Job Posting ERD

4.1.5.10 Job Discovery & Alerts

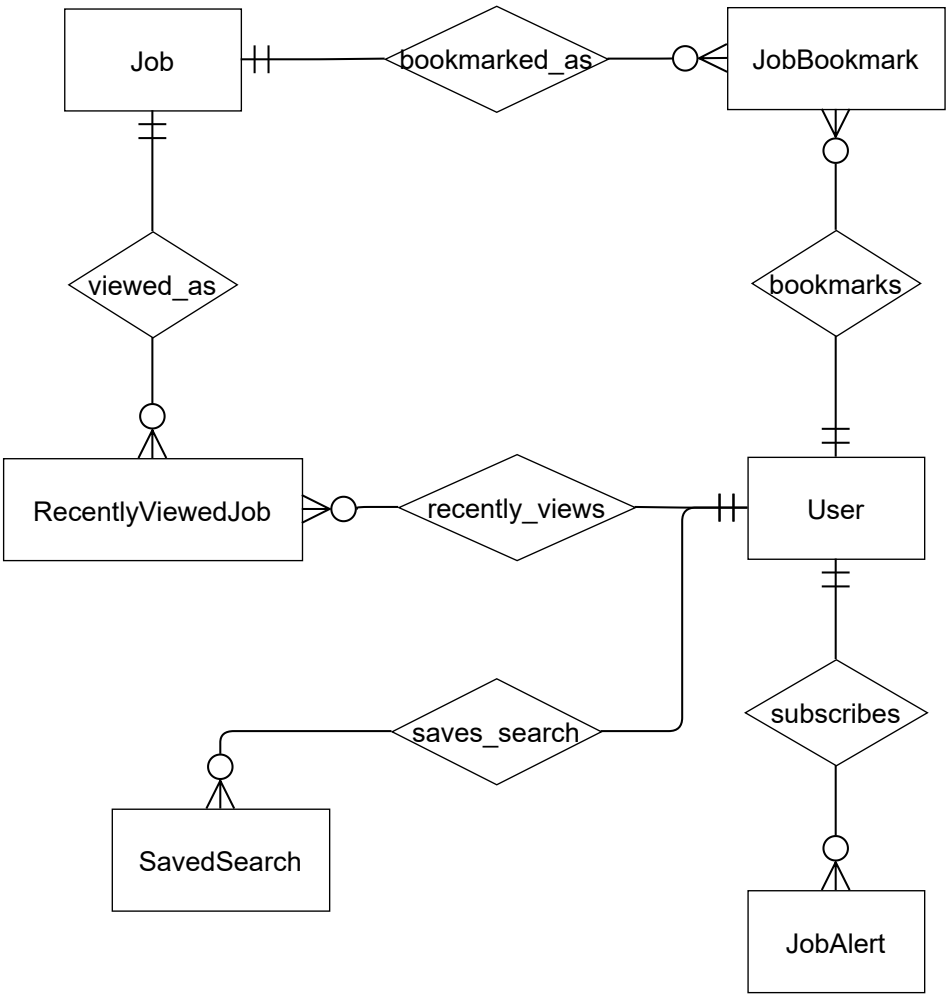


Figure 22: Job Discovery & Alerts ERD

4.1.5.11 Proposal

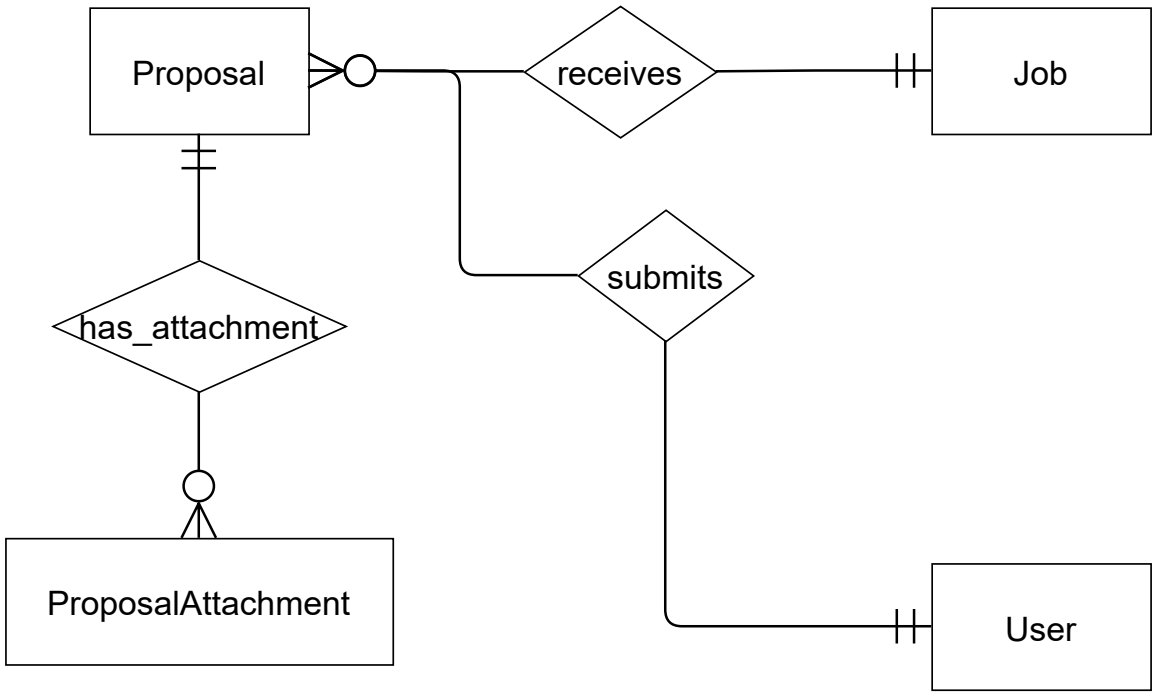


Figure 23: Proposal ERD

4.1.5.12 Contract

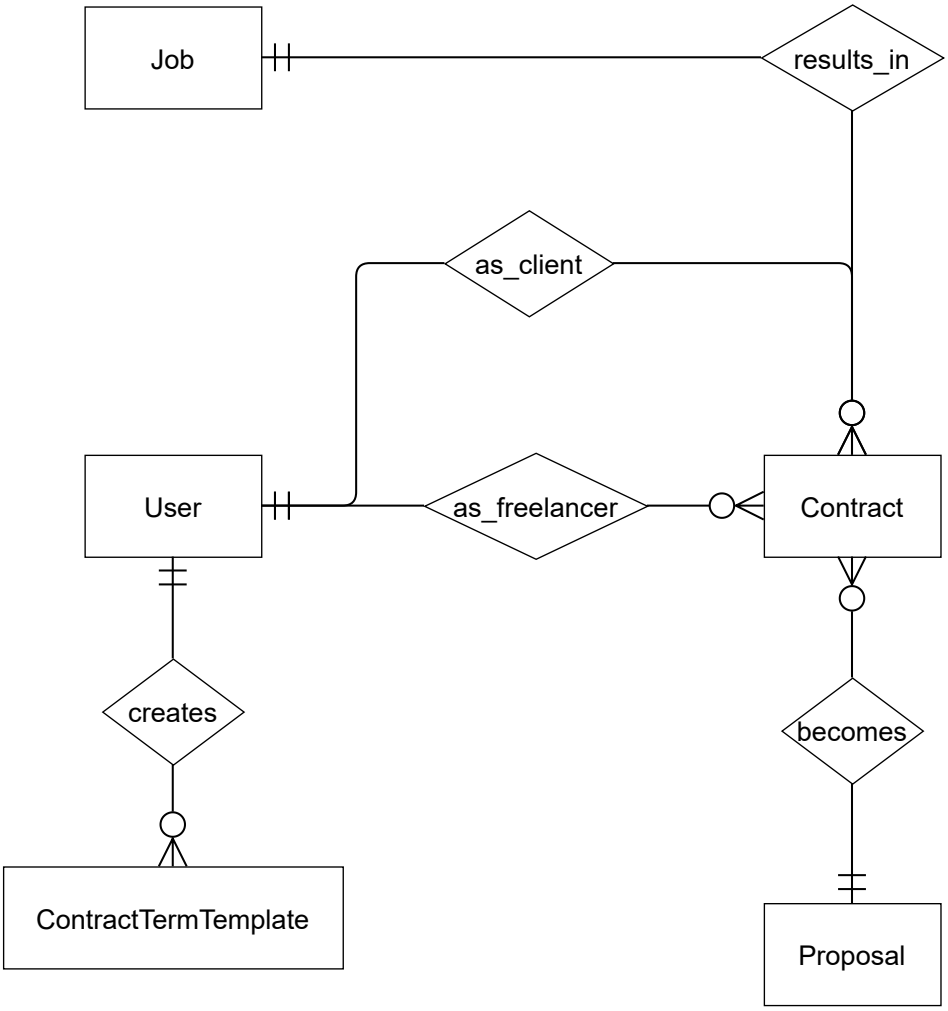


Figure 24: Contract ERD

4.1.5.13 Milestone & Project Progress

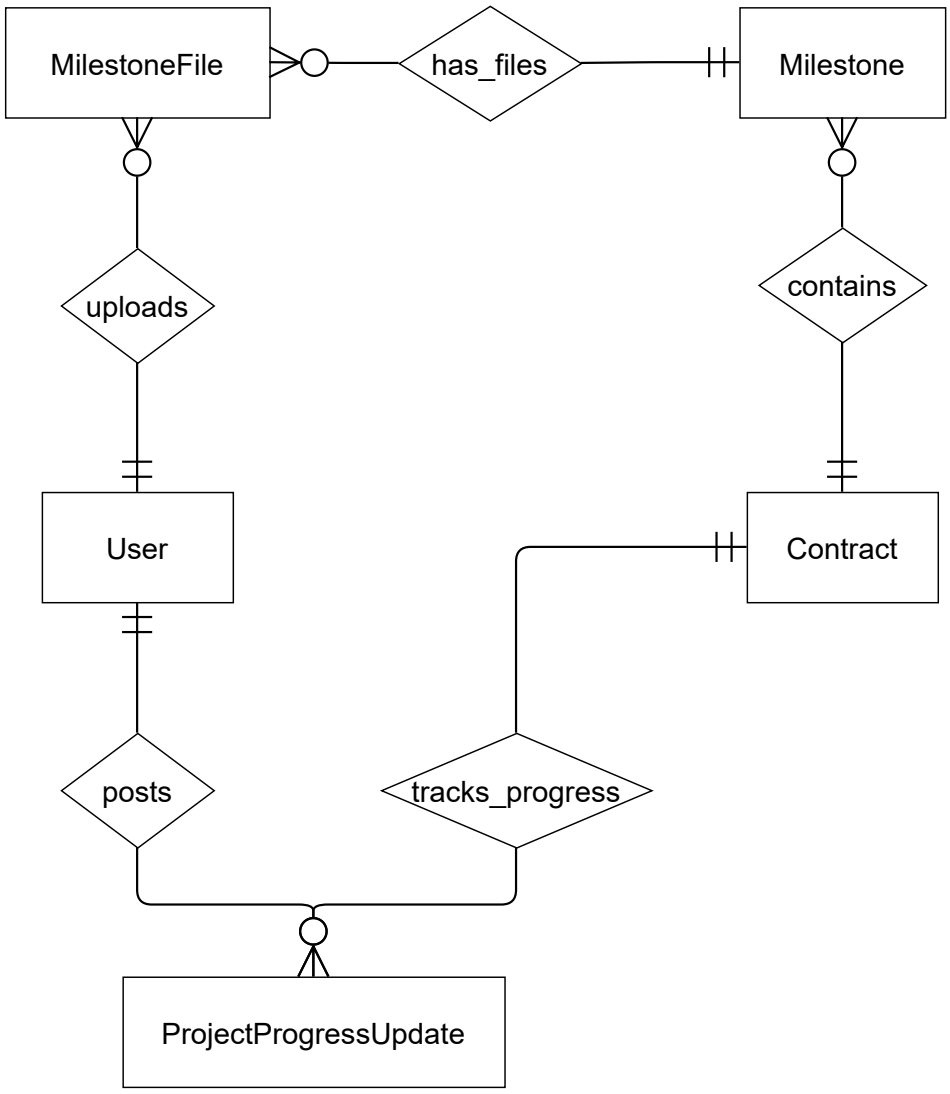


Figure 25: Milestone & Project Progress ERD

4.1.5.14 Milestone Submission

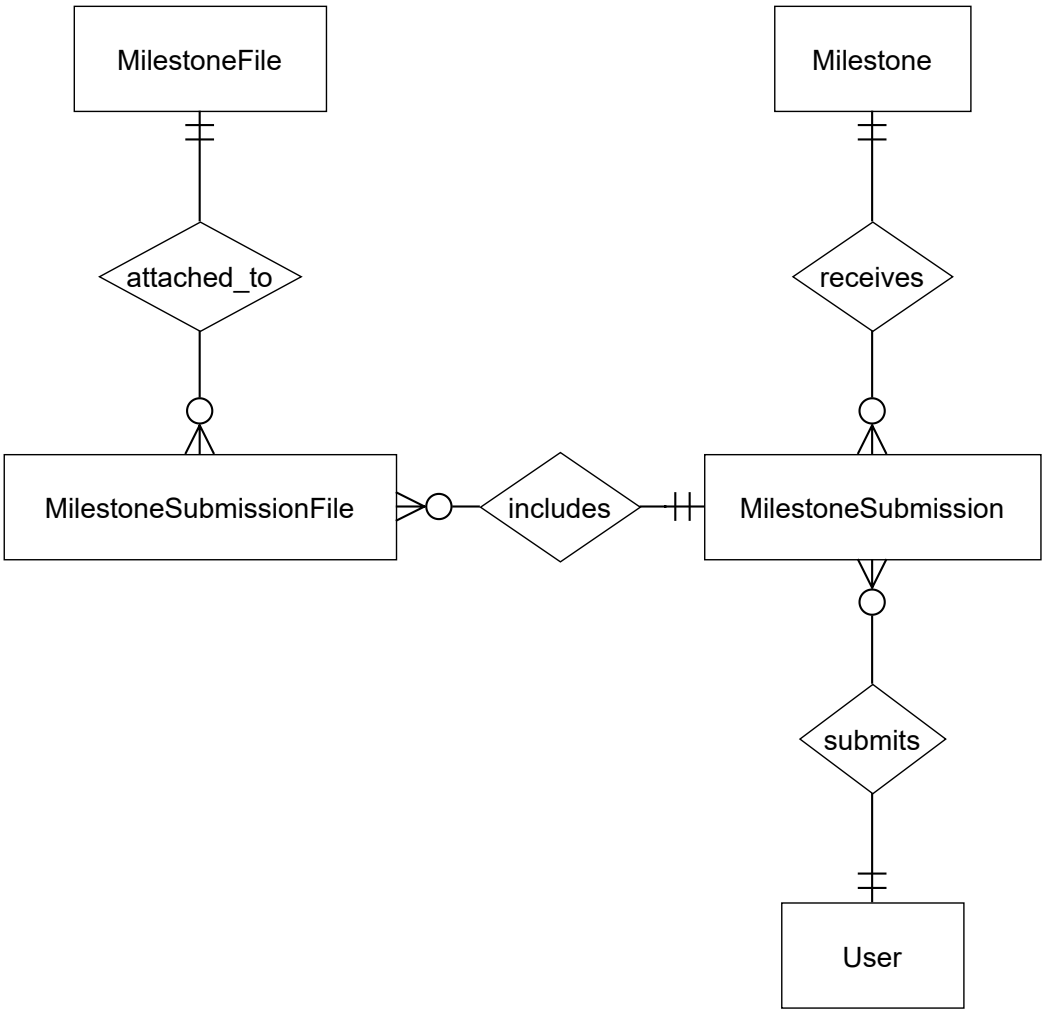


Figure 26: Milestone Submission ERD

4.1.5.15 Escrow, Payments & Certificates

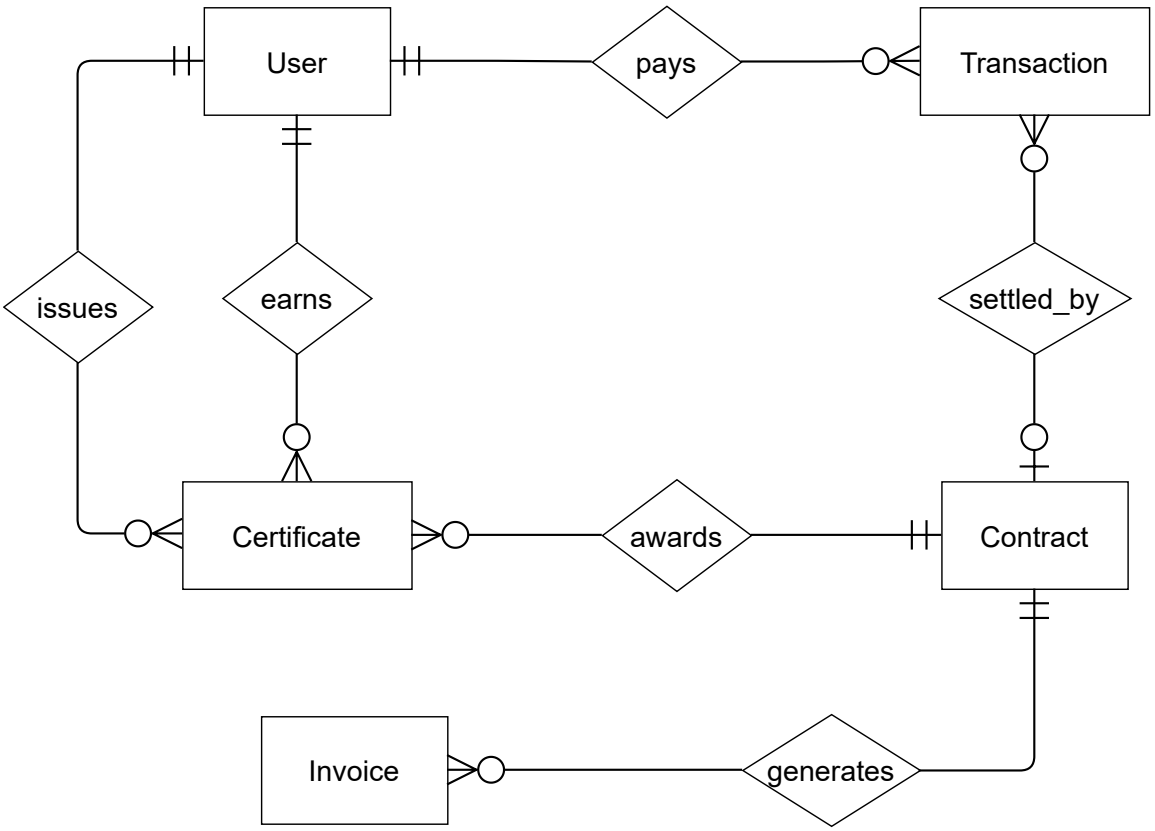


Figure 27: Escrow, Payments & Certificates ERD

4.1.5.16 Dispute Core

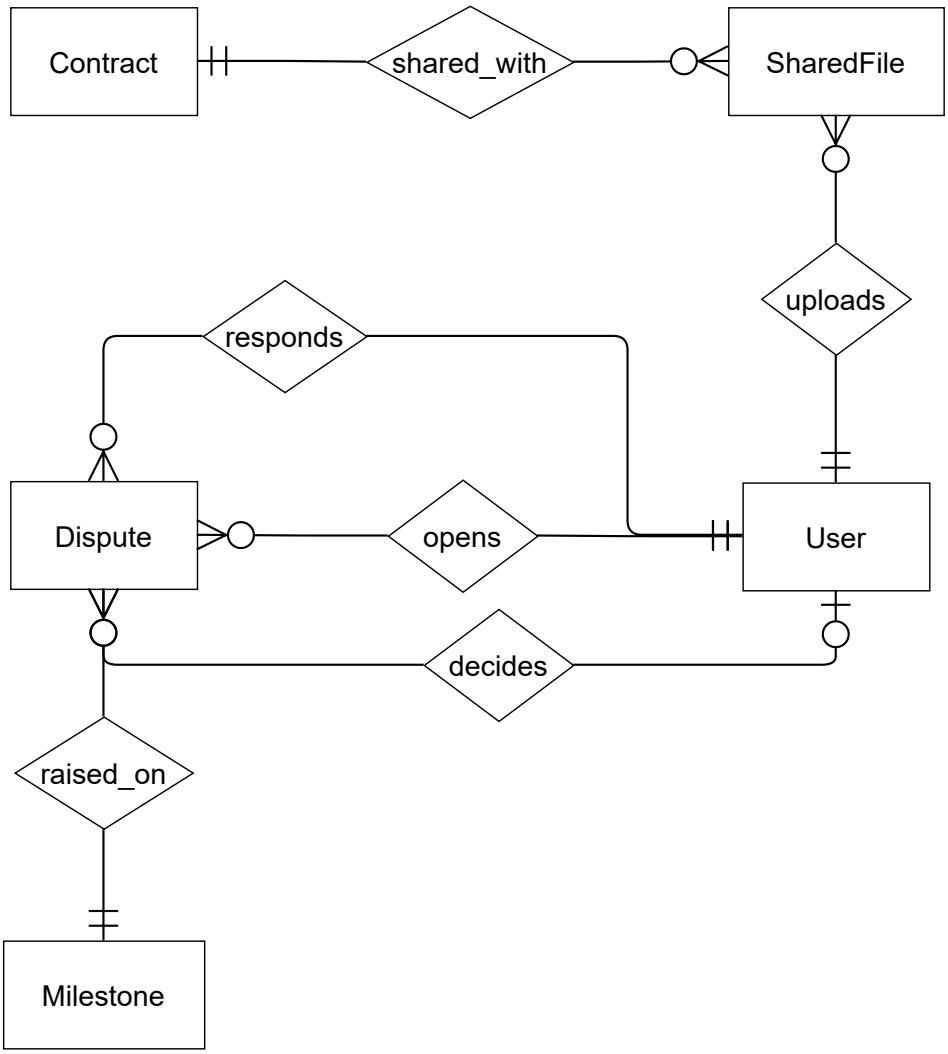


Figure 28: Dispute Core ERD

4.1.5.17 Dispute Resolution

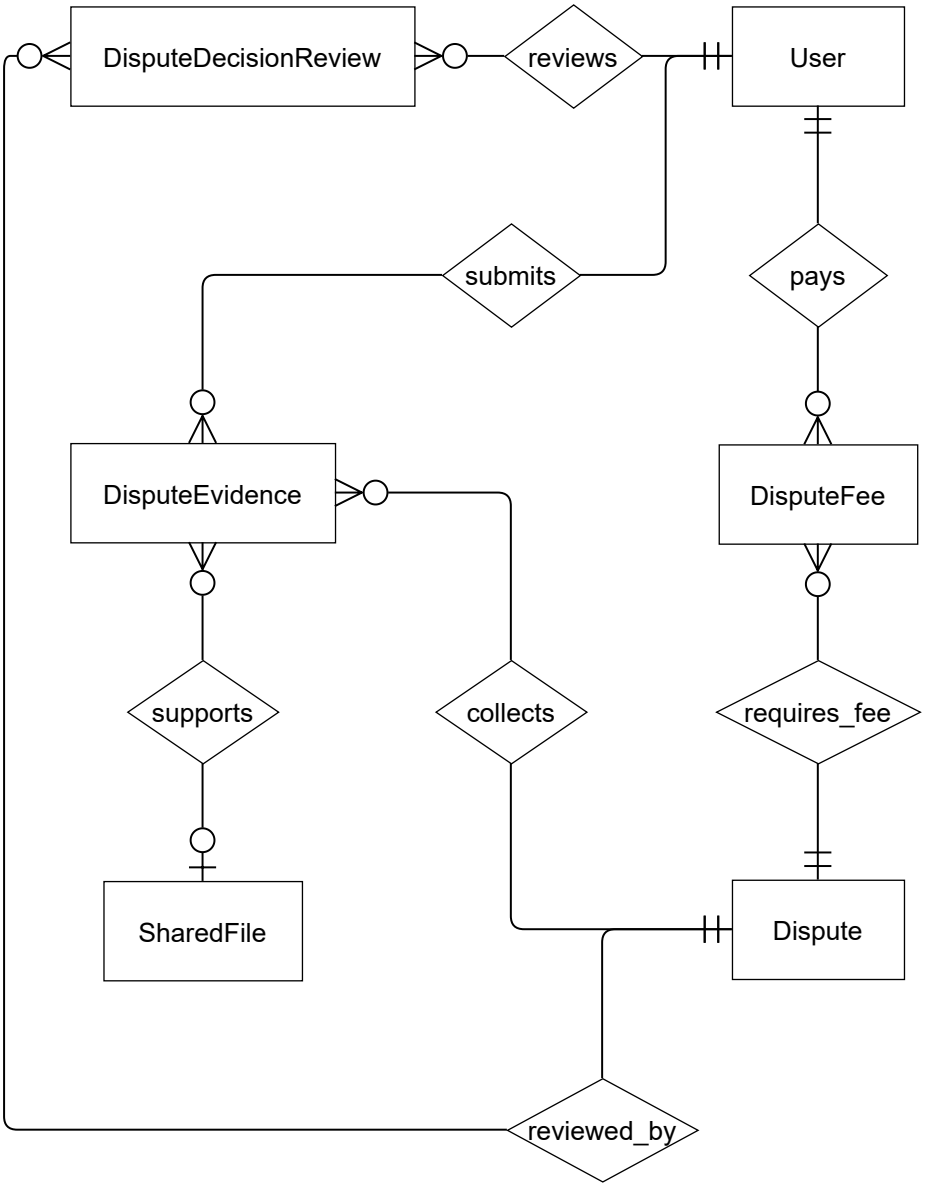


Figure 29: Dispute Resolution ERD

4.1.5.18 Reviews & Reputation

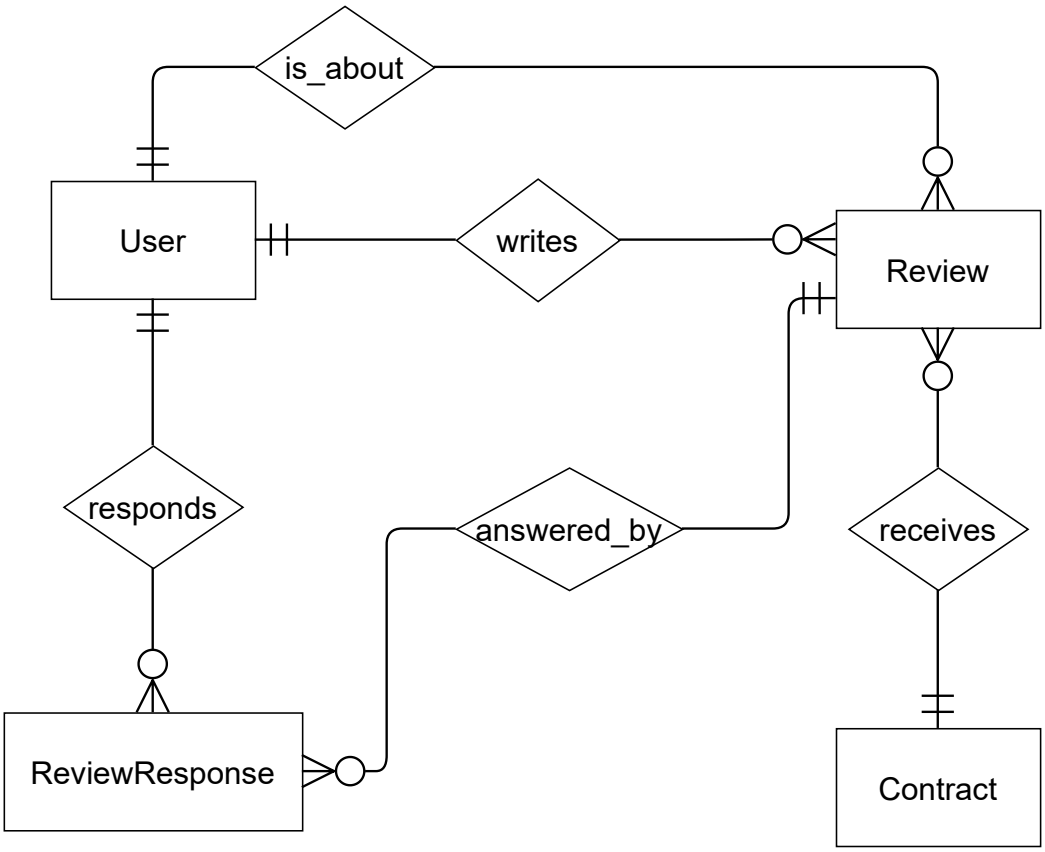


Figure 30: Reviews & Reputation ERD

4.1.5.19 Forum Posts & Comments

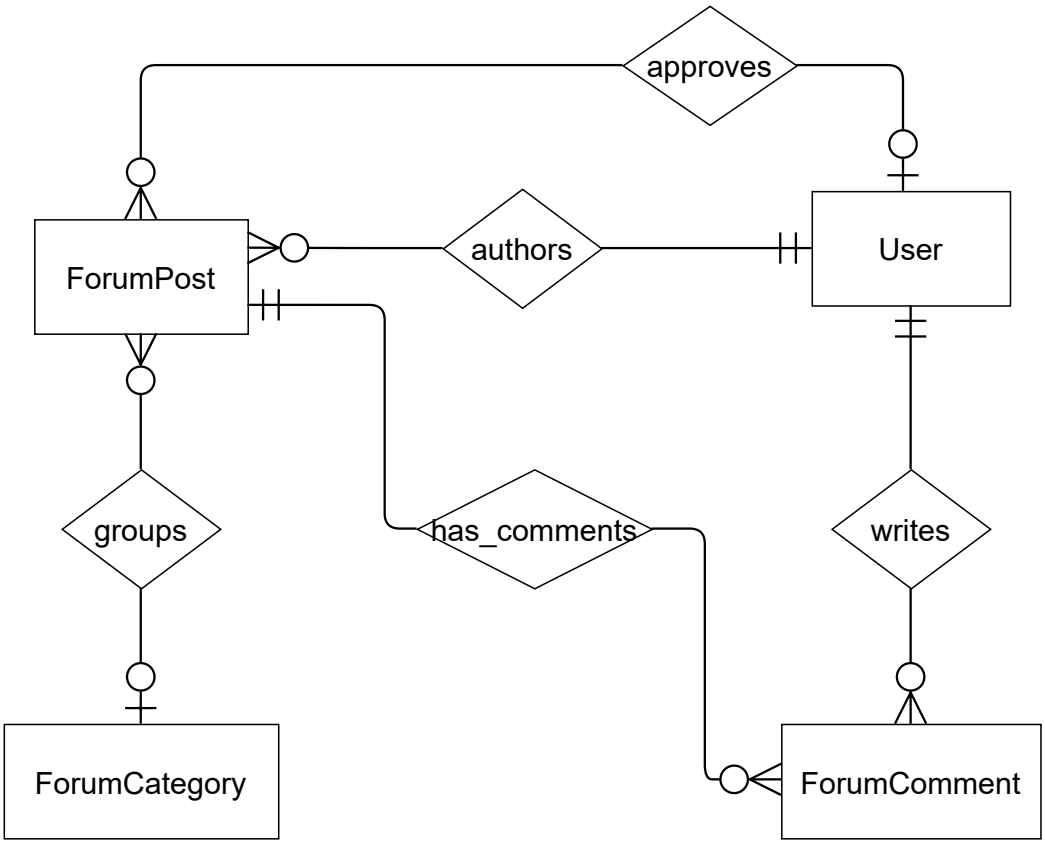


Figure 31: Forum Posts & Comments ERD

4.1.5.20 Forum Likes

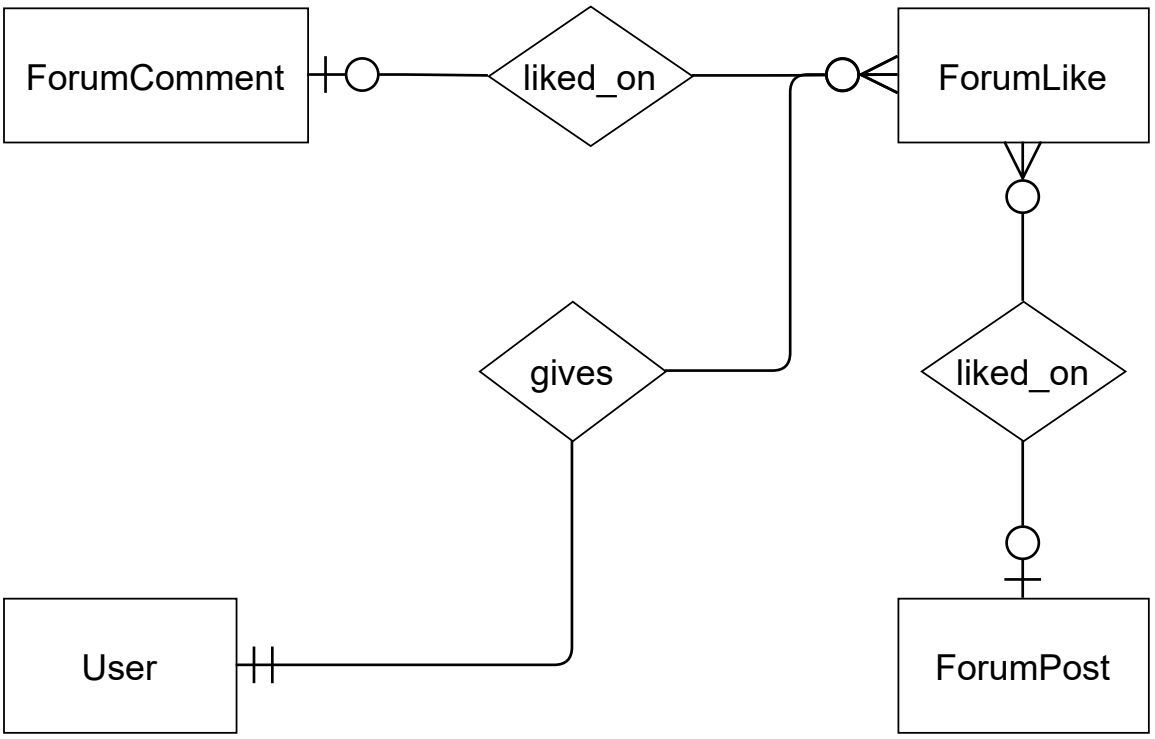


Figure 32: Forum Likes ERD

4.1.5.21 Forum Reports & Moderation

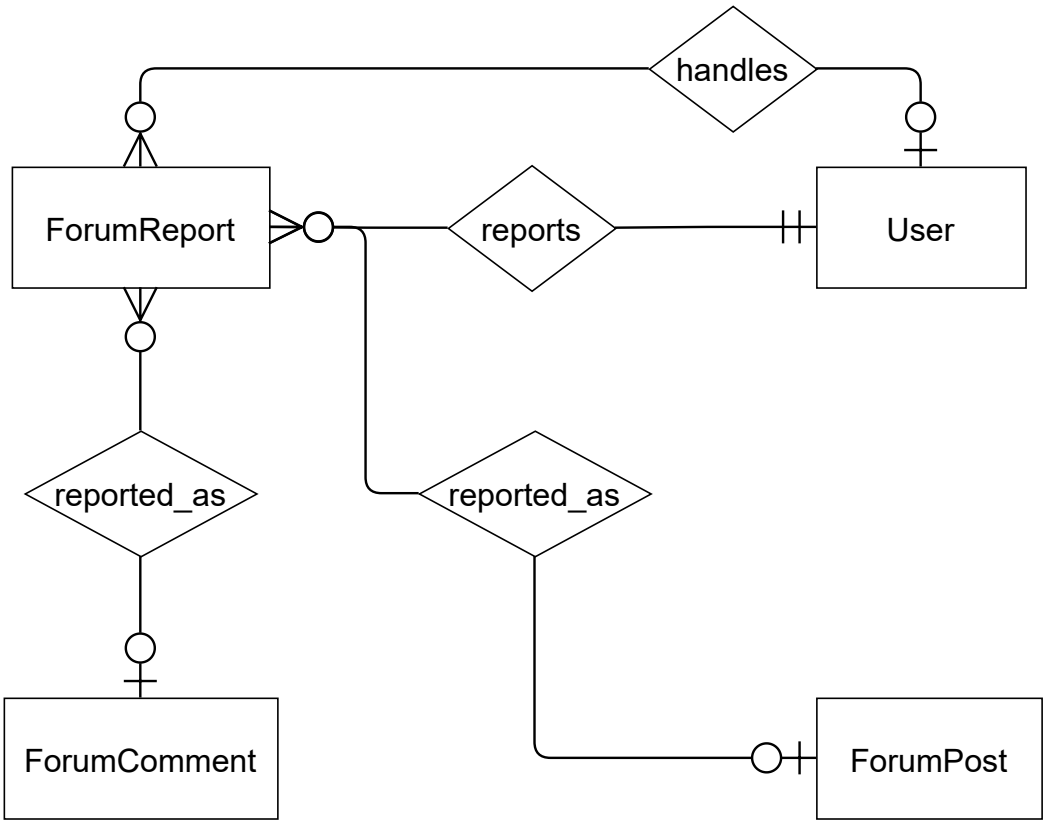


Figure 33: Forum Reports & Moderation ERD

4.1.5.22 Messaging & Notifications

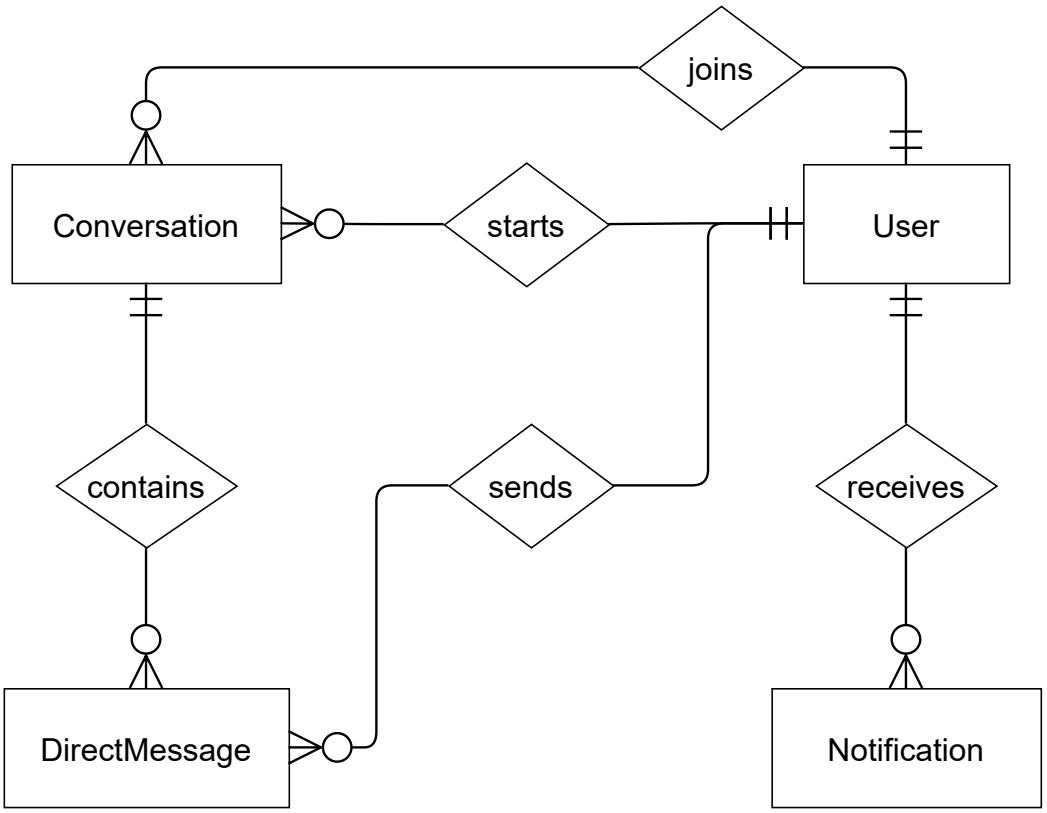


Figure 34: Messaging & Notifications ERD

4.1.5.23 Administration & System

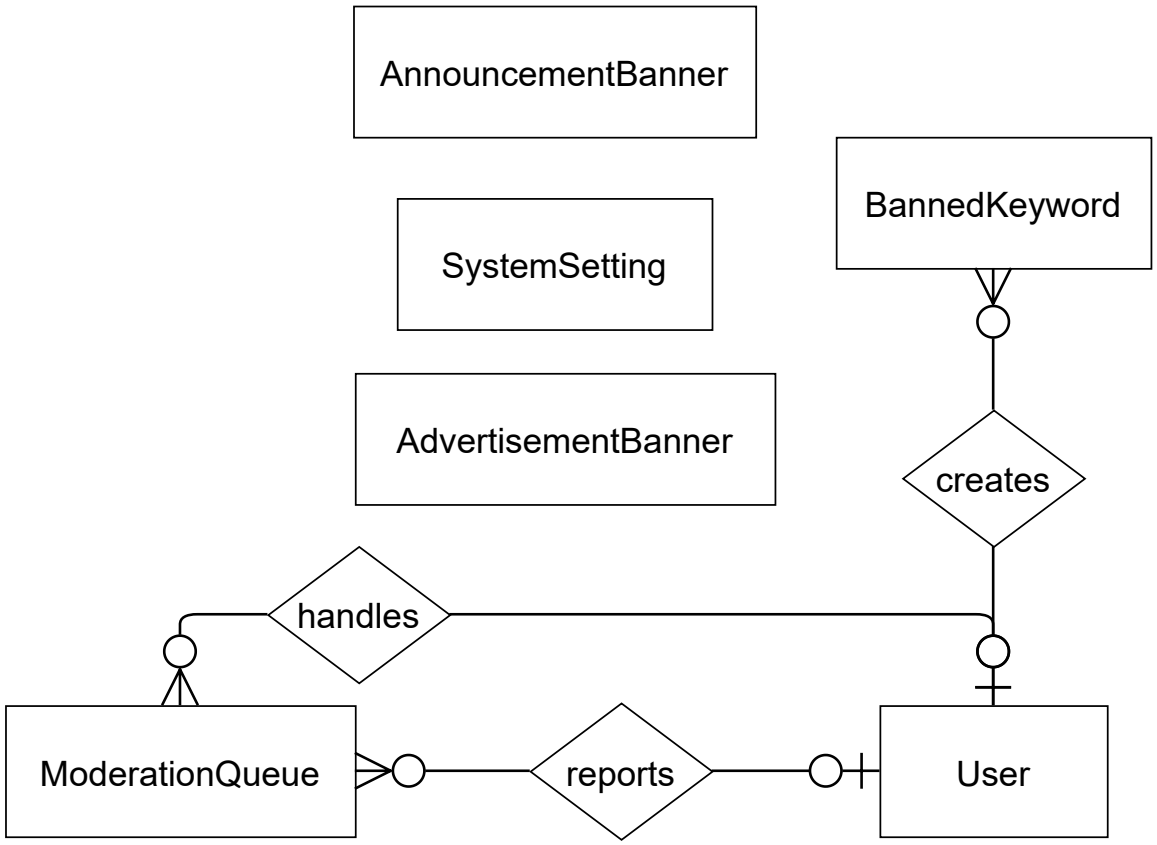


Figure 35: Administration & System ERD

4.2 Manage Forum

4.3 Manage Conversation

4.3.1 Start Conversation

Function Trigger: A logged-in user starts a new chat, either by clicking the “Connect with client” button on the job detail page in Find Work, or by clicking the “New message” button in the conversation list.

Function description: This function allows a logged-in user to start a direct conversation with another user. When the chat is started from a job, the other participant is the client who posted that job. When the chat is started from the conversation list, the user chooses the other participant. The conversation is created and the user can write the first message. If a conversation with that user already exists, the existing conversation is reopened instead of creating a duplicate.

Screen layout:

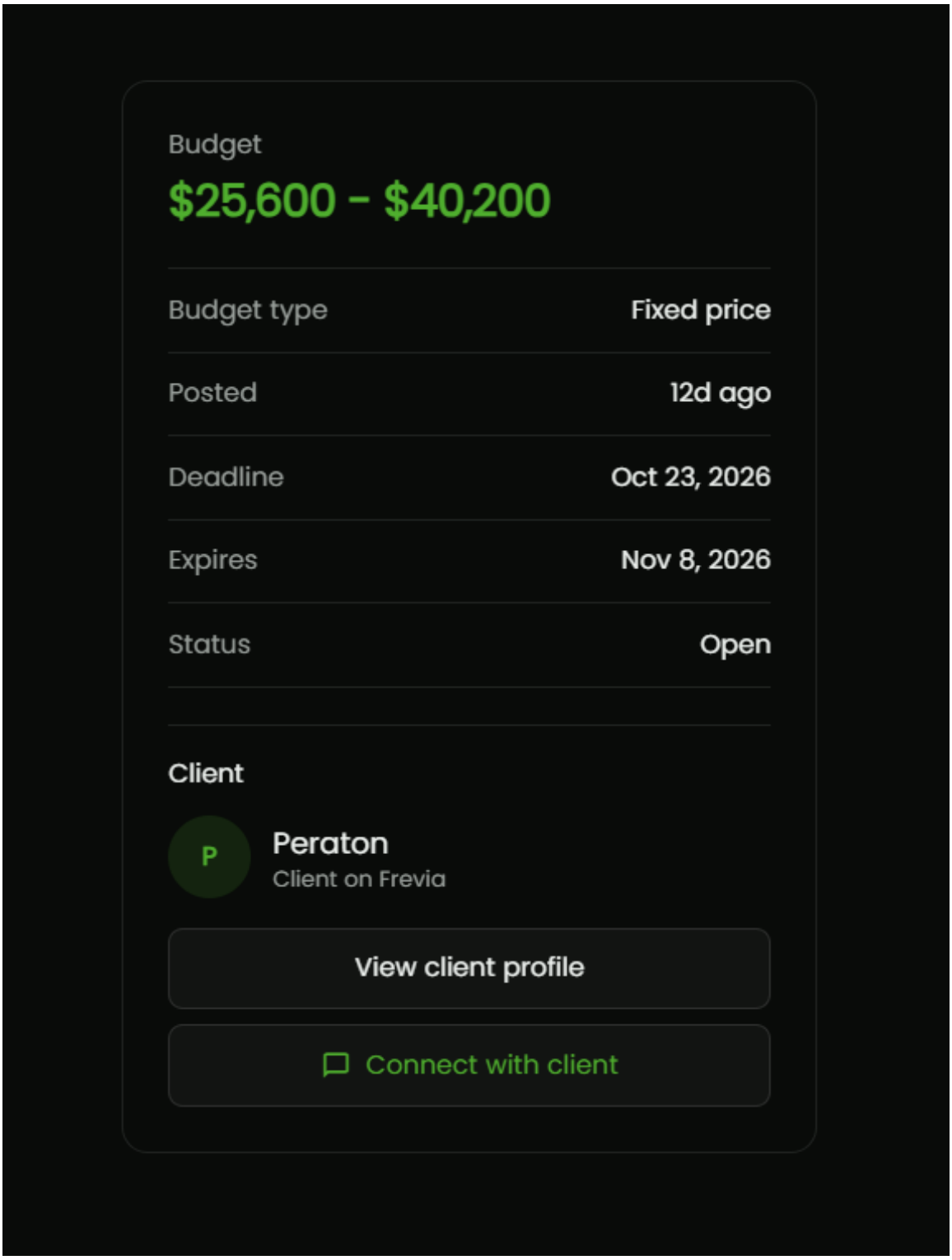


Figure 36: Start conversation screen

Function details:

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.

- The user selected another user who is different from themselves.

- **Process**
  - **Start a New Conversation**: The user starts the chat in one of two ways.
    - From a job in Find Work, the user clicks the "Connect with client" button on the job detail page.
    - From the conversation list, the user clicks the "New message" button.
  - **Choose the Participant**: The user provides or confirms the other participant.
    - When the chat is started from a job, the client of that job is already selected, so the user does not have to choose anyone.
    - When the chat is started from the conversation list, the user enters the identifier of the other participant in the dialog and confirms.
  - **Create or Reopen the Conversation**: The system checks whether a conversation between the two users already exists.
    - When no conversation exists, a new conversation between the two users is created.
    - When a conversation already exists, that conversation is reopened, including a conversation that the user had previously deleted from their own list.
  - **Write the First Message**: The compose screen shows the name and avatar of the other participant, and the user can type and send the first message (see **Send Message**).
- **Alternative Flow**
  - **Conversation Already Exists**: When a conversation with the chosen user already exists, the system opens the existing conversation instead of creating a second one, so there is only one thread per pair of users.
  - **Not Logged In**: When a guest clicks the "Connect with client" button, the system asks the user to log in before the conversation can be started.
  - **Cancel**: When the user leaves the compose screen or closes the dialog without confirming, no conversation is created and nothing changes.
- **Postconditions**
  - A conversation between the two users exists and is open on the screen.
  - The conversation is visible in the conversation list of both participants.
- **Error Handling**
  - **Cannot Chat with Yourself**: When the user chooses their own account as the participant, the conversation is not created and a message tells the user that a chat with themselves is not allowed.
  - **Participant Not Found**: When the chosen user cannot be found, the conversation is not created and a message tells the user that the participant cannot be found.

- **Cannot Start Chat**: When the conversation cannot be created, the screen shows a start-chat failure message so that the user can try again.
- **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.

- **Security Measures**
  - Only logged-in users can start a conversation, and the user is always recorded as one of the two participants.
  - When the chat is started from a job, the other participant is always the client of that job, so the button cannot be used to start a chat with an unrelated account.
  - Only one conversation can exist for a pair of users, so a user cannot create duplicated threads with the same person.

4.3.2 View Conversation List

Function Trigger: The logged-in user opens the Messages area by clicking the chat icon in the main navigation.

Function description: This function allows a logged-in user (Client, Freelancer) to see all of their conversations in one list. Each row shows the other participant, a preview of the last message, the time of that message and the number of unread messages. Conversations that the user has pinned are always shown at the top of the list.

Screen layout:

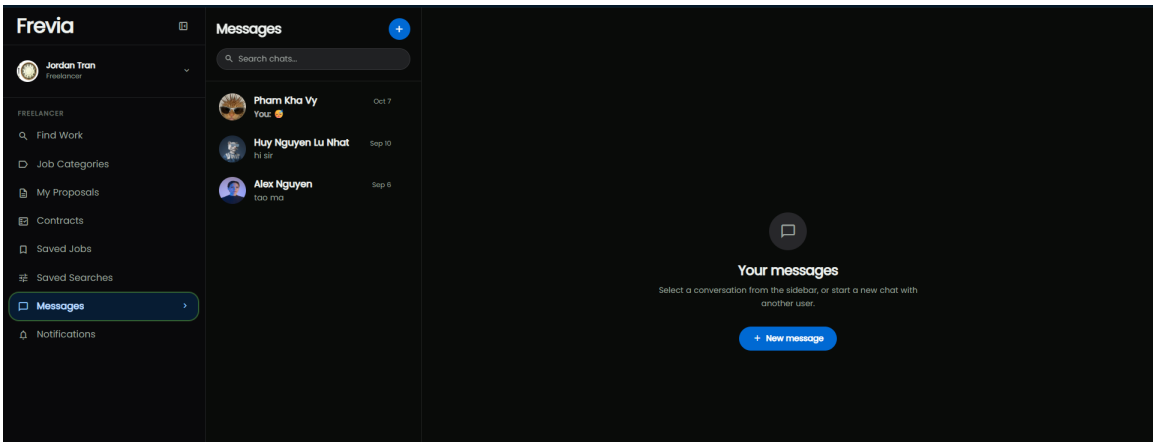


Figure 37: Conversation list screen

Function details:

- **Precondition**
  - The user is logged in.

- The user is connected to the internet.
- The user has at least one conversation, or the empty state is displayed instead.

- **Process**
  - **Open the Messages Area**: The user clicks the chat icon in the main navigation and the conversation list is displayed.
  - **Read Each Conversation Row**: Every row in the list presents the following information.
    - The avatar and the display name of the other participant.
    - A short preview of the last message, with a paperclip icon when the last message contains a file.
    - The relative time of the last message.
    - A blue unread dot on the avatar and an unread count badge when there are unread messages.
    - A pin icon when the conversation is pinned.
  - **Search for a Conversation**: The user types a keyword in the search box and the list shows only the conversations whose participant name or last message contains that keyword.
  - **Sorting**: Pinned conversations are shown first, and the remaining conversations are ordered by the time of the last message, newest first.
  - **Open a Conversation**: The user clicks a conversation row to open the chat thread (see **View Conversation Detail**).
- **Alternative Flow**
  - **No Conversation**: When the user has no conversation at all, the screen shows an empty state with a "New message" button, and the user can start a new chat from there (see **Create Conversation**).
  - **No Search Result**: When the search keyword does not match any conversation, the list is replaced by a message telling the user that no chat matches the keyword.
- **Postconditions**
  - The user sees all of their conversations, with the pinned ones at the top and the unread counts for each conversation.
  - Opening the list does not change any message; unread messages are only cleared when the user opens a thread (see **Mark Conversation as Read**).
- **Error Handling**
  - **Cannot Load Conversations**: When the conversation list cannot be loaded, the screen shows a load-failure message so that the user can try again.

- **Lost Connection**: When the connection to the server is lost, the screen shows a “Lost Connection” state and asks the user to reconnect.

- **Security Measures**
  - Only the conversations of the logged-in user are shown, so a user can never see the chats of another account.
  - Conversations that the user has deleted from their own list are not shown again until a new message arrives.

4.3.3 View Conversation Detail

Function Trigger: The user clicks a conversation row in the conversation list.

Function description: This function allows the user to read the whole chat thread with the other participant. The thread shows all messages in chronological order, the avatar and display name of the other user, the attachments that were exchanged and the read state of the messages that the user has sent.

Screen layout:

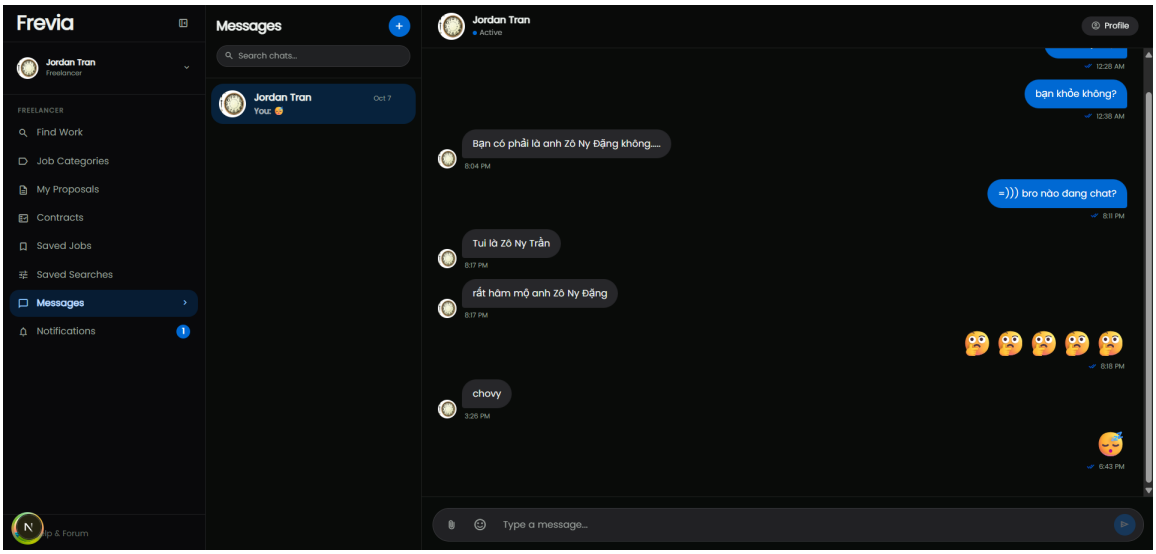


Figure 38: Conversation detail screen (chat thread)

Function details:

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The user is a participant of the conversation that is being opened.

- **Process**

- **Open the Conversation**: The user clicks a conversation row and the chat thread is displayed.
- **Read the Thread Header**: The header of the thread shows the following information.
  - The avatar and the display name of the other participant.
  - A button that opens the profile of the other participant.
  - A back button that returns to the conversation list.
- **Read the Messages**: The messages are shown in chronological order, oldest at the top and newest at the bottom.
  - Messages sent by the user are aligned on one side, and messages received from the other user on the other side.
  - Image attachments are shown as pictures that can be opened in a new tab.
  - Other files are shown as a download card with the file name and the file size.
  - A message that contains only emojis is displayed with a larger font.
- **Check the Read State**: Messages that the user has sent show a read indicator.
  - A single check mark means that the message has been delivered but not read.
  - A double check mark means that the other user has read the message.
- **Use the Thread Actions**: The user can send a message (see **Send Message**), attach a file (see **Send File**) or delete one of their own messages (see **Delete Message**).

- **Alternative Flow**
  - **Empty Thread**: When the conversation has no message yet, the thread shows an empty state with a hint to write the first message.
  - **User Is Typing**: When the other user is typing, an animated typing indicator is shown at the bottom of the thread.
- **Postconditions**
  - The user sees the whole chat thread with the other participant, including the attachments and the read state of their own messages.
  - The messages of the other user are marked as read while the thread is open (see **Mark Conversation as Read**).
- **Error Handling**
  - **Conversation Not Found**: When the conversation no longer exists, the screen shows a not-found state and offers the user a way back to the conversation list.
  - **Access Denied**: When the user is not a participant of the conversation, the thread is not opened and a permission message is shown.
  - **Cannot Load Messages**: When the messages cannot be loaded, the thread shows a load-failure message so that the user can try again.

- **Lost Connection**: When the connection to the server is lost, the screen shows a “Lost Connection” state and asks the user to reconnect.

- **Security Measures**
  - Only the two participants of a conversation can read its messages; another user is refused even when the conversation link is known.
  - Messages that have been deleted are never shown again in the thread.

4.3.4 Send Message

Function Trigger: The user types a message in the input box at the bottom of the chat thread and clicks the send button.

Function description: This function allows a logged-in user to send a text message to the other participant of the conversation. The user types the text and sends it, and the message appears in the thread of both users immediately.

Screen layout:

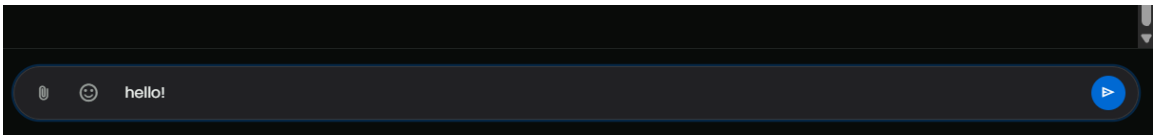


Figure 39: Sending a message in the chat thread

Function details:

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The user is a participant of the conversation and the chat thread is open.

- **Process**
  - **Type the Message**: The user types the text in the input box at the bottom of the thread. The send button stays disabled while the input box is empty.
  - **Add an Emoji if Wanted**: The user can click the emoji button to pick an emoji, and the chosen emoji is added to the input box.
  - **Send the Message**: The user clicks the send button or presses the Enter key.
  - **View the Result**: When the message is sent, the screen shows the result.
    - The message appears at the bottom of the thread on the user’s side.
    - The input box is cleared.

        - The message shows a single check mark, meaning that it has been sent but not read yet.
    - **Message Arrives for the Other User**: The other participant receives the message in real time, and the conversation list of that user shows the new message preview and the unread count.

- **Alternative Flow**
    - **Enter Key**: When the user presses the Enter key without holding Shift, the message is sent in the same way as clicking the send button.
    - **Empty Message**: When the input box is empty, the send button remains disabled and the user cannot send an empty message.
    - **Emoji Only**: When the message contains only emojis, the message is displayed with a larger font in the thread.
- **Postconditions**
    - The new message is stored in the conversation and is shown to both participants.
    - The conversation list of both users is updated with the new last message.
- **Error Handling**
    - **Conversation Not Found**: When the conversation no longer exists, the message is not sent and the thread shows a message telling the user that the conversation cannot be found.
    - **Access Denied**: When the user is not a participant of the conversation, the message is refused and a permission message is shown.
    - **Cannot Send Message**: When the message cannot be sent, the thread shows a send-failure message and the text is kept so that the user can try again.
    - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.
- **Security Measures**
    - Only a participant of the conversation can send a message into it, and the sender is always the account that is currently logged in.
    - The length of a message is limited, so very large text cannot be sent.

4.3.5 Send File

Function Trigger: The user clicks the attach-file button in the input box of the chat thread and chooses a file from their device.

Function description: This function allows a logged-in user to send a file, such as a

picture or a document, to the other participant of the conversation. The user chooses a file, the system uploads it and the file is sent as a message with a preview or a download card.

Screen layout:

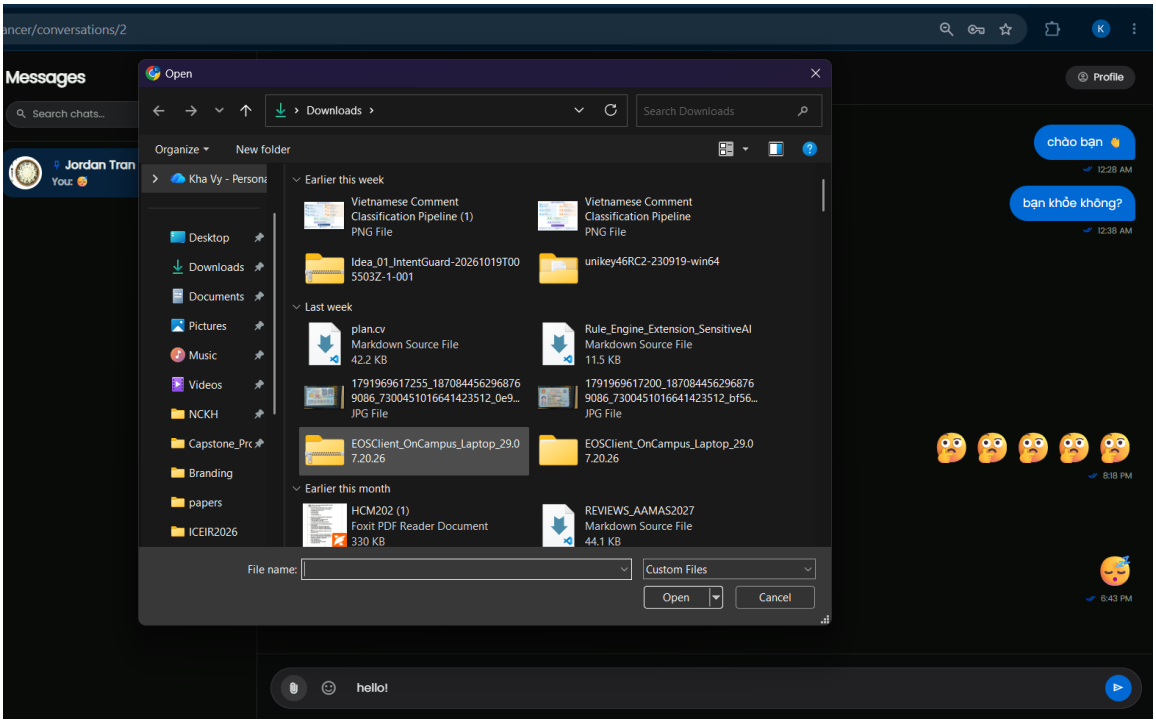


Figure 40: Sending a file in the chat thread

Function details:

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The user is a participant of the conversation and the chat thread is open.
- **Process**
  - **Click the Attach Button**: The user clicks the paperclip button in the input box, and the file chooser of the device is opened.
  - **Choose the File**: The user selects a file. The file chooser accepts images, documents, archives and other supported file types.
  - **Upload the File**: The system uploads the file before sending it, and a loading state is shown while the upload is running.
  - **Send the File as a Message**: When the upload finishes, the file is sent to the conversation as a message.
    - When the file is an image, it is displayed as a picture that can be opened in a new tab.

        - When the file is not an image, it is displayed as a download card with the file name and the file size.
    - **View the Result**: The file message appears at the bottom of the thread, and the conversation list of both users shows the file preview with a paperclip icon.
- **Alternative Flow**
    - **Cancel the File Chooser**: When the user closes the file chooser without choosing a file, no upload is started and nothing changes.
    - **Image Preview**: When the chosen file is an image, the picture is rendered directly inside the message bubble instead of a download card.
- **Postconditions**
    - The file is uploaded and attached to a new message in the conversation.
    - Both participants can see the file message in the thread and can open or download the file.
- **Error Handling**
    - **File Too Large**: When the chosen file is larger than the allowed size, the upload is stopped and a message tells the user that the file is too large.
    - **File Type Not Allowed**: When the chosen file type is not supported, the upload is stopped and a message tells the user that this file type is not allowed.
    - **Cannot Upload File**: When the upload fails, the file message is not sent and a failure message is shown so that the user can try again.
    - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.
- **Security Measures**
    - Only supported file types can be uploaded, and the size of the file is limited before the upload starts.
    - Only a participant of the conversation can send a file into it, and the sender is always the account that is currently logged in.

4.3.6 Mark Conversation as Read

Function Trigger: The user opens a conversation from the conversation list, or a new message arrives while the thread is already open.

Function description: This function allows a logged-in user to clear the unread messages of a conversation. The unread badge disappears from the conversation list and the double check mark appears on the messages of the other user in the thread, so the other participant can see that the messages have been read.

Screen layout:

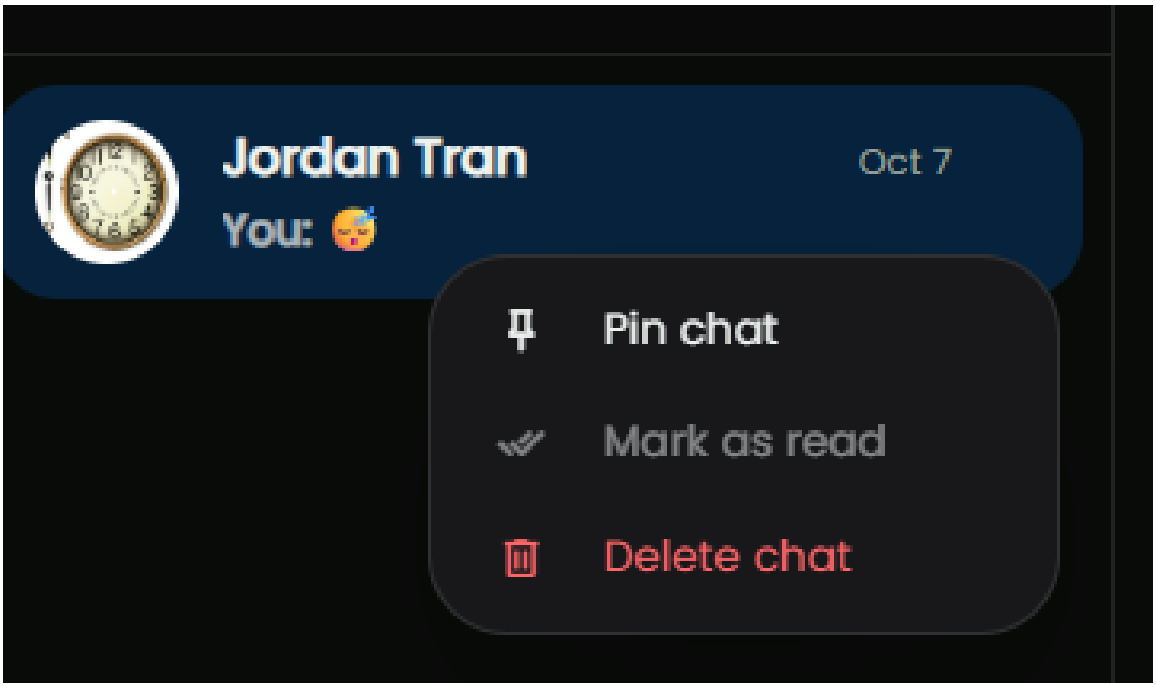


Figure 41: Conversation after being marked as read

Function details:

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The conversation contains at least one unread message from the other participant.
- **Process**
  - **Open the Conversation**: The user opens the conversation, and the system automatically marks the received messages as read.
  - **View the Result in the List**: The conversation row in the conversation list is updated.
    - The blue unread dot disappears from the avatar.
    - The unread count badge is removed from the row.
  - **Mark Manually if Wanted**: The user can also choose the "Mark as read" action from the "..." menu of a conversation row.
  - **Notify the Sender**: The other participant sees the read state change in real time.
    - Messages that were shown with a single check mark are updated to a double check mark.
- **Alternative Flow**

- **Nothing to Read**: When the conversation has no unread message, the “Mark as read” action is disabled in the menu.
- **New Message While Reading**: When a new message arrives while the thread is open, that message is marked as read immediately.

- **Postconditions**
  - All messages received from the other participant in this conversation are marked as read.
  - The unread count of the conversation is cleared for the current user, and the other participant sees the double check mark on their sent messages.
- **Error Handling**
  - **Conversation Not Found**: When the conversation no longer exists, the read state is not updated and a message tells the user that the conversation cannot be found.
  - **Cannot Mark as Read**: When the read state cannot be updated, the unread badge stays on the row and a failure message is shown so that the user can try again.
  - **Lost Connection**: When the connection to the server is lost, the screen shows a “Lost Connection” state and asks the user to reconnect.
- **Security Measures**
  - Only the messages received by the current user are marked as read, so a user cannot change the read state of their own sent messages through this action.
  - Only a participant of the conversation can mark it as read.

4.3.7 Pin Conversation

Function Trigger: The user opens the “...” menu of a conversation row and chooses “Pin chat”.

Function description: This function allows a logged-in user to keep an important conversation at the top of the conversation list. The user pins a conversation and the row is moved to the pinned group, which is always shown above the other conversations.

Screen layout:

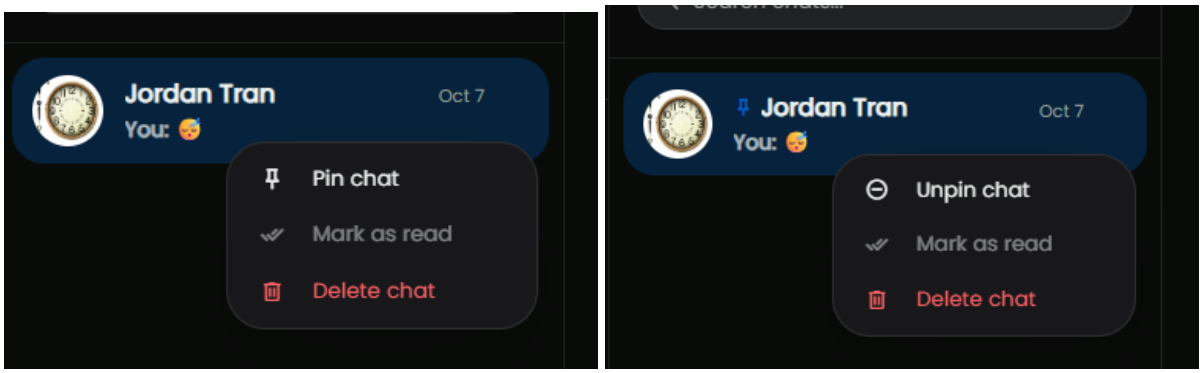


Figure 42: Conversation list before pinning (left) and after pinning (right)

Function details:

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The conversation that the user wants to pin is shown in the conversation list.
- **Process**
  - **Open the Row Menu**: The user clicks the "..." button of the conversation row and the menu is displayed.
  - **Choose Pin Chat**: The user clicks the "Pin chat" action in the menu.
  - **View the Result**: The conversation list is updated.
    - The conversation is moved to the top of the list, above the not-pinned conversations.
    - A pin icon is shown on the conversation row.
  - **Unpin if Wanted**: The user can open the same menu again and choose "Unpin chat" to remove the conversation from the pinned group.
- **Alternative Flow**
  - **Unpin Conversation**: When the user chooses "Unpin chat", the pin icon is removed and the conversation returns to the normal order of the list.
  - **Pin State Is Personal**: The pin of one user does not change the order of the conversation list of the other participant.
- **Postconditions**
  - The pinned conversation stays at the top of the conversation list of the current user.
  - The content of the conversation and the messages are not changed by this action.
- **Error Handling**

- **Conversation Not Found**: When the conversation no longer exists, the pin is not applied and a message tells the user that the conversation cannot be found.
- **Cannot Pin Conversation**: When the pin state cannot be saved, the row returns to its previous position and a failure message is shown so that the user can try again.
- **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.

- **Security Measures**
  - The pin belongs to the current user only, so pinning a chat never affects the conversation list of the other participant.
  - Only a participant of the conversation can pin it.

4.3.8 Delete Message

Function Trigger: The user moves the pointer over one of their own messages in the chat thread and clicks the delete button.

Function description: This function allows a logged-in user to remove a message that they have sent. The user deletes their own message and the message disappears from the chat thread of both participants. The user can only delete messages that they sent themselves.

Screen layout:

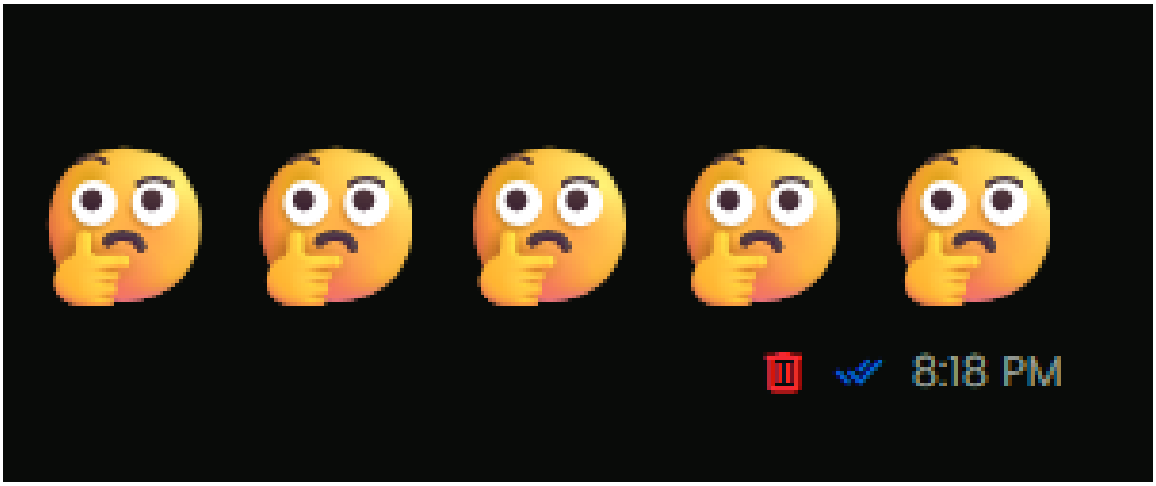


Figure 43: Deleting a message in the chat thread

Function details:

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.

- The message belongs to the current user and is shown in the chat thread.

- **Process**
  - **Point at the Message**: The user moves the pointer over one of their own messages and the delete button appears.
  - **Click the Delete Button**: The user clicks the delete button of that message.
  - **View the Result**: The message is removed from the thread.
    - The message disappears from the thread of the current user.
    - The message also disappears from the thread of the other participant in real time.
  - **Conversation List Update**: When the deleted message was the last message of the conversation, the preview in the conversation list is refreshed to show the previous message.

- **Alternative Flow**
  - **Delete a File Message**: When the deleted message contains a file, the whole file message is removed from the thread in the same way.
  - **Message Already Deleted**: When the message has already been deleted, the delete button is not available for it.

- **Postconditions**
  - The message no longer appears in the chat thread of either participant.
  - The message is kept in the system for record-keeping but is never shown to users again.

- **Error Handling**
  - **Message Not Found**: When the message no longer exists or has already been deleted, the page shows a message telling the user that the message cannot be found.
  - **Not the Sender**: When the user tries to delete a message that they did not send, the action is refused and a permission message is shown.
  - **Cannot Delete Message**: When the message cannot be deleted, the message stays in the thread and a delete-failure message is shown so that the user can try again.
  - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.

- **Security Measures**
  - The delete button is shown only on the messages that the current user sent.
  - The system also checks the sender when the deletion is confirmed, so a user can never delete a message that belongs to the other participant.

4.4 Moderation

4.4.1 View Moderation Queue

Function Trigger: The logged-in administrator opens the Moderation screen by choosing **Forum** → **Moderation** in the admin sidebar.

Function description: This function allows an administrator to see all forum posts that are waiting to be reviewed. Every row shows the identifier of the post, its title, the automatic moderation score, the categories that were detected, the author and the creation date. The list only contains posts that are still pending, so the administrator always works on the items that need a decision.

Screen layout:

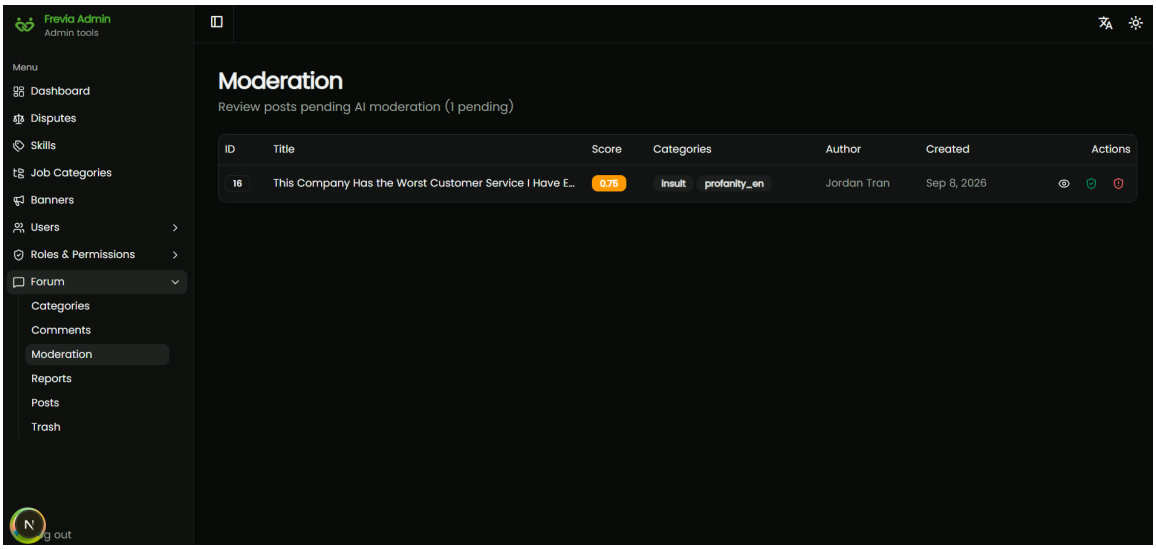


Figure 44: Moderation queue screen

Function details:

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The user is on the admin area and is allowed to open the Moderation screen.
- **Process**
  - **Open the Moderation Screen**: The user selects **Forum** in the sidebar and then **Moderation**, and the screen titled “Moderation” is displayed.
  - **Read the Summary Line**: The subtitle shows the total number of posts that are waiting, in the form “Review posts pending AI moderation (n pending)”.

- **Read Each Row**: Every row of the table presents the following information.
  - **ID**: The identifier of the post, shown as a small badge.
  - **Title**: The title of the post, shortened when it is too long.
  - **Score**: The automatic moderation score of the post, shown with two decimals, or "N/A" when no score was computed.
  - **Categories**: The categories detected by the automatic check, shown as badges, or "—" when none were detected.
  - **Author**: The display name of the user who wrote the post, or "User #id" when the user has no display name.
  - **Created**: The date on which the post was created.
- **Read the Score Colour**: The score badge uses a colour that reflects the risk level.
  - A score of 0.80 or more is shown in red, a score from 0.30 to 0.79 is shown in amber and a score below 0.30 is shown in green.
- **Move Between Pages**: When there is more than one page, the user uses the page numbers or the **Previous** and **Next** buttons at the bottom of the list, and the counter shows "Page x of y (n total)".
- **Choose an Item**: The user opens, approves or rejects a post with the action buttons at the end of the row (see **View Moderation Item Detail** and **Resolve Moderation Item**).

- **Alternative Flow**
  - **No Pending Post**: When no post is waiting, the table shows the message "No posts awaiting moderation." instead of rows.
  - **Only One Page**: When all posts fit on a single page, the page numbers are hidden and only the table is shown.
- **Postconditions**
  - The administrator sees the complete list of posts that are waiting for a decision.
  - No data is changed by this function, so the queue stays exactly as it was.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator opens the screen, the user is sent to the access-denied page, because only administrators may read the moderation queue.
  - **Lost Connection**: When the queue cannot be loaded, the screen shows an empty list so that the administrator can reload the page and try again.
- **Security Measures**
  - The whole admin area is protected: a guest or a banned account is sent to the login page, and a logged-in user without the administrator role is sent to the access-denied

page.

- The moderation queue is only returned to administrator accounts, so ordinary users can never see posts that are waiting for review, no matter how they call the screen.

4.4.2 View Moderation Item Detail

x

Function Trigger: The logged-in administrator clicks the view button (the eye icon) at the end of a row in the moderation queue.

Function description: This function allows an administrator to read a pending post in full before making a decision. The dialog shows the title, the author, the creation date, the moderation score and the complete content of the post, together with the **Approve** and **Reject** buttons so that the decision can be taken from the same screen.

Screen layout:

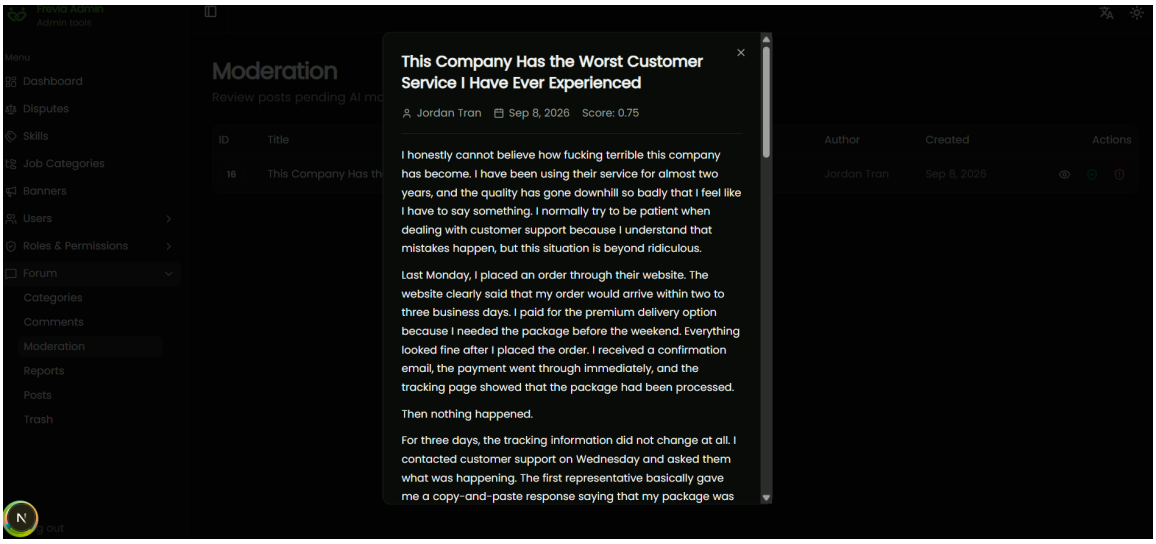


Figure 45: Moderation item detail dialog

Function details:

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The user is on the Moderation screen and there is at least one pending post.
- **Process**
  - **Open the Detail**: The user clicks the view button on a row, and a dialog opens with the title of the post as its heading.

- **Read the Item Summary**: Under the title, the dialog shows the author of the post, the creation date and the moderation score (with "N/A" when no score exists).
- **Read the Content**: The dialog shows the whole content of the post with its formatting, exactly as it would appear to readers.
- **Scroll When Needed**: When the post is long, the dialog becomes scrollable so that the whole content can be read without leaving the screen.
- **Decide**: The user gives the decision with the **Approve** or **Reject** button at the bottom of the dialog (see **Resolve Moderation Item**).
- **Close the Detail**: The user closes the dialog with the close button or by clicking outside it, and returns to the queue.

- **Alternative Flow**
  - **No Score**: When the automatic check produced no score, the badge shows "N/A" and the rest of the dialog is unchanged.
  - **Close Without Deciding**: When the user closes the dialog without pressing a button, no decision is recorded and the post stays in the queue.
- **Postconditions**
  - The administrator has read the complete post and knows its author, date, score and content.
  - Nothing has changed while only viewing the item.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to open the detail, the action is refused because only administrators may review pending posts.
  - **Item Already Handled**: When the post has already been approved or rejected by another administrator, the decision fails and a message tells the user that the post cannot be found or updated any more.
  - **Lost Connection**: When the connection to the server is lost, the decision cannot be saved and the screen asks the user to try again.
- **Security Measures**
  - The detail is only available inside the admin area, so a pending post can never be read by an ordinary user.
  - Every decision is recorded together with the administrator who made it and the moment it was made, so each review can be traced.

4.4.3 Resolve Moderation Item

Function Trigger: The logged-in administrator clicks the approve button or the reject button on a row of the moderation queue, or presses **Approve** or **Reject** inside the detail dialog.

Function description: This function allows an administrator to close a pending review by approving or rejecting a post. An approved post becomes visible to everyone in the forum. A rejected post stays hidden from the public and is moved to the trash, where it can be restored later. Before a rejection is applied, the administrator is asked to confirm it.

Screen layout:

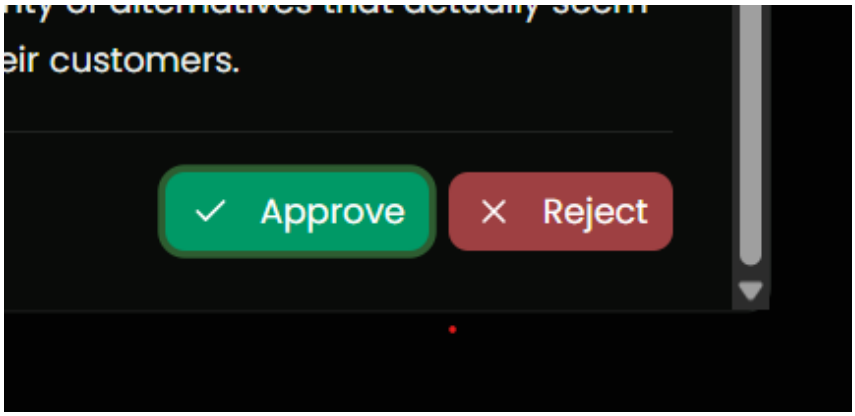


Figure 46: Approve action on a moderation item

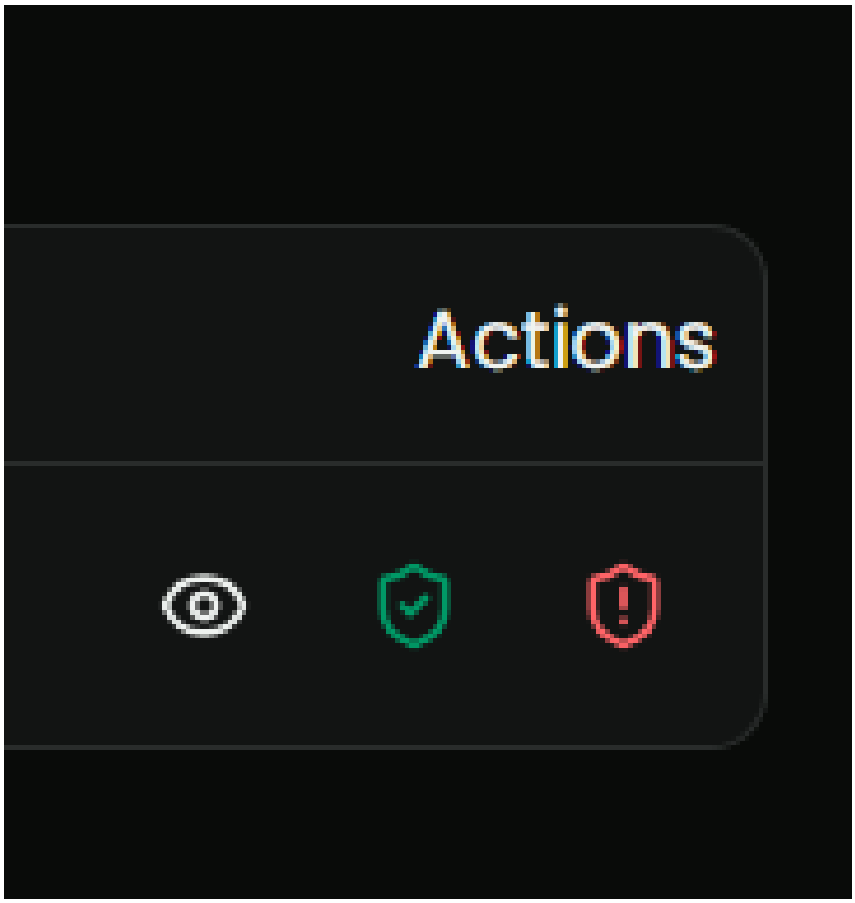


Figure 47: Reject confirmation dialog

Function details:

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The post that is being resolved is still waiting in the moderation queue.
- **Process**
  - **Approve a Post**: The user clicks the approve button on a row, and the post is accepted straight away.
    - The post leaves the moderation queue and becomes visible to everyone in the forum.
    - The screen shows the message “Post approved and now visible publicly”.
  - **Start a Rejection**: The user clicks the reject button on a row, or presses **Reject** inside the detail dialog.
  - **Confirm the Rejection**: When the reject button of a row is used, a confirmation dialog titled “Reject Post” appears with the question “Reject “title”? The post will not be visible publicly and will be moved to trash.”.

        - The user presses **Reject** to confirm, or **Cancel** to keep the post in the queue.
    - **Apply the Rejection**: When the rejection is confirmed, the post is refused and is moved to the trash.
        - The post is never shown to ordinary users while it stays rejected.
        - The screen shows the message "Post rejected and moved to trash".
    - **Update the Queue**: The resolved post disappears from the list and the remaining items move up, so the queue always shows only the posts that still need a decision.
- **Alternative Flow**
    - **Cancel the Rejection**: When the user closes the confirmation dialog, the post stays pending and nothing changes.
    - **Reject from the Detail Dialog**: When **Reject** is pressed inside the detail dialog, the rejection is applied immediately without an extra confirmation dialog.
    - **Empty Queue**: When the last pending post is resolved, the table shows the message "No posts awaiting moderation.".
- **Postconditions**
    - The post is no longer pending and has been marked as approved or rejected.
    - An approved post is visible in the forum, while a rejected post is hidden and can be found in the trash.
    - The decision is recorded with the administrator who made it and the moment it was made.
- **Error Handling**
    - **Not Allowed**: When a user who is not an administrator tries to approve or reject a post, the action is refused because only administrators may resolve moderation items.
    - **Item Not Found**: When the post has already been resolved by another administrator, the action fails and the screen shows the message "Couldn't approve post. Try again." or "Couldn't reject post. Try again.".
    - **Cannot Save the Decision**: When the decision cannot be saved, the post stays in the queue and a message asks the administrator to try again.
    - **Lost Connection**: When the connection to the server is lost, the decision is not applied and the screen asks the user to try again.
- **Security Measures**
    - Only administrators may approve or reject a post, and the check is made again on the server for every decision.
    - The post can only be resolved while it is still pending, so a decision cannot be applied twice to the same post.
    - A rejected post is hidden from the public and kept in the trash instead of being erased,

so the decision can be reviewed and undone later.

4.5 ID Verification

4.5.1 View Verification Request List

Function Trigger: The logged-in administrator opens the ID Verification screen by choosing **Users** → **Identity Verification** in the admin sidebar.

Function description: This function allows an administrator to see all the identity verification requests that users have submitted. Every row shows the identifier, the user, the type of document, the status and the submission date. The administrator can filter the requests by status and search by email or display name.

Screen layout:

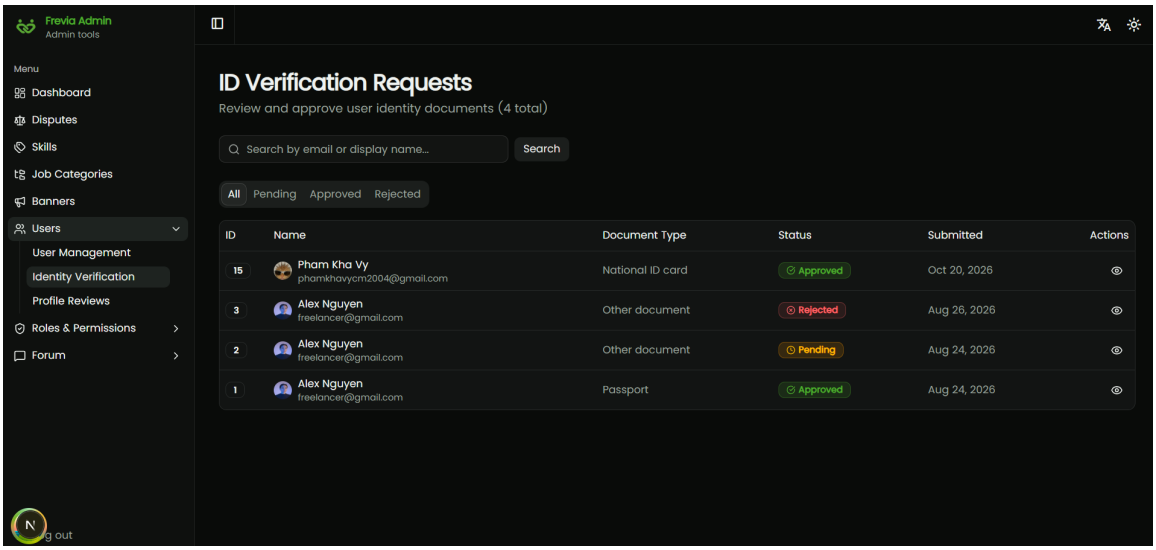


Figure 48: ID verification request list screen

Function details:

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - At least one user has already submitted an identity document, otherwise the list is empty.
- **Process**
  - **Open the ID Verification Screen**: The user selects **Users** in the sidebar and then **Identity Verification**, and the screen titled “ID Verification Requests” is displayed.

- **Read the Summary Line**: The subtitle shows the total number of requests in the form “Review and approve user identity documents (n total)”.
- **Read Each Row**: Every row of the table presents the following information.
  - **ID**: The identifier of the request, shown as a small badge.
  - **Name**: The avatar, the display name and the email of the user who submitted the document.
  - **Document Type**: The kind of document, shown as National ID card, Passport, Driver license, Residence permit or Other document.
  - **Status**: The current status, shown as a badge with the labels Pending, Approved or Rejected.
  - **Submitted**: The date on which the document was submitted.
- **Read the Status Colour**: A pending request is shown in amber, an approved request in green and a rejected request in red.
- **Filter by Status**: The user chooses one of the tabs **All**, **Pending**, **Approved** or **Rejected** to show only the requests with that status.
- **Search**: The user types a text in the search box with the placeholder “Search by email or display name...” and submits it, and only the matching requests are shown.
- **Move Between Pages**: When there is more than one page, the user uses the page numbers or the **Previous** and **Next** buttons at the bottom of the list.
- **Choose a Request**: The user opens the request with the view button (see **View ID Verification Request Detail**).

- **Alternative Flow**
  - **No Request**: When no request matches the current filter or search, the table shows the message “No verification requests found.”.
  - **Clear the Search**: When the user clears the search box, all requests are listed again.
- **Postconditions**
  - The administrator sees the requests that match the chosen status and search text.
  - No data is changed by this function, so the requests stay exactly as they were.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator opens the screen, the user is sent to the access-denied page, because only administrators may read identity documents.
  - **Lost Connection**: When the requests cannot be loaded, the screen shows an empty list so that the administrator can reload the page and try again.
- **Security Measures**

- The list of requests is only returned to administrator accounts, so the identity data of a user is never exposed to other users.
- Only the basic information is listed, while the images of the document are shown only inside the detail of a request.

4.5.2 View ID Verification Request Detail

Function Trigger: The logged-in administrator clicks the view button (the eye icon) at the end of a row in the ID verification request list.

Function description: This function allows an administrator to examine an identity verification request in full. The dialog shows the user, the submission date, the type of document, the front and back images of the document, the review notes if a decision has already been made, and the buttons that let the administrator approve or reject a pending request.

Screen layout:

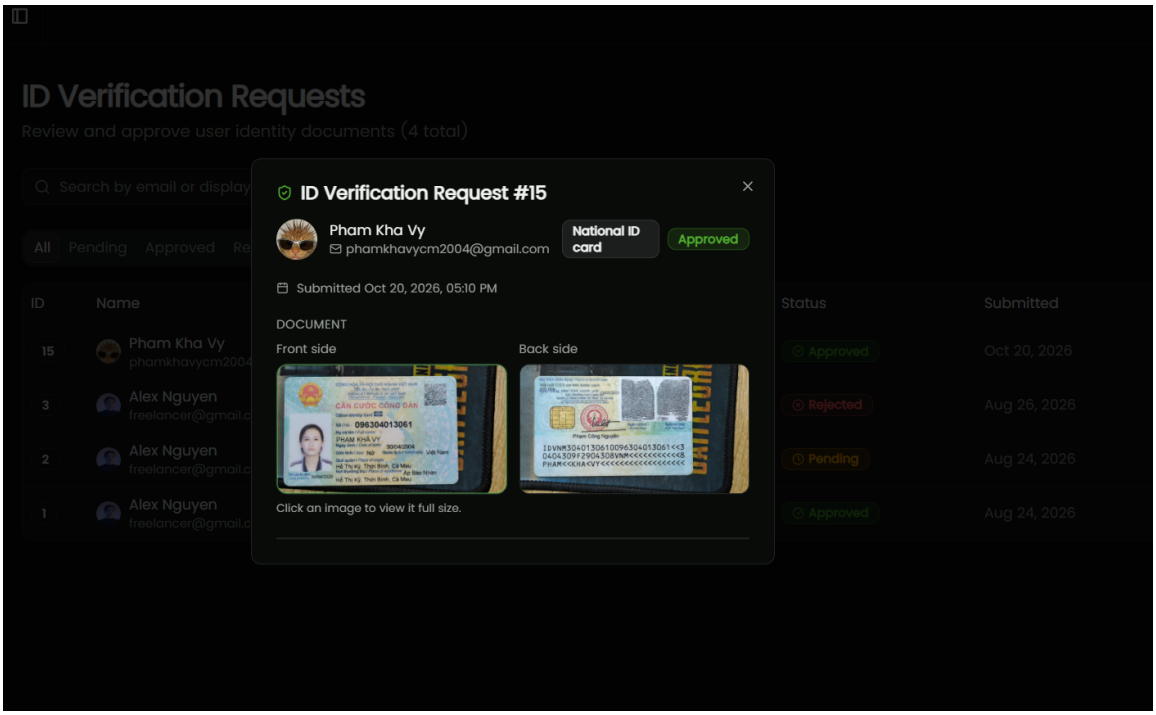


Figure 49: ID verification request detail dialog

Function details:

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The user is on the ID Verification screen and the request exists.

- **Process**
  - **Open the Detail**: The user clicks the view button on a row, and a dialog titled “ID Verification Request #id” opens.
  - **Read the Header**: The header shows the avatar, the display name and the email of the user, together with the badge of the document type and the badge of the status.
  - **Read the Submission Date**: The line “Submitted …” shows the moment at which the document was sent, including the time.
  - **Read the Document**: The section **Document** shows the front side of the document and, when it was uploaded, the back side as well.
    - When the back side was not submitted, a dashed placeholder shows the message “No back side uploaded with this submission.”.
  - **Zoom an Image**: The user clicks an image to open it in full size in a larger dialog, and closes the larger dialog to come back to the request.
  - **Read the Review Notes**: When a decision has already been made, the section **Review Notes** shows the note that was left by the administrator who handled the request.
  - **Decide on a Pending Request**: When the request is pending, the dialog also shows a review notes box and the **Approve** and **Reject** buttons (see **Approve or Reject ID Verification**).
  - **Close the Detail**: The user closes the dialog and returns to the list, where the row reflects the current status.
- **Alternative Flow**
  - **Already Decided**: When the request has already been approved or rejected, the dialog only shows the images, the status and the review notes, and no decision button appears.
  - **No Back Side**: When only the front side exists, the dialog shows the placeholder instead of the second image and the rest is unchanged.
  - **Close Without Deciding**: When the user closes the dialog without pressing a button, no decision is recorded and the request stays pending.
- **Postconditions**
  - The administrator has seen the user, the document, the date and any previous review notes.
  - Nothing has changed while only viewing the request.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to open the detail, the action is refused because only administrators may read identity documents.
  - **Request Not Found**: When the request no longer exists, the detail cannot be opened and a message tells the user that the request cannot be found.

- **Document Not Available**: When an image of the document can no longer be read, the image is not shown and a message tells the user that the document is not available.
- **Lost Connection**: When the detail cannot be loaded, the screen asks the user to try again.

- **Security Measures**
  - The images of the document are delivered only to an administrator account, so the document of a user is never exposed to anyone else.
  - The document images are opened inside the admin area as read-only content, so they cannot be saved or edited from the screen.
  - The decision buttons are hidden as soon as the request has been handled, so a request can be reviewed only while it is pending.

4.5.3 Approve or Reject ID Verification

Function Trigger: The logged-in administrator presses the **Approve** or **Reject** button inside the detail of a pending ID verification request.

Function description: This function allows an administrator to close an identity verification request by approving or rejecting the document that the user submitted. The administrator can write an optional review note that is stored with the decision. Approving a request marks the user as verified, while rejecting a request leaves the user unverified.

Screen layout:

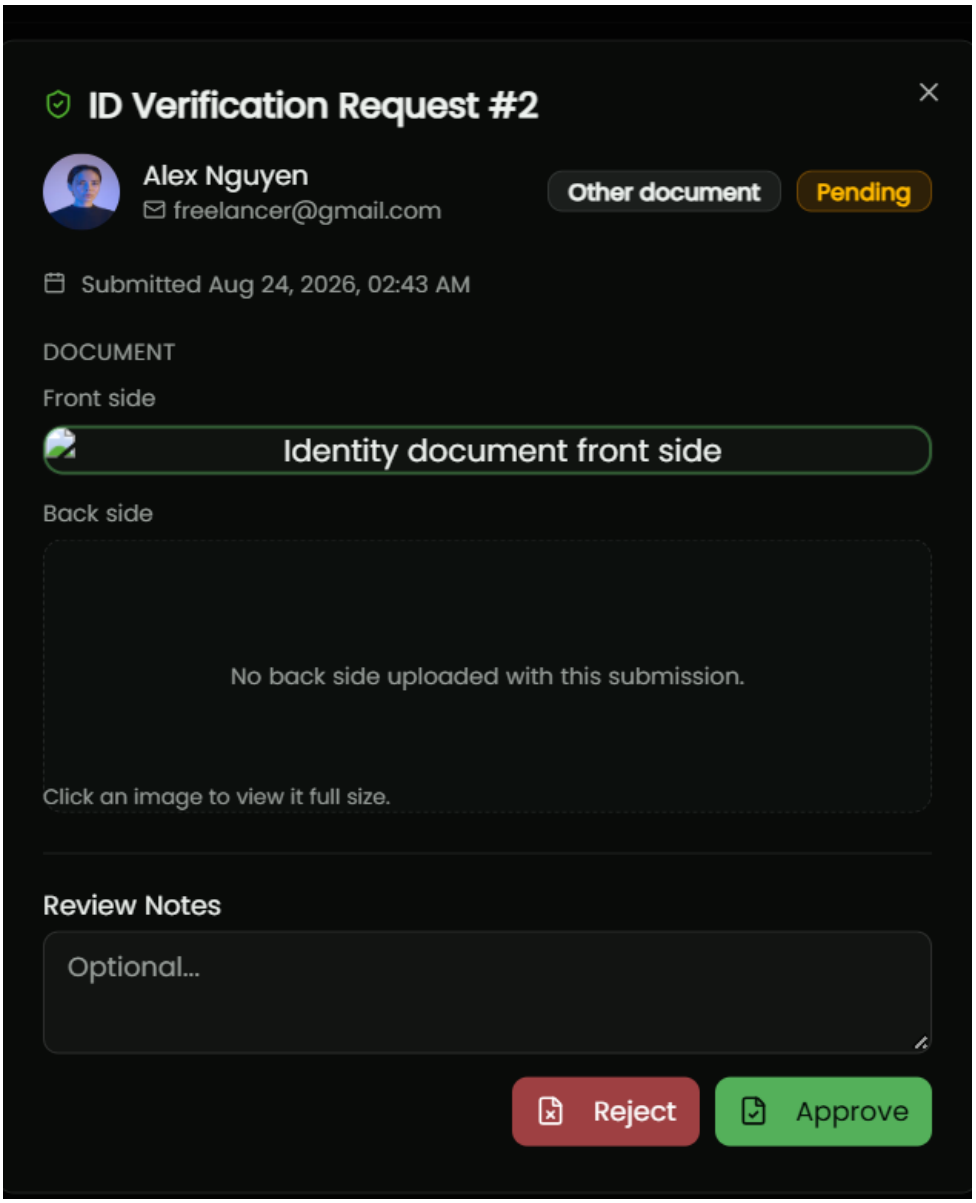


Figure 50: Approving or rejecting an ID verification request

Function details:

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The request is pending and its document has been examined in the detail dialog.
- **Process**
  - **Write a Review Note**: The user may type a note in the box labeled “Review Notes”, which is optional.
    - The note is stored with the decision and is shown again the next time the request is opened.

- **Approve the Request**: The user presses **Approve**, and the button shows “Approving…” while the decision is being saved.
  - The request is marked as approved, the user is marked as verified and the screen shows the message “Identity verification approved”.
- **Reject the Request**: The user presses **Reject**, and the button shows “Rejecting…” while the decision is being saved.
  - The request is marked as rejected, the user stays unverified and the screen shows the message “Identity verification rejected”.
- **See the Result**: When the decision is saved, the status badge of the request changes and the decision buttons disappear, so the request is now read-only.

- **Alternative Flow**
  - **No Review Note**: When the user leaves the box empty, the decision is saved without a note.
  - **Note Too Long**: When the note is longer than the allowed length, the note is refused and the administrator is asked to shorten it before the decision can be saved.
  - **Reopen the Detail Later**: When the request is opened again, it shows the chosen status, the review note and no decision buttons.
- **Postconditions**
  - The request has the status that the administrator chose, and it is no longer pending.
  - An approved user is marked as verified, while a rejected user stays unverified.
  - The decision, the note and the moment of the decision are stored with the request.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to approve or reject a request, the action is refused because only administrators may decide on identity documents.
  - **Request Not Found**: When the request no longer exists, the decision is refused and the screen shows the message “Couldn’t update the request. Try again.”.
  - **Already Reviewed**: When another administrator has already handled the request, the decision is refused and a message tells the user that the request has already been reviewed.
  - **Lost Connection**: When the connection to the server is lost, the decision is not saved and the screen asks the user to try again.
- **Security Measures**
  - Only administrators may approve or reject a request, and the check is made again on the server for every decision.
  - A request can only be decided while it is still pending, so a decision cannot be applied

twice to the same request.

- Each decision is recorded together with the administrator who made it, the note and the moment of the decision, so every review can be traced.

4.6 Trash

4.6.1 View Deleted Post List

Function Trigger: The logged-in administrator opens the Trash screen by choosing **Forum** → **Trash** in the admin sidebar while the **Posts** tab is selected.

Function description: This function allows an administrator to see all the posts that have been removed from the forum, including the posts that were rejected during moderation. Every row shows the identifier, the title, the status, the author and the moment of deletion, so the administrator can decide which posts to restore.

Screen layout:

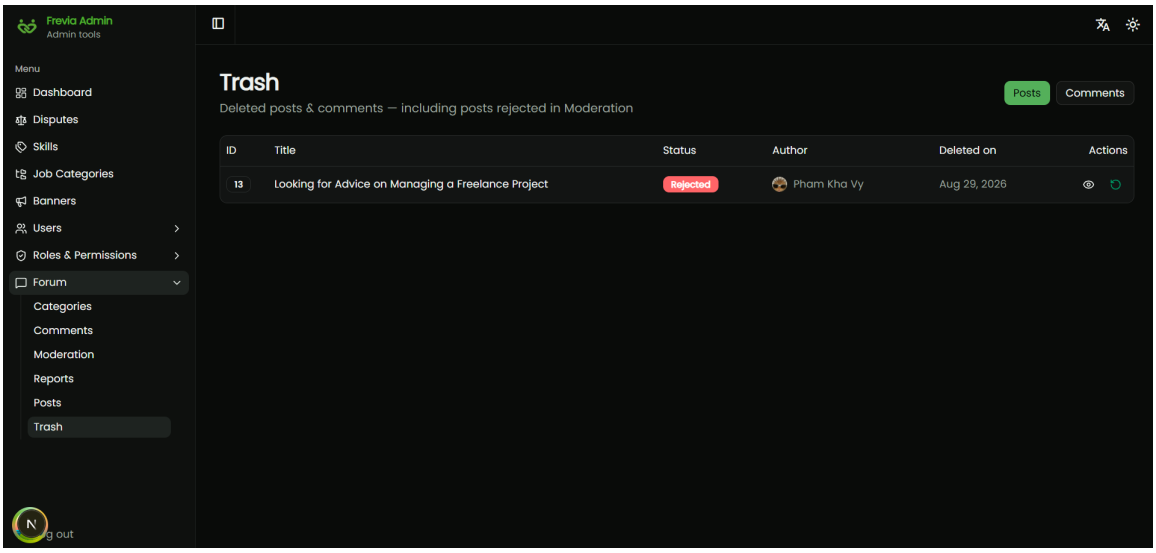


Figure 51: Deleted post list screen

Function details:

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - At least one post has been deleted or rejected, otherwise the trash is empty.
- **Process**
  - **Open the Trash Screen**: The user selects **Forum** in the sidebar and then **Trash**, and

the screen titled “Trash” is displayed with the tab **Posts** active.

- **Read the Summary Line**: The subtitle explains that the trash contains deleted posts and comments, including the posts that were rejected during moderation.
- **Read Each Row**: Every row of the table presents the following information.
  - **ID**: The identifier of the post, shown as a small badge.
  - **Title**: The title of the post, shortened when it is too long.
  - **Status**: The moderation status of the post, shown as the badge Rejected, Pending or Approved.
  - **Author**: The avatar and the display name of the user who wrote the post, or “User #id” when the user has no display name.
  - **Deleted on**: The date on which the post was deleted, or “—” when the post is in the trash because it was rejected.
- **Switch to Comments**: The user presses the **Comments** tab at the top right of the screen to see the deleted comments instead (see **View Deleted Comment List**).
- **Move Between Pages**: When there is more than one page, the user uses the page numbers or the **Previous** and **Next** buttons at the bottom of the list.
- **Choose a Post**: The user opens a post with the view button, or restores it with the restore button in the row (see **View Deleted Post Detail** and **Restore Deleted Post**).

- **Alternative Flow**
  - **Empty Trash**: When nothing has been deleted or rejected, the table shows the message “Trash is empty.”.
  - **Rejected Post Without a Deletion Date**: When a post is in the trash because it was rejected, the column shows “—” while the status badge shows Rejected.
- **Postconditions**
  - The administrator sees all the posts that are currently out of the forum.
  - No data is changed by this function, so the trash stays exactly as it was.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator opens the screen, the user is sent to the access-denied page, because only administrators may read the trash.
  - **Lost Connection**: When the trash cannot be loaded, the screen shows an empty list so that the administrator can reload the page and try again.
- **Security Measures**
  - The trash is only returned to administrator accounts, so deleted and rejected posts are never visible to ordinary users.
  - A rejected post is kept in the trash instead of being erased, so it can be restored later

if the decision was wrong.

4.6.2 View Deleted Post Detail

Function Trigger: The logged-in administrator clicks the view button (the eye icon) at the end of a row in the deleted post list.

Function description: This function allows an administrator to read a deleted or rejected post in full before restoring it. The dialog shows the title, the status, the author, the creation date, the moment of deletion and the complete content of the post.

Screen layout:

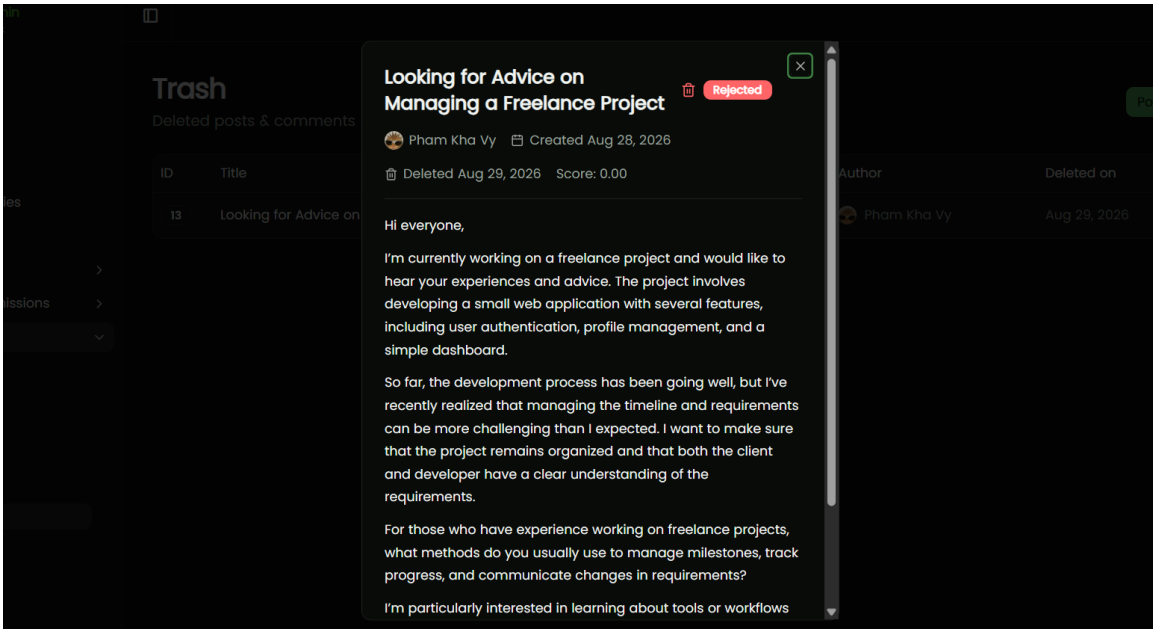


Figure 52: Deleted post detail dialog

Function details:

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The user is on the Trash screen with the **Posts** tab active and the post exists in the trash.
- **Process**
  - **Open the Detail**: The user clicks the view button on a row, and a dialog opens with the title of the post and its status badge as its heading.
  - **Read the Summary**: Under the title, the dialog shows the author of the post, the creation date, the moment of deletion and the moderation score.

        - When the post is in the trash because it was rejected and not because it was deleted, the line reads “In trash (rejected)” instead of the deletion date.
    - **Read the Content**: The dialog shows the whole content of the post with its formatting, exactly as it was written by the author.
    - **Scroll When Needed**: When the post is long, the dialog becomes scrollable so that the whole content can be read.
    - **Close the Detail**: The user closes the dialog and returns to the trash list.
- **Alternative Flow**
    - **No Score**: When the automatic check produced no score, the badge shows “N/A” and the rest of the dialog is unchanged.
    - **Rejected Post**: When the post was rejected during moderation, the dialog shows the Rejected badge and the line “In trash (rejected)”.
- **Postconditions**
    - The administrator has read the complete post and knows why it is out of the forum.
    - Nothing has changed while only viewing the post.
- **Error Handling**
    - **Not Allowed**: When a user who is not an administrator tries to open the detail, the action is refused because only administrators may read the trash.
    - **Post Not Found**: When the post is no longer in the trash, the detail cannot be opened and a message tells the user that the post cannot be found.
    - **Lost Connection**: When the detail cannot be loaded, the screen asks the user to try again.
- **Security Measures**
    - The detail is only available inside the admin area, so the content of a deleted post is never shown to ordinary users.
    - The dialog is read-only, so the content of a deleted post cannot be edited from this screen.

4.6.3 Restore Deleted Post

Function Trigger: The logged-in administrator clicks the restore button (the round arrow icon) at the end of a row in the deleted post list.

Function description: This function allows an administrator to bring a post back into the forum. A post that was only deleted becomes visible again straight away, while a post that was rejected goes back to the moderation queue so that it can be reviewed again. Before the post is restored, the administrator is asked to confirm the action.

Screen layout:

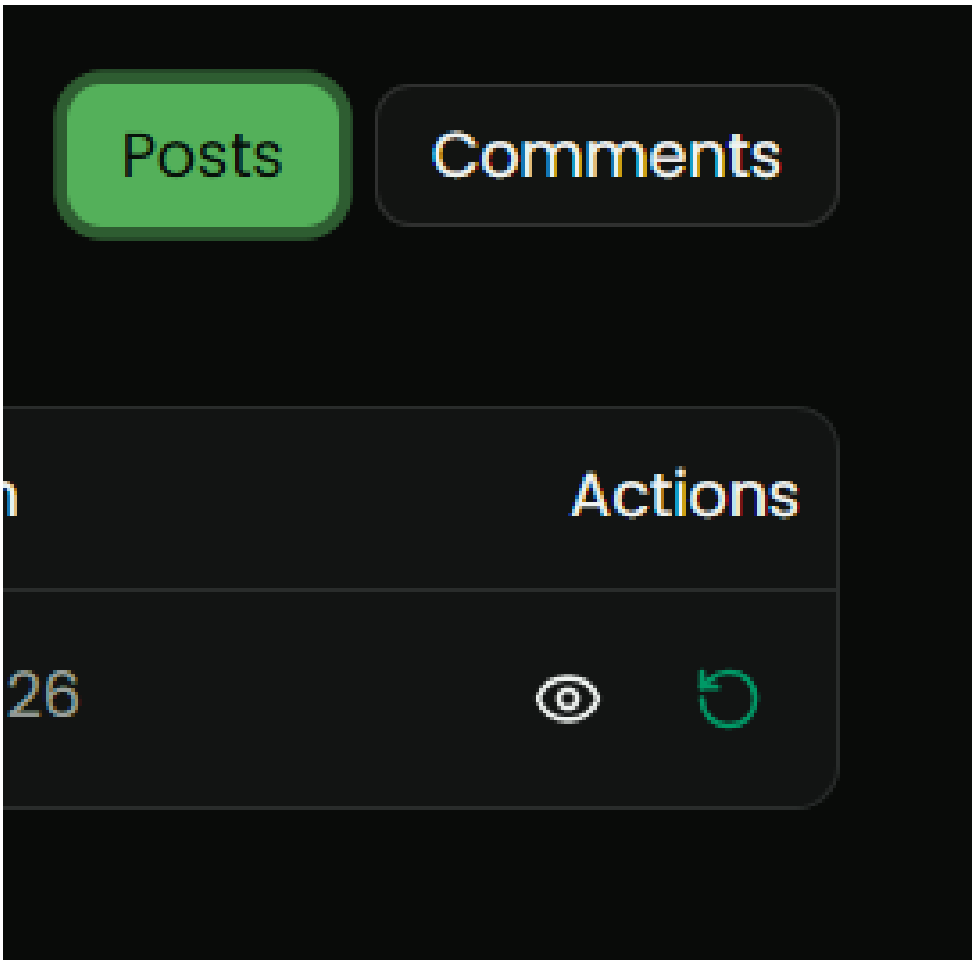


Figure 53: Restore post confirmation dialog

Function details:

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The post is currently in the trash, either because it was deleted or because it was rejected during moderation.
- **Process**
  - **Start the Restore**: The user clicks the restore button on a row, and a confirmation dialog titled “Restore Post” appears.
  - **Read the Confirmation**: The dialog shows the question “Restore “title” back to the forum?” and explains what will happen.
    - When the post was rejected, the dialog adds “It will return to the moderation queue for review.”.

        - When the post was only deleted, the dialog adds “It will be visible publicly again immediately.”.
    - **Confirm the Restore**: The user presses **Restore**, and the button shows “Restoring…” while the action is being applied.
    - **Apply the Restore**: The post is taken out of the trash and the screen shows the message “Post restored successfully”.
        - A post that was only deleted becomes visible in the forum again straight away.
        - A post that was rejected is put back into the moderation queue with the status Pending, so the administrator can review it again in the Moderation screen.
    - **Update the Trash**: The restored post disappears from the trash list and the remaining rows move up.
- **Alternative Flow**
    - **Cancel the Restore**: When the user presses **Cancel** or closes the dialog, the post stays in the trash and nothing changes.
    - **Empty Trash**: When the last post is restored, the table shows the message “Trash is empty.”.
    - **Restore Several Posts**: The user repeats the action on other rows, one post at a time.
- **Postconditions**
    - The post is no longer in the trash.
    - A restored post that was only deleted is visible in the forum again.
    - A restored post that was rejected is waiting again in the moderation queue with the status Pending and without a previous review.
- **Error Handling**
    - **Not Allowed**: When a user who is not an administrator tries to restore a post, the action is refused because only administrators may restore content.
    - **Post Not Found**: When the post is no longer in the trash, the restore fails and the screen shows the message “Couldn’t restore post. Try again.”.
    - **Cannot Restore**: When the post cannot be restored, it stays in the trash and a message asks the administrator to try again.
    - **Lost Connection**: When the connection to the server is lost, the restore is not applied and the screen asks the user to try again.
- **Security Measures**
    - Only administrators may restore a post, and the check is made again on the server for every restore.
    - Only a post that is really in the trash can be restored, so the action cannot be used on a post that is already visible in the forum.

- A restored post that had been rejected is never made public directly: it is put back into the moderation queue, so it must pass a new review before it can be seen by users.

4.6.4 View Deleted Comment List

Function Trigger: The logged-in administrator opens the Trash screen by choosing **Forum** → **Trash** in the admin sidebar and then presses the **Comments** tab.

Function description: This function allows an administrator to see all the comments that have been removed from the forum. Every row shows the identifier, the text of the comment, the post it belongs to, the author and the moment of deletion, so the administrator can decide which comments to restore.

Screen layout:

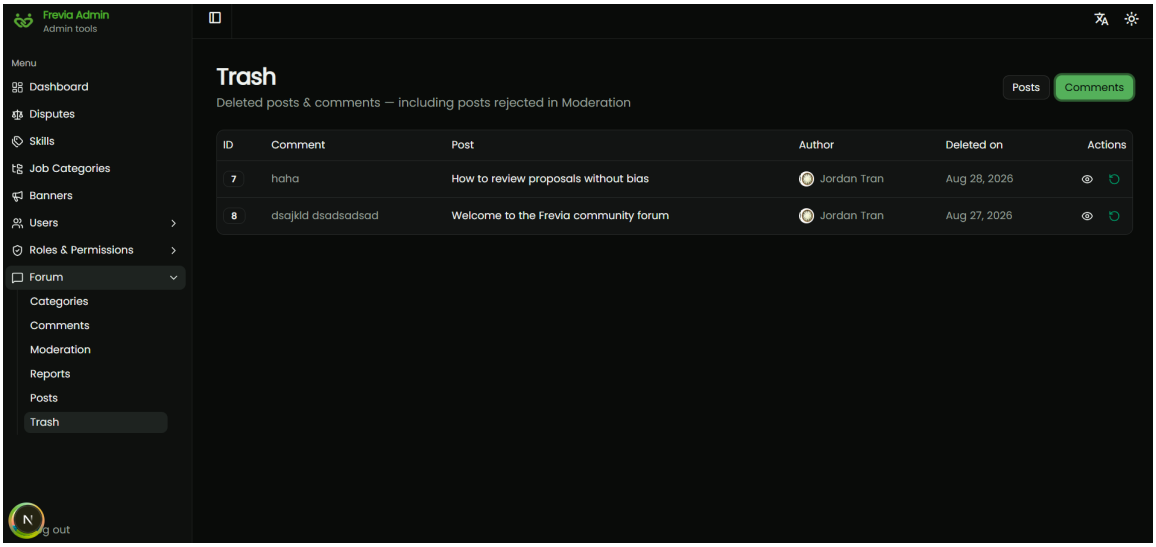


Figure 54: Deleted comment list screen

Function details:

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - At least one comment has been deleted, otherwise the trash is empty.
- **Process**
  - **Open the Comments Tab**: The user opens the Trash screen and presses the **Comments** tab at the top right, and the screen titled "Trash" is displayed with the comments of the trash.
  - **Read Each Row**: Every row of the table presents the following information.

        - **ID**: The identifier of the comment, shown as a small badge.
        - **Comment**: The text of the comment, shortened when it is too long.
        - **Post**: The title of the post that the comment belongs to, or “Post #id” when the title is not available.
        - **Author**: The avatar and the display name of the user who wrote the comment, or “User #id” when the user has no display name.
        - **Deleted on**: The date on which the comment was deleted.
    - **Switch to Posts**: The user presses the **Posts** tab at the top right of the screen to see the deleted posts instead (see **View Deleted Post List**).
    - **Move Between Pages**: When there is more than one page, the user uses the page numbers or the **Previous** and **Next** buttons at the bottom of the list.
    - **Choose a Comment**: The user opens a comment with the view button, or restores it with the restore button in the row (see **View Deleted Comment Detail** and **Restore Deleted Comment**).
- **Alternative Flow**
    - **Empty Trash**: When no comment has been deleted, the table shows the message “Trash is empty.”.
    - **Missing Post Title**: When the post of the comment is not available, the row shows only the identifier of the post.
- **Postconditions**
    - The administrator sees all the comments that are currently out of the forum.
    - No data is changed by this function, so the trash stays exactly as it was.
- **Error Handling**
    - **Not Allowed**: When a user who is not an administrator opens the screen, the user is sent to the access-denied page, because only administrators may read the trash.
    - **Lost Connection**: When the trash cannot be loaded, the screen shows an empty list so that the administrator can reload the page and try again.
- **Security Measures**
    - The list of deleted comments is only returned to administrator accounts, so a deleted comment is never shown to ordinary users.
    - The delete action of a comment does not erase its content, so the administrator can still read what was removed before deciding to restore it.

4.6.5 View Deleted Comment Detail

Function Trigger: The logged-in administrator clicks the view button (the eye icon) at the end of a row in the deleted comment list.

Function description: This function allows an administrator to read a deleted comment in full before restoring it. The dialog shows the post that the comment belongs to, the author, the complete text of the comment, the identifier of the comment and the moment of deletion.

Screen layout:

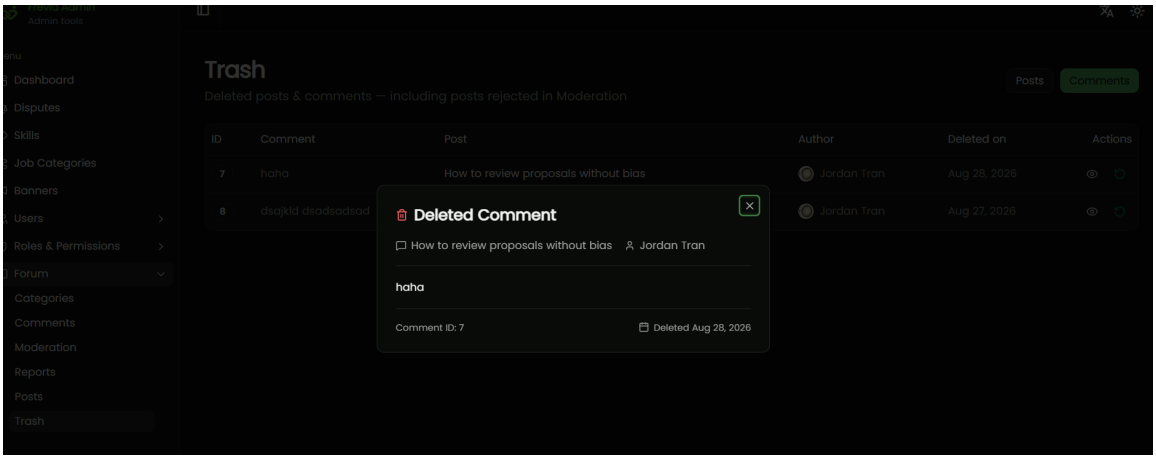


Figure 55: Deleted comment detail dialog

Function details:

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The user is on the Trash screen with the **Comments** tab active and the comment exists in the trash.
- **Process**
  - **Open the Detail**: The user clicks the view button on a row, and a dialog titled “Deleted Comment” opens.
  - **Read the Summary**: Under the title, the dialog shows the title of the post that the comment belongs to and the name of the author.
  - **Read the Content**: The dialog shows the complete text of the comment, exactly as it was written by the author.
  - **Read the Footer**: The footer of the dialog shows the identifier of the comment and the moment at which it was deleted.
  - **Close the Detail**: The user closes the dialog and returns to the trash list.

- **Alternative Flow**
  - **Missing Post Title**: When the title of the post is not available, the dialog shows only the identifier of the post.
  - **Missing Deletion Date**: When the deletion date is not available, the footer shows "—".
- **Postconditions**
  - The administrator has read the complete comment and knows to which post it belongs.
  - Nothing has changed while only viewing the comment.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to open the detail, the action is refused because only administrators may read the trash.
  - **Comment Not Found**: When the comment is no longer in the trash, the detail cannot be opened and a message tells the user that the comment cannot be found.
  - **Lost Connection**: When the detail cannot be loaded, the screen asks the user to try again.
- **Security Measures**
  - The detail is only available inside the admin area, so the content of a deleted comment is never shown to ordinary users.
  - The dialog is read-only, so the content of a deleted comment cannot be edited from this screen.

4.6.6 Restore Deleted Comment

Function Trigger: The logged-in administrator clicks the restore button (the round arrow icon) at the end of a row in the deleted comment list.

Function description: This function allows an administrator to bring a deleted comment back to its post. The comment is shown again to users when its post is still visible in the forum. Before the comment is restored, the administrator is asked to confirm the action.

Screen layout:

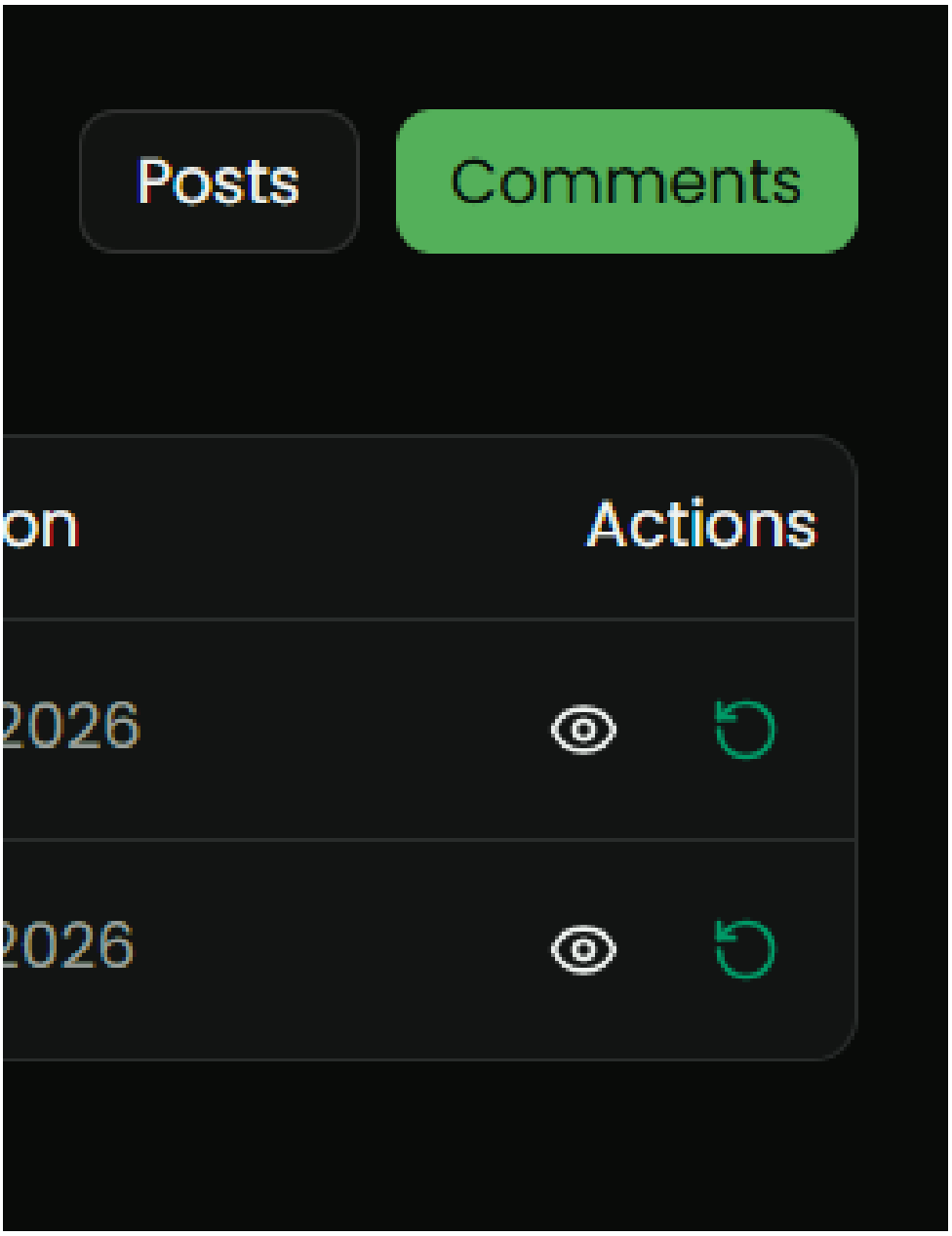


Figure 56: Restore comment confirmation dialog

Function details:

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The comment is currently in the trash because it was deleted.
- **Process**
  - **Start the Restore**: The user clicks the restore button on a row, and a confirmation dialog titled “Restore Comment” appears.
  - **Read the Confirmation**: The dialog shows the question “Restore this comment on

post “title”? It will be visible again (as long as its post is not deleted).”.

- **Confirm the Restore**: The user presses **Restore**, and the button shows “Restoring. . . ” while the action is being applied.
- **Apply the Restore**: The comment is taken out of the trash and the screen shows the message “Comment restored successfully”.
  - The comment appears again under its post when the post itself is still visible in the forum.
  - When the post has also been deleted, the comment stays hidden until the post is restored as well.
- **Update the Trash**: The restored comment disappears from the trash list and the remaining rows move up.

- **Alternative Flow**
  - **Cancel the Restore**: When the user presses **Cancel** or closes the dialog, the comment stays in the trash and nothing changes.
  - **Deleted Post**: When the post of the comment has also been deleted, the comment is restored but remains hidden until the post comes back to the forum.
  - **Empty Trash**: When the last comment is restored, the table shows the message “Trash is empty.”.
- **Postconditions**
  - The comment is no longer in the trash and is visible again under its post when the post is visible.
  - The rest of the comments and the post itself are unchanged.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to restore a comment, the action is refused because only administrators may restore content.
  - **Comment Not Found**: When the comment is no longer in the trash, the restore fails and the screen shows the message “Couldn’t restore comment. Try again.”.
  - **Cannot Restore**: When the comment cannot be restored, it stays in the trash and a message asks the administrator to try again.
  - **Lost Connection**: When the connection to the server is lost, the restore is not applied and the screen asks the user to try again.
- **Security Measures**
  - Only administrators may restore a comment, and the check is made again on the server for every restore.
  - Only a comment that is really in the trash can be restored, so the action cannot be used on a comment that is already visible.

- A restored comment can never become visible outside a visible post, so the owner of a deleted post cannot see comments that belong to removed content.

5 Non-Functional Requirement

This section describes the non-functional requirements of the Frevia platform, covering performance, security, availability, usability, and Web3-specific requirements.

5.1 Performance & Scalability

- **Response Time:** The system's API read operations must respond within 200ms under normal load (up to 500 concurrent requests). Write/update operations, excluding blockchain transactions, must complete within 500ms.
- **Page Load Time:** The web application must achieve a Largest Contentful Paint (LCP) of less than 2.5 seconds, adhering to Google's Core Web Vitals guidelines.
- **Concurrency:** The system backend must support at least 1,000 concurrent active users and handle a database connection pool efficiently.
- **Search and Filtering Efficiency:** The job search page must employ debouncing (300ms delay) and pagination (default 10 items per page) to optimize query performance and reduce server load.

5.2 Security & Privacy

- **Authentication and Authorization:** All network communications must be encrypted using HTTPS/TLS 1.3. User passwords must be hashed using bcrypt with a work factor of 12.
- **Session Management:** JWT Access Tokens must have a short lifespan (e.g., 15 minutes) and be stored in memory. Refresh Tokens must have a longer lifespan (e.g., 7 days), be stored in HTTP-only, Secure, and SameSite=Strict cookies, and be invalidated in the database upon user logout.
- **Role-Based Access Control (RBAC):** The system must strictly enforce role boundaries (Guest, Freelancer, Client, Admin). Administrative features, such as changing smart contract addresses or viewing audit logs, must be blocked by a server-side gateway for non-admin accounts.

- **Web3 Wallet and Transaction Safety:** Input wallet addresses must be verified on the client and server side to conform to the standard EVM format (starts with `0x` followed by 40 hexadecimal characters).

5.3 Availability & Reliability

- **System Uptime:** The system core services must maintain a 99.9% monthly uptime, excluding scheduled maintenance.
- **Blockchain Fault Tolerance:** In case of RPC node connection failures, the platform must gracefully fall back to backup RPC endpoints within 3 seconds to avoid breaking the wallet balance display or transaction signing flows.
- **Escrow Reliability:** Funds locked in the Escrow Smart Contracts must be secure. Contracts must be verified on Etherscan/BscScan and use standard libraries (e.g., OpenZeppelin) to prevent vulnerabilities.

5.4 Usability & User Experience

- **Responsive Design:** The user interface must be fully responsive, supporting mobile devices, tablets, and desktop viewports (minimum width 320px).
- **Toast Notification System:** Interactive toast notifications (success, warning, error) must have the highest z-index to overlay all content, auto-dismiss after 3000ms, and must not freeze or block other user interactions.
- **Client-side Image Processing:** The avatar cropping tool must handle images locally before upload. The system must restrict upload sizes to under 5MB and validate formats (.jpg, .jpeg, .png, .webp) before initiating API requests.